

LLM 기반 데이터 분석 실무 입문

챕터 0~15 통합 강의안

생성일: 2026-07-11

목차

목차 항목을 찾을 수 없습니다.

0장. 이 책의 방향과 전체 흐름

데이터 분석을 배운다는 것은 pandas 문법을 외우거나 그래프 몇 개를 그리는 일에 머물지 않습니다. 실제 분석 프로젝트에서는 데이터를 이해하고, 정리하고, 질문을 만들고, 지표를 계산하고, 결과를 해석해야 합니다. 여기에 머신러닝, 대규모 언어 모델(LLM), 외부 데이터, 자동화 도구까지 연결되면 하나의 실무형 분석 과정이 만들어집니다.

이 책은 이러한 과정을 처음부터 끝까지 자연스럽게 경험하기 위한 입문서입니다. Python과 VS Code를 기반으로 분석 환경을 준비하고, pandas와 시각화로 데이터를 탐색한 뒤, 회귀와 분류 모델링으로 예측 분석의 기본을 살펴봅니다. 이후에는 LLM을 분석 질문 정리, 코드 초안 작성, 코드 검증, 결과 해석의 보조 도구로 활용하고, 외부 데이터 수집과 자동화 파이프라인으로 프로젝트를 확장합니다.

이 책에서 강조하는 관점은 분명합니다. AI가 분석을 대신하는 것이 아니라, **사람이 분석 목적과 판단 기준을 세우고, AI와 자동화 도구를 활용해 반복 작업과 초안 작성의 효율을 높이는 것**입니다. LLM이 만든 코드와 문장은 항상 사람이 검토해야 하며, 데이터에 없는 원인을 단정하지 않는 태도가 중요합니다.

1. 이 책에서 다루는 분석 흐름

이 책의 전체 흐름은 하나의 데이터 분석 프로젝트가 점점 확장되는 과정과 비슷합니다.

먼저 AI 시대의 데이터 분석을 어떤 관점으로 바라볼지 정리하고, VS Code 중심의 Python 분석 환경을 준비합니다. 그다음 CSV 파일의 구조를 이해하고, pandas로 데이터에 질문을 던지며, 결측치와 중복, 타입 문제를 전처리합니다. 이어서 탐색적 데이터 분석(EDA) 질문을 만들고, 시각화로 결과를 더 직관적으로 확인합니다. 중간 프로젝트에서는 지금까지의 내용을 하나의 작은 분석 프로젝트로 묶어 봅니다.

후반부에서는 머신러닝으로 넘어갑니다. 회귀 분석에서는 숫자를 예측하고, 분류 분석에서는 범주를 예측합니다. 이후 LLM을 활용해 분석 질문을 다듬고, LLM이 만든 분석 코드를 검증하는 방법을 다룹니다. 마지막으로 공공데이터, 네이버 API, 크롤링을 통해 외부 데이터로 분석을 확장하고, Make,

n8n, Airflow를 활용해 반복되는 분석 과정을 자동화하는 방법을 살펴봅니다. 크롤링과 외부 데이터 수집 시에는 대상 서비스의 이용약관, 저작권, 개인정보 보호 기준도 함께 확인해야 합니다.

2. 이 책을 읽기 전에 알면 좋은 것

이 책은 데이터 분석을 처음 시작하는 독자도 따라올 수 있도록 구성되어 있습니다. 다만 모든 내용을 더 편하게 이해하려면 Python 기본 문법을 어느 정도 알고 있는 것이 좋습니다. 특히 변수, 리스트, 딕셔너리, 조건문, 반복문, 함수, 모듈 불러오기(import), 파일 경로 개념에 익숙하면 pandas와 머신러닝 실습을 훨씬 수월하게 따라갈 수 있습니다.

Python 문법을 완벽하게 알고 시작할 필요는 없습니다. 실습을 진행하면서 필요한 문법을 그때그때 확인해도 충분합니다. 다만 코드의 의미가 잘 보이지 않거나 오류 메시지를 읽기 어렵다면, Python 기본 문법 학습을 병행하는 것이 좋습니다. 이 책에서 다루는 데이터 분석 코드는 대부분 기본 문법 위에 pandas, matplotlib, scikit-learn 같은 라이브러리를 얹어 사용하는 형태이기 때문입니다.

다음 내용을 알고 있다면 이 책을 읽는 데 도움이 됩니다.

선수 내용	알고 있으면 좋은 이유
변수와 자료형	숫자, 문자열, 날짜, 표 형태 데이터를 구분하는 데 필요합니다.
리스트와 딕셔너리	여러 값을 묶거나 컬럼 목록, 설정값을 다룰 때 자주 사용합니다.
조건문과 반복문	데이터 처리 흐름을 이해하고 반복 작업을 줄이는 데 도움이 됩니다.
함수	전처리나 분석 코드를 재사용 가능한 형태로 정리할 때 필요합니다.
모듈 불러오기(import)	pandas, matplotlib, scikit-learn 같은 라이브러리를 불러올 때 사용합니다.
파일과 폴더 경로	CSV 파일을 읽고 결과 파일을 저장할 때 자주 마주칩니다.

Python 기본 문법이 아직 익숙하지 않다면, 이 책을 읽기 전에 짧게 복습하거나 2~4장을 학습하면서 필요한 문법을 함께 익히는 방식을 권장합니다. 중요한 것은 모든 문법을 먼저 끝내는 것이 아니라, 데이터 분석 과정에서 필요한 문법을 반복해서 익히는 것입니다.

3. 전체 목차

장	주제	핵심 내용
0장	이 책의 방향과 전체 흐름	도서 개요, 선수 내용, 분석 흐름, 전체 목차

장	주제	핵심 내용
1장	AI와 함께하는 데이터 분석의 시작	AI 시대의 데이터 분석 관점, LLM의 역할, 분석가의 판단 책임
2장	VS Code에서 시작하는 데이터 분석 환경	Python, VS Code, Git-GitHub, 가상환경, Jupyter Notebook 실행 준비
3장	데이터의 첫인상 읽기	CSV 파일 구조, 행과 열의 구조, 각 컬럼의 의미, 데이터 타입, 기본 점검
4장	pandas로 데이터에 질문하기	컬럼 선택, 필터링, 정렬, 파생 컬럼, 병합, 기본 집계
5장	분석을 믿을 수 있게 만드는 데이터 전처리	결측치, 중복, 타입 변환, 이상값, 파생 컬럼, 전처리 결과 저장
6장	데이터를 보며 질문을 만드는 EDA	탐색적 데이터 분석(EDA) 질문 만들기, 지표 설계, pandas 집계와 해석 주의점
7장	데이터 시각화	막대그래프, 선그래프, 분포 그래프, 시각화 해석
8장	작은 데이터 분석 프로젝트 완성하기	EDA, 전처리, 시각화, 보고서를 하나의 프로젝트로 통합
9장	회귀 분석으로 숫자 예측하기	숫자 예측 문제, 입력값과 정답 분리, 훈련/테스트 데이터 분할, 회귀 평가
10장	분류 분석으로 주문 취소 여부 예측하기	범주 예측, 주문 취소 여부, 분류 모델, 정밀도(precision), 재현율(recall), F1 점수
11장	LLM과 함께 분석 질문을 다듬기	데이터 구조 요약, 분석 질문 프롬프트, LLM 답변 검토
12장	LLM이 만든 분석 코드를 검증하는 방법	코드 생성, 실행 전 검증, 실행 후 검증, 데이터 누수 점검
13장	외부 데이터로 분석을 확장하기	공공데이터, 네이버 API, 크롤링, 외부 데이터 연결과 검증, 이용약관·저작권·개인정보 확인
14장	반복되는 분석 흐름을 자동화하기	Make, n8n, Airflow를 활용한 분석 자동화와 파이프라인 설계
15장	하나의 데이터 분석 프로젝트로 완성하기	EDA, 시각화, 머신러닝, LLM, 외부 데이터, 자동화 아이디어 통합

이 목차는 데이터 분석의 기본기에서 출발해 예측 모델링, LLM 활용, 외부 데이터 수집, 자동화, 최종 프로젝트로 이어지도록 구성되어 있습니다. 순서대로 읽으면 하나의 프로젝트가 점차 확장되는

과정을 경험할 수 있고, 필요한 장만 골라 읽어도 특정 주제의 기본 개념과 실습 과정을 확인할 수 있습니다.

4. 이 책의 도구 구성

이 책에서 사용하는 도구는 크게 네 가지 역할로 나눌 수 있습니다.

역할	도구	사용 목적
분석 환경	Python, VS Code, Jupyter Notebook	코드 작성, 실행, 결과 확인
데이터 분석	pandas, matplotlib, scikit-learn	전처리, EDA, 시각화, 머신러닝
협업과 기록	Git-GitHub, Markdown	실습 파일 관리, 보고서 작성, 프로젝트 기록
AI와 자동화	LLM, Make, n8n, Airflow	코드 초안, 결과 검토, 외부 연결, 반복 실행 자동화

Jupyter Notebook은 별도의 웹 환경을 중심으로 사용하기보다, VS Code 안에서 간단히 열고 실행하는 방식으로 다룹니다. 주요 작업 공간은 VS Code이며, 프로젝트 폴더와 터미널, Jupyter Notebook, Git 변경 사항과 GitHub 저장소를 한곳에서 관리하는 방식을 권장합니다.

5. LLM을 사용하는 관점

이 책에서 LLM은 분석을 대신 수행하는 존재가 아닙니다. LLM은 다음 작업을 도와주는 보조 도구입니다.

- 분석 질문을 더 구체적인 지표로 바꾸기
- 데이터 구조를 설명 가능한 형태로 정리하기
- pandas 코드 초안 작성하기
- 오류 메시지의 원인 설명하기
- 시각화 코드나 모델링 코드의 구조 제안하기
- 보고서 문장을 더 자연스럽게 다듬기
- 코드와 해석에서 빠진 검증 항목 찾기

하지만 LLM의 답변은 항상 검증해야 합니다. 데이터에 없는 컬럼명을 만들거나, 원인을 단정하거나, 모델 평가 지표를 잘못 선택할 수 있습니다. 특히 개인정보, API 키, 내부 데이터, 민감한 파일 경로는 LLM 프롬프트에 포함하지 않아야 합니다.

6. 프로젝트 중심으로 읽기

이 책은 각 장을 따로 읽어도 되지만, 전체적으로는 하나의 프로젝트가 점차 확장되는 학습 과정으로 읽는 것이 좋습니다.

처음에는 온라인 쇼핑몰 데이터를 사용해 고객, 상품, 주문, 매출을 이해합니다. 중간 프로젝트에서는 이 데이터를 바탕으로 기본 분석 보고서를 완성합니다. 후반부에는 이 데이터를 회귀와 분류 문제로 확장하고, LLM을 활용해 분석 질문과 코드를 검토하며, 외부 데이터와 자동화 도구를 연결해 더 실무적인 프로젝트 구조로 발전시킵니다.

따라서 각 장의 목표는 단순히 기능 하나를 익히는 것이 아니라, 다음 질문에 답할 수 있는 분석 체계를 만드는 것입니다.

- 현재 데이터로 무엇을 알 수 있는가?
- 그 결과를 어떤 코드와 지표로 확인할 수 있는가?
- 해석할 때 조심해야 할 점은 무엇인가?
- LLM이나 자동화 도구를 어디까지 활용할 수 있는가?
- 최종 판단은 어떤 근거로 사람이 내려야 하는가?

7. 이 책을 마친 뒤 기대할 수 있는 것

이 책을 끝까지 따라가면 간단한 데이터 분석 프로젝트를 스스로 구성할 수 있습니다. 원본 데이터를 불러오고, 구조를 확인하고, 전처리하고, EDA 질문을 만들고, 시각화와 기본 머신러닝 모델을 적용한 뒤, LLM과 자동화 도구를 보조적으로 활용하는 전체 과정을 경험하게 됩니다.

완성도 높은 분석은 화려한 모델에서 시작되지 않습니다. 좋은 질문, 정리된 데이터, 재현 가능한 코드, 신중한 해석, 그리고 검증 가능한 결과에서 시작됩니다. 이 책의 목적은 바로 그 기본 과정을 자연스럽게 익히는 것입니다.

1장. AI와 함께하는 데이터 분석의 시작

데이터 분석은 숫자를 계산하는 일로만 보이기 쉽습니다. 하지만 실제 분석은 숫자보다 질문에서 시작됩니다. 무엇이 궁금한지, 어떤 의사결정에 도움이 되어야 하는지, 지금 가진 데이터로 그 질문에 답할 수 있는지를 차례대로 확인하는 과정이 데이터 분석입니다.

예를 들어 “매출이 줄었다”는 말은 분석의 출발점일 뿐입니다. 분석을 하려면 먼저 매출을 어떤 기준으로 볼지 정해야 합니다. 전체 매출인지, 결제 완료 주문의 매출인지, 취소·환불을 제외한 순매출인지 구분해야 합니다. 기간도 정해야 합니다. 최근 1주일인지, 최근 1개월인지, 전월 대비인지, 전년 동월 대비인지에 따라 분석 결과는 달라집니다.

대규모 언어 모델(LLM)의 등장은 이 과정을 크게 바꾸고 있습니다. ChatGPT나 Gemini 같은 도구는 분석 질문을 정리하고, pandas 코드 초안을 작성하고, 오류 메시지를 설명하고, 그래프 해석이나 보고서 문장을 다듬는 데 도움을 줄 수 있습니다. 그러나 분석의 결론을 책임지는 주체는 여전히 사람입니다. AI는 분석가를 대신한다기보다, 분석가가 더 빠르게 생각하고 더 넓게 검토하도록 돕는 도구에 가깝습니다.

이 책은 Python 데이터 분석의 기본 과정을 익힌 뒤, 간단한 머신러닝과 LLM 활용, 외부 데이터 수집, 보고서 작성, 자동화 개념까지 자연스럽게 확장하도록 구성되어 있습니다. 하나의 온라인 쇼핑몰 데이터를 바탕으로 탐색적 데이터 분석(EDA), 시각화, 회귀·분류 모델, LLM 프롬프트, 보고서, 자동화 아이디어까지 연결해 봅니다.

이 장은 본격적인 코딩에 들어가기 전의 안내판입니다. 데이터 분석을 어떤 관점으로 바라봐야 하는지, LLM을 어디까지 믿고 어디서부터 검증해야 하는지, 그리고 실습 환경을 어떻게 준비하면 좋은지 먼저 정리합니다.

이 장에서 생각해 볼 질문

이 장은 본격적인 코딩에 들어가기 전에, AI 시대의 데이터 분석을 어떤 관점으로 바라볼지 정리하는 도입부입니다. 다음 질문을 염두에 두고 읽어 보겠습니다.

- 데이터 분석은 단순히 코드를 실행하는 일일까?

- 업무 질문은 어떻게 분석 가능한 질문으로 바뀔까?
- LLM은 데이터 분석에서 어디까지 도와줄 수 있을까?
- AI가 만든 코드와 해석을 그대로 믿어도 될까?
- 좋은 데이터 분석가는 어떤 역할을 해야 할까?
- 실습 프로젝트는 어떤 폴더 구조로 정리하면 좋을까?
- 개인정보와 API 키는 어떻게 보호해야 할까?

1. 데이터 분석은 질문에서 시작된다

온라인 쇼핑몰 운영자가 다음과 같이 말한다고 가정해 보겠습니다.

- 요즘 매출이 줄어든 것 같은데 왜 그런가요?

이 문장은 자연스러운 업무 질문입니다. 하지만 이 상태로는 바로 코드를 작성하기 어렵습니다. “요즘”이 어느 기간인지, “매출”이 주문 금액인지 결제 완료 금액인지, “줄어든 것”을 전월 대비로 볼지 전년 동월 대비로 볼지 정해야 합니다.

데이터 분석가는 이런 질문을 더 구체적인 분석 질문으로 바꿉니다.

- 최근 6개월 동안 월별 총매출은 어떻게 변했는가?
- 매출 감소가 특정 상품 카테고리에서 두드러지게 나타나는가?
- 주문 수가 줄었는가, 객단가가 줄었는가?
- 신규 고객 수나 재구매 고객 수의 변화와 관련이 있는가?
- 결제 수단이나 주문 상태별로 차이가 있는가?

이렇게 질문을 바꾸면 어떤 데이터가 필요한지, 어떤 컬럼을 확인해야 하는지, 어떤 그래프를 그려야 하는지 판단할 수 있습니다. 데이터 분석은 막연한 궁금증을 측정 가능한 질문으로 바꾸는 일에서 시작됩니다.

질문을 구체화할 때는 다음 네 가지를 함께 생각하면 좋습니다.

구분	확인할 내용	예시
대상	무엇을 분석할 것인가?	전체 매출, 카테고리별 매출, 고객별 구매 금액
기준	어떤 단위로 볼 것인가?	월별, 지역별, 카테고리별, 고객별
비교	무엇과 비교할 것인가?	전월 대비, 카테고리 간 비교, 상위 고객과 일반 고객 비교
목적	결과를 어디에 쓸 것인가?	매출 변화 요인 파악, 캠페인 기획, 재고 관리, 고객 관리

분석 질문이 모호하면 코드도 모호해집니다. 반대로 질문이 명확하면 필요한 데이터와 코드가 자연스럽게 정리됩니다.

예를 들어 다음 두 질문은 비슷해 보이지만 분석 방식이 다릅니다.

- 어떤 상품이 잘 팔리나요?

이 질문은 판매 수량 기준으로 볼 수도 있고, 매출 기준으로 볼 수도 있습니다. 판매 수량이 많은 상품과 매출이 높은 상품은 다를 수 있습니다. 단가가 낮은 상품은 많이 팔려도 매출 기여도가 낮을 수 있고, 단가가 높은 상품은 적게 팔려도 매출 기여도가 높을 수 있습니다.

따라서 더 좋은 질문은 다음과 같습니다.

- 상품별 총 판매 수량과 총매출을 각각 계산했을 때, 상위 상품은 어떻게 달라지는가?

이렇게 질문을 바꾸면 `quantity`, `unit_price`, `product_id`, `product_name` 같은 컬럼이 필요하다는 점이 분명해집니다. 총매출을 계산할 때는 `quantity`와 `unit_price`를 곱해 `line_total`과 같은 파생 컬럼을 만들 수도 있습니다.

2. 데이터 분석의 기본 흐름

전통적인 데이터 분석은 대체로 다음 흐름으로 진행됩니다.

1. 문제 정의
2. 데이터 수집
3. 데이터 구조 이해
4. 데이터 전처리
5. 탐색적 데이터 분석(EDA)
6. 데이터 시각화
7. 시사점 도출
8. 보고서 작성
9. 의사결정 또는 후속 실험 제안

AI와 LLM을 활용하는 분석에서도 이 기본 흐름은 크게 달라지지 않습니다. 달라지는 것은 각 단계에서 도움을 받을 수 있는 방식입니다. 예를 들어 LLM은 분석 질문 후보를 제안하거나, pandas 코드 초안을 작성하거나, 그래프 해석 문장을 정리하는 데 도움을 줄 수 있습니다.

이 책에서는 여기에 두 가지 요소를 더합니다. 하나는 머신러닝입니다. EDA와 시각화가 “무슨 일이 있었는가”를 이해하는 데 가깝다면, 머신러닝은 “앞으로 어떤 일이 일어날 가능성이 있는가”를 예측하는 방향으로 확장됩니다. 다른 하나는 자동화입니다. 반복되는 분석 과정을 보고서 생성이나 파이프라인 형태로 연결하면 분석 결과를 더 안정적으로 재사용할 수 있습니다.

- 문제 정의
- 데이터 수집
- 데이터 구조 이해
- 전처리
- EDA
- 시각화
- 머신러닝 기초 모델링
- LLM을 활용한 코드 해석 보조
- 보고서 작성
- 외부 데이터 확장
- 자동화와 파이프라인

이 순서는 이 책의 학습 구성을 보여 줍니다. 실제 분석에서 LLM 활용은 반드시 한 번만 거치는 독립 단계라기보다, 질문 정리, 코드 작성, 오류 해결, 결과 해석 등 여러 단계에서 선택적으로 활용하는 보조 활동에 가깝습니다.

아래 그림은 이 책에서 다룰 전체 데이터 분석 흐름을 요약한 것입니다.



그림 1-1. 데이터 분석 전체 흐름도

각 단계의 의미를 조금 더 자세히 보면 다음과 같습니다.

단계	핵심 질문	이 책에서 다루는 내용
문제 정의	무엇을 알고 싶은가?	업무 질문을 분석 질문으로 바꾸기
데이터 수집	어떤 데이터가 필요한가?	샘플 CSV, 외부 API, 크롤링 개념
데이터 구조 이해	행과 열은 무엇을 의미하는가?	CSV 읽기, 컬럼 확인, 데이터 타입 확인
전처리	분석 가능한 상태인가?	결측치, 중복, 날짜, 문자열, 이상값 처리
EDA	어떤 패턴이 보이는가?	그룹별 집계, 비교, 가설 만들기
시각화	결과를 어떻게 보여줄 것인가?	막대그래프, 선그래프, 히스토그램, 산점도
머신러닝	예측할 수 있는가?	회귀, 분류, 훈련/테스트 데이터 분할, 평가 지표
LLM 활용	더 빠르게 시작할 수 있는가?	프롬프트 작성, 코드 초안, 해석 문장 검토
보고서	어떻게 전달할 것인가?	결과 요약, 한계, 다음 단계 제안
자동화	반복할 수 있는가?	스크립트, 파이프라인, Make, n8n, Airflow 개념

외부 데이터나 크롤링 결과를 사용할 때는 제공 기관과 웹사이트의 이용약관, 저작권, 개인정보 보호 기준을 먼저 확인해야 합니다.

중요한 점은 LLM이 분석 과정의 일부를 빠르게 도와줄 수는 있지만, 분석 질문을 정의하고 최종 결과를 검증하는 책임은 사람에게 있다는 것입니다.

3. LLM은 데이터를 분석을 어떻게 바꾸는가

LLM을 잘 활용하면 분석을 시작하는 속도를 높일 수 있습니다. 특히 처음 데이터를 받았을 때 “무엇부터 봐야 할지” 막막한 상황에서 유용합니다.

예를 들어 온라인 쇼핑몰 데이터가 있다고 할 때, 다음과 같은 요청을 할 수 있습니다.

온라인 쇼핑몰의 고객, 상품, 주문, 주문 상세 데이터가 있습니다.
 매출 현황과 고객 구매 패턴을 분석하려고 합니다.
 초보 데이터 분석자가 먼저 확인해야 할 분석 질문 10개를 제안해 주세요.
 각 질문마다 필요한 데이터 파일과 분석 방법을 함께 설명해 주세요.

LLM은 이 요청에 대해 월별 매출 추이, 카테고리별 매출, 고객군별 구매 패턴, 주문 취소율, 재구매 여부 같은 분석 관점을 제안할 수 있습니다. 이런 답변은 분석을 시작할 때 생각을 정리하는 데 도움이 됩니다.

LLM이 특히 잘 도와주는 일은 다음과 같습니다.

- 막연한 업무 질문을 분석 질문으로 바꾸기
- 데이터 컬럼의 의미를 추정하고 확인할 항목 제안하기
- pandas, matplotlib, scikit-learn 코드 초안 만들기
- 오류 메시지를 쉬운 말로 설명하기
- 그래프와 표를 해석하는 초안 작성하기
- 보고서 문장을 더 명확하게 다듬기
- 빠뜨린 검토 항목을 체크리스트로 만들기

하지만 LLM의 답변은 분석의 결론이 아니라 출발점입니다. LLM은 데이터의 업무 맥락과 품질을 스스로 보장하지 못하며, 사용자가 제공한 설명, 일부 샘플, 일반적인 패턴을 바탕으로 그럴듯한 답을 생성할 수 있습니다. 따라서 답변이 자연스럽게 보인다고 해서 곧바로 맞는 분석이라고 판단해서는 안 됩니다.

LLM을 사용할 때는 다음처럼 역할을 나누어 생각하는 것이 좋습니다.

작업	LLM에게 맡길 수 있는 부분	사람이 해야 하는 부분
질문 만들기	분석 질문 후보 제안	현재 데이터로 답할 수 있는지 판단
코드 작성	pandas·시각화·머신러닝 코드 초안	컬럼명, 데이터 타입, 실행 결과 검증
오류 해결	오류 메시지 원인 설명	실제 환경과 파일 경로 확인
해석 작성	보고서 문장 초안	데이터에 없는 원인 단정 제거
발표 준비	발표 구성과 문장 정리	핵심 수치와 근거 확인

좋은 활용 방식은 “LLM이 답을 내고 사람이 복사한다”가 아니라, “LLM이 초안을 만들고 사람이 검토한다”에 가깝습니다.

4. LLM이 대신할 수 없는 것

LLM은 분석을 도와줄 수 있지만, 다음을 자동으로 보장하지는 못합니다.

- 실제 데이터가 정확한지 확인하기
- 컬럼명이 현재 코드와 일치하는지 검증하기
- 결측치나 이상값이 분석에 미치는 영향 판단하기
- 회사나 프로젝트의 내부 맥락 이해하기
- 머신러닝 모델의 성능이 충분한지 판단하기

- 그래프와 보고서 해석이 데이터보다 과장되지 않았는지 확인하기
- 개인정보와 민감정보를 안전하게 처리하기

예를 들어 LLM이 “고객 연령대별 매출을 분석하세요”라고 제안했다고 해도, 실제 `customers.csv`에 `age` 컬럼이 없다면 그 분석은 바로 수행할 수 없습니다. 이 경우 연령대 분석을 제외하거나, 적법하게 사용할 수 있는 다른 데이터에서 나이 정보를 가져올 수 있는지 확인해야 합니다.

LLM이 작성한 코드는 실행해 봐야 합니다. LLM이 작성한 해석은 실제 집계 결과와 비교해야 합니다.

LLM이 작성한 보고서는 데이터가 말하는 범위를 넘어 과장하고 있지 않은지 사람이 확인해야 합니다.

AI 시대의 데이터 분석가는 단순히 코드를 많이 작성하는 사람이 아닙니다. 더 중요한 역할은 질문을 정의하고, 분석 과정을 설계하고, 결과를 검증하는 것입니다. 좋은 분석가는 계속해서 다음과 같은 질문을 던집니다.

- 이 분석은 어떤 의사결정에 도움이 되는가?
- 이 질문에 답하려면 어떤 데이터가 필요한가?
- 현재 데이터로 정말 답할 수 있는가?
- LLM이 제안한 분석이 우리 상황에 맞는가?
- 같은 결과를 다르게 해석할 가능성은 없는가?
- 보고서 문장이 데이터보다 앞서가고 있지는 않은가?

5. 이 책에서 다룰 온라인 쇼핑몰 데이터

이 책에서는 하나의 가상 온라인 쇼핑몰 데이터를 중심으로 분석을 진행합니다. 데이터는 `Faker` 라이브러리로 생성했으며, 실제 개인과 무관한 실습용 데이터입니다.

온라인 쇼핑몰 운영자가 궁금해할 만한 질문은 다음과 같습니다.

- 이번 달 매출은 증가했는가?
- 어떤 상품 카테고리가 잘 팔리는가?
- 어떤 고객군이 많이 구매하는가?
- 매출 감소는 어떤 지표 변화와 함께 나타나는가?
- 주문이 취소될 가능성이 높은 조건은 무엇인가?
- 고객의 향후 구매 금액이나 매출을 예측할 수 있는가?
- 어떤 마케팅 방안을 제안할 수 있는가?

처음에는 단순한 집계와 시각화로 시작하지만, 뒤로 갈수록 머신러닝과 LLM 활용으로 확장합니다. 예를 들어 회귀 모델로 고객의 과거 구매 이력 등을 바탕으로 향후 구매 금액을 예측하고, 분류 모델로 주문 취소 가능성이나 구매 가능성을 예측해 볼 수 있습니다. 이후 LLM을 활용해 분석 질문을 정리하고, 코드 초안을 생성하고, 결과 해석과 보고서 문장을 다듬는 방식으로 연결합니다.

이 책에서 사용할 주요 데이터 파일은 다음 4개입니다.

파일	역할	주요 컬럼 예시
data/raw/customers.csv	고객 정보	customer_id, name, gender, age, city, signup_date
data/raw/products.csv	상품 정보	product_id, product_name, category, price
data/raw/orders.csv	주문 정보	order_id, customer_id, order_date, payment_method, order_status
data/raw/order_items.csv	주문 상세 정보	order_item_id, order_id, product_id, quantity, unit_price

이 4개 파일은 서로 연결되어 있습니다. orders.csv의 customer_id는 customers.csv의 customer_id와 연결됩니다. order_items.csv의 order_id는 orders.csv의 order_id와 연결됩니다. order_items.csv의 product_id는 products.csv의 product_id와 연결됩니다.

```
customers.customer_id — orders.customer_id
orders.order_id      — order_items.order_id
products.product_id  — order_items.product_id
```

파일을 연결한다는 것은 두 표에서 같은 의미를 가진 기준 컬럼을 사용해 하나의 분석용 표로 합친다는 뜻입니다. 예를 들어 고객별 구매 금액을 분석하려면 customers.csv, orders.csv, order_items.csv를 연결해야 합니다. 상품 카테고리별 매출을 분석하려면 products.csv와 order_items.csv를 연결해야 합니다.



그림 1-2. 온라인 쇼핑몰 데이터 관계도

이 장에서는 아직 모든 파일을 코드로 분석하지 않습니다. 먼저 데이터가 어떤 업무 질문과 연결되는지 이해하는 데 집중합니다. 실제 CSV 읽기, 데이터 구조 확인, 전처리, 시각화는 다음 장부터 차례대로 다룹니다.

6. EDA에서 머신러닝까지

초기 데이터 분석에서는 주로 “무슨 일이 있었는가”를 확인합니다. 월별 매출이 늘었는지 줄었는지, 어떤 카테고리가 많이 팔렸는지, 어떤 고객군의 구매 금액이 높은지 살펴봅니다. 이런 과정을 탐색적 데이터 분석, 즉 EDA라고 부릅니다.

머신러닝은 여기에서 한 걸음 더 나아갑니다. 이미 발생한 데이터를 바탕으로 새로운 데이터의 값을 예측하거나, 특정 상태에 속할 가능성을 판단합니다.

이 책에서 다룰 머신러닝 실습은 복잡한 이론보다 분석 과정을 이해하는 데 초점을 둡니다.

구분	예시 질문	대표 모델
회귀	고객의 향후 구매 금액을 예측할 수 있을까?	Linear Regression, RandomForestRegressor
분류	주문이 취소될 가능성이 높은가?	Logistic Regression, RandomForestClassifier

회귀는 숫자를 예측하는 문제입니다. 예를 들어 고객의 과거 구매 이력, 가입 기간, 최근 구매 시점처럼 예측 시점 이전에 확인할 수 있는 정보를 바탕으로 향후 구매 금액을 예측할 수 있습니다. 분류는 정해진 범주를 예측하는 문제입니다. 예를 들어 주문 상태가 `completed`인지 `cancelled`인지 예측하는 문제를 생각해 볼 수 있습니다.

머신러닝을 배울 때 중요한 것은 모델 이름을 많이 외우는 것이 아닙니다. 어떤 컬럼을 입력값으로 사용할지, 무엇을 예측할지, 학습 데이터와 테스트 데이터를 어떻게 나눌지, 결과를 어떤 지표로 평가할지 이해하는 것이 더 중요합니다.

머신러닝 실습에서 특히 조심해야 할 개념은 데이터 누수입니다. 예를 들어 주문 취소 여부를 예측하면서 `order_status`를 입력값으로 사용하면, 모델은 정답을 미리 보고 학습하는 셈입니다. 겉으로는 성능이 높아 보이지만 실제 예측 모델로는 사용할 수 없습니다. 이 책에서는 이런 실수를 피하기 위해 입력값과 예측 대상을 분리하는 연습을 반복합니다.

7. LLM과 함께 분석할 때의 기록 습관

LLM을 분석에 사용했다면 어떤 질문을 입력했고 어떤 답변을 참고했는지 기록해 두는 것이 좋습니다. 이를 프롬프트 로그라고 부를 수 있습니다.

프롬프트 로그가 필요한 이유는 다음과 같습니다.

- 분석 과정의 재현성을 높일 수 있습니다.
- LLM이 어떤 가정을 했는지 확인할 수 있습니다.
- 잘못된 답변이 분석에 반영되었는지 추적할 수 있습니다.
- 팀 프로젝트에서 다른 사람이 분석 과정을 이해할 수 있습니다.
- 본인의 판단과 AI의 도움을 구분할 수 있습니다.

프롬프트 로그에는 모든 대화를 길게 복사할 필요는 없습니다. 분석에 영향을 준 주요 질문, LLM의 핵심 답변, 실제 반영 여부를 간단히 적어 두면 충분합니다. 단, 로그에도 개인정보, API 키, 내부 기밀정보를 남기지 않아야 합니다.

항목	기록 예시
사용 목적	카테고리별 매출 분석 질문 생성
입력 프롬프트	"온라인 쇼핑몰 주문 데이터로 확인할 분석 질문을 제안해 주세요."
LLM 답변 요약	월별 매출, 카테고리별 매출, 고객군별 구매 패턴 제안
실제 반영 여부	월별 매출과 카테고리별 매출만 반영
검증 내용	실제 컬럼 존재 여부와 집계 결과 확인

기록 습관은 단순한 형식이 아닙니다. AI를 활용한 분석에서 신뢰를 확보하기 위한 최소한의 장치입니다. 특히 팀 프로젝트나 과제에서는 "LLM을 사용했다"는 사실보다 "어떻게 검증했는가"가 더 중요합니다.

8. 개인정보와 보안 원칙

실제 고객 이름, 전화번호, 이메일, 주소, 결제 정보 같은 개인정보를 LLM에 그대로 입력하면 안 됩니다. 회사 내부 매출 자료, 계약 정보, 비공개 전략 자료도 조직에서 사용을 승인하지 않은 외부 LLM 서비스에 입력해서는 안 됩니다.

실무에서는 다음 원칙을 지켜야 합니다.

- 개인정보는 제거하거나 비식별화합니다.
- 내부 기밀 데이터는 승인되지 않은 외부 LLM 서비스에 입력하지 않습니다.
- 필요한 경우 전체 데이터가 아니라 일부 샘플 구조나 요약 정보만 제공합니다.
- API 키, 비밀번호, 토큰은 절대 프롬프트에 넣지 않습니다.
- 조직의 보안 정책과 허용된 도구를 먼저 확인합니다.

이 책의 실습 데이터는 가상으로 생성한 데이터입니다. 하지만 수업이나 실무 프로젝트에서 실제 데이터를 사용할 때는 항상 보안과 개인정보 보호를 먼저 생각해야 합니다.

LLM에게 데이터를 설명할 때는 원본 행 전체를 넣기보다 다음과 같은 요약 정보를 사용하는 것이 좋습니다.

```
데이터셋: orders.csv
행 수: 300
컬럼: order_id, customer_id, order_date, payment_method, order_status
분석 목적: 월별 주문 수와 주문 상태별 비중 확인
주의: 실제 고객명이나 결제 정보는 제공하지 않음
```

이 정도 정보만으로도 LLM은 분석 방향이나 코드 초안을 제안할 수 있습니다. 필요한 정보만 최소한으로 제공하는 습관은 AI 시대의 기본 데이터 보안 원칙입니다.

9. 실습 프로젝트는 어떻게 구성할까

이 책의 실습은 하나의 프로젝트 폴더를 기준으로 진행하는 것이 좋습니다. 프로젝트 폴더는 단순히 파일을 모아 두는 공간이 아니라, 코드와 데이터, Jupyter Notebook, 보고서, 환경 설정을 함께 관리하는 작업 공간입니다.

실습 프로젝트에서 자주 사용하게 될 폴더와 파일은 다음과 같습니다.

```

my-llm-data-analysis-course/
├── data/
│   ├── raw/
│   ├── processed/
│   └── external/
├── notebooks/
├── scripts/
├── reports/
│   └── figures/
├── prompts/
├── src/
├── .env.example
├── .gitignore
├── README.md
└── requirements.txt

```

각 위치의 역할은 다음과 같습니다.

위치	역할	사용 예시
data/raw/	원본 또는 샘플 CSV 저장	생성된 CSV 확인
data/processed/	전처리된 CSV 저장	결측치·타입 정리 후 정제된 파일 저장
data/external/	외부 데이터 저장	공공데이터, API, 크롤링 결과 저장
notebooks/	장별 실습 노트북	코드를 실행하고 결과와 해석 기록
scripts/	반복 실행용 Python 스크립트	샘플 데이터 생성, 자동화 코드 실행
reports/	분석 결과와 보고서 저장	CSV 요약표, Markdown 보고서 저장
reports/figures/	그래프 이미지 저장	시각화 결과 PNG 저장
prompts/	LLM 프롬프트 예시 저장	분석 질문, 코드 생성, 검증 프롬프트 참고
src/	재사용 가능한 Python 코드	함수나 모듈 형태 코드 관리
.env.example	환경변수 예시	API 키 변수 이름을 확인한 뒤 .env 작성
.gitignore	Git에서 제외할 파일 목록	.env, .venv, 캐시 파일 제외 확인
requirements.txt	필요한 패키지 목록	가상환경에 패키지 설치
README.md	프로젝트 안내	설치, 실행, 작업 과정 확인

분석을 진행하다 보면 파일이 계속 늘어납니다. 이때 폴더 역할을 지키면 나중에 결과를 찾기 쉽고, 다른 사람에게 프로젝트를 설명하기도 쉬워집니다.

Git과 GitHub 사전 학습 안내

이 책의 실습 파일과 변경 이력을 안정적으로 관리하려면 Git과 GitHub의 기본 개념을 함께 익혀두는 것이 좋습니다.

Git과 GitHub의 차이, Git 설치, 저장소 생성, Clone, Add, Commit, Push, Pull 등 기본적인 설치 및 사용 방법은 다음 가이드를 참고합니다.

- [Git-GitHub 개념, 설치 및 기본 사용 가이드](#)

이 책에서는 Git과 GitHub 자체를 자세히 설명하기보다, 준비된 저장소를 내려받고 실습 코드와 분석 결과의 변경 이력을 관리하는 용도로 활용합니다.

단, `.env`, API 키, 비밀번호, 개인정보, 회사 내부 데이터와 대용량 원본 데이터는 Git 저장소에 커밋하지 않도록 주의해야 합니다.

10. 실습을 시작하는 기본 흐름

이 장에서는 실습을 자세히 진행하기보다, 앞으로 어떤 순서로 실습이 이어지는지만 먼저 확인합니다. 실제 VS Code 실행, 가상환경 생성, 패키지 설치, Jupyter Notebook 커널 선택, 첫 셀 실행 확인은

2장. VS Code에서 시작하는 데이터 분석 환경에서 단계별로 진행합니다.

1장에서 기억해야 할 핵심은 다음과 같습니다.

- 실습은 하나의 프로젝트 폴더를 기준으로 진행합니다.
- 데이터는 `data/` 폴더, 노트북은 `notebooks/` 폴더, 보고서는 `reports/` 폴더에 정리합니다.
- Python 패키지는 프로젝트별 가상환경인 `.venv` 안에 설치합니다.
- 노트북을 실행할 때는 VS Code에서 `.venv` 커널을 선택해야 합니다.
- 파일 경로 오류가 나면 현재 작업 폴더와 데이터 파일 위치를 함께 확인해야 합니다.

따라서 지금 단계에서는 “이후 장에서 직접 실행할 분석 프로젝트가 어떤 구조로 준비되는지”를 이해하는 데 집중합니다. 실제로 `notebooks/ch01_ai_data_analysis_intro.ipynb`를 열고 첫 번째 셀을 실행하는 과정은 2장에서 화면 흐름과 함께 다시 안내합니다.

11. LLM과 함께 실습할 때의 좋은 질문 예시

실습하다 보면 파일 경로 오류, 패키지 오류, 컬럼명 오류, 그래프 한글 깨짐 같은 문제가 자주 발생합니다. 이때 LLM에게 질문할 수 있지만, 질문에는 상황을 구체적으로 담아야 합니다.

파일 경로 오류가 발생했다면 다음처럼 질문할 수 있습니다.

```
VS Code에서 Jupyter Notebook을 실행하고 있습니다.
프로젝트 폴더 이름은 my-llm-data-analysis-course입니다.
데이터 파일은 data/raw/customers.csv에 있습니다.
현재 실행한 코드는 pd.read_csv('data/raw/customers.csv')입니다.
FileNotFoundError가 발생했습니다.
현재 작업 폴더를 확인하고 상대 경로를 고치는 방법을 설명해 주세요.
```

pandas 컬럼명 오류가 발생했다면 다음처럼 질문할 수 있습니다.

```
customers DataFrame의 실제 컬럼은 다음과 같습니다.
customer_id, name, gender, age, city, signup_date

LLM이 작성한 코드에서는 customer_age라는 컬럼을 사용해 KeyError가 발생했습니다.
실제 컬럼명에 맞게 코드를 수정하고, 왜 오류가 발생했는지 설명해 주세요.
```

그래프 한글 깨짐이 발생했다면 다음처럼 질문할 수 있습니다.

```
Windows 환경에서 matplotlib 그래프 제목의 한글이 깨집니다.
VS Code의 Jupyter Notebook에서 실행 중입니다.
한글 폰트를 설정하는 코드를 알려 주세요.
음수 기호가 깨지지 않게 하는 설정도 포함해 주세요.
```

좋은 질문은 오류 해결 시간을 줄여 줍니다. 반대로 "안 돼요", "오류가 나요"처럼 상황이 부족한 질문은 LLM도 정확히 답하기 어렵습니다.

12. API 키와 비밀번호는 Git 저장소에 포함하지 않는다

11장 이후에는 LLM API나 외부 API를 활용하는 과정이 등장합니다. 이때 `.env.example` 파일을 복사해 `.env` 파일을 만들고, 실제 키는 `.env`에만 저장합니다. `.env.example`에는 변수 이름과 예시 값만 기록하고 실제 키를 입력하지 않습니다.

Windows PowerShell에서는 다음처럼 복사할 수 있습니다.

```
Copy-Item .env.example .env
```

macOS나 Linux에서는 다음 명령을 사용할 수 있습니다.

```
cp .env.example .env
```

.env 파일에는 다음처럼 개인 API 키를 입력합니다.

```
GEMINI_API_KEY=여기에_개인_API_KEY_입력
NAVER_CLIENT_ID=여기에_네이버_CLIENT_ID_입력
NAVER_CLIENT_SECRET=여기에_네이버_CLIENT_SECRET_입력
```

하지만 .env 파일은 절대 GitHub에 올리면 안 됩니다. .gitignore에 .env가 포함되어 있는지 확인해야 합니다.

```
.env
.venv/
__pycache__/
.ipynb_checkpoints/
```

GitHub에 올리기 전에는 다음 명령으로 어떤 파일이 커밋 대상인지 확인합니다.

```
git status
```

.env가 이미 Git의 추적 대상인지 확인하려면 다음 명령을 사용할 수 있습니다.

```
git ls-files .env
```

명령 실행 결과에 .env가 표시되면 .gitignore에 추가하는 것만으로는 추적이 중단되지 않습니다. 실제 파일은 로컬에 남겨 둔 채 Git의 추적 대상에서만 제거합니다.

```
git rm --cached .env
```

그런 다음 다시 git status로 상태를 확인하고 변경 사항을 커밋합니다. API 키가 원격 저장소에 한 번이라도 올라갔다면 파일을 삭제하는 것만으로는 충분하지 않습니다. 해당 키를 즉시 폐기하거나 재발급하고, 필요하면 Git 기록에서도 비밀정보를 제거해야 합니다.

2장. VS Code에서 시작하는 데이터 분석 환경

데이터 분석은 코드 한 줄을 실행하는 일에서 시작되는 것처럼 보이지만, 실제로는 그보다 앞서 준비해야 할 것이 많습니다. 같은 코드를 실행해도 Python 버전이 다르거나, 패키지가 설치된 위치가 다르거나, 작업 폴더가 달라지면 전혀 다른 오류가 발생할 수 있습니다. 분석 자체보다 환경 설정에서 더 많은 시간을 쓰게 되는 경우도 흔합니다.

이 장에서는 앞으로의 분석 프로젝트를 안정적으로 이어 가기 위한 기본 환경을 준비합니다. 핵심은 여러 도구를 따로따로 익히는 것이 아니라, **VS Code를 중심으로 Python, GitHub, 가상환경, Notebook, 샘플 데이터를 하나의 작업 흐름으로 연결하는 것**입니다.

이 책에서는 Jupyter Notebook을 별도의 웹 환경으로 길게 다루기보다, VS Code 안에서 간단히 열고 실행하는 방식으로 사용합니다. 분석 코드 작성, 터미널 명령 실행, 파일 관리, Git 변경사항 확인은 대부분 VS Code에서 진행합니다. Notebook은 코드와 결과를 함께 확인해야 할 때 사용하는 보조적인 실습 형식으로 이해하면 됩니다.

1. 왜 환경 설정이 먼저인가

데이터 분석 프로젝트는 보통 CSV 파일 하나를 읽는 간단한 작업에서 시작합니다. 하지만 프로젝트가 조금만 커져도 코드, 데이터, 결과물, 설정 파일, 보고서가 함께 움직입니다. 이 파일들이 흩어져 있으면 분석 흐름을 다시 실행하기 어렵고, 다른 사람이 같은 결과를 재현하기도 어렵습니다.

분석 환경을 먼저 정리해 두면 다음과 같은 장점이 있습니다.

- 어떤 Python 환경에서 코드를 실행하는지 명확해집니다.
- 필요한 패키지를 한 번에 설치하고 다시 재현할 수 있습니다.
- 데이터 파일과 코드 파일의 위치를 일관되게 관리할 수 있습니다.
- GitHub를 통해 작업 이력과 결과물을 남길 수 있습니다.
- LLM이나 AI 코딩 도구에 오류를 질문할 때 현재 상황을 정확히 설명할 수 있습니다.

분석 환경은 단순한 설치 과정이 아니라, 앞으로 15주 동안 반복해서 사용할 작업 공간을 만드는 일입니다.

구간	주요 흐름	이 장에서 준비하는 것
1주차	AI 기반 데이터 분석 개요와 개발 환경 설정	VS Code, Python, GitHub, 가상환경
2~3주차	데이터 구조 이해와 pandas 기초	CSV 파일 로드, DataFrame 확인
4~6주차	전처리, EDA, 시각화	pandas, matplotlib, seaborn 실행 환경
7~10주차	머신러닝 기초, 회귀, 분류	scikit-learn 기반 모델링 환경
11~12주차	LLM 분석 프롬프트와 코드 검증	.env, API Key 보안, LLM 질문 방식
13~14주차	외부 데이터 수집과 자동화	프로젝트 구조, 스크립트, 실행 흐름 관리
15주차	기말 프로젝트	개인 저장소와 재현 가능한 분석 환경

2. VS Code를 중심에 두는 이유

데이터 분석 입문 단계에서는 Notebook 하나만 열어도 충분해 보입니다. 하지만 실제 프로젝트에서는 데이터 파일, Python 스크립트, Notebook, 환경 설정 파일, 보고서, README가 함께 필요합니다. VS Code는 이러한 파일을 하나의 프로젝트 폴더 안에서 관리하기에 적합합니다.

VS Code에서는 다음 작업을 한 화면에서 처리할 수 있습니다.

- 프로젝트 폴더 구조 확인
- Python 코드와 Markdown 문서 작성
- 터미널에서 명령어 실행
- Jupyter Notebook 파일 열기와 셀 실행
- Git 변경사항 확인과 commit/push
- .env, .gitignore, requirements.txt 같은 설정 파일 관리
- LLM 또는 AI 코딩 도구를 활용한 코드 검토

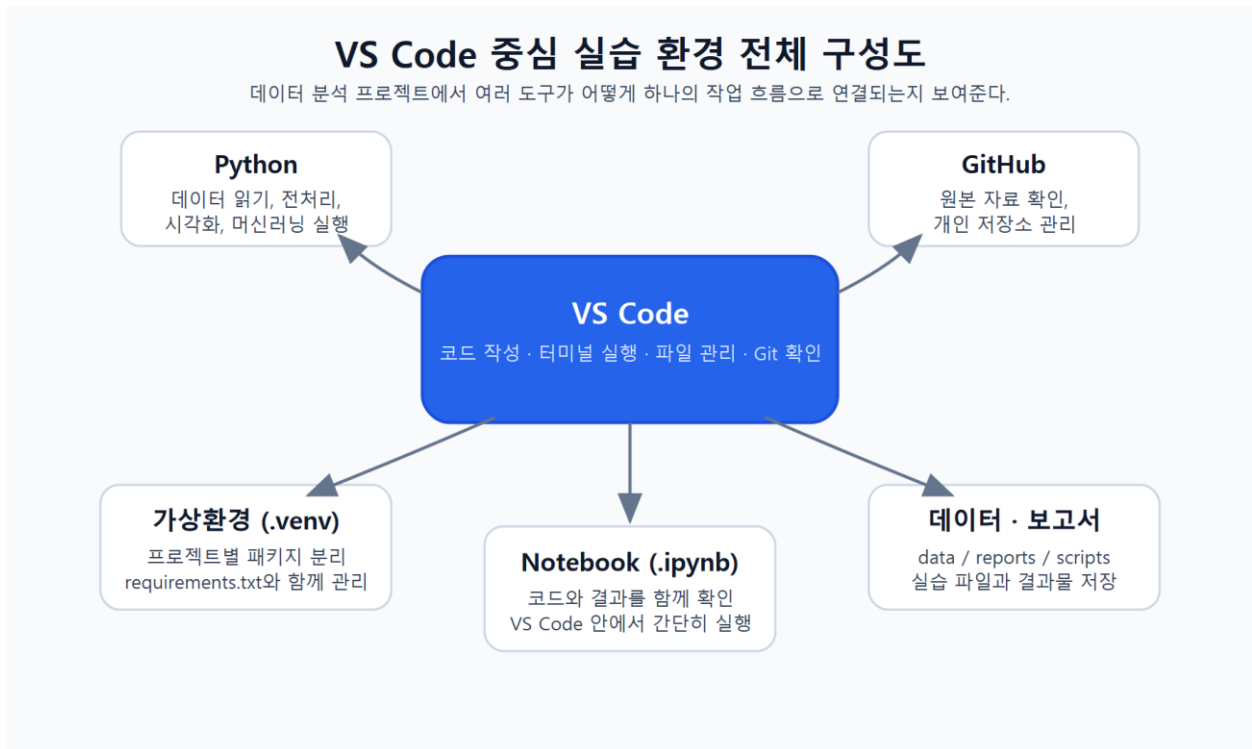


그림 2-1. VS Code 중심 실습 환경 전체 구성도

이번 책에서 사용하는 도구의 역할은 다음처럼 정리할 수 있습니다.

도구	역할	이 책에서의 사용 방식
VS Code	기본 작업 공간	코드 작성, 터미널 실행, 파일 관리, Git 관리
Python	분석 코드 실행 언어	데이터 읽기, 전처리, 시각화, 머신러닝 실행
Jupyter Notebook	코드와 결과를 함께 보는 파일 형식	VS Code 안에서 .ipynb 파일을 열어 간단히 실행
GitHub	코드와 결과물 저장소	원본 자료 확인, 개인 저장소 관리, 과제·프로젝트 제출
.venv	프로젝트별 Python 가상환경	패키지 충돌 방지
requirements.txt	필요한 패키지 목록	같은 환경을 다시 만들기 위한 기준
.env	API Key 등 민감한 설정 저장	LLM/API 실습을 위한 보안 관리

Jupyter Notebook은 어디에 쓰는가

Jupyter Notebook은 코드, 실행 결과, 설명 문장을 한 파일에 함께 기록할 수 있는 형식입니다.

데이터의 모양을 빠르게 확인하거나 그래프 결과를 바로 보고 싶을 때 유용합니다.

다만 이 책에서는 Notebook 자체를 중심 도구로 삼지 않습니다. Notebook은 VS Code 안에서 열어 실행합니다. 이렇게 하면 Notebook의 장점은 활용하면서도 프로젝트 폴더, Python 환경, GitHub 작업 흐름을 한곳에서 관리할 수 있습니다.

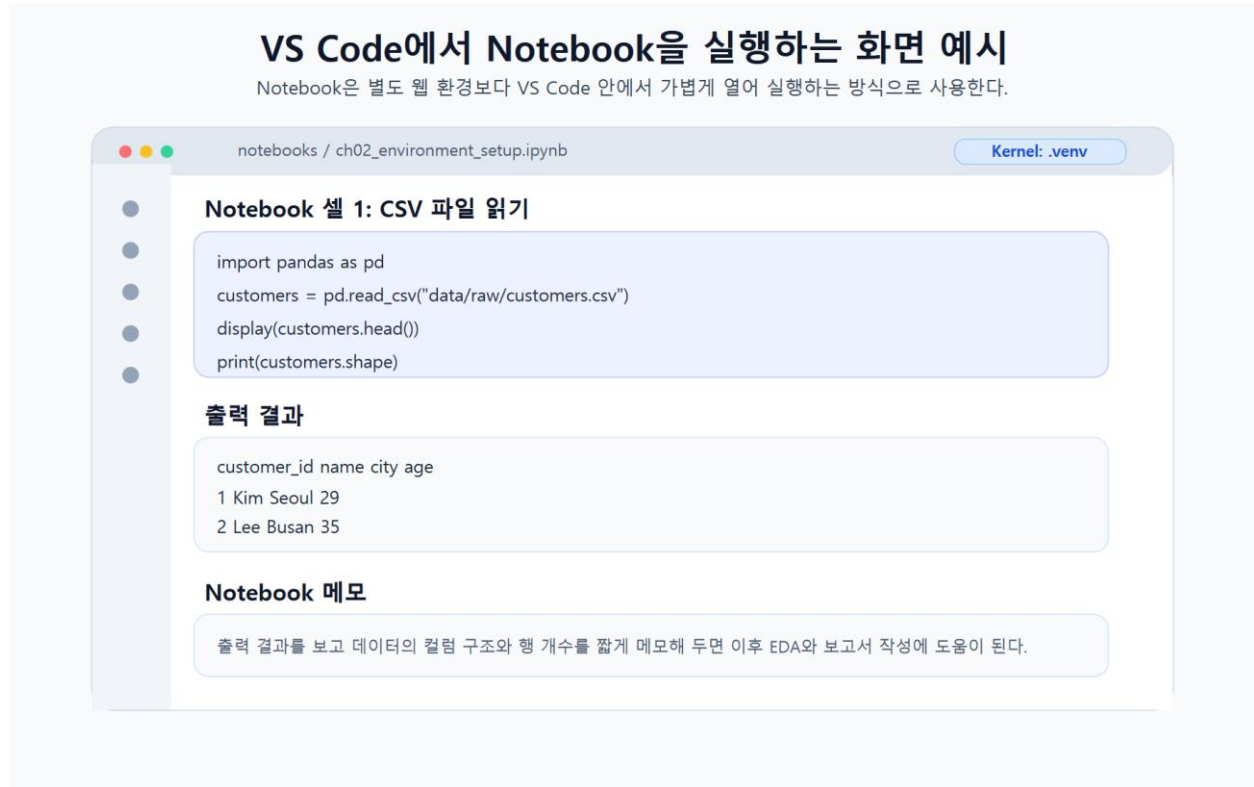


그림 2-2. VS Code에서 Notebook을 실행하는 화면 예시

Notebook을 사용할 때 중요한 점은 셀을 실행하는 것에서 멈추지 않는 것입니다. 출력 결과가 무엇을 의미하는지 짧게라도 기록해 두면, 이후 EDA나 프로젝트 보고서를 작성할 때 큰 도움이 됩니다.

3. 프로젝트 폴더는 분석의 뼈대다

분석 프로젝트는 폴더 구조가 정리되어 있을수록 다루기 쉽습니다. 데이터 파일은 `data` 폴더에, 실습 Notebook은 `notebooks` 폴더에, 반복 실행할 코드는 `scripts` 폴더에 두면 파일을 찾는 시간이 줄어듭니다.

VS Code 프로젝트 폴더 구조 예시

데이터, 코드, 결과물의 위치를 미리 정해 두면 실습과 프로젝트 관리가 쉬워진다.



그림 2-3. VS Code 프로젝트 폴더 구조 예시

예를 들어 이 책의 실습 저장소는 다음과 같은 구조를 가질 수 있습니다.

```
my-llm-data-analysis-course/
├── data/
│   └── raw/
├── notebooks/
├── scripts/
├── reports/
├── book/
├── .env.example
├── .gitignore
├── README.md
└── requirements.txt
```

각 폴더의 역할은 단순합니다.

위치	역할
data/raw	원본 또는 샘플 CSV 파일 저장
notebooks	주차별 Notebook 실습 파일 저장
scripts	데이터 생성, 자동화, 반복 실행 코드 저장
reports	분석 결과 보고서나 산출물 저장

위치	역할
book	ebook 원고와 관련 자료 저장
requirements.txt	필요한 Python 패키지 목록
.env.example	환경변수 예시 파일
.gitignore	GitHub에 올리지 않을 파일 목록

폴더 구조를 처음부터 완벽하게 외울 필요는 없습니다. 중요한 것은 코드와 데이터가 어디에 있는지, 그리고 현재 터미널이 어느 폴더에서 실행되고 있는지를 확인하는 습관입니다.

4. Python과 가상환경

Python은 데이터 분석 코드를 실행하는 기본 언어입니다. CSV 파일을 읽고, 데이터를 정리하고, 그래프를 그리고, 머신러닝 모델을 학습하는 대부분의 작업은 Python 코드로 이루어집니다.

먼저 터미널에서 Python이 실행되는지 확인합니다.

```
python --version
```

버전이 출력되면 Python 명령을 사용할 수 있는 상태입니다. Windows에서는 PC 환경에 따라 `python` 대신 `py` 명령을 사용해야 할 수도 있고, macOS나 Linux에서는 `python3` 명령을 사용하는 경우가 많습니다.

```
python3 --version
```

가상환경이 필요한 이유

Python 패키지는 프로젝트마다 필요한 버전이 다를 수 있습니다. 모든 패키지를 PC 전체 환경에 설치하면 프로젝트 간 충돌이 발생하기 쉽습니다.

가상환경은 프로젝트별로 분리된 Python 실행 공간입니다. 이 책에서는 프로젝트 폴더 안에 `.venv`라는 이름의 가상환경을 만들어 사용합니다.

```
python -m venv .venv
```

Windows PowerShell에서는 다음 명령으로 가상환경을 활성화합니다.

```
.\venv\Scripts\Activate.ps1
```

macOS나 Linux에서는 다음 명령을 사용합니다.

```
source .venv/bin/activate
```

가상환경이 활성화되면 터미널 앞부분에 보통 (.venv) 표시가 나타납니다.

```
(.venv) PS D:\WDEV\my-llm-data-analysis-course>
```

이 표시가 없으면 패키지를 설치해도 다른 Python 환경에 설치될 수 있습니다. 실습을 시작하기 전에는 항상 가상환경이 활성화되어 있는지 확인하는 습관을 들이는 것이 좋습니다.

PowerShell 실행 정책 오류

Windows PowerShell에서 가상환경을 활성화할 때 다음과 같은 오류가 나타날 수 있습니다.

```
running scripts is disabled on this system
```

이 경우 현재 PowerShell 창에서만 임시로 실행 정책을 완화한 뒤 다시 활성화할 수 있습니다.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

보안 정책으로 명령 실행이 제한된 기관이나 회사 PC에서는 임의로 시스템 전체 정책을 바꾸지 말고, 관리자 또는 담당 부서의 안내를 따릅니다.

5. 패키지 설치와 requirements.txt

데이터 분석에는 기본 Python만으로는 부족한 경우가 많습니다. 표 형태 데이터를 다루려면 pandas가 필요하고, 수치 계산에는 numpy가 자주 쓰입니다. 그래프를 그릴 때는 matplotlib이나 seaborn을 사용하고, 머신러닝 실습에는 scikit-learn이 필요합니다.

이런 패키지 목록을 한 파일에 모아 둔 것이 requirements.txt입니다. 가상환경을 활성화한 뒤 다음 명령으로 필요한 패키지를 설치합니다.

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

python -m pip 형식을 사용하면 현재 선택한 Python 환경의 pip를 실행한다는 점이 더 명확합니다. macOS나 Linux에서 python3로 가상환경을 만들었더라도, 가상환경을 활성화한 뒤에는 일반적으로 python -m pip를 사용할 수 있습니다.

처음부터 모든 패키지의 사용법을 외울 필요는 없습니다. 지금은 어떤 패키지가 어떤 역할을 하는지 정도만 가볍게 이해해 두면 충분합니다.

패키지	주된 역할
pandas	표 형태 데이터 처리
numpy	수치 계산
matplotlib	기본 시각화
seaborn	통계 기반 시각화
scikit-learn	머신러닝 모델링
jupyter	Notebook 실행 지원
openpyxl	Excel 파일 처리
python-dotenv	.env 파일에서 환경변수 읽기
google-genai	Gemini API 사용
faker	가상 샘플 데이터 생성
python-docx	Word 보고서 생성
ipykernel	VS Code Notebook 커널 연결

설치가 끝난 뒤에는 VS Code에서 Python 인터프리터가 프로젝트의 .venv를 사용하고 있는지 확인해야 합니다. VS Code 오른쪽 아래 또는 명령 팔레트에서 Python 인터프리터를 선택할 수 있습니다.

Python: Select Interpreter

목록에서 프로젝트 폴더 안의 .venv를 선택합니다. Notebook 파일을 열 때도 오른쪽 위 커널 선택 메뉴에서 같은 .venv 환경을 선택해야 합니다.

6. GitHub와 개인 저장소

이 책에서 제공하는 GitHub 저장소는 실습 원본 자료를 확인하기 위한 공간입니다. 독자는 이 원본 저장소를 직접 수정하지 않고, 자신의 GitHub 계정에 개인 실습 저장소를 만들어 작업합니다.

원본 저장소와 개인 저장소를 분리하면 다음과 같은 장점이 있습니다.

구분	원본 GitHub 저장소	개인 GitHub 저장소
역할	실습 자료와 기본 구조 제공	자신의 코드, 결과, 보고서 저장
수정 권한	보통 읽기 중심	자유롭게 수정 가능

구분	원본 GitHub 저장소	개인 GitHub 저장소
포함 내용	기본 코드, 샘플 데이터 생성 스크립트, 원고	수정한 Notebook, 분석 결과, 프로젝트 산출물
활용	기준 자료 확인	실습 기록, 제출, 포트폴리오

GitHub를 처음 사용할 때는 다음 용어를 먼저 구분해 두면 좋습니다.

용어	의미
Repository	코드와 문서를 저장하는 GitHub 저장소
Clone	GitHub 저장소를 내 PC로 내려받는 작업
Commit	변경 내용을 하나의 기록으로 저장하는 작업
Push	로컬 commit을 GitHub 원격 저장소에 업로드하는 작업

Template Repository 방식

원본 저장소가 Template Repository로 제공된다면 이 방식이 가장 간단합니다. GitHub에서 `Use this template` 버튼을 눌러 자신의 계정에 같은 구조의 새 저장소를 만들 수 있습니다.

개인 저장소를 만든 뒤에는 로컬 PC로 내려받습니다.

```
git clone https://github.com/본인아이디/my-llm-data-analysis-course.git
cd my-llm-data-analysis-course
```

VS Code가 설치되어 있다면 프로젝트 폴더에서 다음 명령으로 바로 열 수 있습니다.

```
code .
```

Fork 방식

원본 저장소가 Template Repository로 설정되어 있지 않다면 `Fork` 버튼을 사용해 자신의 계정에 복사본을 만들 수 있습니다. Fork한 저장소를 clone한 뒤에는 개인 저장소에서 자유롭게 실습하고 변경 이력을 관리할 수 있습니다.

Download ZIP 방식

GitHub 사용이 익숙하지 않다면 처음에는 ZIP 파일로 내려받아 시작할 수도 있습니다. 다만 ZIP 방식은 Git 변경 이력을 바로 관리하기 어렵기 때문에, 프로젝트 결과를 지속적으로 남기려면 개인 저장소와 연결하는 과정이 추가로 필요합니다.

가능하다면 Template Repository 또는 Fork 방식으로 시작하는 것이 좋습니다. 개인 저장소에 작업을 남기면 이후 프로젝트 결과를 정리하거나 포트폴리오로 활용하기도 쉽습니다.

7. VS Code에서 처음 실행해 보기

이제 프로젝트 폴더를 VS Code에서 열고, 터미널을 사용해 기본 실행 환경을 준비해 보겠습니다. Windows PowerShell 기준 흐름은 다음과 같습니다.

```
python -m venv .venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python scripts/generate_sample_data.py
```

macOS나 Linux에서는 가상환경 생성과 활성화 명령이 조금 다릅니다.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python scripts/generate_sample_data.py
```

`generate_sample_data.py` 스크립트는 이후 실습에서 사용할 샘플 CSV 파일을 생성합니다. 실행 후 `data/raw` 폴더에 다음 파일들이 만들어졌는지 확인합니다.

```
customers.csv
products.csv
orders.csv
order_items.csv
```

파일이 보이지 않는다면 스크립트가 실행된 위치를 먼저 확인해야 합니다. VS Code 터미널이 프로젝트 루트 폴더에서 열려 있어야 합니다. 현재 위치는 터미널에서 다음 명령으로 확인할 수 있습니다.

```
pwd
```

Windows 명령 프롬프트에서는 다음 명령을 사용합니다.

```
cd
```

Python 코드 안에서는 다음처럼 확인할 수 있습니다.


```
from pathlib import Path

print(Path.cwd())
```

8. Notebook 첫 실행과 커널 선택

Notebook을 사용할 때는 별도로 jupyter notebook 명령을 실행해 웹 브라우저를 열 수도 있지만, 이 책에서는 VS Code 안에서 Notebook 파일을 여는 방식을 기본으로 합니다. 특히 처음 실행할 때는 **파일 열기, 커널 선택, 첫 셀 실행**이 서로 다른 단계라는 점을 이해해야 합니다.

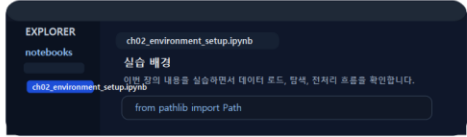
환경 점검용으로는 notebooks/ch01_ai_data_analysis_intro.ipynb 또는
notebooks/ch02_environment_setup.ipynb를 열 수 있습니다. 이 장에서는 2장 실습 파일인 ch02_environment_setup.ipynb 기준으로 설명합니다.

VS Code에서 Notebook을 열고 실행하는 흐름

처음 실행할 때 햇갈리는 지점을 실제 화면 흐름에 가깝게 정리했습니다. 핵심은 파일 열기 → .venv 커널 선택 → 첫 셀 실행 → 정상 실행 확인입니다.

1 Notebook 파일 열기

Explorer에서 notebooks 폴더를 열고 ch02_environment_setup.ipynb를 클릭합니다.
상단 탭에 Notebook 이름과 Markdown, Code 셀이 보이면 정상입니다.
실제 의미: 파일을 여는 것과 코드를 실행하는 것은 다른 단계입니다.



2 커널 선택하기

첫 셀 실행 시 Select Kernel 창이 뜨면 먼저 Python Environments...를 선택합니다.
그 다음 목록에서 프로젝트의 .venv (Python 3.11.x)를 선택합니다.
중요: 터미널에 (.venv)가 보여도 Notebook 커널은 자동 선택되지 않습니다. 오른쪽 위 커널도 별도로 맞춰야 합니다.

Select Kernel

Python Environments...

Existing Jupyter Server...

Python Environments

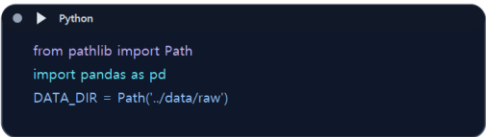
Type to filter...

.venv (Python 3.11.9)

base (Anaconda)

3 첫 번째 셀 실행하기

첫 코드 셀은 import와 기본 경로 설정입니다.
셀 왼쪽 실행 버튼(▶) 또는 Shift+Enter로 실행합니다.
이 셀은 표를 출력하지 않아도 정상입니다.
패키지 import와 DATA_DIR = Path("../data/raw") 설정이 핵심입니다.



4 정상 실행 확인

초록 체크 표시, 실행 시간, 오류 없이 이동한 상태를 확인합니다.
처음에는 Matplotlib is building the font cache 메시지가 잠깐 보일 수 있으며, 오류가 아니라 정상 동작입니다.

✔ 30.8s
 Matplotlib is building the font cache;
 this may take a moment.

핵심: Notebook 열기 → .venv 커널 선택 → 첫 셀 실행 → 초록 체크 확인

그림 2-4. VS Code에서 Notebook을 열고 실행하는 흐름

8-1. Notebook 파일 열기

10. VS Code 왼쪽 Explorer에서 notebooks 폴더를 엽니다.
11. ch02_environment_setup.ipynb 파일을 클릭합니다.
12. Notebook 화면에 Markdown 셀과 Code 셀이 함께 보이면 정상입니다.

Notebook은 일반 .py 파일과 다르게 셀 단위로 실행합니다. 따라서 파일을 열었다고 해서 코드가 자동으로 실행되는 것은 아닙니다.

8-2. 커널 선택하기

첫 번째 셀을 실행하려고 하거나 오른쪽 위 Select Kernel을 누르면 커널 선택 창이 나타납니다. 여기서는 다음 순서로 선택합니다.

1. Python Environments... 선택
2. 목록에서 프로젝트 폴더 안의 .venv 환경 또는 .venv\Scripts\python.exe 선택

Python의 세부 버전 표시는 설치 시점에 따라 달라질 수 있으므로, Python 3.11.x처럼 특정 버전 번호보다 프로젝트 폴더의 .venv인지를 기준으로 선택합니다.

Existing Jupyter Server..., base (Anaconda), Python Global Env는 지금 실습에서는 선택하지 않는 것이 좋습니다. 다른 환경을 선택하면 패키지를 설치했는데도 ModuleNotFoundError가 나거나, Notebook이 다른 Python을 바라보는 문제가 생길 수 있습니다.

초보자가 특히 헷갈리는 부분은 다음입니다.

- 터미널에 (.venv)가 보여도 Notebook 커널은 별도로 선택해야 합니다.
- python -m pip install -r requirements.txt를 실행한 환경과 Notebook 커널이 같아야 합니다.
- 처음 실행할 때 커널 선택 창이 뜨는 것은 정상 동작입니다.

만약 .venv가 목록에 보이지 않는다면 VS Code 창을 한 번 새로고침하거나, 명령 팔레트에서 Developer: Reload Window를 실행한 뒤 다시 확인합니다. 그래도 보이지 않을 때만 다음 명령으로 커널을 등록합니다.

```
python -m ipykernel install --user --name llm-data-analysis-course --display-name "llm-data-analysis-course"
```

위 명령으로 등록한 커널은 프로젝트 폴더가 삭제되어도 사용자 환경에 남을 수 있습니다. 일반적으로는 VS Code에서 .venv 인터프리터를 직접 선택하는 방식을 우선합니다.

8-3. 첫 번째 셀 실행하기

첫 코드 셀은 보통 패키지를 불러오고 기본 경로를 설정합니다.

```
from pathlib import Path

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

DATA_DIR = Path('../data/raw')
sns.set_theme(style='whitegrid')
```

이 경로는 Notebook의 현재 작업 폴더가 notebooks일 때 정상적으로 동작합니다. 실행 환경에 따라 작업 폴더가 프로젝트 루트로 잡히면 Path('data/raw')를 사용해야 할 수도 있습니다. 현재 작업 폴더는 Path.cwd()로 확인합니다.

셀 왼쪽의 실행 버튼(▶)을 누르거나 Shift + Enter를 누르면 실행됩니다. 이 셀은 표나 그래프를 출력하지 않기 때문에, 실행 후 아무 결과가 화면에 바로 보이지 않아도 이상한 것이 아닙니다.

다음 중 하나가 보이면 정상 실행으로 판단할 수 있습니다.

- 셀 왼쪽에 실행 완료 표시가 보임
- 실행 시간이 표시됨
- 오류 메시지 없이 다음 셀로 이동함

VS Code 버전에 따라 완료 표시의 모양이나 실행 시간 표시는 달라질 수 있습니다.

처음 실행할 때는 아래 메시지가 잠깐 보일 수 있습니다.

```
Matplotlib is building the font cache; this may take a moment.
```

이 메시지는 오류가 아니라 matplotlib가 폰트 정보를 처음 캐시로 만드는 정상 동작입니다. 보통 첫 실행 때만 나타나고, 다음부터는 훨씬 빠르게 실행됩니다.

8-4. 데이터가 실제로 읽히는지 확인하기

첫 셀까지 성공했다면, 아래 코드를 새 셀에 넣거나 예시 셀의 주석을 해제해서 실행해 봅니다.

```
customers = pd.read_csv(DATA_DIR / 'customers.csv')
customers.head()
```

표가 정상적으로 보이면 Notebook, 커널, 패키지, 경로가 기본적으로 맞게 연결된 것입니다.

만약 `ModuleNotFoundError`가 발생하면, 대개 현재 선택한 Notebook 커널에 패키지가 설치되지 않은 경우입니다. 이때는 `.venv` 커널이 맞는지 다시 확인합니다.

만약 `FileNotFoundError`가 발생하면, 다음 항목을 우선 확인합니다.

1. `python scripts/generate_sample_data.py`를 이미 실행했는가?
2. `data/raw/customers.csv` 파일이 실제로 존재하는가?
3. `Path.cwd()`의 결과가 프로젝트 루트인가, `notebooks` 폴더인가?
4. 현재 작업 폴더에 맞게 `data/raw` 또는 `./data/raw`를 사용했는가?

상대 경로 오류는 데이터 분석 입문자가 가장 자주 만나는 문제 중 하나입니다. 오류가 발생했을 때는 파일이 실제로 어디에 있는지, 코드가 어느 위치에서 실행되고 있는지부터 확인하면 대부분 해결의 실마리를 찾을 수 있습니다.

9. `.env` 파일과 API Key 보안

11주차 이후에는 LLM API나 외부 API를 활용하는 실습으로 확장됩니다. 이때 API Key를 코드에 직접 적거나 GitHub에 올리면 안 됩니다. API Key는 비밀번호와 비슷하게 다뤄야 합니다.

저장소에는 보통 `.env.example` 파일이 포함됩니다. 이 파일은 어떤 환경변수가 필요한지 보여 주는 예시입니다.

```
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL_NAME=gemini-2.5-flash-lite
```

모델 이름과 사용 가능 여부는 API 제공사의 정책에 따라 변경될 수 있습니다. 실습 시점에 모델 관련 오류가 발생하면 Google AI for Developers의 현재 모델 목록과 저장소 안내를 확인합니다.

실제 키를 사용할 때는 `.env.example`을 복사해 `.env` 파일을 만들고, 그 안에 개인 키를 입력합니다.

Windows PowerShell에서는 다음처럼 복사할 수 있습니다.

```
Copy-Item .env.example .env
```

macOS나 Linux에서는 다음 명령을 사용합니다.

```
cp .env.example .env
```

.env 파일은 절대 GitHub에 올리지 않습니다. 이를 위해 .gitignore에 .env가 포함되어 있는지 확인해야 합니다.

```
.env
.venv/
__pycache__/
.ipynb_checkpoints/
```

커밋하기 전에는 다음 명령으로 .env가 추적 대상에 포함되지 않았는지 확인합니다.

```
git status
```

이미 실수로 API Key를 GitHub에 올렸다면 파일만 삭제하는 것으로 끝나지 않습니다. 해당 키를 즉시 폐기하고 새 키를 발급받아야 합니다.

LLM에게 오류를 질문할 때도 API Key, 비밀번호, 토큰, 개인 이메일, 내부 서버 주소가 포함되어 있지 않은지 먼저 확인해야 합니다. LLM은 오류 해결을 도와주는 도구이지만, 민감정보를 안전하게 다루는 책임은 사용자에게 있습니다.

10. LLM은 환경 오류를 정리하는 도구가 될 수 있다

LLM은 분석을 대신해 주는 도구가 아닙니다. 특히 데이터 분석에서는 최종 판단과 검증을 사람이 해야 합니다. 다만 설치 오류, 패키지 오류, 파일 경로 오류처럼 원인을 찾기 어려운 문제를 정리할 때는 좋은 보조 도구가 될 수 있습니다.

좋은 질문은 현재 상황을 구체적으로 설명합니다. 단순히 “오류가 났어요”라고 묻는 것보다, 실행한 명령어, 현재 폴더, 사용 중인 운영체제, 오류 메시지를 함께 제공하는 편이 좋습니다.

예를 들어 가상환경 활성화 오류가 발생했다면 다음처럼 질문할 수 있습니다.

당신은 Python 데이터 분석 프로젝트의 실습 조교입니다.

Windows PowerShell에서 가상환경을 활성화하려고 했는데 오류가 발생했습니다.

현재 상황:

- 운영체제: Windows
- 편집기: VS Code
- 프로젝트 폴더: my-llm-data-analysis-course
- 실행한 명령어: .\venv\Scripts\Activate.ps1

오류 메시지:

[여기에 오류 메시지를 붙여넣기]

요청:

- 초보자가 이해할 수 있게 원인을 설명해 주세요.
- 위험한 명령어는 제안하지 말아 주세요.
- 현재 PowerShell 창에서만 해결하는 방법을 우선 알려 주세요.
- API Key, 비밀번호, 토큰을 공유하지 말라고 안내해 주세요.

파일 경로 오류는 다음처럼 질문할 수 있습니다.

VS Code에서 Notebook 파일을 실행하는 중 FileNotFoundError가 발생했습니다.

현재 상황:

- 프로젝트 폴더 이름: my-llm-data-analysis-course
- 데이터 파일 위치: data/raw/customers.csv
- Notebook 위치: notebooks/ch02_environment_setup.ipynb
- 사용한 코드: pd.read_csv('data/raw/customers.csv')
- Path.cwd() 실행 결과: [여기에 결과 입력]

요청:

- 현재 작업 폴더와 상대 경로의 관계를 설명해 주세요.
- 현재 작업 폴더에 맞는 수정 코드 예시를 제시해 주세요.
- 파일 존재 여부를 안전하게 확인하는 방법을 알려 주세요.

LLM의 답변은 바로 실행하지 말고 한 번 더 확인해야 합니다. 특히 파일 삭제, 환경 초기화, 권한 변경, 시스템 설정 변경과 관련된 명령어는 신중하게 다뤄야 합니다.

11. Git으로 작업 흐름을 남긴다

분석 프로젝트는 한 번에 완성되지 않습니다. 데이터를 읽고, 전처리하고, 시각화하고, 모델을 바꾸고, 보고서를 고치는 과정이 반복됩니다. Git은 이러한 변경 과정을 기록하는 도구입니다.

처음에는 많은 명령을 외우기보다 기본 흐름만 이해하면 됩니다.

```
git status
git add .
git status
git commit -m "Set up analysis environment"
git push
```

git add . 뒤에 git status를 한 번 더 실행하면 실제로 커밋될 파일을 확인할 수 있습니다. .env나 불필요한 대용량 파일이 포함되어 있다면 커밋 전에 제외합니다.

각 명령의 의미는 다음과 같습니다.

명령어	의미
git status	변경된 파일과 커밋 준비 상태 확인
git add .	현재 폴더 아래 변경 파일을 커밋 준비 상태로 올림
git commit -m "메시지"	변경 내용을 하나의 기록으로 저장
git push	로컬 commit을 GitHub 저장소에 업로드

GitHub에 올리면 안 되는 파일이 포함되지 않았는지 확인하는 것도 중요합니다. 특히 .env, .venv, 캐시 폴더, 개인정보가 포함된 데이터, 불필요한 대용량 데이터 파일은 주의해야 합니다.

12. 환경 점검하기

모든 설정을 한 번에 완벽하게 끝낼 필요는 없습니다. 다만 이후 장으로 넘어가기 전에 다음 항목이 준비되어 있으면 데이터 분석 실습을 안정적으로 시작할 수 있습니다.

확인할 항목	확인 방법
Python 실행 가능	python --version 또는 python3 --version
VS Code에서 프로젝트 열림	프로젝트 루트 폴더가 Explorer에 표시됨
가상환경 생성	.venv 폴더 존재
가상환경 활성화	터미널 앞에 (.venv) 표시
패키지 설치	python -m pip install -r requirements.txt 완료
VS Code Python 인터프리터	프로젝트 .venv 선택
VS Code Notebook 커널	Notebook 오른쪽 위 커널이 .venv
샘플 데이터 생성	data/raw에 CSV 파일 존재
CSV 로드 확인	pd.read_csv() 실행 성공

확인할 항목	확인 방법
.env 보안	.env가 .gitignore에 포함되고 git status에 나타나지 않음
GitHub 연결	git status, git push 흐름 확인

문제가 생겼을 때는 오류 메시지를 그대로 받아들이기보다, 어느 단계에서 발생했는지 나누어 보는 것이 좋습니다.

- Python 명령이 실행되지 않는 문제인가?
- 가상환경이 활성화되지 않는 문제인가?
- 패키지가 설치되지 않은 문제인가?
- VS Code가 다른 Python 인터프리터를 보고 있는 문제인가?
- Notebook 커널이 .venv가 아닌 문제인가?
- 데이터 파일 경로가 잘못된 문제인가?
- GitHub 인증이나 권한 문제인가?

이렇게 문제를 분리하면 LLM에게 질문할 때도 훨씬 정확한 답변을 얻을 수 있습니다.

13. 다음 장으로 이어지는 흐름

환경 설정이 끝나면 이제 실제 데이터를 읽고 구조를 살펴볼 수 있습니다. 다음 장에서는 CSV 파일을 pandas로 불러오고, 행과 열, 데이터 타입, 결측치, 기본 통계량을 확인하는 방법을 다룹니다.

데이터 분석은 복잡한 모델에서 시작되지 않습니다. 먼저 데이터가 어떤 모양인지 확인하고, 각 컬럼이 무엇을 의미하는지 이해하는 것에서 출발합니다. VS Code와 Python 환경이 준비되어 있다면, 이제 분석의 첫 단계인 데이터 구조 이해로 넘어갈 수 있습니다.

3장. 데이터의 첫인상 읽기

데이터 분석은 복잡한 모델이나 화려한 그래프에서 시작하지 않습니다. 먼저 해야 할 일은 데이터가 어떤 모양인지 차분히 살펴보는 것입니다. 파일은 몇 개인지, 각 파일은 어떤 역할을 하는지, 어떤 컬럼이 있고 어떤 값이 들어 있는지 확인해야 이후 분석이 흔들리지 않습니다.

CSV 파일을 pandas로 불러오는 일 자체는 어렵지 않습니다. 하지만 파일을 불러왔다고 해서 곧바로 분석을 시작할 수 있는 것은 아닙니다. 날짜가 문자열로 저장되어 있을 수도 있고, 숫자처럼 보이는 값이 실제로는 문자열일 수도 있습니다. 어떤 컬럼에는 값이 비어 있을 수 있고, 어떤 ID는 중복되어 있을 수도 있습니다. 이런 부분을 확인하지 않고 분석을 시작하면 그래프나 모델 결과가 그럴듯해 보여도 잘못된 결론으로 이어질 수 있습니다.

이번 장에서는 온라인 쇼핑몰 고객·매출 분석 프로젝트에서 사용할 4개의 CSV 파일을 불러오고, 데이터의 기본 구조를 읽는 방법을 살펴봅니다. 이 과정은 이후 pandas 기초, 전처리, 탐색적 데이터 분석(EDA), 시각화, 머신러닝, LLM 기반 분석으로 이어지는 출발점입니다.

이 장에서 생각해 볼 질문

데이터를 처음 열었을 때는 다음 질문을 먼저 던져 보는 것이 좋습니다.

- 데이터 파일은 몇 개인가?
- 각 파일은 어떤 역할을 하는가?
- 각 파일은 몇 행, 몇 열로 구성되어 있는가?
- 어떤 컬럼이 있고, 각 컬럼은 어떤 의미를 가지는가?
- 숫자형, 문자열, 날짜형 컬럼은 무엇인가?
- 결측치나 중복 데이터는 없는가?
- 각 데이터셋에서 고유해야 하는 ID는 무엇인가?
- 여러 파일을 연결할 수 있는 기준 컬럼은 무엇인가?
- LLM이 설명한 데이터 구조가 실제 데이터와 일치하는가?

이 질문들에 답할 수 있으면 본격적인 분석에 들어갈 준비가 된 것입니다.



그림 3-1. CSV 파일을 pandas DataFrame으로 불러오는 흐름

1. 데이터를 보기 전에 확인해야 할 것

데이터 구조를 파악한다는 것은 데이터를 분석하기 전에 데이터가 어떤 형태로 구성되어 있는지 확인하는 과정입니다. 단순히 파일이 열리는지 보는 것이 아니라, 그 데이터가 분석 가능한 상태인지 점검하는 일에 가깝습니다.

예를 들어 orders.csv에 order_date라는 컬럼이 있다고 가정해 보겠습니다. 이름만 보면 주문일처럼 보입니다. 하지만 이 컬럼이 실제 날짜 타입인지, 문자열인지, 일부 값이 비어 있는지 확인해야 합니다. 날짜가 문자열로 저장되어 있다면 월별 매출 분석을 하기 전에 날짜 타입으로 변환해야 합니다.

데이터를 처음 볼 때는 다음 항목을 순서대로 확인합니다.

확인 항목	의미
파일 목록	분석에 필요한 파일이 모두 있는지 확인합니다.
행과 열 개수	데이터 규모와 구조를 파악합니다.
컬럼명	어떤 정보가 들어 있는지 확인합니다.
데이터 타입	숫자형, 문자열, 날짜형 등 값의 종류를 확인합니다.
결측치	비어 있는 값이 있는지 확인합니다.
중복 데이터	같은 행이나 고유해야 하는 ID가 반복되는지 확인합니다.
기본 통계	숫자형 컬럼의 범위와 이상 가능성을 살펴봅니다.
키 컬럼	각 행을 구분하거나 여러 파일을 연결할 기준 컬럼을 확인합니다.

이 과정은 지루해 보일 수 있지만, 실제 분석에서 가장 중요한 습관 중 하나입니다. 데이터를 제대로 확인한 사람은 이후 전처리와 시각화, 머신러닝 단계에서 발생할 오류를 훨씬 빠르게 발견할 수 있습니다.

2. CSV 파일과 DataFrame

CSV는 Comma-Separated Values의 줄임말입니다. 값을 쉼표로 구분해 저장한 텍스트 파일입니다. 메모장으로 열면 쉼표로 구분된 텍스트처럼 보이고, 스프레드시트 프로그램으로 열면 표처럼 보입니다.

pandas는 CSV 파일을 읽어 DataFrame이라는 표 형태의 데이터로 다룹니다. DataFrame은 스프레드시트의 시트와 비슷하게 행과 열로 구성되어 있습니다.

구성 요소	설명
행(row)	하나의 관측값 또는 기록입니다.
열(column)	데이터의 속성 또는 변수입니다.
인덱스(index)	DataFrame 안에서 각 행을 구분하는 번호 또는 이름입니다.
값(value)	행과 열이 만나는 위치에 있는 실제 데이터입니다.

예를 들어 고객 데이터에서 한 행은 고객 한 명을 의미할 수 있습니다. customer_id, gender, age, city 같은 열은 고객의 속성을 의미합니다.

인덱스와 customer_id는 역할이 다를 수 있습니다. 인덱스는 pandas가 DataFrame 내부에서 행을 구분하기 위해 사용하는 값이고, customer_id는 업무 데이터에서 고객을 식별하는 ID입니다. 따라서 인덱스가 고유하다고 해서 customer_id까지 고유하다고 단정해서는 안 됩니다.

pandas DataFrame 구조
DataFrame은 행(row), 열(column), 인덱스(index), 값(value)으로 구성된 표 형태 데이터입니다

index	customer_id	gender	age	city
0	C001	F	34	Seoul
1	C002	M	28	Busan
2	C003	F	41	Incheon

구성 요소

- 행(row): 한 고객 기록
- 열(column): 고객 속성
- 인덱스: 행 번호
- 값(value): 셀의 실제 값

그림 3-2. pandas DataFrame 구조 예시

DataFrame을 이해하면 이후 pandas 문법도 훨씬 쉽게 받아들일 수 있습니다. 대부분의 데이터 분석 코드는 결국 “어떤 행을 볼 것인가”, “어떤 열을 사용할 것인가”, “어떤 기준으로 묶고 계산할 것인가”의 조합입니다.

3. 온라인 쇼핑몰 데이터 살펴보기

이 책에서 사용할 데이터는 가상의 온라인 쇼핑몰 운영 데이터입니다. 분석에 사용할 파일은 4개입니다.

파일	설명	주요 확인 항목
customers.csv	고객 정보	고객 수, 연령, 성별, 지역, 가입일
products.csv	상품 정보	상품 수, 카테고리, 가격
orders.csv	주문 정보	주문 수, 주문일, 결제 수단, 주문 상태
order_items.csv	주문 상세 정보	주문별 상품, 수량, 단가

처음에는 이 파일들을 따로 살펴봅니다. 각 파일이 어떤 구조인지 이해한 뒤에야 고객별 구매 금액, 상품 카테고리별 매출, 주문 취소율 같은 분석으로 확장할 수 있습니다.

온라인 쇼핑몰 데이터에서는 여러 파일의 관계를 이해하는 것이 중요합니다. 고객 정보만으로는 매출을 알 수 없고, 상품 정보만으로는 누가 무엇을 샀는지 알 수 없습니다. 주문과 주문 상세 데이터를 연결해야 실제 구매 과정을 볼 수 있습니다.

기본 생성 스크립트는 결측치와 주요 ID 중복이 없도록 샘플 데이터를 만듭니다. 그렇더라도 실제 분석에서는 파일을 다시 생성하거나 외부 데이터로 교체할 수 있으므로, 구조 점검 과정을 생략하지 않는 것이 좋습니다.



그림 3-3. 데이터 구조 점검 흐름도

4. 코드로 데이터 구조 확인하기

이번 장의 코드는 `notebooks/ch03_data_overview.ipynb`에서 실행할 수 있습니다. 여기서는 핵심 과정을 살펴봅니다.

먼저 `pandas`와 파일 경로 관리를 위한 `Path`를 불러옵니다.

```
from pathlib import Path
import pandas as pd
```

`pandas`는 CSV 파일을 읽고 표 형태 데이터를 다루는 핵심 패키지입니다. `Path`는 현재 작업 폴더와 파일 경로를 다룰 때 사용합니다.

현재 작업 폴더 확인하기

CSV 파일을 불러올 때 가장 흔한 오류 중 하나는 파일 경로 오류입니다. 먼저 현재 작업 폴더를 확인합니다.

```
print("현재 작업 폴더:", Path.cwd())
```

현재 작업 폴더가 프로젝트 루트라면 다음과 같은 파일과 폴더가 보여야 합니다.

```
README.md
requirements.txt
data/
notebooks/
scripts/
book/
```

Jupyter Notebook을 notebooks 폴더를 작업 폴더로 하여 실행한 경우에는 데이터 경로가 달라질 수 있습니다. 파일을 찾지 못하는 오류가 발생하면 코드를 임의로 바꾸기 전에 Path.cwd()의 출력과 실제 데이터 파일 위치를 함께 확인합니다.

프로젝트 루트와 데이터 경로 설정하기

현재 작업 폴더가 프로젝트 루트이거나 notebooks 폴더인 경우를 모두 처리하도록 다음처럼 작성할 수 있습니다.

```
project_root = Path.cwd()

if project_root.name == "notebooks":
    project_root = project_root.parent

data_dir = project_root / "data" / "raw"
print("데이터 폴더:", data_dir.resolve())
```

이 코드는 현재 작업 폴더 이름이 notebooks이면 한 단계 위 폴더를 프로젝트 루트로 사용합니다. 다른 위치에서 노트북을 실행하고 있다면 project_root를 실제 프로젝트 폴더 경로로 직접 지정해야 합니다.

파일이 실제로 존재하는지도 확인합니다.

```
required_files = [
    "customers.csv",
    "products.csv",
    "orders.csv",
    "order_items.csv",
]

for file_name in required_files:
    file_path = data_dir / file_name
    print(f"{file_name}: {file_path.exists()}")
```

예상 출력은 다음과 같습니다.

```
customers.csv: True
products.csv: True
orders.csv: True
order_items.csv: True
```

하나라도 False가 나오면 경로가 잘못되었거나 샘플 데이터가 아직 생성되지 않은 것입니다. 샘플 데이터가 없다면 터미널에서 프로젝트 루트로 이동한 뒤 다음 명령을 실행합니다.

```
python scripts/generate_sample_data.py
```

명령을 다른 폴더에서 실행하면 scripts/generate_sample_data.py 경로를 찾지 못할 수 있으므로, 실행 전에 터미널의 현재 위치도 확인합니다.

CSV 파일 불러오기

이제 4개의 CSV 파일을 DataFrame으로 불러옵니다.

```
customers = pd.read_csv(data_dir / "customers.csv")
products = pd.read_csv(data_dir / "products.csv")
orders = pd.read_csv(data_dir / "orders.csv")
order_items = pd.read_csv(data_dir / "order_items.csv")
```

정상적으로 실행되면 각 CSV 파일이 pandas DataFrame으로 저장됩니다. 이제부터는 이 변수들을 사용해 데이터 구조를 확인할 수 있습니다.

데이터 크기 확인하기

shape는 데이터의 행과 열 개수를 알려 줍니다.

```
print("customers:", customers.shape)
print("products:", products.shape)
print("orders:", orders.shape)
print("order_items:", order_items.shape)
```

현재 제공되는 생성 스크립트를 기본 설정으로 실행하면 다음과 같이 출력됩니다.

```
customers: (150, 6)
products: (100, 4)
orders: (300, 5)
order_items: (764, 5)
```

shape 결과는 (행 개수, 열 개수) 형식입니다. 예를 들어 (150, 6)은 150개의 행과 6개의 열이 있다는 뜻입니다. 생성 스크립트의 옵션이나 데이터가 바뀌면 실제 숫자도 달라질 수 있으므로, 보고서에는 직접 실행한 결과를 기록해야 합니다.

여러 데이터셋의 크기를 하나의 표로 정리하면 전체 구조를 보기 쉽습니다.

```
summary = pd.DataFrame({
    "dataset": ["customers", "products", "orders", "order_items"],
    "rows": [
        customers.shape[0],
        products.shape[0],
        orders.shape[0],
        order_items.shape[0],
    ],
    "columns": [
        customers.shape[1],
        products.shape[1],
        orders.shape[1],
        order_items.shape[1],
    ],
})

summary
```

예상 결과는 다음과 같은 형태입니다.

dataset	rows	columns
customers	150	6
products	100	4
orders	300	5
order_items	764	5

이 표는 보고서나 발표 자료에서 데이터 개요를 설명할 때도 사용할 수 있습니다.

데이터 앞부분과 마지막 부분 확인하기

`head()`는 기본적으로 데이터의 앞부분 5행을 보여줍니다. 괄호 안에 숫자를 넣으면 확인할 행 수를 바꿀 수 있습니다.

```
customers.head()
customers.head(3)
```

다른 데이터도 같은 방식으로 확인합니다.

```
products.head()
orders.head()
order_items.head()
```


`head()`를 볼 때는 컬럼명이 예상과 일치하는지, 값의 형태가 자연스러운지, 날짜와 숫자가 어떤 형식으로 저장되어 있는지 살펴봅니다.

앞부분만 보고 전체 데이터가 정상이라고 판단하기는 어렵습니다. `tail()`은 기본적으로 마지막 5행을 보여 줍니다.

```
customers.tail()
```

`head()`와 `tail()`은 일부 행만 보여 주므로, 이 결과만으로 결측치나 중복이 없다고 판단해서는 안 됩니다.

컬럼명 확인하기

컬럼명은 코드 작성에서 매우 중요합니다. LLM이 생성한 코드가 실제 컬럼명과 다르면 오류가 발생합니다.

```
print(customers.columns)
print(products.columns)
print(orders.columns)
print(order_items.columns)
```

`customers.columns`는 pandas의 `Index` 형태로 표시됩니다. 일반적인 Python 리스트 형태로 보고 싶다면 다음처럼 작성합니다.

```
list(customers.columns)
```

예를 들어 실제 컬럼은 `customer_id`인데 LLM이 `cust_id`라고 코드를 작성하면 `KeyError`가 발생합니다. 따라서 LLM의 도움을 받더라도 실제 컬럼명은 반드시 데이터에서 직접 확인해야 합니다.

데이터 타입 확인하기

`info()`를 사용하면 각 컬럼의 데이터 타입과 비어 있지 않은 값의 개수를 확인할 수 있습니다.

```
customers.info()
```

다른 데이터도 같은 방식으로 확인합니다.

```
products.info()
orders.info()
order_items.info()
```

`info()`에서 특히 확인할 항목은 다음과 같습니다.

- 전체 행 수와 컬럼 수

- 각 컬럼의 데이터 타입
- 비어 있지 않은 값의 개수인 Non-Null Count
- 날짜처럼 보이지만 object로 저장된 컬럼
- 대략적인 메모리 사용량

예를 들어 전체 행이 150개인데 어떤 컬럼의 Non-Null Count가 147이라면 해당 컬럼에는 결측치가 3개 있다는 뜻입니다. object는 보통 문자열 데이터를 의미하지만, 날짜나 숫자가 문자열로 저장된 경우도 있으므로 값의 실제 형태를 함께 확인해야 합니다.

데이터 타입만 간단히 확인하고 싶을 때는 dtypes를 사용할 수 있습니다.

```
customers.dtypes
```

여러 DataFrame에 같은 점검 코드를 반복하기 위해 이름과 DataFrame을 딕셔너리로 묶을 수 있습니다.

```
datasets = {
    "customers": customers,
    "products": products,
    "orders": orders,
    "order_items": order_items,
}
```

딕셔너리는 이름과 값을 한 쌍으로 저장하는 Python 자료형입니다. 여기서는 데이터셋 이름을 키로, DataFrame을 값으로 저장합니다. .items()를 사용하면 이름과 DataFrame을 하나씩 꺼내 반복할 수 있습니다.

```
for name, df in datasets.items():
    print(f"\n[{name}]")
    print(df.dtypes)
```

5. 결측치, 중복, 데이터 타입 읽기

데이터 구조를 볼 때 우선 확인해야 할 문제는 결측치와 중복입니다. 결측치는 값이 비어 있는 상태이고, 중복은 같은 행이나 같은 ID가 반복되는 상태입니다. 둘 다 분석 결과에 영향을 줄 수 있지만, 발견했다고 해서 무조건 삭제해야 하는 것은 아닙니다.

이번 장에서는 결측치와 중복을 직접 처리하기보다, 어느 데이터셋의 어떤 컬럼에서 문제가 발견되는지 기록하는 데 집중합니다. 삭제·대체·보정 방법은 5장 데이터 전처리에서 자세히 다룹니다.

결측치 확인하기

결측치는 `isna()`와 `sum()`을 사용해 확인할 수 있습니다.

```
customers.isna().sum()
```

여러 데이터셋의 결측치를 한 번에 확인할 수도 있습니다.

```
for name, df in datasets.items():
    print(f"\n[{name}]\n")
    print(df.isna().sum())
```

결측치 개수와 비율을 함께 보면 컬럼별 상태를 더 쉽게 이해할 수 있습니다.

```
missing_summary = pd.DataFrame({
    "missing_count": customers.isna().sum(),
    "missing_ratio_pct": customers.isna().mean() * 100,
})

missing_summary
```

현재 기본 생성 스크립트로 만든 `customers.csv`에는 결측치가 없으므로 모든 값이 0으로 나오는 것이 정상입니다. 결과가 다르다면 실제 출력값을 기준으로 원인을 확인합니다.

결측치 비율이 높다면 이후 전처리 단계에서 처리 방법을 결정해야 합니다. 예를 들어 `age` 컬럼에 결측치가 있으면 연령대별 분석에서 일부 고객이 제외될 수 있습니다.

처리 방법	설명
행 제외	결측치가 있는 행을 특정 분석에서 제외합니다.
대표값 대체	평균, 중앙값, 최빈값 등으로 채웁니다.
별도 범주 처리	Unknown 같은 범주로 표시합니다.
컬럼 제외	분석 목적에 적합하지 않은 컬럼은 사용하지 않습니다.
원인 확인	원본 데이터 수집 과정에서 문제가 있었는지 확인합니다.

어떤 방법이 적절한지는 컬럼의 의미, 결측 원인, 분석 목적에 따라 달라집니다.

전체 행 중복 확인하기

전체 행이 중복되어 있는지 확인합니다.

```
customers.duplicated().sum()
```

여러 데이터셋의 중복 행 개수를 확인할 수도 있습니다.

```
for name, df in datasets.items():
    duplicated_count = df.duplicated().sum()
    print(f"{name}: {duplicated_count}")
```

단, 모든 값의 반복이 오류는 아닙니다. 예를 들어 `order_items`에서는 같은 `order_id`가 여러 번 나올 수 있습니다. 한 주문에 여러 상품이 포함될 수 있기 때문입니다.

주요 ID의 결측치와 중복 확인하기

각 데이터셋에서 한 행을 고유하게 구분해야 하는 ID는 따로 확인해야 합니다.

```
key_columns = {
    "customers": "customer_id",
    "products": "product_id",
    "orders": "order_id",
    "order_items": "order_item_id",
}

for name, key_column in key_columns.items():
    df = datasets[name]
    null_count = df[key_column].isna().sum()
    duplicated_count = df[key_column].duplicated().sum()

    print(
        f"{name}.{key_column} - "
        f"결측치: {null_count}, 중복: {duplicated_count}"
    )
```

현재 샘플 데이터에서는 모든 결과가 0이어야 합니다. 반면 `order_items["order_id"]`의 중복은 자연스럽습니다. 한 주문에 여러 주문 상세 행이 연결될 수 있기 때문입니다.

중복 여부는 숫자만 보고 판단하지 말고, 해당 컬럼이 고유해야 하는 ID인지 여러 행에서 반복될 수 있는 연결 ID인지 구분해 해석해야 합니다.

숫자형 컬럼 기본 통계 확인하기

`describe()`는 숫자형 컬럼의 기본 통계 정보를 보여 줍니다. ID는 숫자로 저장되어 있어도 크기나 평균에 업무적 의미가 없는 식별자이므로, 분석할 숫자 컬럼을 명시적으로 선택하는 것이 좋습니다.

```
customers[["age"]].describe()
```

상품 가격과 주문 상세의 수량·단가도 확인합니다.

```
products[["price"]].describe()
order_items[["quantity", "unit_price"]].describe()
```

describe()에서 확인할 주요 항목은 다음과 같습니다.

항목	의미
count	결측치를 제외한 값의 개수
mean	평균
std	값이 평균 주변에서 얼마나 퍼져 있는지를 나타내는 표준편차
min	최소값
25%	값을 작은 순서로 놓았을 때 하위 25% 지점인 1사분위수
50%	가운데 값인 중앙값
75%	하위 75% 지점인 3사분위수
max	최대값

최대값이나 최소값이 업무적으로 가능한 범위를 벗어나면 이상치 가능성을 의심할 수 있습니다. 평균과 중앙값의 차이가 크면 값의 분포가 한쪽으로 치우쳐 있을 가능성도 있습니다. 다만 describe()만으로 이상치를 확정해서는 안 되며, 원본 값과 업무 규칙을 함께 확인해야 합니다.

현재 샘플 데이터의 가격 컬럼은 숫자형이므로 별도 변환이 필요하지 않습니다. 다만 실무 데이터에서는 숫자가 "10,000"처럼 쉼표가 포함된 문자열로 저장되는 경우가 있습니다. 다음 코드는 이러한 상황을 설명하기 위한 참고 예시입니다.

```
price_text = pd.Series(["10,000", "25,500", "3000", "확인 필요"])

price_number = pd.to_numeric(
    price_text.str.replace(",", "", regex=False),
    errors="coerce",
)

price_number
```

errors="coerce"는 숫자로 변환할 수 없는 값을 오류로 중단하지 않고 NaN으로 바꿉니다. 변환 후에는 새로 생긴 결측치가 있는지 반드시 확인해야 합니다.

```
price_number.isna().sum()
```

범주형 컬럼 고유값 확인하기

문자열 또는 범주형 컬럼은 고유값 개수와 값별 빈도를 확인합니다.

```
customers["city"].nunique(dropna=False)
customers["city"].value_counts(dropna=False)
```

상품 카테고리나 주문 상태도 확인합니다.

```
products["category"].value_counts(dropna=False)
orders["order_status"].value_counts(dropna=False)
```

`nunique()`는 서로 다른 값이 몇 개인지 알려 줍니다. `value_counts()`는 각 값이 몇 번 등장했는지 보여 줍니다. `dropna=False`를 지정하면 결측치도 결과에서 제외하지 않고 함께 확인할 수 있습니다.

범주형 컬럼을 볼 때는 다음을 확인합니다.

- 고유값이 몇 개인가?
- 특정 값에 데이터가 지나치게 몰려 있는가?
- 오타나 대소문자·띄어쓰기 차이가 있는가?
- 결측치가 있는가?
- 분석에 사용할 수 있는 그룹 기준인가?

예를 들어 `Seoul`, `seoul`, `SEOUL`이 함께 있다면 같은 지역이 다르게 기록된 것일 수 있습니다. 이런 문제는 전처리 단계에서 정리해야 합니다.

날짜 컬럼 확인하기

날짜 컬럼은 분석에서 매우 중요합니다. 월별, 분기별, 요일별 분석을 하려면 날짜 타입으로 변환해야 합니다.

먼저 주문일 컬럼의 앞부분과 데이터 타입을 확인합니다.

```
orders["order_date"].head()
orders["order_date"].dtype
```

날짜 타입으로 변환합니다.

```
orders["order_date"] = pd.to_datetime(
    orders["order_date"],
    errors="coerce",
)
```

변환 후에는 타입, 변환 실패 건수, 날짜 범위를 함께 확인합니다.

```
print("데이터 타입:", orders["order_date"].dtype)
print("날짜 변환 실패 건수:", orders["order_date"].isna().sum())
print("가장 빠른 주문일:", orders["order_date"].min())
print("가장 최근 주문일:", orders["order_date"].max())
```

errors="coerce"를 사용하면 날짜로 변환할 수 없는 값이 NaT로 바뀝니다. 날짜 변환 실패 건수가 0보다 크면 원본 값에 오타나 형식 차이가 있는지 확인해야 합니다.

고객 가입일인 signup_date도 같은 방식으로 확인할 수 있습니다.

```
customers["signup_date"] = pd.to_datetime(
    customers["signup_date"],
    errors="coerce",
)
```

이 변환은 현재 메모리에 있는 DataFrame을 변경하며 원본 CSV 파일을 자동으로 덮어쓰지는 않습니다. 전처리 결과를 파일로 저장하는 방법은 5장에서 다룹니다.

날짜 범위는 분석 질문을 결정하는 데도 중요합니다. 데이터가 3개월치인지, 1년치인지에 따라 월별 매출 분석의 해석이 달라질 수 있습니다.

6. 여러 파일의 관계 확인하기

온라인 쇼핑몰 데이터는 하나의 파일만으로 분석하기 어렵습니다. 고객, 상품, 주문, 주문 상세 데이터가 서로 연결되어 있기 때문입니다.

CSV 파일은 데이터베이스처럼 기본 키와 외래 키 제약조건을 자동으로 검사하지 않습니다. 따라서 부모 데이터의 ID가 고유한지, 연결 대상 ID가 실제로 존재하는지를 코드로 확인해야 합니다.

연결 관계	의미
customers.customer_id ↔ orders.customer_id	어떤 고객이 주문했는지 연결합니다.
orders.order_id ↔ order_items.order_id	주문과 주문 상세를 연결합니다.
products.product_id ↔ order_items.product_id	주문 상세와 상품 정보를 연결합니다.



그림 3-4. 4개 CSV 파일 간 키 관계도

앞에서 `customer_id`, `product_id`, `order_id`, `order_item_id`의 결측치와 중복을 확인했습니다. 부모 데이터의 ID가 고유하다는 전제 아래, 연결 대상에 존재하지 않는 ID가 있는지 확인합니다.

먼저 `orders`의 `customer_id`가 `customers`에 모두 존재하는지 단계별로 확인합니다.

```
customer_exists = orders["customer_id"].isin(customers["customer_id"])
invalid_customers = orders.loc[~customer_exists]

print("customers에 없는 customer_id 행 수:", len(invalid_customers))
```

`isin()`은 각 값이 다른 값 목록에 포함되어 있는지를 `True` 또는 `False`로 반환합니다. `~`는 이 참-거짓 조건을 반대로 바꿉니다. 따라서 `~customer_exists`는 고객 데이터에 존재하지 않는 `customer_id`를 가진 주문만 선택합니다.

`order_items`의 `order_id`가 `orders`에 모두 존재하는지도 확인합니다.

```
order_exists = order_items["order_id"].isin(orders["order_id"])
invalid_orders = order_items.loc[~order_exists]

print("orders에 없는 order_id 행 수:", len(invalid_orders))
```

마지막으로 `order_items`의 `product_id`가 `products`에 모두 존재하는지 확인합니다.

```
product_exists = order_items["product_id"].isin(products["product_id"])
invalid_products = order_items.loc[~product_exists]

print("products에 없는 product_id 행 수:", len(invalid_products))
```

부모 데이터의 주요 ID에 결측치와 중복이 없고, 세 검사 결과가 모두 0이면 현재 샘플 데이터의 기본적인 연결 관계가 유지되고 있다고 볼 수 있습니다.

검사 결과가 0보다 크다면 해당 행을 바로 삭제하지 말고 다음을 먼저 확인합니다.

- 데이터 생성이나 수집 과정에서 일부 파일이 누락되었는가?
- ID의 데이터 타입이 서로 다른가? 예: 숫자 1과 문자열 "1"
- 앞뒤 공백이나 표기 차이가 있는가?
- 기준 데이터보다 거래 데이터가 먼저 들어온 시점 차이가 있는가?
- 어떤 병합 방식이 분석 목적에 적합한가?

실제 병합과 조인 방식은 4장에서 자세히 다룹니다.

7. 반복 점검을 함수로 정리하기

데이터 구조 확인은 여러 파일에 반복해서 적용하는 작업입니다. 이럴 때는 간단한 함수를 만들어 두면 편리합니다.

```
def check_data_overview(name, df, key_column=None):
    print(f"===== {name} =====")
    print("shape:", df.shape)

    print("\ncolumns:")
    print(list(df.columns))

    print("\ndtypes:")
    print(df.dtypes)

    print("\nmissing values:")
    print(df.isna().sum())

    print("\nmissing ratio (%):")
    print((df.isna().mean() * 100).round(2))

    print("\nduplicated rows:", df.duplicated().sum())

    if key_column is not None:
        print(f"\n{key_column} missing:", df[key_column].isna().sum())
        print(f"{key_column} duplicated:", df[key_column].duplicated().sum())
```

def는 사용자가 직접 함수를 만들 때 사용하는 Python 문법입니다. 위 함수는 데이터셋 이름, DataFrame, 선택적인 ID 컬럼명을 입력받아 크기, 컬럼, 데이터 타입, 결측치, 중복을 한 번에 출력합니다.

각 데이터셋에 적용할 주요 ID를 딕셔너리로 정리합니다.

```
key_columns = {
    "customers": "customer_id",
    "products": "product_id",
    "orders": "order_id",
    "order_items": "order_item_id",
}
```

여러 데이터셋에 반복 적용합니다.

```
for name, df in datasets.items():
    check_data_overview(
        name=name,
        df=df,
        key_column=key_columns[name],
    )
print()
```

Jupyter Notebook에서는 코드, 실행 결과, 해석 메모를 함께 남기면 이후 분석 과정을 다시 확인하기 쉽습니다.



그림 3-5. Jupyter Notebook 데이터 구조 점검 결과 화면 예시

이 함수는 간단하지만 실무에서도 유용한 기본 패턴입니다. 새로운 데이터를 받을 때마다 같은 형식으로 구조를 점검하면 분석 초기에 발생하는 실수를 줄일 수 있습니다.

8. LLM을 활용해 데이터 구조 점검하기

LLM은 데이터 구조를 설명하고 분석 계획을 세우는 데 도움을 줄 수 있습니다. 하지만 실제 고객명, 주문 상세, 거래 정보처럼 개인이나 거래를 식별할 수 있는 값을 승인되지 않은 외부 LLM 서비스에 그대로 입력해서는 안 됩니다.

원본 행 대신 다음과 같은 구조 정보만 제공하는 것이 좋습니다.

- 컬럼명과 데이터 타입
- 행과 열 개수
- 결측치 개수 또는 비율
- 전체 중복 행 개수
- 주요 ID의 결측치와 중복 개수
- 범주형 컬럼의 고유값 목록 또는 요약
- 날짜 범위

예를 들어 `customers.head()` 결과를 그대로 입력하기보다 실제 점검 결과를 다음처럼 요약해서 질문할 수 있습니다.

customers.csv 구조 요약

컬럼:

- customer_id: int64
- name: object
- gender: object
- age: int64
- city: object
- signup_date: object

데이터 크기: 150행 6열

결측치: 없음

전체 중복 행: 0개

customer_id 중복: 0개

이 데이터 구조를 보고 분석 전에 추가로 확인해야 할 점을 알려 주세요.

실제 데이터 확인 없이 단정하지 말고, 추정과 확인이 필요한 내용을 구분해 주세요.

위 숫자는 현재 기본 생성 스크립트의 예상 결과입니다. 데이터가 바뀌었다면 예시를 그대로 복사하지 말고 직접 실행한 결과로 교체해야 합니다.

여러 파일의 역할 설명 요청 예시

온라인 쇼핑몰 데이터 분석을 시작하기 전에 다음 CSV 파일들의 구조를 이해하려고 합니다.

파일 목록:

- customers.csv: 고객 정보
- products.csv: 상품 정보
- orders.csv: 주문 정보
- order_items.csv: 주문 상세 정보

각 파일의 역할과 예상되는 연결 키를 설명하고,
분석 전에 확인해야 할 항목을 체크리스트로 정리해 주세요.

실제 데이터 확인 없이 단정하지 말고,
추정한 내용과 실제로 확인해야 할 내용을 구분해 주세요.

컬럼 의미 추정 요청 예시

다음은 customers.csv의 컬럼 목록입니다.

- customer_id
- name
- gender
- age
- city
- signup_date

각 컬럼의 의미를 추정하고,
데이터 분석에서 어떻게 활용할 수 있는지 표로 정리해 주세요.

컬럼명만으로 확정할 수 없는 내용은
반드시 '확인 필요'라고 표시해 주세요.

데이터 타입 점검 요청 예시

다음은 orders.csv의 컬럼과 데이터 타입입니다.

```
order_id: int64
customer_id: int64
order_date: object
payment_method: object
order_status: object
```

이 데이터 타입을 보고 분석 전에 확인해야 할 점을 알려 주세요.
특히 order_date 컬럼의 변환 방법과 변환 후 검증 항목을 설명해 주세요.

LLM 답변 검증 요청 예시

LLM이 다음과 같이 답했습니다.

"고객 데이터에 age 컬럼이 있으므로 연령대별 매출 분석을 바로 수행하면 됩니다."

이 답변이 충분히 안전한지 검토해 주세요.
실제 분석 전에 추가로 확인해야 할 사항을 알려 주세요.
결측치, 이상치, 고객 데이터와 주문 데이터의 연결 가능성을 포함해 설명해 주세요.

LLM이 제안한 분석이 실제 데이터와 맞는지 확인하는 과정은 반드시 필요합니다. 예를 들어 연령대별 매출 분석을 제안했다면 다음을 확인해야 합니다.

- age 컬럼이 실제로 존재하는가?
- age가 숫자형이며 업무적으로 가능한 범위인가?
- 결측치가 있는가?
- customer_id가 고객 데이터에서 고유한가?
- 고객 데이터와 주문 데이터가 customer_id로 정상적으로 연결되는가?
- 취소·환불 주문을 매출에 포함할 것인가?

LLM은 초안을 빠르게 만들 수 있지만, 보고서에 들어가는 숫자와 해석은 반드시 실제 실행 결과를 기준으로 확인해야 합니다.

9. 구조 점검에서 다음 분석으로

이번 장에서 확인한 결과는 최종 분석 결론이 아닙니다. 데이터 분석을 시작하기 전의 사전 점검 결과입니다. 데이터가 몇 행과 몇 열인지, 어떤 컬럼이 있는지, 결측치와 중복이 있는지, 주요 ID가 고유한지, 파일 간 연결 관계가 유지되는지를 확인해야 이후 분석이 안정적으로 이어집니다.

데이터 구조 점검을 마칠 때는 다음 항목을 확인합니다.

점검 항목	확인
필요한 CSV 파일이 모두 존재하는가?	☐
현재 작업 폴더와 데이터 경로를 확인했는가?	☐
각 데이터셋의 행과 열 개수를 확인했는가?	☐
컬럼명이 예상과 일치하는가?	☐
숫자형, 문자열, 날짜형 컬럼을 구분했는가?	☐
날짜 컬럼의 변환 실패 건수와 범위를 확인했는가?	☐
결측치가 있는 컬럼과 비율을 확인했는가?	☐
전체 중복 행이 있는지 확인했는가?	☐
주요 ID 컬럼의 결측치와 중복 여부를 확인했는가?	☐
ID를 일반 숫자형 변수처럼 해석하지 않았는가?	☐
여러 파일을 연결할 키 컬럼을 확인했는가?	☐
연결 대상에 존재하지 않는 ID가 있는지 확인했는가?	☐
LLM에 원본 행 대신 구조 요약을 제공했는가?	☐
LLM이 제안한 설명을 실제 데이터와 비교해 검증했는가?	☐

직접 더 연습해 보고 싶다면 다음을 수행해 봅니다.

- 4개 CSV 파일의 행과 열 개수를 하나의 표로 정리합니다.
- 각 파일의 결측치 비율과 전체 중복 행 개수를 확인합니다.
- 주요 ID 컬럼의 결측치와 중복 여부를 표로 정리합니다.
- 주문일과 가입일을 날짜 타입으로 변환하고 데이터 기간을 확인합니다.
- 범주형 컬럼의 고유값과 빈도를 확인합니다.
- LLM에게 데이터 구조 요약을 요청한 뒤 실제 데이터와 맞는지 검증합니다.
- `check_data_overview()` 함수에 날짜 범위와 범주형 고유값 요약을 추가합니다.

데이터 구조를 이해하면 다음 단계가 훨씬 쉬워집니다. 다음 장에서는 이번 장에서 불러온 데이터를

바탕으로 pandas의 선택, 필터링, 정렬, 집계 기초를 다룹니다.

4장. pandas로 데이터에 질문하기

3장에서는 데이터를 불러오고 구조를 살펴보았습니다. 파일이 몇 개인지, 어떤 컬럼이 있는지, 결측치와 중복은 없는지, 여러 파일이 어떤 키로 연결되는지 확인했습니다. 이제부터는 데이터를 단순히 바라보는 단계를 넘어, 데이터에 질문을 던지고 pandas로 답을 찾아갑니다.

pandas는 Python 데이터 분석에서 가장 기본적이면서도 강력한 도구입니다. 필요한 컬럼만 고르고, 조건에 맞는 행을 추출하고, 특정 기준으로 정렬하고, 그룹별로 합계나 평균을 계산할 수 있습니다. 대부분의 분석은 이 기본 기능들의 조합에서 시작됩니다.

이번 장의 핵심은 복잡한 머신러닝 모델을 만드는 것이 아닙니다. 데이터를 정확히 선택하고, 조건에 맞게 걸러내고, 새로운 계산 컬럼을 만들고, 여러 파일을 안전하게 연결해 요약표를 만드는 능력을 익히는 것입니다. 이 능력이 있어야 이후 전처리, EDA, 시각화, 머신러닝, LLM 기반 분석도 안정적으로 이어질 수 있습니다.

이 장에서 생각해 볼 질문

- 고객 데이터에서 필요한 컬럼만 보고 싶다면 어떻게 해야 할까?
- 30세 이상 고객이나 특정 지역 고객만 추출하려면 어떻게 해야 할까?
- 가격이 높은 상품이나 최근 주문을 빠르게 확인하려면 어떻게 정렬해야 할까?
- 수량과 단가를 이용해 주문 상세 금액을 만들 수 있을까?
- 취소·환불 주문을 제외한 완료 주문 매출은 어떻게 계산할까?
- 카테고리별 매출이나 월별 매출은 어떻게 계산할까?
- 여러 CSV 파일을 연결할 때 결과가 부풀려지지 않았는지 어떻게 검증할까?
- LLM이 만들어 준 pandas 코드는 어떻게 검증해야 할까?

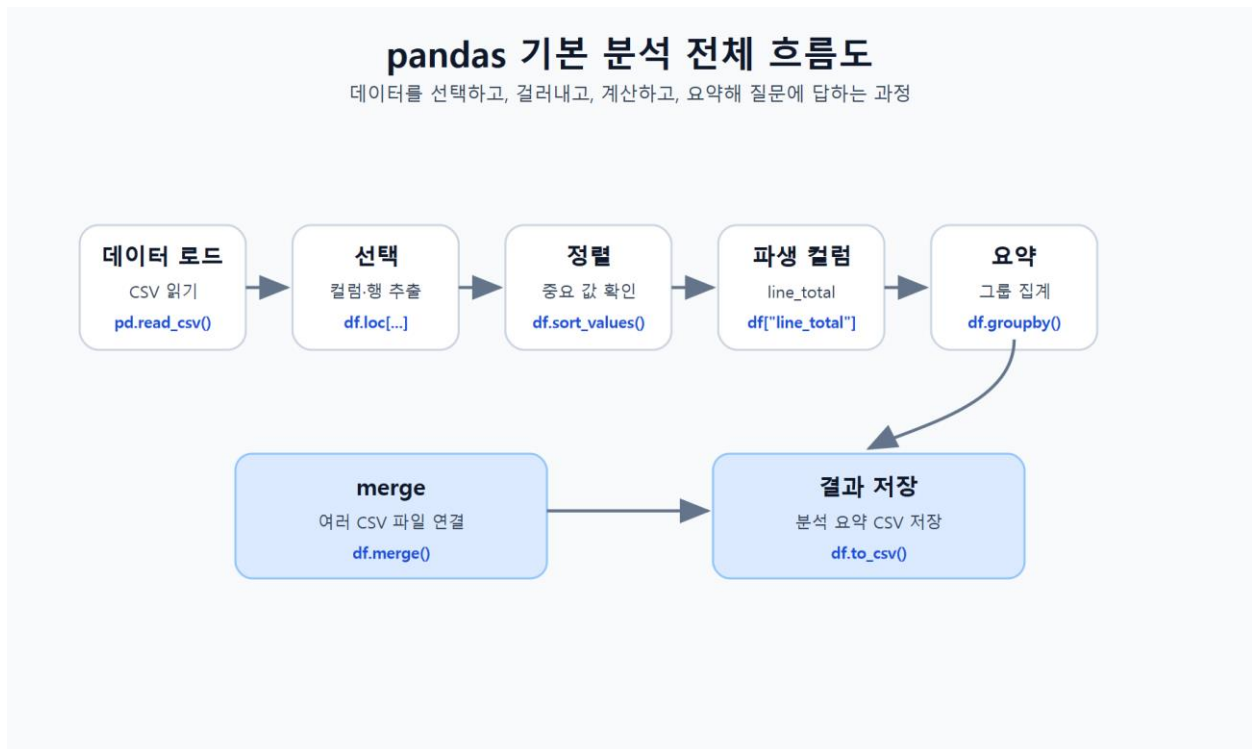


그림 4-1. pandas 기본 분석 전체 흐름도

1. pandas 기본 분석의 흐름

pandas 기본 분석은 DataFrame에서 필요한 데이터를 선택하고, 조건에 맞게 걸러내고, 기준으로 요약하는 작업입니다.

작업	pandas 기능	예시
컬럼 선택	df["컬럼명"], df[...]	고객 ID와 나이만 선택
행 필터링	조건식, isin()	30세 이상 고객만 추출
정렬	sort_values()	가격이 높은 상품순 정렬
파생 컬럼 생성	새 컬럼 대입	수량 × 단가로 주문 상세 금액 계산
빈도 확인	value_counts()	주문 상태별 건수 확인
그룹 집계	groupby(), agg()	카테고리별 매출 합계
파일 연결	merge()	주문 상세와 상품 데이터 연결
병합 검증	validate, indicator	키 중복과 미매칭 확인
결과 저장	to_csv()	분석 요약 결과 CSV 저장

이 기능들은 실제 분석에서 연결해서 사용합니다. 예를 들어 카테고리별 완료 주문 매출을 계산하려면 다음 순서가 필요합니다.

5. 주문 상세 데이터에 `line_total`을 만듭니다.
6. 주문 데이터와 병합해 주문 상태를 붙입니다.
7. `completed` 주문만 남깁니다.
8. 상품 데이터와 병합해 카테고리를 붙입니다.
9. 카테고리 기준으로 매출을 집계합니다.
10. 병합 전후 행 수와 미매칭 건수를 검증합니다.

2. 데이터를 선택하고 걸러내는 법

분석은 보통 “무엇을 볼 것인가”에서 시작합니다. 모든 컬럼과 모든 행을 한꺼번에 보는 대신, 질문에 필요한 부분만 선택하면 데이터가 훨씬 읽기 쉬워집니다.

컬럼 선택

컬럼 하나를 선택하면 Series가 됩니다.

```
customers["city"].head()
```

컬럼 여러 개를 선택하면 DataFrame이 됩니다.

```
customers[["customer_id", "gender", "age", "city"]].head()
```

분석 전에는 항상 실제 컬럼명을 먼저 확인합니다.

```
customers.columns.tolist()
```

LLM이 작성한 코드에서 존재하지 않는 컬럼명을 사용하면 `KeyError`가 발생합니다. 컬럼 이름의 대소문자, 공백, 언어가 실제 데이터와 일치하는지도 확인해야 합니다.

행 필터링

30세 이상 고객만 추출합니다.

```
customers_over_30 = customers[customers["age"] >= 30]
```

조건이 여러 개인 경우에는 `&`, `|`, `~`를 사용합니다.

연산자	의미	예시
<code>&</code>	그리고	30세 이상이면서 서울 거주
<code> </code>	또는	서울 또는 부산 거주

연산자	의미	예시
~	아니다	완료 상태가 아닌 주문

각 조건은 괄호로 감싸는 습관을 들이는 것이 좋습니다.

```
customers[
    (customers["age"] >= 30)
    & (customers["city"] == "서울")
]
```

여러 값 중 하나에 해당하는 데이터를 찾을 때는 `isin()`이 읽기 쉽습니다.

```
customers[customers["city"].isin(["서울", "부산"])]
```

이 책의 샘플 데이터 생성 스크립트는 도시명을 서울, 부산처럼 한글로 만듭니다. 다른 데이터에서는 Seoul, Busan처럼 영어로 저장될 수도 있으므로 필터링 전에 실제 값을 확인합니다.

```
customers["city"].value_counts()
```



그림 4-2. 컬럼 선택·필터링·정렬 개념도

3. 정렬과 파생 컬럼

정렬

가격이 높은 상품부터 확인합니다.

```
products.sort_values("price", ascending=False).head(10)
```

ascending=False는 내림차순, ascending=True는 오름차순입니다. 가격, 매출, 주문 수처럼 큰 값을 먼저 보고 싶을 때는 내림차순을 자주 사용합니다.

파생 컬럼 만들기

파생 컬럼은 기존 컬럼을 활용해 새로 만든 컬럼입니다. 주문 상세 데이터의 수량과 단가를 곱하면 한 행의 주문 상세 금액을 계산할 수 있습니다.

```
order_items = order_items.copy()
order_items["line_total"] = (
    order_items["quantity"] * order_items["unit_price"]
)
```

기존 컬럼	파생 컬럼	의미
quantity, unit_price	line_total	주문 상세 1행의 금액
order_date	order_month	주문 월
age	age_group	연령대
order_status	is_completed	완료 주문 여부

파생 컬럼 line_total 생성 개념도

기존 컬럼을 계산해 분석에 필요한 새로운 컬럼을 만든다.



그림 4-3. 파생 컬럼 line_total 생성 개념도

line_total의 합계가 곧바로 확정 매출을 의미하는 것은 아닙니다. 주문 상세 데이터에는 취소되거나 환불된 주문의 상품도 포함될 수 있습니다. 따라서 주문 상태를 연결하기 전에는 **전체 주문 상세 금액**으로 표현하는 것이 정확합니다.

```
all_order_amount = order_items["line_total"].sum()
all_order_amount
```

4. 그룹별로 요약하기

그룹별 집계는 데이터를 특정 기준으로 묶고 합계, 평균, 개수 등을 계산하는 작업입니다.

```
products.groupby("category")["price"].mean()
```

여러 집계를 한 번에 수행할 때는 agg()를 사용합니다.

```
products.groupby("category", as_index=False).agg(
    product_count=("product_id", "nunique"),
    average_price=("price", "mean"),
)
```

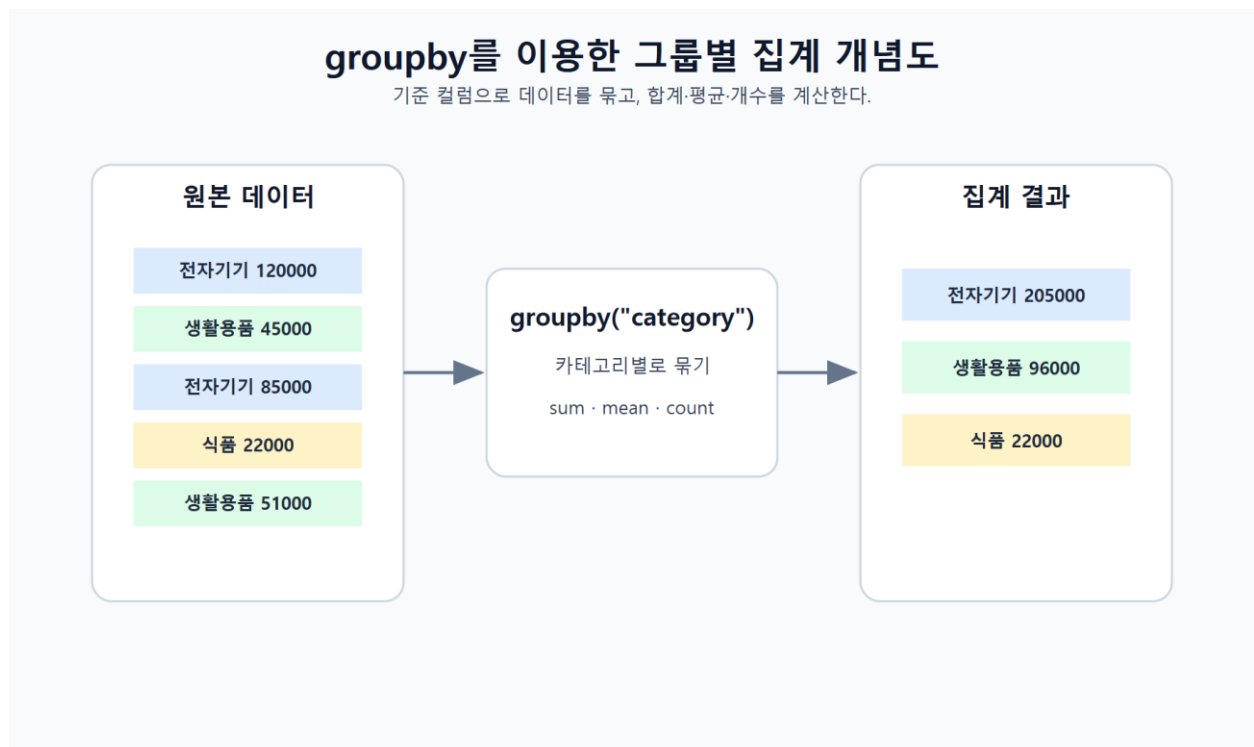


그림 4-4. groupby를 이용한 그룹별 집계 개념도

단일 컬럼의 값별 빈도는 value_counts()로 확인할 수 있습니다.

```
orders["order_status"].value_counts()
```

비율까지 확인하려면 `normalize=True`를 사용합니다.

```
orders["payment_method"].value_counts(normalize=True).mul(100).round(1)
```

5. 여러 파일을 안전하게 연결하는 merge

온라인 쇼핑몰 데이터는 하나의 파일만으로 충분히 분석하기 어렵습니다.

- `order_items.csv`: 주문별 상품, 수량, 단가
- `products.csv`: 상품명, 카테고리, 가격
- `orders.csv`: 주문일, 고객 ID, 주문 상태
- `customers.csv`: 고객 속성

예를 들어 `order_items`와 `products`는 `product_id`를 기준으로 연결합니다.

```
sales_items = order_items.merge(
    products,
    on="product_id",
    how="left",
    validate="many_to_one",
)
```

`validate="many_to_one"`은 왼쪽의 `product_id`는 여러 번 나올 수 있지만 오른쪽 `products`의 `product_id`는 한 번만 나와야 한다는 뜻입니다. 상품 테이블의 키가 중복되어 있으면 오류를 발생시켜 잘못된 병합을 조기에 발견할 수 있습니다.

`left merge`가 왼쪽 행 수를 항상 보장하는 것은 아닙니다. 오른쪽 기준 키가 중복되면 왼쪽 한 행이 여러 행으로 늘어날 수 있습니다. 따라서 다음 항목을 확인해야 합니다.

- 양쪽 데이터에 연결 기준 컬럼이 있는가?
- 오른쪽 기준 키가 기대한 대로 고유한가?
- 병합 후 행 수가 예상과 같은가?
- 오른쪽 정보가 연결되지 않은 행은 몇 개인가?
- 같은 이름의 컬럼이 `_x`, `_y`로 중복되지 않았는가?

병합 출처를 직접 확인하려면 `indicator=True`를 사용할 수 있습니다.

```

sales_items_check = order_items.merge(
    products,
    on="product_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)

sales_items_check["_merge"].value_counts()

```

모든 행이 `both`라면 양쪽 데이터가 정상적으로 연결된 것입니다.

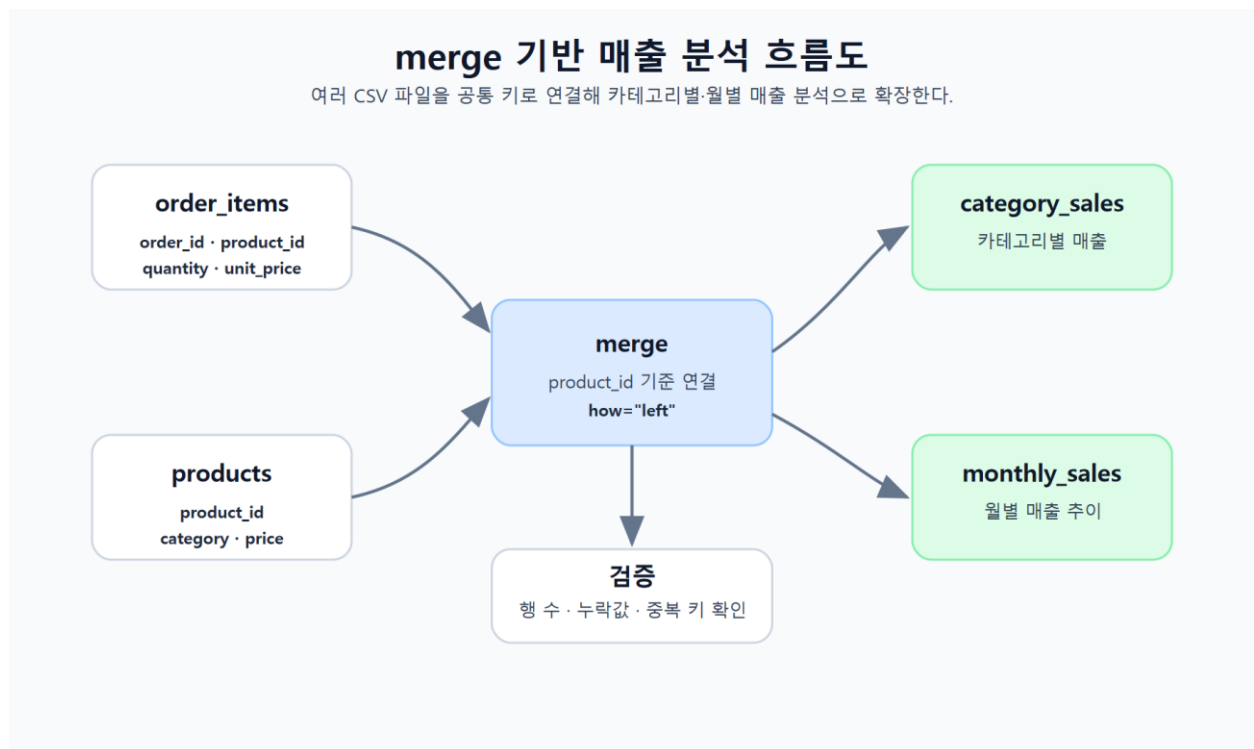


그림 4-5. merge 기반 매출 분석 흐름도

6. 실습 환경과 데이터 불러오기

이번 장의 전체 코드는 `notebooks/ch04_pandas_basic.ipynb`에서 실행할 수 있습니다.

프로젝트 루트 찾기

VS Code에서 Notebook을 실행할 때 현재 작업 폴더가 프로젝트 루트일 수도 있고 notebooks 폴더일 수도 있습니다. 고정된 상대 경로만 사용하면 실행 위치에 따라 FileNotFoundError가 발생할 수 있습니다.

```
from pathlib import Path

import pandas as pd

def find_project_root(start_path):
    start_path = Path(start_path).resolve()
    for candidate in [start_path, *start_path.parents]:
        if (
            (candidate / "requirements.txt").exists()
            and (candidate / "scripts").exists()
        ):
            return candidate
    raise FileNotFoundError("프로젝트 루트 폴더를 찾을 수 없습니다.")

PROJECT_ROOT = find_project_root(Path.cwd())
DATA_DIR = PROJECT_ROOT / "data" / "raw"
REPORT_DIR = PROJECT_ROOT / "reports"
REPORT_DIR.mkdir(parents=True, exist_ok=True)
```

파일 존재 여부 확인

```
required_files = [
    "customers.csv",
    "products.csv",
    "orders.csv",
    "order_items.csv",
]

missing_files = [
    filename
    for filename in required_files
    if not (DATA_DIR / filename).exists()
]

if missing_files:
    raise FileNotFoundError(
        "필요한 데이터 파일이 없습니다: " + ", ".join(missing_files)
        + ". 프로젝트 루트에서 "
        + "python scripts/generate_sample_data.py를 실행하세요."
    )
```

데이터 불러오기

```
customers = pd.read_csv(DATA_DIR / "customers.csv")
products = pd.read_csv(DATA_DIR / "products.csv")
orders = pd.read_csv(DATA_DIR / "orders.csv")
order_items = pd.read_csv(DATA_DIR / "order_items.csv")
```

각 데이터의 크기와 컬럼명을 확인합니다.

```
for name, df in {
    "customers": customers,
    "products": products,
    "orders": orders,
    "order_items": order_items,
}.items():
    print(name, df.shape, df.columns.tolist())
```


7. 기본 선택·필터링·정렬 실습

```
customer_basic = customers[
    ["customer_id", "gender", "age", "city"]
]
customer_basic.head()

customers_over_30 = customers[customers["age"] >= 30]
customers_over_30.head()

seoul_customers = customers[customers["city"] == "서울"]
seoul_customers.head()

city_customers = customers[
    customers["city"].isin(["서울", "부산"])
]
city_customers.head()
```

주문 상태는 필터링 전에 실제 값을 확인합니다.

```
orders["order_status"].value_counts()

completed_orders = orders[
    orders["order_status"] == "completed"
]
completed_orders.head()
```

8. 완료 주문 기준 분석 데이터 만들기

먼저 주문 상세 금액을 만듭니다.

```
order_items = order_items.copy()
order_items["line_total"] = (
    order_items["quantity"] * order_items["unit_price"]
)
```

주문 상태와 주문일을 붙입니다.

```

order_sales = order_items.merge(
    orders[
        [
            "order_id",
            "customer_id",
            "order_date",
            "order_status",
        ]
    ],
    on="order_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)

print(order_sales["_merge"].value_counts())
order_sales = order_sales.drop(columns="_merge")

```

주문일을 날짜 타입으로 변환하고 실패 건수를 확인합니다.

```

order_sales["order_date"] = pd.to_datetime(
    order_sales["order_date"],
    errors="coerce",
)

print("날짜 변환 실패:", order_sales["order_date"].isna().sum())

```

완료 주문만 추출합니다.

```

completed_order_sales = order_sales[
    order_sales["order_status"] == "completed"
].copy()

completed_order_sales["order_month"] = (
    completed_order_sales["order_date"]
    .dt.to_period("M")
    .astype(str)
)

```

이후 이 장에서 사용하는 매출은 별도 설명이 없는 한 완료 주문 기준입니다. 취소·환불 주문을 포함한 주문 금액과 완료 주문 매출을 구분해야 결과를 정확하게 해석할 수 있습니다.

9. 카테고리별·상품별 매출

완료 주문 상세에 상품 정보를 붙입니다.

```
completed_sales_items = completed_order_sales.merge(
    products,
    on="product_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)

print(completed_sales_items["_merge"].value_counts())
completed_sales_items = completed_sales_items.drop(columns="_merge")
```

카테고리별 매출

```
category_sales = (
    completed_sales_items
    .groupby("category", as_index=False)
    .agg(
        total_quantity=("quantity", "sum"),
        total_sales=("line_total", "sum"),
    )
    .sort_values("total_sales", ascending=False)
)

category_sales["sales_ratio"] = (
    category_sales["total_sales"]
    / category_sales["total_sales"].sum()
    * 100
).round(2)

category_sales
```

카테고리 매출이 높은 이유가 판매 수량 때문인지, 평균 단가 때문인지는 이 결과만으로 알 수 없습니다. 수량과 단가를 함께 확인해야 합니다.

상품별 매출

```
product_sales = (
    completed_sales_items
    .groupby(
        ["product_id", "product_name", "category"],
        as_index=False,
    )
    .agg(
        total_quantity=("quantity", "sum"),
        total_sales=("line_total", "sum"),
    )
    .sort_values("total_sales", ascending=False)
)

product_sales.head(10)
```

10. 월별 매출과 고객별 구매 금액

월별 매출

```
monthly_summary = (
    completed_order_sales
    .groupby("order_month", as_index=False)
    .agg(
        total_sales=("line_total", "sum"),
        order_count=("order_id", "nunique"),
    )
    .sort_values("order_month")
)

monthly_summary["average_order_value"] = (
    monthly_summary["total_sales"]
    / monthly_summary["order_count"]
).round(0)

monthly_summary
```

`order_count=("order_id", "nunique")`는 주문 상세 행 수가 아니라 고유한 주문 건수를 계산한다는 뜻입니다.

고객별 구매 금액

고객 식별자인 `customer_id`를 먼저 기준으로 집계한 뒤 고객 속성을 붙입니다. 고객 이름을 그룹 기준에 직접 포함하면 이름 변경이나 중복 이름 때문에 결과가 흔들릴 수 있습니다.

```
customer_sales = (
    completed_order_sales
    .groupby("customer_id", as_index=False)
    .agg(
        order_count=("order_id", "nunique"),
        total_sales=("line_total", "sum"),
    )
    .sort_values("total_sales", ascending=False)
)

customer_sales = customer_sales.merge(
    customers[["customer_id", "city"]],
    on="customer_id",
    how="left",
    validate="one_to_one",
)

customer_sales["customer_label"] = (
    "Customer " + customer_sales["customer_id"].astype(str)
)

customer_sales[
    [
        "customer_label",
        "city",
        "order_count",
        "total_sales",
    ]
].head(10)
```

보고서나 LLM 입력에는 실제 고객 이름, 이메일, 전화번호보다 익명화된 고객 ID를 사용하는 것이 안전합니다.

11. 분석 결과 저장하기

Windows Excel에서 한글 CSV를 바로 열 계획이라면 `encoding="utf-8-sig"`를 사용합니다.

```

category_sales.to_csv(
    REPORT_DIR / "ch04_category_sales.csv",
    index=False,
    encoding="utf-8-sig",
)

product_sales.to_csv(
    REPORT_DIR / "ch04_product_sales.csv",
    index=False,
    encoding="utf-8-sig",
)

monthly_summary.to_csv(
    REPORT_DIR / "ch04_monthly_sales.csv",
    index=False,
    encoding="utf-8-sig",
)

customer_sales.to_csv(
    REPORT_DIR / "ch04_customer_sales.csv",
    index=False,
    encoding="utf-8-sig",
)

```

저장 후에는 파일 존재 여부도 확인할 수 있습니다.

```

for path in sorted(REPORT_DIR.glob("ch04_*.csv")):
    print(path.name, path.exists(), path.stat().st_size)

```

12. 반복되는 점검을 함수로 정리하기

```

def summarize_dataframe(name, df):
    print(f"===== {name} =====")
    print("shape:", df.shape)
    print("columns:", df.columns.tolist())
    print("missing values:", int(df.isna().sum().sum()))
    print("duplicated rows:", int(df.duplicated().sum()))

```

```
datasets = {
    "customers": customers,
    "products": products,
    "orders": orders,
    "order_items": order_items,
}

for name, df in datasets.items():
    summarize_dataframe(name, df)
    print()
```

이 함수는 기본적인 이상 신호를 빠르게 찾는 용도입니다. 결측치의 처리 방법이나 이상값 판단은 다음 장의 전처리 과정에서 더 자세히 다룹니다.

13. LLM에게 pandas 코드를 요청하고 검증하기

LLM은 pandas 코드 작성과 분석 아이디어 정리에 도움을 줄 수 있습니다. 하지만 실제 컬럼명, 데이터 타입, 키의 고유성, 주문 상태 범위와 반드시 비교해야 합니다.

실제 고객명, 이메일, 주문 원본을 그대로 입력하지 말고 컬럼명, 데이터 크기, 요약 통계, 결측치 개수처럼 구조화된 정보만 전달하는 것이 좋습니다.

필터링 코드 요청 예시

다음 customers DataFrame에서 조건에 맞는 데이터를 필터링하는 pandas 코드를 작성해 주세요.

컬럼:

- customer_id
- gender
- age
- city

조건:

- age가 30 이상
- city가 서울 또는 부산

실제 데이터가 아니라 컬럼 구조만 보고 작성해 주세요.

각 조건을 괄호로 감싸고 isin()을 사용하는 이유도 설명해 주세요.

안전한 병합 코드 요청 예시

order_items와 products를 product_id 기준으로 left merge하려고 합니다.

요구사항:

- products.product_id가 고유한지 확인
- validate="many_to_one" 사용
- indicator=True로 미매칭 행 확인
- 병합 전후 행 수 비교
- 중복 컬럼 확인

초보자가 이해할 수 있도록 코드와 검증 순서를 작성해 주세요.

완료 주문 기준 월별 매출 요청 예시

orders와 order_items를 사용해 완료 주문 기준 월별 매출을 계산하려고 합니다.

orders 컬럼:

- order_id
- customer_id
- order_date
- order_status

order_items 컬럼:

- order_id
- product_id
- quantity
- unit_price

요구사항:

1. quantity와 unit_price로 line_total 생성
2. order_id 기준 many-to-one 병합
3. order_date 날짜 변환 실패 건수 확인
4. order_status가 completed인 주문만 필터링
5. 월별 total_sales와 고유 order_count 계산
6. 취소·환불 주문이 제외되었는지 확인

LLM 코드 검증 예시

LLM이 다음 코드를 제안했습니다.

```
category_sales = order_items.groupby("category")["line_total"].sum()
```

현재 데이터 구조:

- order_items: order_id, product_id, quantity, unit_price, line_total
- products: product_id, product_name, category, price
- orders: order_id, order_status

이 코드가 바로 실행 가능한지 검토해 주세요.

카테고리 연결 과정과 완료 주문 필터링이 빠져 있다면 이유를 설명하고, merge 검증을 포함한 수정 코드를 제안해 주세요.

LLM이 만든 코드가 짧고 그럴듯해 보여도 실제 데이터 구조와 분석 범위가 맞지 않으면 실행되지 않거나 잘못된 결과를 만들 수 있습니다.

14. 결과를 읽는 방법

결과	확인할 질문
카테고리별 매출	완료 주문만 포함했는가? 판매 수량과 단가를 함께 봐야 하는가?
상품별 매출	특정 상품에 매출이 집중되는가?
월별 매출	월별 기간이 완전한가? 최근 월이 일부 기간만 포함된 것은 아닌가?
고객별 구매 금액	일부 고객에게 집중되는가? 개인정보 표현은 안전한가?
병합 결과	키 중복과 미매칭 행을 확인했는가?

숫자를 계산하는 것에서 멈추지 말고, 어떤 조건과 범위로 계산했는지 함께 기록해야 합니다.

데이터에 없는 원인을 단정하지 않고 추가 확인이 필요한 사항을 구분하는 것이 중요합니다.

15. 실습 점검표

- 실제 컬럼명과 필터 값의 언어를 확인했는가?
- 원본 DataFrame을 수정하기 전에 필요한 경우 `.copy()`를 사용했는가?
- `line_total`과 완료 주문 매출을 구분했는가?
- 병합에 `validate`를 사용했는가?
- 병합 전후 행 수와 미매칭 건수를 확인했는가?

- 주문 수는 상세 행 수가 아니라 `nunique()`로 계산했는가?
- 고객 결과에서 개인정보를 안전하게 처리했는가?
- 저장된 CSV 파일을 다시 열 수 있는지 확인했는가?

16. 다음 장으로 이어지는 흐름

이번 장에서는 pandas로 데이터를 선택하고, 필터링하고, 정렬하고, 파생 컬럼을 만들고, 그룹별 집계를 수행했습니다. 또한 여러 CSV 파일을 `merge()`로 연결하고 병합 결과를 검증한 뒤, 완료 주문 기준으로 카테고리별·상품별·월별·고객별 매출 요약표를 만들었습니다.

다음 장에서는 이런 분석을 더 믿을 수 있게 만들기 위해 데이터 전처리를 다룹니다. 결측치, 중복, 타입 오류, 날짜 형식, 이상값 후보를 정리해야 분석 결과의 신뢰도를 높일 수 있습니다.

5장. 분석을 믿을 수 있게 만드는 데이터 전처리

데이터 분석은 데이터를 불러오는 순간 바로 시작되는 것처럼 보이지만, 실제 분석의 신뢰도는 그전에 결정되는 경우가 많습니다. 값이 비어 있거나, 같은 고객이 여러 번 기록되어 있거나, 낱자가 문자열로 저장되어 있거나, 숫자처럼 보이는 가격이 문자로 들어 있으면 분석 결과는 쉽게 왜곡됩니다.

데이터 전처리는 이러한 문제를 찾아내고 분석 목적에 맞게 정리하는 과정입니다. 단순히 데이터를 “깨끗하게” 만드는 작업이 아니라, **데이터의 상태를 확인하고 처리 기준과 영향을 기록하여 분석 결과를 검증할 수 있게 만드는 일**에 가깝습니다.

이 장에서는 온라인 쇼핑물의 고객·상품·주문 데이터를 사용해 문자열과 빈 값, 데이터 타입, 결측치, 중복, 이상값, 파생 컬럼, 파일 간 연결 관계를 차례대로 점검합니다. 전처리 결과는 원본과 분리해 data/processed 폴더에 저장하고, 이후 탐색적 데이터 분석(EDA)과 시각화에서 사용할 수 있는 형태로 준비합니다.



그림 5-1. 데이터 전처리 전체 흐름도

이 장에서 생각해 볼 질문

- 결측치는 모두 삭제하거나 대표값으로 채워도 되는가?
- 전체 행 중복과 고유 ID 중복은 어떻게 다른가?
- 공백 문자열이나 숫자 변환 실패는 언제 결측치가 되는가?
- 이상값은 어떤 기준으로 제외해야 하는가?

- 주문 상세 금액의 합계를 바로 매출이라고 불러도 되는가?
- 전처리 후에도 파일 간 연결 관계가 유지되는가?
- 전처리 기준과 제외된 데이터는 어떻게 기록해야 하는가?
- LLM이 제안한 전처리 코드를 어떤 기준으로 검증해야 하는가?

1. 왜 전처리가 필요한가

현실의 데이터는 처음부터 분석하기 좋은 형태로 주어지지 않습니다. 같은 도시명이 `Seoul`, `seoul`, `SEOUL`처럼 다르게 기록될 수 있고, 날짜가 2024-01-15라는 문자열로 저장되어 있을 수도 있습니다. 숫자처럼 보이는 "10,000"도 실제로는 쉼표가 포함된 문자열일 수 있습니다.

이 상태에서 바로 집계하거나 그래프를 그리면 분석 결과가 달라질 수 있습니다. 고객의 평균 나이를 계산하려는데 결측치가 특정 연령대에 집중되어 있다면, 단순 평균 대체로 분포가 왜곡될 수 있습니다. 주문 수량이 음수인데 반품을 의미하는지 입력 오류인지 확인하지 않으면 금액 합계도 잘못 해석할 수 있습니다. 주문 상태가 완료, `complete`, `COMPLETED`처럼 섞여 있으면 같은 상태가 서로 다른 범주로 집계됩니다.

전처리에서 중요한 것은 문제를 무조건 삭제하거나 자동으로 고치는 것이 아닙니다. **왜 그런 값이 생겼는지, 분석 목적에 어떤 영향을 주는지, 어떤 기준으로 처리했는지를 설명할 수 있어야 합니다.**

문제 유형	예시	확인할 질문
결측치	나이가 비어 있는 고객	누락 원인은 무엇이며 분석에서 제외해도 되는가?
빈 문자열	도시 값이 공백만 포함	실제 결측치로 처리해야 하는가?
중복	같은 고객 ID가 여러 번 등장	자연스러운 반복인가, 고유 키 오류인가?
타입 오류	가격이 문자열로 저장됨	계산 가능한 숫자형으로 안전하게 변환할 수 있는가?
날짜 오류	주문일이 문자열로 저장됨	변환 실패 없이 기간 분석에 사용할 수 있는가?
문자열 표기 차이	<code>Seoul</code> , <code>Seoul</code> , <code>SEOUL</code>	하나의 표기로 통일해야 하는가?
이상값	수량이 음수이거나 가격이 0원	반품·행사·오류 중 무엇을 의미하는가?
파생 컬럼	수량과 단가만 존재	계산식과 포함 범위를 명확히 정의할 수 있는가?

문제 유형	예시	확인할 질문
연결 오류	존재하지 않는 상품 ID	부모 데이터 누락인지 거래 데이터 오류인지 확인했는가?

2. 원본 데이터와 전처리 데이터는 분리한다

전처리에서 가장 먼저 지켜야 할 원칙은 원본 파일을 직접 덮어쓰지 않는 것입니다. 원본 데이터는 `data/raw`에 보관하고, 전처리 결과는 `data/processed`에 따로 저장합니다.

```
data/
├── raw/
│   ├── customers.csv
│   ├── products.csv
│   ├── orders.csv
│   └── order_items.csv
└── processed/
    ├── customers_clean.csv
    ├── products_clean.csv
    ├── orders_clean.csv
    └── order_items_clean.csv
```

원본과 전처리 데이터를 분리하면 다음과 같은 장점이 있습니다.

- 원본 데이터를 언제든지 다시 확인할 수 있습니다.
- 전처리 기준을 바꾸어 전체 과정을 다시 실행할 수 있습니다.
- 전처리 전후의 행 수, 열 수, 결측치 변화를 비교할 수 있습니다.
- 다른 사람이 같은 과정을 재현하기 쉽습니다.
- 제외된 데이터와 처리 기준을 보고서에 명확히 남길 수 있습니다.

raw / processed 데이터 분리 구조

원본은 보존하고, 분석에 사용할 데이터는 별도 폴더에 저장합니다

data/raw

원본 데이터 보관

- customers.csv
- products.csv
- orders.csv
- order_items.csv
- 수정하지 않고 그대로 유지

data/processed

전처리 완료 데이터 저장

- customers_clean.csv
- products_clean.csv
- orders_clean.csv
- order_items_clean.csv
- 분석과 시각화에 바로 사용

장점: 원본 복구 가능 · 전후 비교 가능 · 재현성 향상 · 보고서 설명 용이

그림 5-2. raw 데이터와 processed 데이터 분리 구조

DataFrame도 같은 원칙으로 다룹니다. 다음처럼 `.copy()`를 사용하면 원본 DataFrame과 독립된 전처리용 복사본을 만들 수 있습니다.

```
customers_clean = customers.copy()
```

단순히 `customers_clean = customers`라고 대입하면 두 변수가 같은 객체를 가리킬 수 있으므로, 한쪽을 수정했을 때 원본 변수에도 영향이 생길 수 있습니다.

3. 전처리 순서도 결과에 영향을 준다

전처리는 여러 단계를 한꺼번에 실행하는 작업이 아닙니다. 어느 순서로 처리하는지에 따라 결측치 수와 중복 행 수가 달라질 수 있습니다.

예를 들어 공백만 들어 있는 문자열은 처음에는 결측치로 집계되지 않을 수 있습니다. 문자열 공백을 제거한 뒤 빈 문자열을 결측치로 바꾸면 결측치 수가 늘어납니다. 숫자 변환에 실패한 값도 NaN으로 바뀌므로, 타입 변환 이후에 결측치를 다시 확인해야 합니다.

이 장에서는 다음 순서로 진행합니다.

- 원본 보존
- 초기 구조 기록
- 문자열 공백과 빈 값 정리
- 날짜·숫자 타입 변환
- 변환 실패와 결측치 확인
- 결측치 처리
- 전체 중복과 고유 ID 중복 확인
- 범주형 값 표준화
- 이상값 확인과 처리
- 파생 컬럼 생성
- 파일 간 연결 관계 재검증
- 전후 비교·저장·보고

이 순서가 모든 프로젝트의 정답은 아닙니다. 다만 각 단계가 다음 단계에 어떤 영향을 주는지 기록하는 습관이 중요합니다.



그림 5-3. 결측치·중복·이상값 전처리 개념도

4. 작업 공간을 준비하고 원본 데이터를 불러온다

전체 코드는 notebooks/ch05_data_preprocessing.ipynb에서 따라갈 수 있습니다. 먼저 필요한 패키지를 불러옵니다.

```
from pathlib import Path
import pandas as pd
```

현재 작업 폴더가 프로젝트 루트이거나 notebooks 폴더인 경우를 모두 처리하도록 프로젝트 경로를 정합니다.

```
project_root = Path.cwd()

if project_root.name == "notebooks":
    project_root = project_root.parent

raw_dir = project_root / "data" / "raw"
processed_dir = project_root / "data" / "processed"
report_dir = project_root / "reports"

processed_dir.mkdir(parents=True, exist_ok=True)
report_dir.mkdir(parents=True, exist_ok=True)

print("프로젝트 루트:", project_root.resolve())
print("원본 데이터 폴더:", raw_dir.resolve())
```

다른 위치에서 노트북을 실행했다면 project_root를 실제 프로젝트 폴더 경로로 직접 지정해야 합니다.

필요한 파일이 모두 있는지 먼저 확인합니다.

```

required_files = [
    "customers.csv",
    "products.csv",
    "orders.csv",
    "order_items.csv",
]

missing_files = [
    file_name
    for file_name in required_files
    if not (raw_dir / file_name).exists()
]

if missing_files:
    raise FileNotFoundError(
        f"원본 데이터 파일을 찾을 수 없습니다: {missing_files}"
    )

```

파일이 없다면 프로젝트 루트에서 다음 명령을 실행해 기본 샘플 데이터를 생성합니다.

```
python scripts/generate_sample_data.py
```

원본 데이터를 불러옵니다.

```

customers = pd.read_csv(raw_dir / "customers.csv")
products = pd.read_csv(raw_dir / "products.csv")
orders = pd.read_csv(raw_dir / "orders.csv")
order_items = pd.read_csv(raw_dir / "order_items.csv")

```

원본과 전처리용 DataFrame을 이름으로 관리하면 반복 작업을 줄일 수 있습니다.

```

raw_data = {
    "customers": customers,
    "products": products,
    "orders": orders,
    "order_items": order_items,
}

clean_data = {
    name: df.copy()
    for name, df in raw_data.items()
}

```

전처리 전 데이터 크기를 기록합니다.


```
raw_shapes = pd.DataFrame(
    [
        {
            "dataset": name,
            "rows": df.shape[0],
            "columns": df.shape[1],
        }
        for name, df in raw_data.items()
    ]
)

raw_shapes
```

기본 생성 스크립트를 사용했다면 customers 150행, products 100행, orders 300행, order_items 764행이 생성됩니다. 스크립트나 데이터가 바뀌었다면 실제 출력값을 기준으로 판단합니다.



그림 5-4. 데이터 전처리 실습 흐름도

5. 전처리 전 상태를 먼저 기록한다

처리 전 상태를 남겨 두어야 어떤 문제가 원래부터 있었고, 전처리 과정에서 무엇이 바뀌었는지 설명할 수 있습니다.

```
def quality_summary(df):
    return {
        "rows": len(df),
        "columns": df.shape[1],
        "missing_cells": int(df.isna().sum().sum()),
        "duplicated_rows": int(df.duplicated().sum()),
    }

initial_quality = pd.DataFrame(
    [
        {"dataset": name, **quality_summary(df)}
        for name, df in clean_data.items()
    ]
)

initial_quality
```

기본 샘플 데이터에는 결측치와 전체 중복 행이 없도록 생성되어 있습니다. 따라서 결과가 모두 0으로 나오는 것이 정상입니다. 이후의 코드는 실제 데이터나 구조가 변경된 데이터에도 적용할 수 있는 안전한 전처리 패턴을 연습하기 위한 것입니다.

6. 문자열 공백과 빈 값을 정리한다

문자열 컬럼에는 앞뒤 공백이나 공백만 들어 있는 값이 존재할 수 있습니다. "Seoul"과 " Seoul "은 pandas에서 서로 다른 값으로 집계됩니다.

```
def clean_string_columns(df):
    df = df.copy()
    string_columns = df.select_dtypes(
        include=["object", "string"]
    ).columns

    for column in string_columns:
        cleaned = df[column].astype("string").str.strip()
        df[column] = cleaned.mask(cleaned.eq(""), pd.NA)

    return df
```

`select_dtypes()`는 지정한 데이터 타입의 컬럼만 선택합니다. 여기서는 문자열로 처리할 수 있는 `object`와 `string` 타입을 선택합니다.

`mask(condition, value)`는 조건이 참인 위치를 지정한 값으로 바꿉니다. 위 코드에서는 공백 제거 후 빈 문자열이 된 값을 `pd.NA`로 바꿉니다. `astype(str)` 대신 pandas의 "string" 타입을 사용하면 기존 결측치를 문자열 "nan"으로 바꾸지 않고 유지할 수 있습니다.

모든 데이터셋에 적용합니다.

```
clean_data = {
    name: clean_string_columns(df)
    for name, df in clean_data.items()
}

customers_clean = clean_data["customers"]
products_clean = clean_data["products"]
orders_clean = clean_data["orders"]
order_items_clean = clean_data["order_items"]
```

문자열을 정리한 뒤에는 결측치 수가 새로 늘어났는지 다시 확인합니다.

```
for name, df in clean_data.items():
    print(f"\n[{name}]")
    print(df.isna().sum())
```

7. 날짜와 숫자형 컬럼을 안전하게 변환한다

타입 변환은 성공 여부를 함께 기록해야 합니다. `errors="coerce"`를 사용하면 변환할 수 없는 값이 결측치로 바뀌므로, 변환 전후의 결측치 차이를 확인합니다.

숫자형 변환

```
def to_number_with_report(series, column_name):
    missing_before = series.isna()

    text = (
        series.astype("string")
        .str.replace(",", "", regex=False)
    )

    converted = pd.to_numeric(text, errors="coerce")
    new_failures = converted.isna() & ~missing_before

    print(
        f"{column_name} 숫자 변환 실패:",
        int(new_failures.sum()),
    )

    return converted

customers_clean["age"] = to_number_with_report(
    customers_clean["age"],
    "customers.age",
)

products_clean["price"] = to_number_with_report(
    products_clean["price"],
    "products.price",
)

order_items_clean["quantity"] = to_number_with_report(
    order_items_clean["quantity"],
    "order_items.quantity",
)

order_items_clean["unit_price"] = to_number_with_report(
    order_items_clean["unit_price"],
    "order_items.unit_price",
)
```

현재 기본 샘플 데이터에서는 모두 숫자형이므로 변환 실패 건수는 0이어야 합니다. 실패 건수가 발생했다면 원본 값을 별도로 확인해야 하며, 실패값을 곧바로 삭제해서는 안 됩니다.

날짜형 변환

```
def to_datetime_with_report(series, column_name):
    missing_before = series.isna()
    converted = pd.to_datetime(series, errors="coerce")
    new_failures = converted.isna() & ~missing_before

    print(
        f"{column_name} 날짜 변환 실패:",
        int(new_failures.sum()),
    )

    return converted

customers_clean["signup_date"] = to_datetime_with_report(
    customers_clean["signup_date"],
    "customers.signup_date",
)

orders_clean["order_date"] = to_datetime_with_report(
    orders_clean["order_date"],
    "orders.order_date",
)
```

변환 후 날짜 범위를 확인합니다.

```
print(
    "가입일 범위:",
    customers_clean["signup_date"].min(),
    "~",
    customers_clean["signup_date"].max(),
)

print(
    "주문일 범위:",
    orders_clean["order_date"].min(),
    "~",
    orders_clean["order_date"].max(),
)
```

날짜형 변환은 현재 메모리에 있는 DataFrame만 변경하며 원본 CSV 파일을 덮어쓰지 않습니다.

8. 결측치를 확인하고 처리한다

문자열 정리와 타입 변환이 끝난 뒤 결측치를 다시 요약합니다.

```
def missing_summary(df):
    summary = pd.DataFrame({
        "missing_count": df.isna().sum(),
        "missing_ratio_pct": (
            df.isna().mean() * 100
        ).round(2),
    })

    return summary.sort_values(
        "missing_count",
        ascending=False,
    )

missing_summary(customers_clean)
```

기본 샘플 데이터에는 결측치가 없으므로 실제 결과는 모두 0입니다. 다음 처리 코드는 실제 데이터에서 결측치가 발견된 경우를 대비한 예시입니다.

나이 결측치는 분석 목적에 따라 중앙값으로 대체할 수 있습니다.

```
age_median = customers_clean["age"].median()

if pd.isna(age_median):
    customers_clean["age"] = (
        customers_clean["age"]
        .fillna(age_median)
    )
```

중앙값 대체는 전체 고객 수를 유지할 수 있지만, 실제 나이를 모르는 고객에게 대표값을 부여하는 방식입니다. 연령대별 분석이나 머신러닝에서 이 사실을 반드시 기록해야 합니다. 필요하다면 대체 전에 결측 여부를 별도 컬럼으로 보존하는 방법도 검토할 수 있습니다.

도시 결측치는 Unknown이라는 별도 범주로 유지할 수 있습니다.

```
customers_clean["city"] = (
    customers_clean["city"]
    .fillna("Unknown")
)
```

ID나 주문일처럼 분석 연결과 기간 정의에 필수적인 값은 대표값으로 임의 대체하지 않습니다. 원본 시스템 확인, 제외, 보류 중 어떤 방법을 사용할지 별도의 기준이 필요합니다.

9. 전체 중복과 고유 ID 중복을 구분한다

전체 행이 완전히 같은 중복을 먼저 확인합니다.

```
for name, df in clean_data.items():
    print(
        name,
        "전체 중복 행:",
        int(df.duplicated().sum()),
    )
```

중복 행의 내용을 확인한 뒤 완전 중복만 제거합니다.

```
customers_clean = (
    customers_clean
    .drop_duplicates()
    .reset_index(drop=True)
)

products_clean = (
    products_clean
    .drop_duplicates()
    .reset_index(drop=True)
)

orders_clean = (
    orders_clean
    .drop_duplicates()
    .reset_index(drop=True)
)

order_items_clean = (
    order_items_clean
    .drop_duplicates()
    .reset_index(drop=True)
)
```

전체 행 중복이 없더라도 고유해야 하는 ID가 중복될 수 있습니다.

```

key_columns = {
    "customers": "customer_id",
    "products": "product_id",
    "orders": "order_id",
    "order_items": "order_item_id",
}

clean_data = {
    "customers": customers_clean,
    "products": products_clean,
    "orders": orders_clean,
    "order_items": order_items_clean,
}

key_quality_rows = []

for name, key_column in key_columns.items():
    df = clean_data[name]

    key_quality_rows.append({
        "dataset": name,
        "key": key_column,
        "missing_key": int(
            df[key_column].isna().sum()
        ),
        "duplicated_key": int(
            df[key_column].duplicated().sum()
        ),
    })

key_quality = pd.DataFrame(key_quality_rows)
key_quality

```

`order_items`의 `order_id`는 여러 상품이 한 주문에 포함될 수 있으므로 반복이 자연스럽습니다. 반면 `order_item_id`는 주문 상세 행을 고유하게 식별해야 하므로 중복되면 오류로 판단할 수 있습니다.

고유 키의 결측치나 중복을 자동으로 한 행만 남겨 해결하면 안 됩니다. 서로 다른 값이 같은 ID에 연결되어 있을 수 있으므로 원본 시스템이나 업무 규칙을 확인해야 합니다. 저장 전에 문제가 남아 있으면 실행을 중단하도록 검사할 수 있습니다.


```

if (
    key_quality["missing_key"].gt(0).any()
    or key_quality["duplicated_key"].gt(0).any()
):
    raise ValueError(
        "고유 키 결측치 또는 중복이 남아 있습니다."
    )

```

10. 범주형 값을 표준화한다

주문 상태값처럼 범주형 컬럼은 실제 분포를 먼저 확인합니다.

```

orders_clean["order_status"].value_counts(
    dropna=False
)

```

대소문자와 일부 동의어를 통일합니다.

```

status_text = (
    orders_clean["order_status"]
    .astype("string")
    .str.strip()
    .str.lower()
)

status_map = {
    "complete": "completed",
    "completed": "completed",
    "완료": "completed",
    "cancel": "cancelled",
    "canceled": "cancelled",
    "cancelled": "cancelled",
    "취소": "cancelled",
    "refund": "refunded",
    "refunded": "refunded",
    "환불": "refunded",
}

orders_clean["order_status"] = (
    status_text.replace(status_map)
)

```

처리 후 다시 분포를 확인합니다.

```
orders_clean["order_status"].value_counts(
    dropna=False
)
```

허용할 상태값을 정했다면 예상하지 못한 값도 확인합니다.

```
allowed_statuses = {
    "completed",
    "cancelled",
    "refunded",
}

unexpected_statuses = orders_clean.loc[
    orders_clean["order_status"].notna()
    & ~orders_clean["order_status"].isin(
        allowed_statuses
    ),
    "order_status",
].value_counts()

unexpected_statuses
```

예상하지 못한 상태값을 임의로 가장 비슷한 값에 매핑하지 않습니다. 의미를 확인한 뒤 매핑표를 보완해야 합니다.

11. 이상값은 처리 기준과 제외 건수를 남긴다

숫자형 컬럼의 기본 통계를 확인합니다.

```
customers_clean[["age"]].describe()
products_clean[["price"]].describe()
order_items_clean[
    ["quantity", "unit_price"]
].describe()
```

ID는 숫자로 저장되어 있어도 평균이나 사분위수에 업무적 의미가 없는 식별자이므로 통계 해석 대상에서 제외합니다.

이번 실습에서는 다음 조건을 정상 판매 분석에서 확인이 필요한 값으로 가정합니다.

```

invalid_age = (
    customers_clean["age"].lt(0)
    | customers_clean["age"].gt(120)
)

invalid_product_price = (
    products_clean["price"].isna()
    | products_clean["price"].le(0)
)

invalid_order_item = (
    order_items_clean["quantity"].isna()
    | order_items_clean["unit_price"].isna()
    | order_items_clean["quantity"].le(0)
    | order_items_clean["unit_price"].le(0)
)

print("나이 이상값 후보:", int(invalid_age.sum()))
print(
    "가격 이상값 후보:",
    int(invalid_product_price.sum()),
)
print(
    "주문 상세 이상값 후보:",
    int(invalid_order_item.sum()),
)

```

기본 샘플 데이터에서는 모두 0이어야 합니다. 실제 데이터에서 값이 발견되면 다음을 먼저 확인합니다.

- 0원 상품이 행사나 증정 상품인가?
- 음수 수량이 반품을 의미하는가?
- 단가가 0인 행이 별도 할인 구조를 의미하는가?
- 나이 범위가 실제 업무 기준과 일치하는가?
- 입력 오류를 원본 시스템에서 수정할 수 있는가?

여기서는 **정상 판매 금액 분석**을 위한 실습 규칙으로 가격·수량·단가가 0 이하이거나 변환에 실패한 행을 분석용 데이터에서 제외한다고 가정합니다. 제외 전에 반드시 건수와 원본 행을 별도로 확인해야 합니다.

```
products_excluded = products_clean.loc[
    invalid_product_price
].copy()

order_items_excluded = order_items_clean.loc[
    invalid_order_item
].copy()

products_clean = products_clean.loc[
    ~invalid_product_price
].copy()

order_items_clean = order_items_clean.loc[
    ~invalid_order_item
].copy()
```

나이 이상값은 실제 고객 확인 없이 대표값으로 덮어쓰지 않습니다. 분석 목적에 따라 제외하거나 별도 범주로 처리할 수 있습니다.

12. 파일 간 연결 관계를 다시 확인한다

CSV 파일은 데이터베이스처럼 기본 키와 외래 키 제약조건을 자동으로 검사하지 않습니다. 전처리 과정에서 상품이나 주문 상세 행을 제외하면 파일 간 관계가 깨질 수도 있습니다.

먼저 연결 대상에 존재하지 않는 ID를 확인합니다.

```

invalid_customers = orders_clean.loc[
    ~orders_clean["customer_id"].isin(
        customers_clean["customer_id"]
    )
].copy()

invalid_orders = order_items_clean.loc[
    ~order_items_clean["order_id"].isin(
        orders_clean["order_id"]
    )
].copy()

invalid_products = order_items_clean.loc[
    ~order_items_clean["product_id"].isin(
        products_clean["product_id"]
    )
].copy()

print(
    "customers에 없는 customer_id 행 수:",
    len(invalid_customers),
)

print(
    "orders에 없는 order_id 행 수:",
    len(invalid_orders),
)

print(
    "products에 없는 product_id 행 수:",
    len(invalid_products),
)

```

문제가 발견되면 바로 삭제하지 말고 다음을 확인합니다.

- 부모 데이터 파일이 누락되었는가?
- ID 타입이 숫자와 문자열로 서로 다른가?
- 앞뒤 공백이나 표기 차이가 있는가?
- 거래 데이터가 기준 데이터보다 먼저 들어온 시점 차이인가?
- 전처리 과정에서 부모 행만 제외했는가?

이번 실습에서는 정상 판매 분석용 데이터의 연결 관계를 유지하기 위해 부모 데이터에 존재하지 않는 주문 상세 행은 별도 확인 대상으로 보관한 뒤 분석용 복사본에서 제외한다고 가정합니다.

```

orphan_order_item_mask = (
    ~order_items_clean["order_id"].isin(
        orders_clean["order_id"]
    )
    | ~order_items_clean["product_id"].isin(
        products_clean["product_id"]
    )
)

orphan_order_items = order_items_clean.loc[
    orphan_order_item_mask
].copy()

order_items_clean = order_items_clean.loc[
    ~orphan_order_item_mask
].copy()

```

고객 데이터에 없는 주문은 주문 자체의 의미를 확인해야 하므로 자동으로 제거하지 않습니다. `invalid_customers`가 남아 있다면 저장 전에 원인을 해결하거나 별도 예외 데이터로 분리합니다.

13. 파생 컬럼과 매출 기준을 명확히 한다

날짜형 컬럼에서 주문 월과 요일을 만들 수 있습니다.

```

orders_clean["order_month"] = (
    orders_clean["order_date"]
    .dt.strftime("%Y-%m")
)

orders_clean["order_dayofweek"] = (
    orders_clean["order_date"]
    .dt.day_name()
)

```

`strftime("%Y-%m")`은 날짜가 없는 위치를 결측치로 유지합니다. 날짜형을 곧바로 `astype(str)`로 바꾸면 결측치가 문자열 "NaT"로 저장될 수 있으므로 주의합니다.

주문 상세 금액을 계산합니다.

```
order_items_clean["line_total"] = (
    order_items_clean["quantity"]
    * order_items_clean["unit_price"]
)
```

line_total은 한 주문 상세 행의 수량과 단가를 곱한 **주문 상세 금액**입니다. 모든 line_total의 합계를 곧바로 매출이라고 부르면 안 됩니다. 취소·환불 주문이 포함될 수 있기 때문입니다.

주문 상태를 연결해 완료 주문 기준 금액을 계산합니다.

```
sales_items = order_items_clean.merge(
    orders_clean[
        ["order_id", "order_status"]
    ],
    on="order_id",
    how="left",
    validate="many_to_one",
)
```

validate="many_to_one"은 여러 주문 상세 행이 하나의 주문 행에 연결되는 구조인지 검사합니다.

orders.order_id가 중복되어 있으면 병합 단계에서 오류가 발생합니다.

```
completed_sales = sales_items.loc[
    sales_items["order_status"].eq("completed"),
    "line_total",
].sum()

print("완료 주문 기준 금액:", completed_sales)
```

이 값도 회사의 회계상 순매출과 반드시 같지는 않습니다. 할인, 배송비, 세금, 부분 환불 등의 컬럼이 없다면 계산 가능한 범위만 명확히 설명해야 합니다.

14. 전처리 전후를 비교하고 저장한다

전처리 결과를 다시 이름별로 묶습니다.

```
clean_data = {
    "customers": customers_clean,
    "products": products_clean,
    "orders": orders_clean,
    "order_items": order_items_clean,
}
```

전처리 후 데이터 크기를 계산합니다.

```
processed_shapes = pd.DataFrame(  
    [  
        {  
            "dataset": name,  
            "rows": df.shape[0],  
            "columns": df.shape[1],  
        }  
        for name, df in clean_data.items()  
    ]  
)  
  
comparison = raw_shapes.merge(  
    processed_shapes,  
    on="dataset",  
    suffixes=("_raw", "_processed"),  
)  
  
comparison["rows_removed"] = (  
    comparison["rows_raw"]  
    - comparison["rows_processed"]  
)  
  
comparison["columns_added"] = (  
    comparison["columns_processed"]  
    - comparison["columns_raw"]  
)  
  
comparison
```

행 수가 줄었다면 제외 기준과 제외 건수를 설명해야 합니다. 열 수가 늘었다면 어떤 파생 컬럼이 추가되었는지 기록합니다.

저장 전 마지막 검증을 수행합니다.


```

if len(invalid_customers) > 0:
    raise ValueError(
        "고객 데이터에 없는 customer_id가 "
        "orders에 남아 있습니다."
    )

if (
    key_quality["missing_key"].gt(0).any()
    or key_quality["duplicated_key"].gt(0).any()
):
    raise ValueError(
        "고유 키 문제가 해결되지 않았습니다."
    )

```

전처리된 데이터를 저장합니다. utf-8-sig는 Windows에서 Excel로 CSV를 열 때 한글이 깨지는 문제를 줄일 수 있습니다.

```

customers_clean.to_csv(
    processed_dir / "customers_clean.csv",
    index=False,
    encoding="utf-8-sig",
    date_format="%Y-%m-%d",
)

products_clean.to_csv(
    processed_dir / "products_clean.csv",
    index=False,
    encoding="utf-8-sig",
)

orders_clean.to_csv(
    processed_dir / "orders_clean.csv",
    index=False,
    encoding="utf-8-sig",
    date_format="%Y-%m-%d",
)

order_items_clean.to_csv(
    processed_dir / "order_items_clean.csv",
    index=False,
    encoding="utf-8-sig",
)

```

제외된 행이 있다면 검토용 파일로 따로 저장할 수 있습니다.

```
rejected_dir = processed_dir / "rejected"
rejected_dir.mkdir(parents=True, exist_ok=True)

if not products_excluded.empty:
    products_excluded.to_csv(
        rejected_dir / "products_excluded.csv",
        index=False,
        encoding="utf-8-sig",
    )

if not order_items_excluded.empty:
    order_items_excluded.to_csv(
        rejected_dir / "order_items_excluded.csv",
        index=False,
        encoding="utf-8-sig",
    )

if not orphan_order_items.empty:
    orphan_order_items.to_csv(
        rejected_dir / "orphan_order_items.csv",
        index=False,
        encoding="utf-8-sig",
    )
```

15. 전처리 보고서를 남긴다

`comparison.to_markdown()`은 `tabulate` 패키지를 사용합니다. 이 프로젝트의 `requirements.txt`에는 `tabulate`가 포함되어 있습니다. 별도 환경에서 오류가 발생하면 `pip install tabulate`로 설치하거나 `to_string()`을 사용할 수 있습니다.

```
try:
    comparison_text = comparison.to_markdown(
        index=False
    )
except ImportError:
    comparison_text = comparison.to_string(
        index=False
    )
```

전처리 기준과 결과를 Markdown 파일로 저장합니다.

```

summary_text = f"""
# Chapter 5 데이터 전처리 요약

## 전처리 결과 파일

- customers_clean.csv
- products_clean.csv
- orders_clean.csv
- order_items_clean.csv

## 전처리 전후 데이터 크기

{comparison_text}

## 주요 처리 기준

- 문자열 앞뒤 공백 제거
- 공백만 있는 문자열을 결측치로 변환
- 날짜·숫자 변환 실패 건수 확인
- 나이 결측치는 중앙값 대체 가능성을 검토
- 도시 결측치는 Unknown 범주로 처리
- 전체 중복 행 제거
- 고유 ID 결측치·중복 검사
- 주문 상태값 표준화
- 0 이하 가격·수량·단가는 정상 판매 분석에서 제외
- 주문 상세 금액 line_total 생성
- 완료 주문만 별도로 집계
- 파일 간 연결 관계 재검증

## 제외 및 검토 대상

- 가격 이상값 후보: {len(products_excluded)}행
- 주문 상세 이상값 후보: {len(order_items_excluded)}행
- 연결 대상이 없는 주문 상세: {len(orphan_order_items)}행

## 완료 주문 기준 금액

- {completed_sales:,0f}

## 주의사항

완료 주문 기준 금액은 할인, 배송비, 세금, 부분 환불을 반영한 회계상 순매출과 다를 수 있습니다.
"""

```

```
report_path = (  
    report_dir  
    / "ch05_preprocessing_summary.md"  
)  
  
report_path.write_text(  
    summary_text,  
    encoding="utf-8",  
)
```

보고서에는 코드가 실행되었다는 사실보다 어떤 기준으로 무엇을 바꾸었고, 어떤 데이터가 제외되었는지를 남기는 것이 중요합니다.

16. 전처리 과정을 함수로 정리한다

반복해서 사용할 전처리 로직은 함수로 분리할 수 있습니다. 다음은 핵심 원칙만 담은 간단한 예시입니다.

```
def preprocess_customers(df):
    result = clean_string_columns(df)

    result["age"] = pd.to_numeric(
        result["age"],
        errors="coerce",
    )

    median_age = result["age"].median()

    if pd.notna(median_age):
        result["age"] = (
            result["age"]
            .fillna(median_age)
        )

    result["city"] = (
        result["city"]
        .fillna("Unknown")
    )

    result["signup_date"] = pd.to_datetime(
        result["signup_date"],
        errors="coerce",
    )

    return (
        result
        .drop_duplicates()
        .reset_index(drop=True)
    )
```

```
def preprocess_products(df):  
    result = clean_string_columns(df)  
  
    result["price"] = to_number_with_report(  
        result["price"],  
        "products.price",  
    )  
  
    result = result.loc[  
        result["price"].gt(0)  
    ]  
  
    return (  
        result  
        .drop_duplicates()  
        .reset_index(drop=True)  
    )
```

```
def preprocess_orders(df):
    result = clean_string_columns(df)

    result["order_date"] = (
        to_datetime_with_report(
            result["order_date"],
            "orders.order_date",
        )
    )

    status = (
        result["order_status"]
        .astype("string")
        .str.lower()
    )

    result["order_status"] = (
        status.replace(status_map)
    )

    result["order_month"] = (
        result["order_date"]
        .dt.strftime("%Y-%m")
    )

    result["order_dayofweek"] = (
        result["order_date"]
        .dt.day_name()
    )

    return (
        result
        .drop_duplicates()
        .reset_index(drop=True)
    )
```



```
def preprocess_order_items(df):
    result = clean_string_columns(df)

    result["quantity"] = to_number_with_report(
        result["quantity"],
        "order_items.quantity",
    )

    result["unit_price"] = to_number_with_report(
        result["unit_price"],
        "order_items.unit_price",
    )

    valid_rows = (
        result["quantity"].gt(0)
        & result["unit_price"].gt(0)
    )

    result = result.loc[valid_rows].copy()

    required_columns = {
        "quantity",
        "unit_price",
    }

    if required_columns.issubset(
        result.columns
    ):
        result["line_total"] = (
            result["quantity"]
            * result["unit_price"]
        )

    return (
        result
        .drop_duplicates()
        .reset_index(drop=True)
    )
```

`issubset()`은 필요한 컬럼이 모두 존재하는지 확인합니다. 다음 표현과 같은 의미입니다.

```
(
    "quantity" in result.columns
    and "unit_price" in result.columns
)
```

함수를 적용한 뒤에도 고유 키와 파일 간 연결 관계 검증은 별도로 수행해야 합니다. 함수가 오류 없이 실행됐다는 사실만으로 데이터 품질이 보장되지는 않습니다.



그림 5-5. 전처리 함수화와 저장 흐름도

17. LLM과 함께 전처리 코드를 검토한다

LLM은 전처리 코드 초안 작성, 오류 설명, 누락된 검증 항목 확인에 도움을 줄 수 있습니다. 그러나 결측치 대체, 중복 제거, 이상값 제외는 분석 결과를 직접 바꾸므로 LLM이 제안한 코드를 그대로 적용해서는 안 됩니다.

실제 고객명, 이메일, 주소, 주문 내역, API 키, 내부 기밀정보는 승인되지 않은 외부 LLM 서비스에 입력하지 않습니다. 다음과 같은 구조 요약만 제공합니다.

customers.csv 구조 요약

- 행 수: 150
- 컬럼: customer_id, name, gender, age, city, signup_date
- age 결측치: 0
- city 결측치: 0
- 전체 중복 행: 0
- customer_id 결측치: 0
- customer_id 중복: 0

분석 목적:

- 고객 연령대별 구매 패턴 분석
- 지역별 고객 분포 확인

실제 데이터 확인 없이 값을 만들어내지 말고,
필요한 전처리와 검증 항목을 구분해 설명해 주세요.

가상의 오류 상황을 검토하려면 실제 샘플 데이터 결과와 혼동되지 않도록 명시합니다.

다음은 전처리 방법을 비교하기 위한 가상의 상황입니다.

- age 결측치: 3건
 - city 결측치: 2건
1. 삭제, 중앙값 대체, Unknown 처리의 장단점을 비교해 주세요.
 2. 분석 결과에 미칠 영향을 설명해 주세요.
 3. 실제 값을 임의로 추정하지 마세요.

LLM이 만든 전처리 코드는 다음 기준으로 검토합니다.

검토 기준	확인할 질문
원본 보존	원본 파일과 DataFrame을 직접 덮어쓰지 않는가?
처리 순서	문자열 정리와 타입 변환 이후 결측치를 다시 확인하는가?
변환 실패	숫자·날짜 변환 실패 건수를 확인하는가?
결측치 처리	컬럼의 의미와 분석 목적에 맞는가?
중복 처리	전체 중복과 고유 ID 중복을 구분하는가?
이상값 처리	삭제 전에 업무적 의미와 제외 건수를 확인하는가?
파일 관계	부모 키 고유성과 외래 키 연결을 다시 검사하는가?
파생 컬럼	계산식과 포함 범위를 명확히 정의하는가?
매출 기준	취소·환불 주문을 구분하는가?
보안	개인정보·API 키·기밀정보를 입력하지 않았는가?

검토 기준	확인할 질문
기록	변경 기준과 제외 데이터를 재현 가능하게 남기는가?

18. 전처리 결과를 어떻게 해석할 것인가

전처리 후에는 “코드가 실행되었다”에서 멈추면 안 됩니다. 어떤 문제가 발견되었고, 어떤 기준으로 처리했으며, 데이터가 어떻게 바뀌었는지를 설명할 수 있어야 합니다.

좋은 기록은 다음과 같습니다.

- 문자열 앞뒤 공백을 제거하고 공백만 있는 값을 결측치로 변환했습니다.
- 숫자·날짜 변환 실패 건수를 확인했으며 새로 발생한 결측치는 없었습니다.
- 전체 중복 행은 제거했지만 고유 ID 중복은 자동으로 해결하지 않았습니다.
- 가격·수량·단가가 0 이하인 행은 정상 판매 분석에서 제외하고 별도 파일로 저장했습니다.
- 주문 상태가 completed인 주문만 완료 주문 기준 금액 계산에 포함했습니다.

다음과 같은 기록은 정보가 부족합니다.

- 데이터를 깨끗하게 정리했습니다.
- 이상값을 제거했습니다.
- 매출을 계산했습니다.

처리 기준, 제외 건수, 포함 범위가 없으면 다른 사람이 같은 결과를 재현하거나 검증하기 어렵습니다.

19. 다음 장으로 이어지는 흐름

전처리는 분석 가능한 데이터를 만드는 과정이고, EDA는 그 데이터에 질문을 던지는 과정입니다. 결측치, 중복, 타입, 이상값, 주문 상태와 파일 관계를 정리해 두면 고객 행동, 주문 패턴, 상품별 금액, 완료 주문 비중 같은 질문을 더 안정적으로 탐색할 수 있습니다.

다음 장에서는 전처리된 데이터를 바탕으로 분석 질문을 만들고, 집계 결과가 무엇을 의미하는지 살펴봅니다.

6장. 데이터를 보며 질문을 만드는 EDA

전처리를 마친 데이터는 이제 분석할 준비가 된 데이터입니다. 하지만 준비된 데이터를 바로 그래프로 만들거나 모델에 넣는다고 해서 좋은 분석이 되는 것은 아닙니다. 데이터 분석에서 중요한 첫 단계는 **무엇을 볼 것인가**를 정하는 일입니다.

EDA는 Exploratory Data Analysis, 즉 탐색적 데이터 분석을 의미합니다. 이름 그대로 데이터를 여러 방향에서 탐색하면서 분포, 패턴, 차이, 관계, 이상한 값을 발견하는 과정입니다. EDA의 목적은 단순히 표와 그래프를 많이 만드는 것이 아니라, **데이터를 보며 더 나은 분석 질문을 찾아가는 것**입니다.

데이터 분석 초보자가 자주 하는 실수는 데이터를 불러오자마자 그래프를 그리거나, LLM에게 바로 “인사이트를 찾아줘”라고 요청하는 것입니다. 하지만 좋은 분석 결과는 좋은 질문에서 시작됩니다. 질문이 모호하면 지표도 모호해지고, 지표가 모호하면 해석도 흔들립니다.

이 장에서는 온라인 쇼핑몰 데이터를 바탕으로 EDA 질문을 만들고, 그 질문을 pandas 코드와 지표로 연결하는 방법을 살펴봅니다. 고객, 상품, 주문, 매출을 각각 다른 관점에서 탐색하면서 **현재 데이터로 답할 수 있는 질문**과 **추가 데이터가 필요한 질문**을 구분해 봅니다.



그림 6-1. EDA 전체 흐름도

1. EDA는 결론보다 질문에 가깝다

EDA는 최종 결론을 내리는 단계가 아닙니다. 오히려 결론을 내리기 전에 데이터를 이해하고, 분석 방향을 잡고, 더 확인해야 할 질문을 만드는 단계입니다.

예를 들어 월별 매출이 증가하는 그래프를 보았다고 해서 바로 “마케팅이 성공했다”라고 말할 수는 없습니다. 매출 증가가 실제로 마케팅 때문인지, 계절성 때문인지, 특정 고가 상품 판매 때문인지, 신규 고객 유입 때문인지는 추가 분석이 필요합니다. EDA에서는 이런 원인을 단정하기보다 **무엇을 더 확인해야 하는가**를 정리하는 것이 중요합니다.

EDA에서 자주 던지는 질문은 다음과 같습니다.

- 데이터는 어떤 변수들로 구성되어 있는가?
- 주요 숫자형 변수의 분포는 어떤가?
- 주요 범주형 변수의 빈도는 어떤가?
- 특정 그룹 간 차이가 있는가?
- 시간에 따른 변화가 있는가?
- 이상하게 큰 값이나 작은 값이 있는가?
- 다음 단계에서 더 깊이 볼 질문은 무엇인가?

EDA 결과를 해석할 때는 **관찰**, **가설**, **결론**을 구분해야 합니다.

구분	의미	예시
관찰	데이터에서 직접 확인한 사실	전자기기 카테고리의 매출 비중이 가장 높다
가설	관찰을 바탕으로 생각해 볼 가능성	전자기기는 단가가 높아 매출 비중이 클 수 있다
결론	추가 검증 후 말할 수 있는 판단	단가와 판매 수량을 함께 분석한 결과 매출 차이의 주요 요인은 단가였다

EDA에서는 관찰과 가설까지는 만들 수 있지만, 결론은 추가 검증을 거쳐 조심스럽게 내려야 합니다.

2. 좋은 분석 질문은 데이터로 답할 수 있다

분석 질문은 데이터를 사용해 답할 수 있는 구체적인 질문이어야 합니다. “매출을 늘리려면 어떻게 해야 하는가?”는 중요한 질문이지만, 너무 넓습니다. 현재 가진 데이터만으로 바로 답하기 어렵습니다. 반면 “월별 매출 합계는 어떻게 변하는가?”는 pandas로 계산할 수 있는 구체적인 질문입니다.

좋은 분석 질문은 다음 조건을 가집니다.

조건	설명	예시
데이터 기반	현재 데이터로 답할 수 있어야 함	월별 매출은 어떻게 변하는가?
구체적	분석 대상과 기준이 명확해야 함	카테고리별 매출 비중은 어떻게 다른가?
측정 가능	지표로 계산할 수 있어야 함	주문 수, 총매출, 평균 구매 금액
해석 가능	결과가 다음 판단과 연결되어야 함	어떤 상품군을 더 자세히 봐야 하는가?
검증 가능	코드로 확인할 수 있어야 함	groupby()로 계산 가능

반대로 다음 질문은 아직 EDA 질문으로는 부족합니다.

질문	부족한 이유	바꿔 볼 수 있는 질문
고객이 왜 이탈했는가?	이탈 여부 데이터가 없을 수 있음	최근 주문이 없는 고객은 얼마나 되는가?
어떤 상품이 인기가 많은가?	인기의 기준이 모호함	상품별 판매 수량 또는 매출은 어떻게 다른가?
매출을 늘리려면 어떻게 해야 하는가?	너무 포괄적임	매출 비중이 높은 카테고리는 무엇인가?
고객 만족도는 어떤가?	만족도 데이터가 없을 수 있음	재구매 횟수나 주문 빈도를 대신 볼 수 있는가?

분석 질문은 지표로 바뀌어야 합니다. 그래야 pandas 코드로 계산하고, 표나 그래프로 확인할 수 있습니다.

원래 질문	구체화된 질문	지표
어떤 카테고리가 잘 팔리는가?	카테고리별 총매출은 얼마인가?	category_sales
고객이 많이 사는가?	고객별 주문 횟수는 몇 회인가?	order_count
매출이 늘고 있는가?	월별 매출 합계는 어떻게 변하는가?	monthly_sales
고가 상품이 팔리는가?	상품별 평균 단가와 판매 수량은 어떤 관계인가?	price, total_quantity

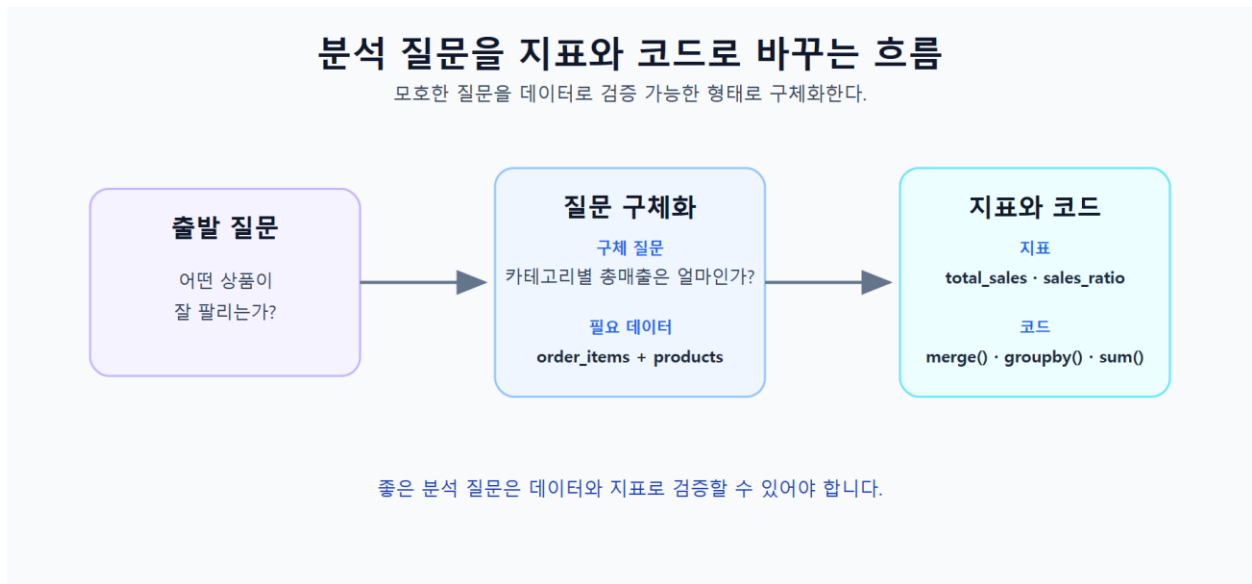


그림 6-2. 분석 질문을 지표와 코드로 바꾸는 흐름

3. EDA의 기본 관점

EDA는 한 번에 모든 것을 보려고 하면 복잡해집니다. 처음에는 하나의 변수부터 보고, 이후 두 변수의 관계, 세 개 이상의 변수를 함께 보는 방식으로 넓혀 가는 것이 좋습니다.

하나의 변수 보기

단변량 EDA는 하나의 변수만 살펴보는 탐색입니다. 데이터의 기본 모양을 파악하는 데 사용합니다.

- 고객 나이 분포는 어떤가?
- 고객은 어느 도시에 많이 분포하는가?
- 상품 카테고리는 몇 종류인가?
- 주문 상태별 주문 수는 어떻게 되는가?
- 상품 가격의 평균과 최댓값은 얼마인가?

두 변수의 관계 보기

이변량 EDA는 두 변수 사이의 관계를 살펴보는 탐색입니다. 그룹 간 차이나 변수 간 관계를 확인할 때 유용합니다.

- 상품 카테고리별 매출은 어떻게 다른가?
- 결제수단별 주문 수는 어떻게 다른가?

- 월별 매출은 어떻게 변하는가?
- 고객 지역별 주문 금액은 어떻게 다른가?
- 상품 가격과 판매 수량은 어떤 관계가 있는가?

세 개 이상의 변수를 함께 보기

다변량 EDA는 세 개 이상의 변수를 함께 살펴보는 탐색입니다. 더 깊은 질문을 만들 수 있지만, 초보 단계에서는 해석이 복잡해질 수 있습니다. 먼저 기본 지표를 확인한 뒤 확장하는 것이 좋습니다.

- 월별·카테고리별 매출은 어떻게 다른가?
- 지역별·연령대별 고객 구매 금액은 어떻게 다른가?
- 주문 상태와 결제수단에 따라 매출 차이가 있는가?
- 고객별 주문 횟수와 총 구매 금액은 어떤 관계가 있는가?

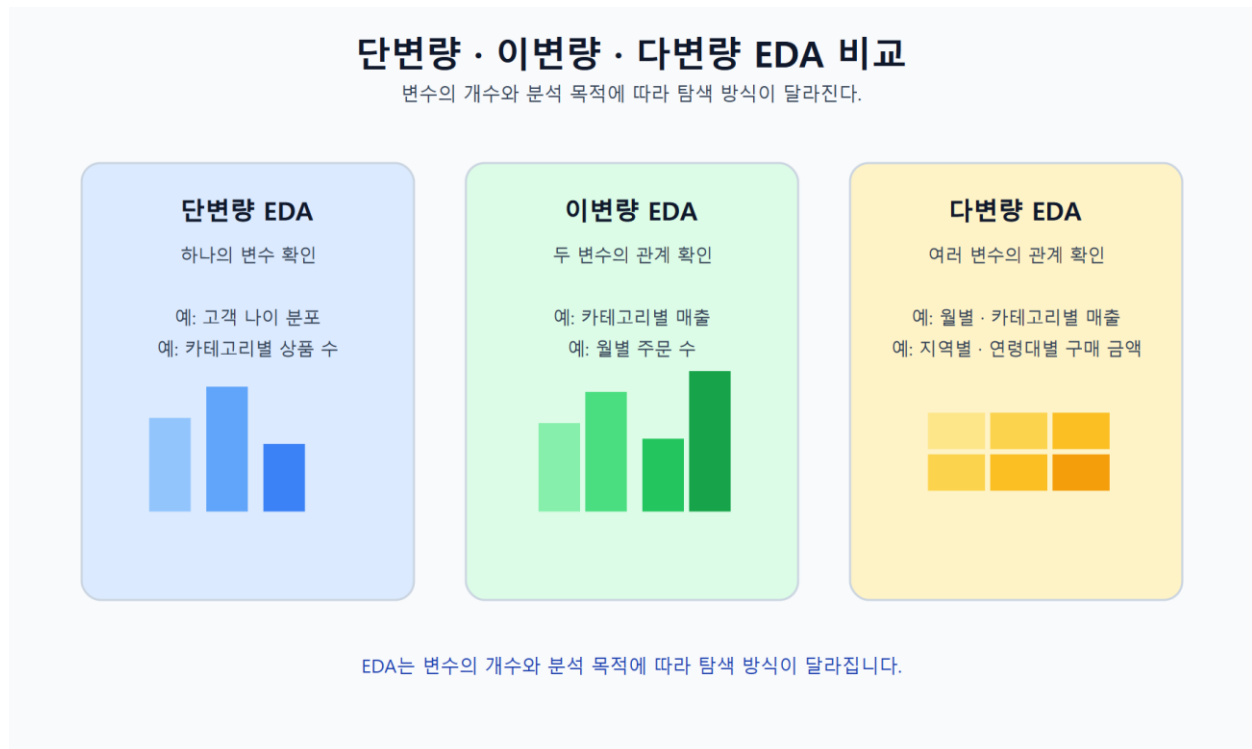


그림 6-3. 단변량·이변량·다변량 EDA 비교

4. 전처리된 데이터를 다시 불러온다

EDA는 5장에서 저장한 전처리 데이터를 사용합니다. 전체 코드는 `notebooks/ch06_eda_questions.ipynb`에서 이어서 실행할 수 있습니다. 여기서는 핵심 흐름만 살펴봅니다.

먼저 필요한 패키지를 불러옵니다.

```
from pathlib import Path
import pandas as pd
```

VS Code에서 Notebook을 실행할 때는 현재 작업 폴더가 프로젝트 루트인지, notebooks 폴더인지에 따라 상대 경로가 달라질 수 있습니다. 다음 코드는 현재 위치를 기준으로 base_dir를 정해 줍니다.

```
current_dir = Path.cwd()

if current_dir.name == "notebooks":
    base_dir = current_dir.parent
else:
    base_dir = current_dir

processed_dir = base_dir / "data" / "processed"
report_dir = base_dir / "reports"
report_dir.mkdir(exist_ok=True)

print("processed_dir:", processed_dir)
print("report_dir:", report_dir)
```

to_markdown()을 사용할 때 환경에 따라 tabulate 패키지가 필요할 수 있습니다. 오류가 발생하면 터미널이나 Notebook에서 다음 명령을 실행합니다.

```
pip install tabulate
```

전처리된 데이터를 불러옵니다.

```
customers = pd.read_csv(processed_dir / "customers_clean.csv")
products = pd.read_csv(processed_dir / "products_clean.csv")
orders = pd.read_csv(processed_dir / "orders_clean.csv")
order_items = pd.read_csv(processed_dir / "order_items_clean.csv")
```

데이터 크기를 확인합니다.

```
print("customers:", customers.shape)
print("products:", products.shape)
print("orders:", orders.shape)
print("order_items:", order_items.shape)
```

CSV로 저장했다가 다시 불러오면 날짜 컬럼이 문자열로 돌아올 수 있습니다. 따라서 EDA 전에 날짜형으로 다시 변환합니다.

```
orders["order_date"] = pd.to_datetime(orders["order_date"], errors="coerce")
```

```
if "signup_date" in customers.columns:
```

```
    customers["signup_date"] = pd.to_datetime(customers["signup_date"], errors="coerce")
```

order_month가 없다면 다시 생성합니다.

```
if "order_month" not in orders.columns:
```

```
    orders["order_month"] = orders["order_date"].dt.to_period("M").astype(str)
```

5. 질문을 먼저 표로 정리한다

EDA를 시작하기 전에 어떤 질문을 볼지 먼저 표로 정리해 두면 분석이 산만해지지 않습니다. 아래 표는 고객, 상품, 주문, 매출, 시간, 고객 가치 관점에서 만들 수 있는 기본 질문입니다.

```

questions = pd.DataFrame({
    "analysis_area": ["고객", "상품", "주문", "매출", "시간", "고객 가치"],
    "question": [
        "고객은 어떤 지역과 연령대에 분포하는가?",
        "어떤 카테고리의 상품이 많은가?",
        "주문 상태와 결제수단 분포는 어떤가?",
        "카테고리별 매출은 어떻게 다른가?",
        "월별 매출과 주문 수는 어떻게 변하는가?",
        "구매 금액이 높은 고객은 누구인가?"
    ],
    "metric": [
        "고객 수, 평균 나이",
        "카테고리별 상품 수",
        "주문 수, 비율",
        "총매출, 매출 비중",
        "월별 매출, 월별 주문 수",
        "고객별 총매출, 주문 횟수"
    ],
    "required_data": [
        "customers",
        "products",
        "orders",
        "order_items, products",
        "orders, order_items",
        "customers, orders, order_items"
    ]
})

questions

```

이 표는 분석 결과표가 아니라 EDA의 방향을 잡는 지도에 가깝습니다. 질문을 먼저 정리해 두면 어떤 데이터를 병합해야 하는지, 어떤 지표를 계산해야 하는지 더 분명해집니다.

6. 고객, 상품, 주문을 각각 살펴본다

먼저 각 데이터셋을 따로 살펴봅니다. 하나의 데이터셋 안에서 분포와 빈도를 확인하면 이후 병합 분석을 할 때 기준이 생깁니다.

고객 나이의 기본 통계를 확인합니다.

```
customers["age"].describe()
```

도시별 고객 수를 확인합니다.

```
customer_city = customers["city"].value_counts().reset_index()
customer_city.columns = ["city", "customer_count"]
customer_city
```

성별 고객 수를 확인합니다.

```
customer_gender = customers["gender"].value_counts().reset_index()
customer_gender.columns = ["gender", "customer_count"]
customer_gender
```

고객 데이터에서 이어질 수 있는 질문은 다음과 같습니다.

- 고객은 어느 도시에 많이 분포하는가?
- 고객의 평균 나이는 어느 정도인가?
- 성별 고객 수는 어떻게 분포하는가?
- 도시별 구매 금액도 차이가 있는가?

상품 데이터에서는 카테고리별 가격을 먼저 봅니다.

```
product_category = products["category"].value_counts().reset_index()
product_category.columns = ["category", "product_count"]
product_category
products["price"].describe()
```

카테고리별 평균 가격도 확인할 수 있습니다.

```
category_price = (
    products
    .groupby("category", as_index=False)
    .agg(
        product_count=("product_id", "count"),
        avg_price=("price", "mean"),
        min_price=("price", "min"),
        max_price=("price", "max")
    )
    .sort_values("avg_price", ascending=False)
)
category_price
```

주문 데이터에서는 주문 상태, 결제수단, 주문 기간을 확인합니다.

```

order_status = orders["order_status"].value_counts().reset_index()
order_status.columns = ["order_status", "order_count"]
order_status

payment_method = orders["payment_method"].value_counts().reset_index()
payment_method.columns = ["payment_method", "order_count"]
payment_method

print("주문 시작일:", orders["order_date"].min())
print("주문 종료일:", orders["order_date"].max())

```

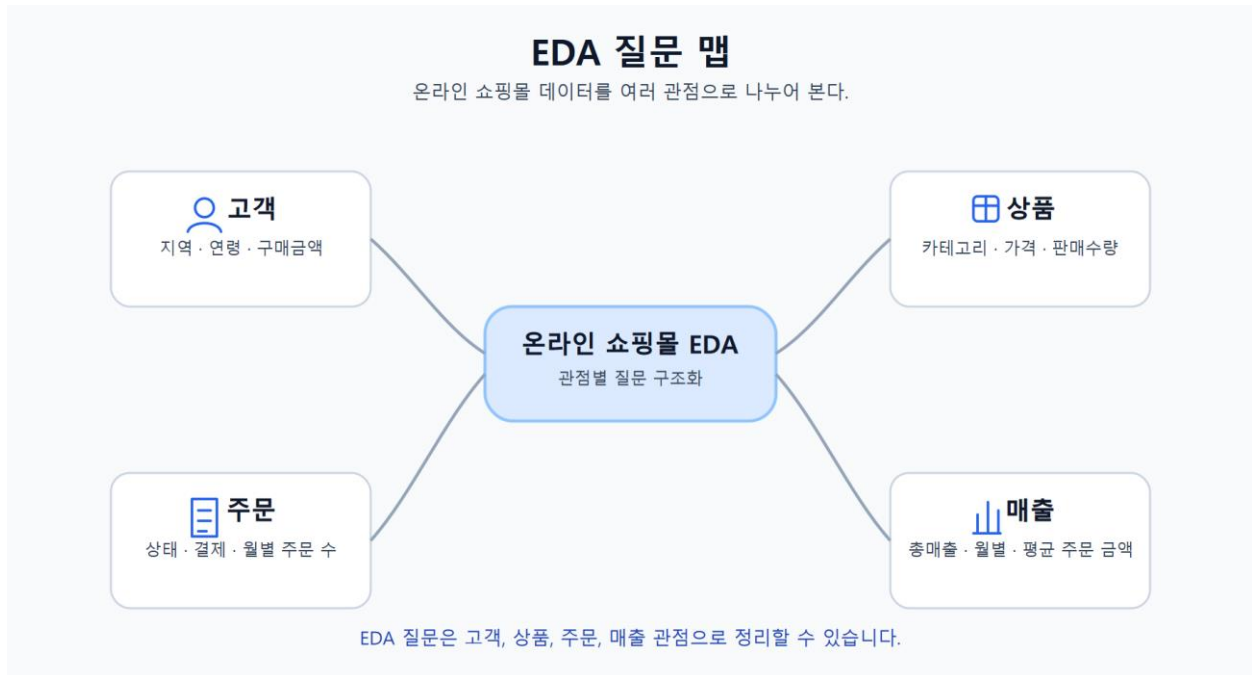


그림 6-4. 고객·상품·주문 관점의 EDA 맵

7. 매출을 계산할 수 있는 형태로 연결한다

고객, 상품, 주문을 각각 확인했다면 이제 매출 분석을 위해 데이터를 연결합니다. 먼저 주문 상세 데이터에 `line_total`이 있는지 확인합니다.

```
"line_total" in order_items.columns
```

없다면 수량과 단가를 곱해 다시 생성합니다.

```

if "line_total" not in order_items.columns:
    order_items["line_total"] = order_items["quantity"] * order_items["unit_price"]

```

주문 상세 금액의 기본 통계를 확인합니다.

```
order_items["line_total"].describe()
```

전체 매출 합계도 확인합니다.

```
total_sales = order_items["line_total"].sum()
total_sales
```

카테고리별 매출을 계산하려면 주문 상세와 상품 정보를 병합해야 합니다.

```
sales_items = order_items.merge(
    products,
    on="product_id",
    how="left"
)
```

병합 후에는 행 수와 누락 여부를 반드시 확인합니다. 병합은 EDA에서 자주 사용하지만, 잘못하면 데이터가 늘어나거나 줄어들 수 있습니다.

```
print("병합 전 order_items:", order_items.shape)
print("병합 후 sales_items:", sales_items.shape)
print("상품명 누락:", sales_items["product_name"].isna().sum())
print("카테고리 누락:", sales_items["category"].isna().sum())
```

카테고리별 매출을 계산합니다.

```
category_sales = (
    sales_items
    .groupby("category", as_index=False)
    .agg(
        total_quantity=("quantity", "sum"),
        total_sales=("line_total", "sum")
    )
    .sort_values("total_sales", ascending=False)
)

category_sales
```

매출 비중도 함께 계산합니다.

```
category_sales["sales_ratio"] = (
    category_sales["total_sales"] / category_sales["total_sales"].sum() * 100
).round(2)

category_sales
```

카테고리별 매출은 어떤 상품군이 매출에 기여하는지 보여 줍니다. 다만 매출이 높은 이유가 판매 수량 때문인지, 단가 때문인지는 추가로 확인해야 합니다.

8. 시간 흐름과 고객 가치를 살펴본다

월별 매출을 보려면 주문 상세와 주문 정보를 연결합니다.

```
order_sales = order_items.merge(
    orders,
    on="order_id",
    how="left"
)
```

날짜를 다시 확인하고 월 정보를 생성합니다.

```
order_sales["order_date"] = pd.to_datetime(order_sales["order_date"], errors="coerce")
order_sales["order_month"] = order_sales["order_date"].dt.to_period("M").astype(str)
```

월별 매출과 주문 수를 계산합니다.

```
monthly_sales = (
    order_sales
    .groupby("order_month", as_index=False)
    .agg(
        total_sales=("line_total", "sum"),
        order_count=("order_id", "nunique")
    )
    .sort_values("order_month")
)

monthly_sales
```

평균 주문 금액도 계산할 수 있습니다.

```
monthly_sales["avg_order_value"] = (
    monthly_sales["total_sales"] / monthly_sales["order_count"]
).round(0)

monthly_sales
```

월별 매출은 시간에 따른 흐름을 보여 주지만, 원인을 바로 설명하지는 않습니다. 특정 월에 매출이 늘었다면 주문 수가 늘었는지, 평균 주문 금액이 늘었는지, 특정 카테고리 매출이 늘었는지를 이어서 확인해야 합니다.

고객별 구매 금액을 보려면 고객 정보를 결합합니다.


```
customer_sales_base = order_sales.merge(
    customers,
    on="customer_id",
    how="left"
)
```

고객별 구매 금액과 주문 횟수를 계산합니다.

```
customer_sales = (
    customer_sales_base
    .groupby(["customer_id", "name", "city"], as_index=False)
    .agg(
        order_count=("order_id", "nunique"),
        total_sales=("line_total", "sum")
    )
    .sort_values("total_sales", ascending=False)
)

customer_sales.head(10)
```

평균 주문 금액을 추가합니다.

```
customer_sales["avg_order_value"] = (
    customer_sales["total_sales"] / customer_sales["order_count"]
).round(0)

customer_sales.head(10)
```

고객별 총 구매 금액은 우수 고객 분석의 출발점입니다. 하지만 한 번의 고객 구매 고객과 여러 번 반복 구매한 고객은 다르게 해석해야 합니다. 그래서 `total_sales`, `order_count`, `avg_order_value`를 함께 봅니다.

9. EDA 결과를 다시 질문으로 연결한다

EDA의 가치는 표를 만드는 데서 끝나지 않습니다. 각 결과가 다음 질문으로 이어져야 합니다.

```
eda_result_summary = pd.DataFrame({
    "question": [
        "고객은 어느 도시에 많이 분포하는가?",
        "어떤 카테고리의 상품이 많은가?",
        "카테고리별 매출은 어떻게 다른가?",
        "월별 매출과 주문 수는 어떻게 변하는가?",
        "구매 금액이 높은 고객은 누구인가?"
    ],
    "result_table": [
        "customer_city",
        "product_category",
        "category_sales",
        "monthly_sales",
        "customer_sales"
    ],
    "next_question": [
        "도시별 구매 금액도 차이가 있는가?",
        "상품 수가 많은 카테고리 매출도 높은가?",
        "매출 차이가 수량 때문인가 단가 때문인가?",
        "특정 월의 매출 변화는 어떤 카테고리 때문인가?",
        "고액 구매 고객은 반복 구매 고객인가?"
    ]
})

eda_result_summary
```

EDA 결과는 필요한 경우 CSV로 저장해 보고서 작성이나 다음 장의 시각화에서 사용할 수 있습니다.

```
category_sales.to_csv(report_dir / "ch06_category_sales.csv", index=False)
monthly_sales.to_csv(report_dir / "ch06_monthly_sales.csv", index=False)
customer_sales.to_csv(report_dir / "ch06_customer_sales.csv", index=False)
eda_result_summary.to_csv(report_dir / "ch06_eda_questions.csv", index=False)
```

저장 결과를 확인합니다.

```
list(report_dir.glob("ch06_*.csv"))
```

간단한 Markdown 요약 보고서를 만들 수도 있습니다.

```

summary_text = f"""
# Chapter 6 EDA 요약 보고서

## 1. 분석 목적

전처리된 온라인 쇼핑몰 데이터를 사용해 고객, 상품, 주문, 매출 관점의 기본 현황을 탐색했습니다.

## 2. 주요 분석 질문

{questions.to_markdown(index=False)}

## 3. 카테고리별 매출 요약

{category_sales.head(10).to_markdown(index=False)}

## 4. 월별 매출 요약

{monthly_sales.to_markdown(index=False)}

## 5. 고객별 구매 금액 상위 10명

{customer_sales.head(10).to_markdown(index=False)}

## 6. 추가 분석 질문

{eda_result_summary.to_markdown(index=False)}

## 7. 해석 시 주의사항

- EDA 결과는 최종 결론이 아니라 추가 분석을 위한 관찰 결과입니다.
- 매출이 높은 카테고리가 반드시 선호도가 높은 카테고리라는 뜻은 아닙니다.
- 월별 매출 변화의 원인을 설명하려면 프로모션, 계절성, 신규 상품 등의 추가 정보가 필요합니다.
- 고객별 구매 금액은 주문 횟수와 함께 해석해야 합니다.
"""

report_path = report_dir / "ch06_eda_summary.md"
report_path.write_text(summary_text, encoding="utf-8")

```

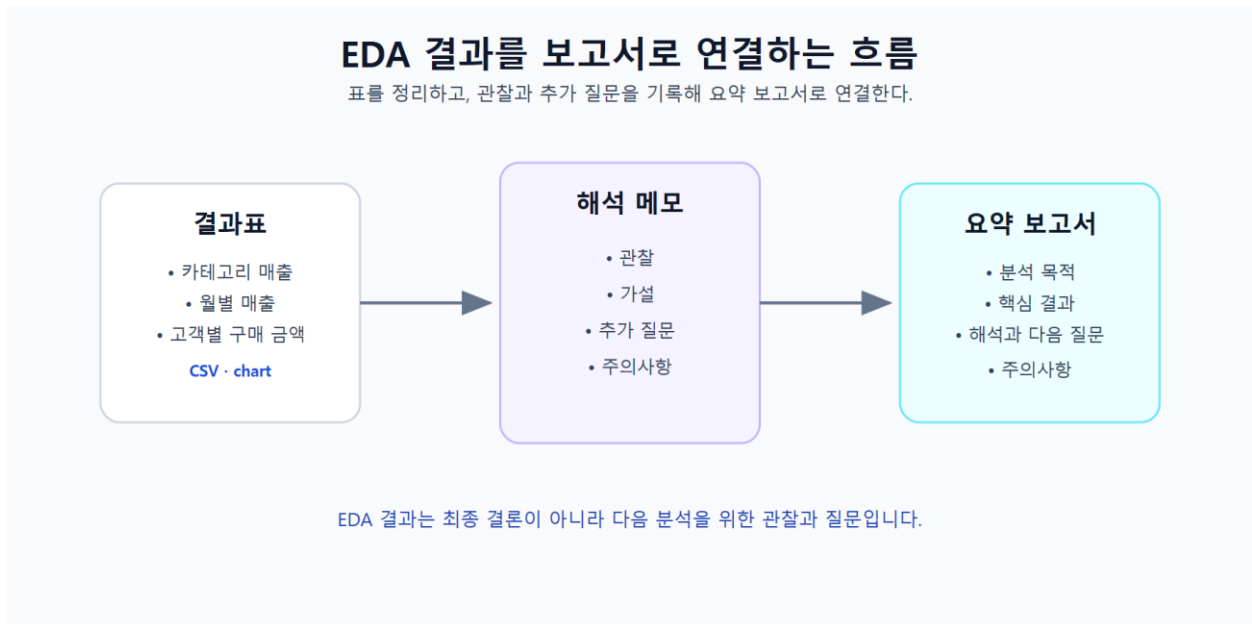


그림 6-5. EDA 결과를 인사이트와 보고서로 연결하는 흐름

10. LLM과 함께 질문을 확장한다

LLM은 EDA 질문을 확장하고, 결과 해석 문장을 다듬는 데 도움이 됩니다. 하지만 LLM이 제안한 질문이 실제 데이터로 답할 수 있는지는 반드시 사람이 확인해야 합니다. 데이터에 없는 이탈 여부, 만족도, 프로모션 정보까지 LLM이 추측해서 말하게 두면 분석이 쉽게 왜곡됩니다.

LLM에게는 원본 고객명, 이메일, 주문 상세 전체 데이터를 넣지 않습니다. 컬럼명, 데이터 타입, 집계 결과, 요약표처럼 구조화된 정보 중심으로 질문합니다.

EDA 질문을 만들 때는 다음처럼 요청할 수 있습니다.

온라인 쇼핑몰 데이터로 EDA를 수행하려고 합니다.

데이터셋:

- customers: 고객 정보
- products: 상품 정보
- orders: 주문 정보
- order_items: 주문 상세 정보

주요 컬럼:

- customers: customer_id, gender, age, city, signup_date
- products: product_id, product_name, category, price
- orders: order_id, customer_id, order_date, payment_method, order_status
- order_items: order_id, product_id, quantity, unit_price, line_total

요청:

1. 현재 데이터로 검증 가능한 EDA 질문 10개를 만들어 주세요.
2. 각 질문에 필요한 데이터셋과 pandas 기능을 함께 정리해 주세요.
3. 데이터에 없는 내용은 추측하지 마세요.
4. 각 질문을 계산 가능한 지표로 바꿔 주세요.

EDA 결과 해석을 요청할 때는 원인 단정을 막는 조건을 넣는 것이 좋습니다.

다음은 온라인 쇼핑몰 카테고리별 매출 요약 결과입니다.

category,total_quantity,total_sales,sales_ratio

전자기기,320,12500000,42.5

생활용품,510,7800000,26.5

패션,260,6200000,21.1

식품,430,2900000,9.9

이 결과를 보고서에 넣을 수 있도록 해석해 주세요.

조건:

- 데이터에 없는 원인을 단정하지 말 것
- 원인 설명은 가설로 표현할 것
- 추가로 확인해야 할 분석 질문을 제안할 것
- 관찰, 가설, 추가 분석을 구분할 것

LLM이 만든 질문을 검토할 때는 다음처럼 현재 데이터로 답할 수 있는지 확인합니다.

LLM이 다음 분석 질문을 제안했습니다.

1. 고객이 왜 이탈했는가?
2. 고객 만족도는 어떤가?
3. 어떤 상품이 가장 인기가 많은가?
4. 매출이 높은 카테고리는 무엇인가?
5. 월별 매출은 어떻게 변하는가?

현재 데이터에는 고객 이탈 여부와 만족도 데이터가 없습니다.

각 질문이 현재 데이터로 분석 가능한지 검토해 주세요.

분석하기 어려운 질문은 현재 데이터로 답할 수 있는 질문으로 수정해 주세요.

LLM은 질문을 넓히는 데 유용하지만, 질문을 좁히고 검증 가능한 형태로 바꾸는 책임은 분석자에게 있습니다.

11. EDA 결과를 읽을 때 조심할 점

EDA 결과는 강력한 출발점이지만, 최종 결론은 아닙니다. 도시별 고객 수가 많다고 해서 그 도시의 매출이 반드시 높다고 말할 수 없습니다. 상품 수가 많은 카테고리가 매출도 높다고 단정할 수 없습니다. 월별 매출이 증가했다고 해서 바로 프로모션 효과라고 말할 수도 없습니다.

EDA 결과를 읽을 때는 다음과 같은 문장을 구분해 사용하는 것이 좋습니다.

- 관찰: 전자기기 카테고리의 매출 비중이 가장 높게 나타났습니다.
- 가설: 전자기기는 평균 단가가 높아 매출 비중이 클 가능성이 있습니다.
- 추가 분석: 카테고리별 판매 수량과 평균 단가를 함께 비교할 필요가 있습니다.

고객별 구매 금액도 마찬가지입니다.

- 관찰: 일부 고객의 총 구매 금액이 다른 고객보다 높게 나타났습니다.
- 가설: 해당 고객은 반복 구매 고객이거나 한 번에 고액 주문을 한 고객일 수 있습니다.
- 추가 분석: 주문 횟수와 평균 주문 금액을 함께 확인해야 합니다.

이처럼 EDA는 분석 결과를 과장하지 않고, 다음 질문을 더 정교하게 만드는 과정입니다.

12. 다음 장으로 이어지는 흐름

EDA를 통해 고객, 상품, 주문, 매출 관점의 기본 질문과 지표를 정리했습니다. 이제 다음 단계는 이 결과를 더 직관적으로 보여 주는 것입니다.

다음 장에서는 데이터 시각화를 다룹니다. EDA에서 만든 질문과 집계표를 바탕으로 막대그래프, 선그래프, 분포 그래프를 만들고, 어떤 그래프가 어떤 질문에 적합한지 살펴봅니다. 좋은 시각화는 단순히 예쁜 그림이 아니라, 분석 질문과 해석을 더 분명하게 보여 주는 도구입니다.

7장. 그래프로 데이터의 이야기를 보여주기

표는 정확한 값을 전달하는 데 유용합니다. 하지만 숫자가 많아지면 변화와 차이를 빠르게 파악하기 어렵습니다. 카테고리별 금액을 표로 보면 값을 하나씩 비교해야 하지만, 막대그래프로 표현하면 어떤 카테고리가 큰지 한눈에 확인할 수 있습니다. 월별 금액도 선 그래프로 그리면 시간에 따른 증가와 감소가 더 분명하게 드러납니다.

데이터 시각화는 분석 결과를 예쁘게 꾸미는 작업이 아닙니다. **분석 질문에 맞는 형태로 데이터를 표현해 패턴, 차이, 추세, 분포, 관계를 이해하고 전달하는 과정**입니다. 좋은 그래프는 무엇을 보여주는지 분명하고, 무엇까지는 설명할 수 없는지도 함께 드러냅니다.

이 장에서는 5장에서 전처리한 온라인 쇼핑몰 데이터를 바탕으로 다음 내용을 시각화합니다.

- 완료 주문 기준 카테고리별 금액
- 완료 주문 기준 월별 금액
- 상품 가격 분포
- 상품 가격과 완료 주문 판매 수량의 관계
- 완료 주문 구매 금액 상위 고객
- 전체 주문 상태별 주문 수

여기서 사용하는 `line_total`은 수량과 단가를 곱한 **주문 상세 금액**입니다. 이 값을 모두 합한 결과를 곧바로 회계상의 순매출이라고 부르면 안 됩니다. 취소·환불 주문, 할인, 배송비, 세금, 부분 환불 정보가 별도로 반영되지 않을 수 있기 때문입니다. 이 장에서는 5장과 같은 기준을 유지해 `order_status == "completed"`인 주문만 완료 주문 기준 금액에 포함합니다.

이 장에서 생각해 볼 질문

- 지금 확인하려는 것은 비교, 추세, 분포, 관계 중 무엇인가?
- 분석 질문에 가장 잘 맞는 그래프는 무엇인가?
- 그래프에 사용한 데이터의 포함·제외 기준은 무엇인가?
- x축과 y축은 무엇을 의미하며 단위는 무엇인가?
- 범주와 시간 순서가 올바르게 정렬되어 있는가?
- 그래프가 관찰된 패턴만 보여 주는가, 원인까지 증명하는가?

- 축 범위나 생략된 데이터가 결과를 과장하지 않는가?
- 개인정보 또는 식별 가능한 값이 노출되지 않았는가?
- LLM이 작성한 해석이 실제 집계 결과와 일치하는가?



그림 7-1. 데이터 시각화 전체 흐름도

1. 시각화는 분석 질문에서 시작된다

좋은 시각화는 그래프 종류를 많이 아는 데서 시작하지 않습니다. 먼저 어떤 질문에 답하려는지 정해야 합니다.

“카테고리별 완료 주문 금액은 어떻게 다른가?”는 범주별 크기를 비교하는 질문이므로 막대그래프가 적합합니다. “월별 완료 주문 금액은 어떻게 변하는가?”는 시간 흐름을 확인하는 질문이므로 선 그래프가 적합합니다. “상품 가격은 어느 구간에 몰려 있는가?”는 분포를 확인하는 질문이므로 히스토그램이 적합합니다.

분석 목적	적합한 그래프	예시 질문
범주별 크기 비교	막대그래프	카테고리별 완료 주문 금액은 어떻게 다른가?
시간 흐름 확인	선 그래프	월별 완료 주문 금액은 어떻게 변하는가?
값의 분포 확인	히스토그램	상품 가격은 어떤 구간에 몰려 있는가?
두 숫자형 변수의 관계 확인	산점도	가격과 완료 주문 판매 수량은 관계가 있는가?
상위 항목 비교	가로 막대그래프	구매 금액 상위 고객군은 어떻게 구성되는가?
비율 또는 빈도 비교	막대그래프	주문 상태별 주문 수는 어떻게 다른가?

파이 차트는 전체에서 각 항목이 차지하는 비율을 보여 줄 수 있습니다. 그러나 항목이 많거나 값 차이가 작으면 비교하기 어렵습니다. 실무에서는 비율도 막대그래프로 표현하는 경우가 많습니다.



그림 7-2. 분석 질문별 그래프 선택 가이드

2. matplotlib 그래프의 기본 구조

matplotlib은 Python에서 널리 사용하는 시각화 라이브러리입니다. 이 장에서는 Figure와 Axes 객체를 명시적으로 사용하는 방식을 중심으로 설명합니다.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 5))

ax.bar(x_data, y_data)
ax.set_title("그래프 제목")
ax.set_xlabel("x축 이름")
ax.set_ylabel("y축 이름")
ax.tick_params(axis="x", rotation=45)

fig.tight_layout()
fig.savefig("graph.png", dpi=150, bbox_inches="tight")
plt.show()
```

각 요소의 의미는 다음과 같습니다.

요소	의미
<code>figsize=(10, 5)</code>	그래프의 가로·세로 크기입니다. 단위는 인치입니다.
<code>fig, ax</code>	전체 그림과 실제 그래프 영역을 나타냅니다.
<code>set_title()</code>	그래프 제목을 설정합니다.
<code>set_xlabel(), set_ylabel()</code>	축 이름과 의미를 표시합니다.
<code>tight_layout()</code>	제목과 레이블이 잘리는 문제를 줄입니다.
<code>dpi=150</code>	저장 이미지의 해상도를 설정합니다.
<code>bbox_inches="tight"</code>	저장할 때 바깥쪽 레이블이 잘리는 문제를 줄입니다.

그래프를 파일로 저장할 때는 일반적으로 `savefig()`를 `show()`보다 먼저 실행합니다. 실행 환경에 따라 `show()` 이후 그래프 상태가 초기화될 수 있으므로, 먼저 저장한 뒤 화면에 표시하는 편이 안전합니다.

스크립트에서 그래프를 여러 개 반복 생성한다면 `plt.close(fig)`로 사용이 끝난 Figure를 닫아 메모리 사용량을 줄일 수 있습니다. Jupyter Notebook에서 몇 개의 그래프만 만드는 실습에서는 반드시 필요한 것은 아닙니다.

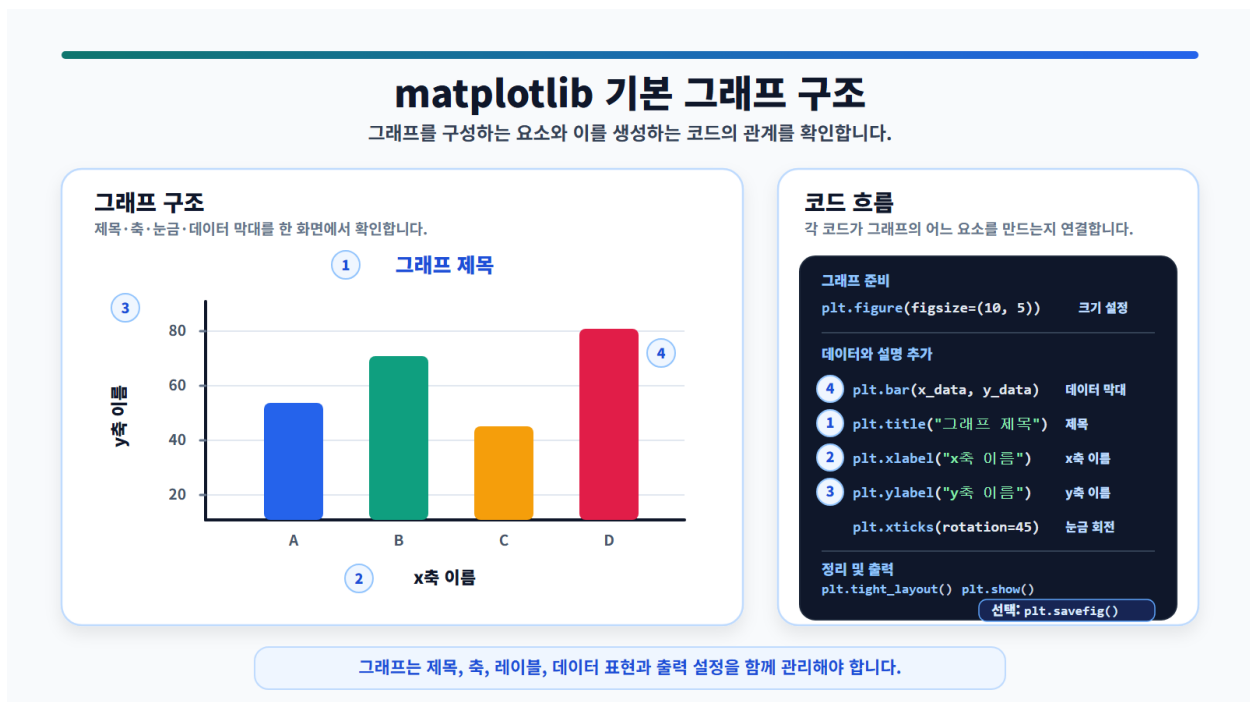


그림 7-3. matplotlib 기본 그래프 구조

3. 한글 폰트와 숫자 표시 설정

matplotlib에서 한글 제목이나 축 이름을 사용하면 실행 환경에 따라 글자가 깨질 수 있습니다. 설치된 폰트 중 사용할 수 있는 한글 폰트를 선택합니다.

```
from matplotlib import font_manager
import matplotlib.pyplot as plt

installed_fonts = {
    font.name
    for font in font_manager.fontManager.ttflist
}

font_candidates = [
    "Malgun Gothic",      # Windows
    "AppleGothic",       # macOS
    "NanumGothic",       # Linux에서 별도 설치 가능
    "Noto Sans CJK KR",
]

selected_font = next(
    (
        font_name
        for font_name in font_candidates
        if font_name in installed_fonts
    ),
    None,
)

if selected_font is not None:
    plt.rcParams["font.family"] = selected_font
    print("사용 폰트:", selected_font)
else:
    print(
        "사용 가능한 한글 폰트를 찾지 못했습니다. "
        "한글 폰트를 설치하거나 영문 레이블을 사용하세요."
    )

plt.rcParams["axes.unicode_minus"] = False
```

axes.unicode_minus를 False로 설정하면 음수 기호가 네모 모양으로 보이는 문제를 줄일 수 있습니다.

금액 축에는 천 단위 구분 기호를 표시하면 읽기 쉽습니다.

```
from matplotlib.ticker import FuncFormatter

money_formatter = FuncFormatter(
    lambda value, position: f"{value:,0f}"
)
```

이후 `ax.yaxis.set_major_formatter(money_formatter)`처럼 적용할 수 있습니다.

4. 그래프 종류별로 읽는 법

막대그래프

막대그래프는 범주별 크기를 비교할 때 사용합니다. 카테고리별 금액, 주문 상태별 주문 수, 고객별 구매 금액 상위 목록 등에 적합합니다.

막대그래프의 길이는 값의 크기를 나타내므로 특별한 이유가 없다면 값 축을 0에서 시작하는 것이 좋습니다. 항목 이름이 길거나 항목 수가 많다면 가로 막대그래프가 더 읽기 좋을 수 있습니다.

선 그래프

선 그래프는 시간에 따른 변화를 확인할 때 사용합니다. 월별 금액이나 월별 주문 수처럼 순서가 중요한 데이터에 적합합니다.

선 그래프를 만들기 전에 날짜가 올바른 타입인지, 누락된 기간이 있는지, 월이 시간 순서대로 정렬되어 있는지 확인해야 합니다. 데이터가 없는 월을 0으로 볼 것인지 결측으로 볼 것인지도 분석 목적에 따라 판단해야 합니다.

히스토그램

히스토그램은 숫자형 데이터의 분포를 확인할 때 사용합니다. 상품 가격이나 고객 나이가 어느 구간에 많이 몰려 있는지 살펴볼 수 있습니다.

bins는 값을 나누는 구간 수입니다. 구간 수가 너무 적으면 분포가 단순하게 보이고, 너무 많으면 작은 변동이 과도하게 강조될 수 있습니다. 한 가지 구간 수만 보고 결론을 내리기보다 몇 가지 값을 비교해 보는 것이 좋습니다.

산점도

산점도는 두 숫자형 변수의 관계를 탐색할 때 사용합니다. 상품 가격과 판매 수량, 고객별 주문 횟수와 구매 금액 등의 관계를 확인할 수 있습니다.

alpha는 점의 투명도입니다. 점이 많이 겹칠 때 0과 1 사이의 값을 사용하면 밀집된 구간을 더 쉽게 확인할 수 있습니다.

산점도에서 관계가 보이더라도 인과관계가 증명되는 것은 아닙니다. 카테고리, 할인, 노출량, 재고, 계절성 같은 다른 요인이 영향을 줄 수 있습니다.

5. 시각화를 위한 데이터 준비

이번 장의 실습 노트북 파일은 `notebooks/ch07_visualization.ipynb`입니다.

먼저 필요한 패키지를 불러오고 프로젝트 경로를 설정합니다.

```
from pathlib import Path

import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.ticker import FuncFormatter
```

현재 작업 폴더가 프로젝트 루트이거나 `notebooks` 폴더인 경우를 모두 처리합니다.

```

project_root = Path.cwd()

if project_root.name == "notebooks":
    project_root = project_root.parent

processed_dir = project_root / "data" / "processed"
report_dir = project_root / "reports"
figure_dir = report_dir / "figures"

figure_dir.mkdir(parents=True, exist_ok=True)

print("프로젝트 루트:", project_root.resolve())
print("전처리 데이터 폴더:", processed_dir.resolve())
print("그래프 저장 폴더:", figure_dir.resolve())

```

다른 위치에서 노트북을 실행하고 있다면 `project_root`를 실제 프로젝트 폴더로 직접 지정해야 합니다.

필요한 파일이 모두 존재하는지 확인합니다.

```

required_files = [
    "customers_clean.csv",
    "products_clean.csv",
    "orders_clean.csv",
    "order_items_clean.csv",
]

missing_files = [
    file_name
    for file_name in required_files
    if not (processed_dir / file_name).exists()
]

if missing_files:
    raise FileNotFoundError(
        "전처리 결과 파일을 찾을 수 없습니다: "
        + ", ".join(missing_files)
        + "\n5장의 전처리 실습을 먼저 실행하세요."
    )

```

전처리 데이터를 불러옵니다.

```
customers = pd.read_csv(  
    processed_dir / "customers_clean.csv"  
)  
products = pd.read_csv(  
    processed_dir / "products_clean.csv"  
)  
orders = pd.read_csv(  
    processed_dir / "orders_clean.csv"  
)  
order_items = pd.read_csv(  
    processed_dir / "order_items_clean.csv"  
)
```

필수 컬럼도 확인합니다.


```

required_columns = {
    "customers": {
        "customer_id",
    },
    "products": {
        "product_id",
        "product_name",
        "category",
        "price",
    },
    "orders": {
        "order_id",
        "customer_id",
        "order_date",
        "order_status",
    },
    "order_items": {
        "order_id",
        "product_id",
        "quantity",
        "unit_price",
    },
}

dataframes = {
    "customers": customers,
    "products": products,
    "orders": orders,
    "order_items": order_items,
}

for name, required in required_columns.items():
    missing = required - set(dataframes[name].columns)

    if missing:
        raise KeyError(
            f"{name}에 필요한 컬럼이 없습니다: "
            f"{sorted(missing)}"
        )

```

날짜와 숫자형 컬럼을 다시 확인합니다. CSV로 저장했다가 다시 불러오면 날짜 타입 정보가 유지되지 않기 때문에 재변환이 필요합니다.

```
orders["order_date"] = pd.to_datetime(
    orders["order_date"],
    errors="coerce",
)

numeric_columns = [
    (products, "price"),
    (order_items, "quantity"),
    (order_items, "unit_price"),
]

for dataframe, column in numeric_columns:
    dataframe[column] = pd.to_numeric(
        dataframe[column],
        errors="coerce",
    )
```

변환 실패와 고유 키 품질을 확인합니다.

```

print(
    "order_date 변환 실패:",
    int(orders["order_date"].isna().sum()),
)
print(
    "price 변환 실패:",
    int(products["price"].isna().sum()),
)
print(
    "quantity 변환 실패:",
    int(order_items["quantity"].isna().sum()),
)
print(
    "unit_price 변환 실패:",
    int(order_items["unit_price"].isna().sum()),
)

print(
    "orders.order_id 중복:",
    int(orders["order_id"].duplicated().sum()),
)
print(
    "products.product_id 중복:",
    int(products["product_id"].duplicated().sum()),
)

```

시각화 전에 변환 실패나 고유 키 중복이 남아 있다면 원인을 먼저 해결해야 합니다.

line_total이 없으면 수량과 단가로 다시 계산합니다.

```

if "line_total" not in order_items.columns:
    order_items["line_total"] = (
        order_items["quantity"]
        * order_items["unit_price"]
    )
else:
    order_items["line_total"] = pd.to_numeric(
        order_items["line_total"],
        errors="coerce",
    )

```

주문 상세와 주문 정보를 연결합니다.

```
order_sales = order_items.merge(
    orders[
        [
            "order_id",
            "customer_id",
            "order_date",
            "order_status",
        ]
    ],
    on="order_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)
```

연결되지 않은 주문 상세가 있는지 확인합니다.

```
unmatched_order_items = order_sales.loc[
    order_sales["_merge"].ne("both")
].copy()

print(
    "주문 정보와 연결되지 않은 주문 상세:",
    len(unmatched_order_items),
)
```

미연결 행이 있다면 그래프를 만들기 전에 원인을 확인합니다. 정상 연결된 행만 남기고, 완료 주문을 별도로 분리합니다.

```
order_sales = order_sales.loc[
    order_sales["_merge"].eq("both")
].drop(columns="_merge")

completed_order_sales = order_sales.loc[
    order_sales["order_status"].eq("completed")
].copy()
```

완료 주문 상세에 상품 정보를 연결합니다.

```
completed_sales_items = completed_order_sales.merge(  
    products[  
        [  
            "product_id",  
            "product_name",  
            "category",  
            "price",  
        ]  
    ],  
    on="product_id",  
    how="left",  
    validate="many_to_one",  
    indicator=True,  
)
```

상품 정보와 연결되지 않은 주문 상세도 확인합니다.

```
unmatched_products = completed_sales_items.loc[  
    completed_sales_items["_merge"].ne("both")  
].copy()  
  
print(  
    "상품 정보와 연결되지 않은 완료 주문 상세:",  
    len(unmatched_products),  
)  
  
completed_sales_items = completed_sales_items.loc[  
    completed_sales_items["_merge"].eq("both")  
].drop(columns="_merge")
```

6. 시각화용 집계 데이터 만들기

카테고리별 완료 주문 금액

```
category_sales = (
    completed_sales_items
    .groupby(
        "category",
        as_index=False,
        dropna=False,
    )
    .agg(
        total_quantity=("quantity", "sum"),
        completed_amount=("line_total", "sum"),
    )
    .sort_values(
        "completed_amount",
        ascending=False,
    )
)

completed_amount_sum = (
    category_sales["completed_amount"].sum()
)

category_sales["amount_ratio_pct"] = (
    category_sales["completed_amount"]
    .div(completed_amount_sum)
    .mul(100)
    .round(2)
)

category_sales
```

월별 완료 주문 금액

날짜 변환에 실패한 행은 월별 분석에 사용할 수 없으므로 별도로 확인합니다.

```
invalid_date_sales = completed_order_sales.loc[
    completed_order_sales["order_date"].isna()
].copy()

print(
    "월별 집계에서 제외되는 날짜 결측 행:",
    len(invalid_date_sales),
)
```

월을 `Period` 타입으로 만들면 시간 순서대로 안정적으로 정렬할 수 있습니다.

```
monthly_sales = (
    completed_order_sales
    .dropna(subset=["order_date"])
    .assign(
        order_month=lambda df: (
            df["order_date"].dt.to_period("M")
        )
    )
    .groupby(
        "order_month",
        as_index=False,
    )
    .agg(
        completed_amount=("line_total", "sum"),
        completed_order_count=(
            "order_id",
            "nunique",
        ),
    )
    .sort_values("order_month")
)

monthly_sales["month_start"] = (
    monthly_sales["order_month"].dt.to_timestamp()
)

monthly_sales
```

데이터가 없는 월도 그래프에 0으로 표시해야 한다면 전체 월 범위를 만들어 재색인할 수 있습니다. 그러나 “거래가 없어서 0인 월”과 “데이터가 누락된 월”은 의미가 다르므로 원본 수집 상태를 먼저 확인해야 합니다.

상품별 완료 주문 판매 수량과 금액

```
product_sales = (
    completed_sales_items
    .groupby(
        [
            "product_id",
            "product_name",
            "category",
            "price",
        ],
        as_index=False,
    )
    .agg(
        completed_quantity=("quantity", "sum"),
        completed_amount=("line_total", "sum"),
    )
    .sort_values(
        "completed_amount",
        ascending=False,
    )
)

product_sales.head()
```

고객별 완료 주문 구매 금액

고객명을 그래프에 표시하지 않고 고객 ID를 기준으로 집계합니다.


```
customer_sales = (
    completed_order_sales
    .groupby(
        "customer_id",
        as_index=False,
    )
    .agg(
        completed_order_count=(
            "order_id",
            "nunique",
        ),
        completed_amount=("line_total", "sum"),
    )
    .sort_values(
        "completed_amount",
        ascending=False,
    )
)

customer_sales["average_completed_order_amount"] = (
    customer_sales["completed_amount"]
    .div(customer_sales["completed_order_count"])
    .round(0)
)

customer_sales.head()
```

7. 그래프 저장을 위한 공통 함수

같은 저장 코드를 반복하지 않도록 간단한 함수를 만듭니다.

```
def save_and_show(
    fig,
    file_name,
    *,
    dpi=150,
):
    output_path = figure_dir / file_name

    fig.tight_layout()
    fig.savefig(
        output_path,
        dpi=dpi,
        bbox_inches="tight",
    )
    plt.show()

    return output_path
```

* 뒤에 있는 dpi는 키워드 이름을 명시해 전달하도록 만든 선택 인자입니다. 예를 들어 `save_and_show(fig, "chart.png", dpi=200)`처럼 사용할 수 있습니다.

금액 축 형식도 준비합니다.

```
money_formatter = FuncFormatter(
    lambda value, position: f"{value:,.0f}"
)
```

8. 주요 그래프 만들기

8.1 카테고리별 완료 주문 금액 막대그래프

```
fig, ax = plt.subplots(figsize=(10, 5))

ax.bar(
    category_sales["category"],
    category_sales["completed_amount"],
)

ax.set_title("카테고리별 완료 주문 금액")
ax.set_xlabel("카테고리")
ax.set_ylabel("완료 주문 금액")
ax.set_ylim(bottom=0)
ax.yaxis.set_major_formatter(money_formatter)
ax.tick_params(
    axis="x",
    rotation=45,
)

for label in ax.get_xticklabels():
    label.set_horizontalalignment("right")

category_chart_path = save_and_show(
    fig,
    "ch07_category_completed_amount_bar.png",
)
```

이 그래프는 완료 주문 기준으로 어떤 카테고리의 금액이 큰지 보여 줍니다. 금액이 높은 이유가 판매 수량, 평균 단가, 상품 수 중 무엇 때문인지는 추가 분석이 필요합니다.

8.2 월별 완료 주문 금액 선 그래프

```
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(
    monthly_sales["month_start"],
    monthly_sales["completed_amount"],
    marker="o",
)

ax.set_title("월별 완료 주문 금액 추이")
ax.set_xlabel("주문 월")
ax.set_ylabel("완료 주문 금액")
ax.set_ylim(bottom=0)
ax.yaxis.set_major_formatter(money_formatter)
ax.grid(
    axis="y",
    alpha=0.3,
)

fig.autofmt_xdate()

monthly_chart_path = save_and_show(
    fig,
    "ch07_monthly_completed_amount_line.png",
)
```

특정 월의 금액이 증가하거나 감소했다는 사실은 관찰 결과입니다. 원인을 설명하려면 완료 주문 수, 평균 주문 금액, 카테고리 구성, 프로모션 여부 등을 추가로 확인해야 합니다.

8.3 상품 가격 분포 히스토그램

상품 가격은 상품 마스터의 분포이므로 주문 상태와 무관합니다.

```
fig, ax = plt.subplots(figsize=(10, 5))

ax.hist(
    products["price"].dropna(),
    bins=20,
    edgecolor="black",
)

ax.set_title("상품 가격 분포")
ax.set_xlabel("상품 가격")
ax.set_ylabel("상품 수")
ax.set_xlim(left=0)
ax.xaxis.set_major_formatter(money_formatter)

price_histogram_path = save_and_show(
    fig,
    "ch07_product_price_hist.png",
)
```

`bins=20`은 가격 범위를 20개 구간으로 나눈다는 뜻입니다. 구간 수를 10, 20, 30 등으로 바꾸어 분포가 어떻게 달라 보이는지 비교해 볼 수 있습니다.

8.4 상품 가격과 완료 주문 판매 수량 산점도

```
fig, ax = plt.subplots(figsize=(10, 5))

ax.scatter(
    product_sales["price"],
    product_sales["completed_quantity"],
    alpha=0.6,
)

ax.set_title(
    "상품 가격과 완료 주문 판매 수량의 관계"
)

ax.set_xlabel("상품 가격")
ax.set_ylabel("완료 주문 판매 수량")
ax.set_xlim(left=0)
ax.set_ylim(bottom=0)
ax.xaxis.set_major_formatter(money_formatter)
ax.grid(
    alpha=0.2,
)

scatter_chart_path = save_and_show(
    fig,
    "ch07_price_completed_quantity_scatter.png",
)
```

산점도는 두 변수 사이의 패턴을 탐색하는 데 사용됩니다. 가격이 높을수록 판매 수량이 줄어드는 것처럼 보여도 가격이 원인이라고 단정할 수 없습니다. 카테고리, 할인, 노출량, 재고, 계절성 등 다른 요인을 함께 확인해야 합니다.

8.5 완료 주문 구매 금액 상위 고객 가로 막대그래프

실제 고객 이름이나 고객 ID를 그래프에 노출하지 않고 순위 기반 익명 라벨을 만듭니다.

```

top_customers = (
    customer_sales
    .head(10)
    .sort_values("completed_amount")
    .copy()
)

top_customers["customer_label"] = [
    f"고객 {rank:02d}"
    for rank in range(
        len(top_customers),
        0,
        -1,
    )
]

```

익명 라벨은 그래프 표시용입니다. 실제 고객과의 대응이 필요한 내부 분석이라면 접근 권한이 제한된 별도 매핑표로 관리해야 합니다.

```

fig, ax = plt.subplots(figsize=(10, 6))

ax.barh(
    top_customers["customer_label"],
    top_customers["completed_amount"],
)

ax.set_title(
    "완료 주문 구매 금액 상위 고객"
)

ax.set_xlabel("완료 주문 구매 금액")
ax.set_ylabel("익명 고객")
ax.set_xlim(left=0)
ax.xaxis.set_major_formatter(money_formatter)

top_customer_chart_path = save_and_show(
    fig,
    "ch07_top_customers_anonymized_barh.png",
)

```

이 그래프는 구매 금액이 큰 고객군을 보여 주지만, 충성 고객을 자동으로 의미하지는 않습니다. 한 번에 크게 구매한 고객과 여러 번 반복 구매한 고객을 구분하려면 주문 횟수와 평균 주문 금액을 함께 확인해야 합니다.

8.6 주문 상태별 주문 수 막대그래프

주문 상태 분포는 완료·취소·환불을 모두 포함한 전체 주문을 기준으로 확인합니다.

```
order_status = (
    orders["order_status"]
    .fillna("missing")
    .value_counts()
    .rename_axis("order_status")
    .reset_index(name="order_count")
)

fig, ax = plt.subplots(figsize=(8, 5))

ax.bar(
    order_status["order_status"],
    order_status["order_count"],
)

ax.set_title("주문 상태별 주문 수")
ax.set_xlabel("주문 상태")
ax.set_ylabel("주문 수")
ax.set_ylim(bottom=0)
ax.tick_params(
    axis="x",
    rotation=30,
)

for label in ax.get_xticklabels():
    label.set_horizontalalignment("right")

order_status_chart_path = save_and_show(
    fig,
    "ch07_order_status_bar.png",
)
```

상태별 건수만으로 취소나 환불의 원인을 알 수는 없습니다. 기간, 결제 수단, 상품 카테고리, 고객군과의 관계를 추가로 확인해야 합니다.

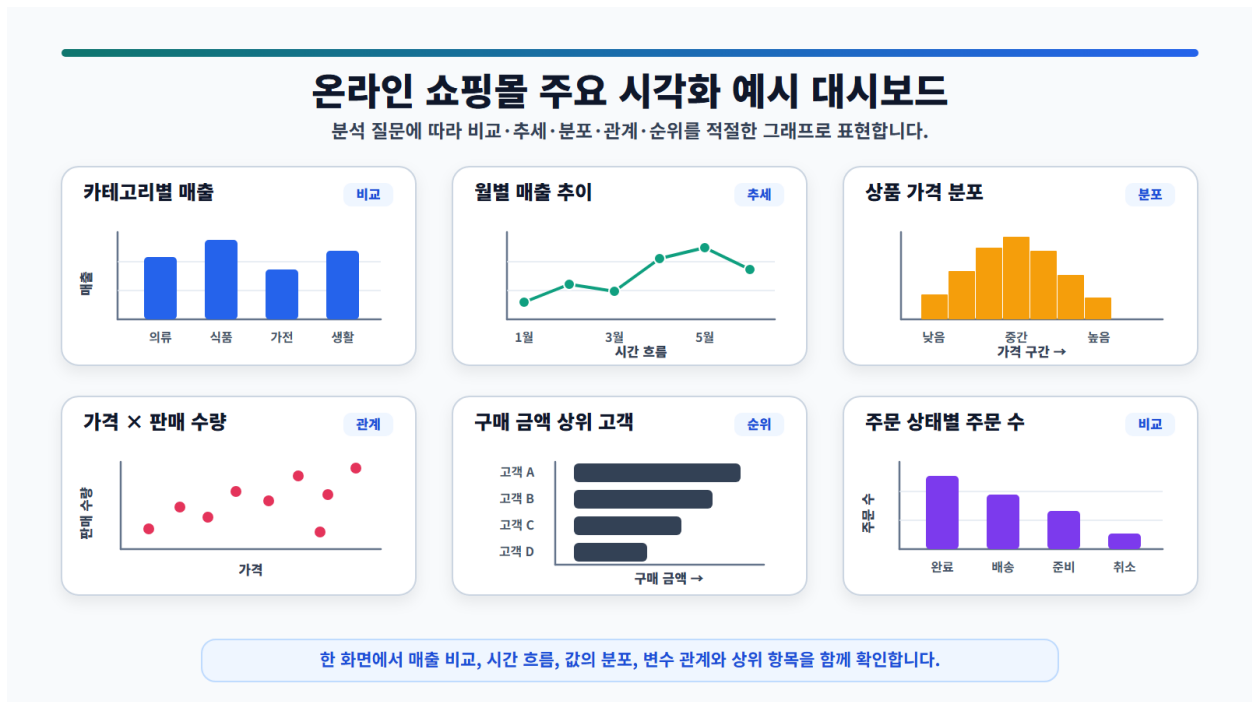


그림 7-4. 온라인 쇼핑몰 주요 시각화 예시 대시보드

9. 그래프를 보고서로 연결하기

저장된 그래프 파일을 확인합니다.

```
sorted(
    path.name
    for path in figure_dir.glob("ch07_*.png")
)
```

이번 장에서 저장한 주요 파일은 다음과 같습니다.

```
ch07_category_completed_amount_bar.png
ch07_monthly_completed_amount_line.png
ch07_product_price_hist.png
ch07_price_completed_quantity_scatter.png
ch07_top_customers_anonymized_barh.png
ch07_order_status_bar.png
```

그래프의 목적과 해석 범위를 표로 정리합니다.

```

visualization_summary = pd.DataFrame(
    {
        "chart": [
            "카테고리별 완료 주문 금액",
            "월별 완료 주문 금액",
            "상품 가격 분포",
            "가격과 완료 주문 판매 수량",
            "완료 주문 구매 금액 상위 고객",
            "주문 상태별 주문 수",
        ],
        "question": [
            "카테고리별 완료 주문 금액은 어떻게 다른가?",
            "월별 완료 주문 금액은 어떻게 변하는가?",
            "상품 가격은 어떤 구간에 몰려 있는가?",
            "가격과 완료 주문 판매 수량은 관계가 있는가?",
            "구매 금액이 큰 고객군은 어떻게 구성되는가?",
            "주문 상태별 주문 수는 어떻게 다른가?",
        ],
        "scope": [
            "completed 주문만 포함",
            "completed 주문과 유효한 날짜만 포함",
            "전체 상품 마스터",
            "completed 주문만 포함",
            "completed 주문만 포함, 고객 익명화",
            "전체 주문 상태",
        ],
        "file_name": [
            category_chart_path.name,
            monthly_chart_path.name,
            price_histogram_path.name,
            scatter_chart_path.name,
            top_customer_chart_path.name,
            order_status_chart_path.name,
        ],
    }
)

```

visualization_summary

`DataFrame.to_markdown()`은 `tabulate` 패키지를 사용합니다. 이 저장소의 `requirements.txt`에는 `tabulate`가 포함되어 있습니다. 다른 환경에서 오류가 발생하면 패키지를 설치하거나 `to_string(index=False)`로 대체할 수 있습니다.

```
try:
    summary_table = (
        visualization_summary.to_markdown(
            index=False
        )
    )
except ImportError:
    summary_table = (
        """text\\n"""
        + visualization_summary.to_string(
            index=False
        )
        + "\\n"""
    )
```

보고서를 저장합니다.

```
summary_text = f"""# Chapter 7 데이터 시각화 요약 보고서

## 1. 시각화 목적

전처리된 온라인 쇼핑몰 데이터를 사용해 비교, 추세, 분포, 관계를 그래프로 확인했습니다.

## 2. 집계 기준

- 완료 주문 금액 관련 그래프는 `order_status == "completed"`인 주문만 포함했습니다.
- `line_total`은 수량과 단가를 곱한 주문 상세 금액입니다.
- 할인, 배송비, 세금, 부분 환불 정보가 없으므로 회계상 순매출과 같다고 단정하지 않습니다.
- 고객 그래프에는 실제 이름과 고객 ID를 표시하지 않았습니다.

## 3. 생성한 그래프 목록

{summary_table}

## 4. 해석 시 주의사항

- 그래프는 관찰된 패턴을 보여 주지만 원인을 자동으로 설명하지 않습니다.
- 범주별 금액 차이는 판매 수량, 단가, 상품 수를 함께 확인해야 합니다.
- 월별 변화는 완료 주문 수와 평균 주문 금액을 함께 확인해야 합니다.
- 산점도에서 관계가 보여도 인과관계라고 단정하지 않습니다.
- 축, 단위, 포함-제외 기준, 익명화 여부를 보고서에 함께 기록합니다.

## 5. 다음 단계

다음 장에서는 데이터 불러오기, 전처리, EDA, 시각화를 종합한 중간 프로젝트를 수행합니다.
"""

summary_path = (
    report_dir
    / "ch07_visualization_summary.md"
)

summary_path.write_text(
    summary_text,
    encoding="utf-8",
)

summary_path
```



그림 7-5. 시각화 결과를 보고서와 발표 자료로 연결하는 흐름

10. LLM에게 그래프 선택과 해석을 요청하기

LLM은 그래프 종류 선택, matplotlib 코드 초안 작성, 해석 문장 정리에 도움을 줄 수 있습니다. 그러나 집계 기준과 실제 수치를 검증하는 책임은 사람에게 있습니다.

원본 고객명, 고객 ID, 주문 상세 행을 승인되지 않은 외부 LLM 서비스에 입력하지 않습니다. 다음과 같은 집계 정보만 제공하는 것이 좋습니다.

- 그래프가 답하려는 질문
- 사용한 집계 기준
- 익명화된 범주명
- 집계된 값과 단위
- 제외된 데이터와 결측치
- 그래프 종류
- 추가로 확인하고 싶은 내용

그래프 선택 요청 예시

다음 분석 질문에 적합한 그래프를 추천해 주세요.

질문:

- 월별 완료 주문 금액은 어떻게 변하는가?
- 카테고리별 완료 주문 금액은 어떻게 다른가?
- 상품 가격은 어느 구간에 많이 분포하는가?
- 상품 가격과 완료 주문 판매 수량은 관계가 있는가?

각 질문마다 다음 내용을 설명해 주세요.

1. 추천 그래프
2. x축과 y축
3. 정렬 방법
4. 포함·제외 기준
5. 해석할 때 주의할 점

그래프 해석 요청 예시

다음은 완료 주문만 포함해 계산한 카테고리별 주문 상세 금액입니다.

category,completed_amount,amount_ratio_pct

전자기기,12500000,42.5

생활용품,7800000,26.5

패션,6200000,21.1

식품,2900000,9.9

보고서용 해석 초안을 작성해 주세요.

조건:

- 수치를 다시 확인해 요약할 것
- 관찰 결과와 원인 가설을 구분할 것
- 데이터에 없는 원인을 단정하지 말 것
- 이 값이 회계상 순매출이라고 단정하지 말 것
- 추가로 확인할 분석 질문을 제안할 것

잘못된 그래프 선택 검토 예시

다음 시각화 제안이 적절한지 검토해 주세요.

1. 월별 완료 주문 금액을 파이 차트로 표현
2. 상품 가격 분포를 선 그래프로 표현
3. 카테고리별 완료 주문 금액을 산점도로 표현
4. 상품 가격과 완료 주문 판매 수량을 산점도로 표현

부적절한 경우 더 적합한 그래프와 이유를 설명해 주세요.

월별 금액은 시간 흐름을 보여야 하므로 선 그래프가 적합합니다. 상품 가격 분포는 히스토그램이 적합합니다. 카테고리별 금액은 범주별 비교이므로 막대그래프가 적합합니다. 가격과 판매 수량의 관계는 두 숫자형 변수의 관계이므로 산점도가 적합합니다.

LLM이 만든 그래프 코드 검토 기준

검토 기준	확인할 질문
데이터 범위	완료·취소·환불 중 무엇을 포함했는가?
집계 단위	주문, 주문 상세, 고객, 상품 중 무엇을 세었는가?
병합 관계	many_to_one 등 예상 관계가 검증되었는가?
시간 정렬	월과 날짜가 시간 순서대로 정렬되었는가?
그래프 선택	질문의 비교·추세·분포·관계 목적과 맞는가?
축과 단위	축 이름과 금액·건수 단위가 명확한가?
축 범위	범주 비교를 과장하는 축 절단이 없는가?
결측치	그래프에서 제외된 결측치가 기록되어 있는가?
저장 순서	savefig()가 show()보다 먼저 실행되는가?
개인정보	고객 이름과 식별 가능한 값이 제거되었는가?
해석	관찰과 원인 가설을 구분했는가?

11. 그래프를 해석할 때 주의할 점

그래프는 패턴을 보여 주지만 원인을 자동으로 설명하지는 않습니다.

카테고리별 완료 주문 금액이 높다는 사실은 관찰 결과입니다. 그 이유가 상품 수가 많아서인지, 판매 수량이 많아서인지, 단가가 높아서인지는 추가 분석이 필요합니다.

월별 완료 주문 금액이 증가하거나 감소했다면 다음을 함께 확인합니다.

- 완료 주문 수
- 주문당 평균 금액
- 카테고리 구성
- 신규·기존 고객 비중
- 데이터 수집 누락 여부
- 프로모션이나 가격 정책 변화

상품 가격 히스토그램에서 고가 상품이 일부 보인다면 실제 고가 상품군인지 입력 오류인지 확인합니다.

산점도에서 가격과 판매 수량 사이의 패턴이 보여도 가격이 판매량을 변화시켰다고 단정하지 않습니다. 카테고리별로 점의 분포가 다른지, 할인이나 노출량 같은 변수가 있는지 추가로 확인합니다.

구매 금액 상위 고객 그래프는 내부 분석에 유용할 수 있지만, 실제 이름이나 식별자를 보고서에 노출해서는 안 됩니다. 총 구매 금액뿐 아니라 주문 횟수와 평균 주문 금액도 함께 확인해야 합니다.

12. 최종 점검

점검 항목	확인
분석 질문에 맞는 그래프를 선택했는가?	☐
데이터의 포함·제외 기준을 기록했는가?	☐
완료 주문 금액과 전체 주문 상태를 구분했는가?	☐
line_total을 회계상 순매출로 단정하지 않았는가?	☐
x축과 y축 이름과 단위를 표시했는가?	☐
범주형 데이터가 읽기 좋은 순서로 정렬되었는가?	☐
시간 데이터가 올바른 순서로 정렬되었는가?	☐
막대그래프 값 축이 특별한 이유 없이 잘리지 않았는가?	☐
한글 폰트와 음수 기호가 정상적으로 표시되는가?	☐
그래프 레이블이 겹치거나 잘리지 않는가?	☐
savefig()를 show()보다 먼저 실행했는가?	☐
그래프 해석에서 관찰과 원인 가설을 구분했는가?	☐
개인정보와 식별 가능한 값을 익명화했는가?	☐

점검 항목	확인
LLM이 제안한 코드와 해석을 실제 데이터로 검증했는가?	☐

직접 더 연습하려면 다음을 수행해 봅니다.

- 카테고리별 완료 주문 금액과 완료 주문 판매 수량을 각각 시각화합니다.
- 월별 완료 주문 수와 월별 완료 주문 금액을 비교합니다.
- 히스토그램의 bins를 10, 20, 30으로 바꾸어 분포를 비교합니다.
- 상품 가격과 완료 주문 판매 수량 산점도를 카테고리별로 나누어 살펴봅니다.
- 익명화한 고객 그래프에 주문 횟수 정보를 함께 제시합니다.
- 생성한 그래프를 reports/figures 폴더에 저장하고 보고서에 포함·제외 기준을 기록합니다.
- LLM에게 그래프 해석을 요청한 뒤 데이터에 없는 원인이나 과장된 표현이 포함되었는지 검토합니다.

이번 장에서는 전처리 결과를 바탕으로 데이터 시각화를 수행했습니다. 막대그래프는 범주별 크기를 비교할 때, 선 그래프는 시간에 따른 변화를 확인할 때, 히스토그램은 숫자형 데이터의 분포를 볼 때, 산점도는 두 숫자형 변수의 관계를 탐색할 때 사용합니다.

좋은 시각화는 단순히 보기 좋은 그래프가 아닙니다. 분석 질문, 데이터 범위, 축과 단위, 개인정보 보호, 해석의 한계가 모두 분명해야 합니다. 다음 장에서는 데이터 불러오기, 전처리, EDA, 시각화를 종합한 중간 프로젝트를 수행합니다.

8장. 작은 데이터 분석 프로젝트 완성하기

지금까지는 데이터 분석의 각 단계를 나누어 살펴보았습니다. 데이터를 불러오고 구조를 확인하고, pandas로 집계하고, 전처리하고, EDA 질문을 만들고, 시각화까지 진행했습니다. 이 장에서는 이 과정을 하나의 재현 가능한 프로젝트로 연결합니다.

온라인 쇼핑몰의 고객, 상품, 주문, 주문 상세 데이터를 사용해 기본 운영 현황을 분석합니다. 복잡한 머신러닝 모델을 만드는 것이 목표는 아닙니다. **질문 설정 → 데이터 점검 → 전처리 → 안전한 병합 → 완료 주문 기준 집계 → 시각화 → 해석 → 보고서 작성 → 검증**의 흐름을 완성하는 것이 핵심입니다.



그림 8-1. 중간 프로젝트 전체 흐름도

1. 프로젝트 목표와 분석 기준

이번 프로젝트에서는 다음 질문에 답합니다.

분석 질문	주요 지표	결과 형태
카테고리별 완료 주문 매출은 어떻게 다른가?	판매 수량, 매출, 매출 비중	집계표, 막대그래프

분석 질문	주요 지표	결과 형태
월별 완료 주문 매출과 주문 수는 어떻게 변하는가?	월별 매출, 주문 수, 평균 주문 금액	집계표, 선그래프
완료 주문 기준 구매 금액 상위 고객은 누구인가?	총 구매 금액, 주문 횟수, 평균 주문 금액	익명화된 집계표, 막대그래프
주문 상태별 주문 수는 어떻게 분포하는가?	상태별 주문 수와 비율	요약표

이 장에서 **매출**은 별도 설명이 없는 한 `order_status == "completed"`인 주문의 금액을 뜻합니다. 취소 또는 환불 주문을 포함한 금액은 확정 매출과 구분해 **전체 주문 상세 금액**으로 표현합니다.

현재 데이터만으로 고객 만족도, 광고 효과, 재고 부족, 이탈 이유를 단정할 수는 없습니다. 분석 질문은 현재 데이터로 계산할 수 있는 지표와 연결되어야 합니다.

2. 재현 가능한 분석 파이프라인

프로젝트형 분석은 다음 단계를 하나의 흐름으로 연결합니다.

단계	수행 내용	주요 산출물
질문 정리	현재 데이터로 답할 질문을 정함	분석 질문 목록
데이터 확인	파일, 컬럼, 타입, 결측치, 중복 확인	데이터 구조 요약
전처리	원본을 보존하고 분석 가능한 형태로 정리	clean CSV
관계 점검	기본키 중복과 외래키 미매칭 확인	관계 점검표
안전한 병합	validate, indicator로 병합 검증	분석용 DataFrame
지표 계산	완료 주문 기준 집계	결과 CSV
시각화	질문에 맞는 그래프 생성	PNG 이미지
해석	관찰, 한계, 다음 질문 정리	해석 메모
보고서	분석 흐름과 결과를 문서화	Markdown 보고서
최종 검증	산출물 존재와 개인정보 노출 점검	제출 체크리스트



그림 8-2. 중간 프로젝트 분석 파이프라인

재현 가능성을 높이려면 실행 위치, 전처리 기준, 병합 관계, 매출 정의, 저장 파일명을 명확히 해야 합니다. Notebook 화면에서 결과를 확인하고 끝내지 말고 주요 표와 그래프를 파일로 남깁니다.

3. 보고서는 코드의 설명서가 아니다

분석 보고서는 코드를 줄 단위로 설명하는 문서가 아닙니다. 읽는 사람이 분석 목적, 데이터, 처리 기준, 주요 결과, 해석, 한계를 이해할 수 있도록 정리해야 합니다.

섹션	포함 내용
분석 목적	무엇을 확인하려는지 설명
데이터 개요	사용 파일, 행·열 수, 주요 컬럼
분석 기준	완료 주문만 매출에 포함한다는 기준
전처리 내용	결측치, 중복, 타입 변환, 이상값 처리
관계와 병합 검증	키 중복, 미매칭, 병합 전후 행 수
주요 결과	표와 그래프
결과 해석	관찰과 추가 확인 사항
한계점	현재 데이터로 알 수 없는 내용
다음 단계	후속 분석 또는 개선 방향



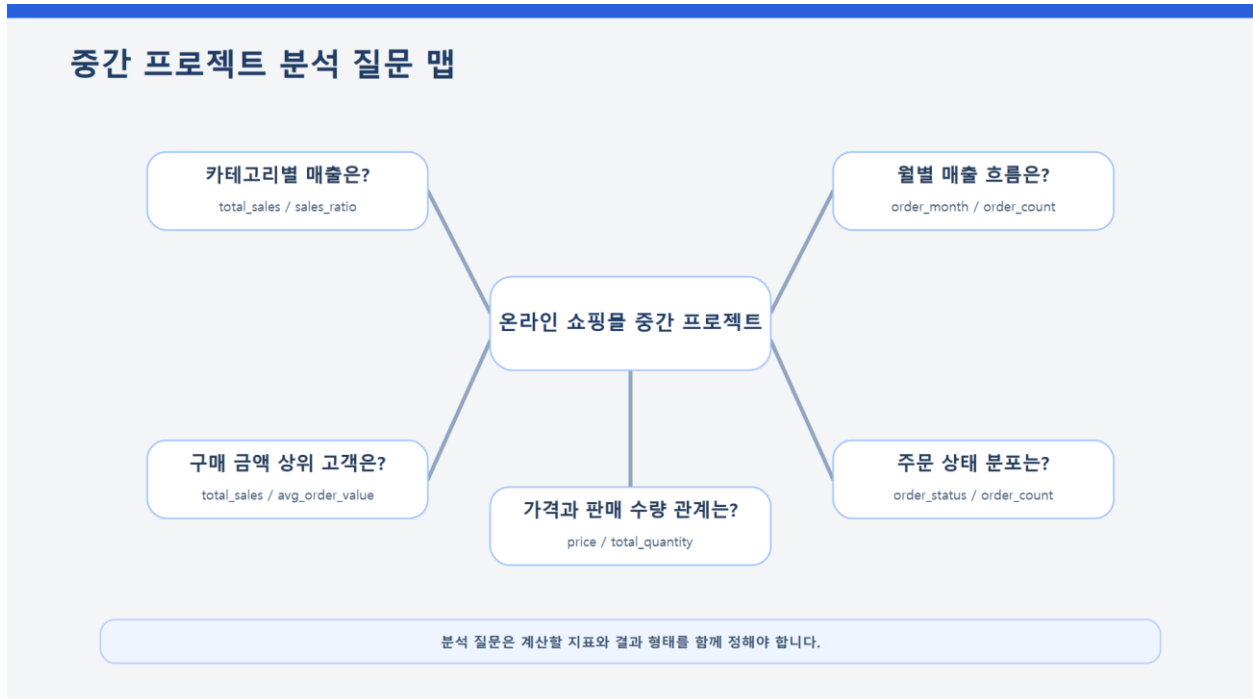


그림 8-4. 중간 프로젝트 분석 질문 맵

예를 들어 “매출이 높은 카테고리는 무엇인가?”라는 질문은 `category`, `quantity`, `line_total`이 필요합니다. “인기 있는 카테고리는 무엇인가?”라는 질문은 매출뿐 아니라 판매 수량, 구매 고객 수, 반복 구매 여부를 함께 정의해야 합니다.

5. 프로젝트 작업 공간 준비하기

전체 실습은 `notebooks/ch08_midterm_project.ipynb`에서 진행하며, 전체 파이프라인은 `python scripts/run_midterm_project.py`로 다시 실행할 수 있습니다.

현재 작업 폴더가 프로젝트 루트 또는 `notebooks` 폴더일 수 있으므로 상위 폴더를 탐색합니다.

```

from pathlib import Path

def find_project_root(start_path):
    start_path = Path(start_path).resolve()

    for candidate in [start_path, *start_path.parents]:
        if (
            (candidate / "requirements.txt").exists()
            and (candidate / "scripts").exists()
        ):
            return candidate

    raise FileNotFoundError(
        "프로젝트 루트 폴더를 찾을 수 없습니다."
    )

PROJECT_ROOT = find_project_root(Path.cwd())
RAW_DIR = PROJECT_ROOT / "data" / "raw"
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
REPORT_DIR = PROJECT_ROOT / "reports"
FIGURE_DIR = REPORT_DIR / "figures"

for path in [PROCESSED_DIR, REPORT_DIR, FIGURE_DIR]:
    path.mkdir(parents=True, exist_ok=True)

```

필요한 패키지는 개별 설치보다 저장소 기준으로 한 번에 설치합니다.

```
python -m pip install -r requirements.txt
```

6. 원본 데이터와 필수 컬럼 확인하기

필요한 파일이 모두 존재하는지 확인한 뒤 불러옵니다.

```
import pandas as pd

required_files = [
    "customers.csv",
    "products.csv",
    "orders.csv",
    "order_items.csv",
]

missing_files = [
    filename
    for filename in required_files
    if not (RAW_DIR / filename).exists()
]

if missing_files:
    raise FileNotFoundError(
        "필요한 파일이 없습니다: "
        + ", ".join(missing_files)
        + ". 프로젝트 루트에서 "
        + "python scripts/generate_sample_data.py를 실행하세요."
    )

customers = pd.read_csv(RAW_DIR / "customers.csv")
products = pd.read_csv(RAW_DIR / "products.csv")
orders = pd.read_csv(RAW_DIR / "orders.csv")
order_items = pd.read_csv(RAW_DIR / "order_items.csv")
```

이번 프로젝트에서 필요한 컬럼도 확인합니다.


```

datasets = {
    "customers": customers,
    "products": products,
    "orders": orders,
    "order_items": order_items,
}

required_columns = {
    "customers": ["customer_id", "age", "city"],
    "products": [
        "product_id",
        "product_name",
        "category",
        "price",
    ],
    "orders": [
        "order_id",
        "customer_id",
        "order_date",
        "order_status",
    ],
    "order_items": [
        "order_item_id",
        "order_id",
        "product_id",
        "quantity",
        "unit_price",
    ],
}

for name, columns in required_columns.items():
    missing = [
        column
        for column in columns
        if column not in datasets[name].columns
    ]
    if missing:
        raise KeyError(
            f"{name}에 필요한 컬럼이 없습니다: {missing}"
        )

```

행·열 수, 결측치, 전체 행 중복을 보고서용 표로 저장합니다.

```
dataset_summary = pd.DataFrame([
    {
        "dataset": name,
        "rows": df.shape[0],
        "columns": df.shape[1],
        "missing_values": int(df.isna().sum().sum()),
        "duplicated_rows": int(df.duplicated().sum()),
    }
    for name, df in datasets.items()
])
```

7. 원본을 보존하며 전처리하기

전처리는 원본 DataFrame을 직접 수정하지 않고 복사본을 사용합니다. 이 저장소에서는 `src.preprocessing`의 공통 함수를 이용할 수 있습니다.

```
from src.preprocessing import (
    compare_shapes,
    preprocess_sales_data,
    save_processed_data,
    validate_relationships,
)

processed_data = preprocess_sales_data(datasets)
save_processed_data(processed_data, PROCESSED_DIR)

preprocessing_comparison = compare_shapes(
    datasets,
    processed_data,
)
relationship_checks = validate_relationships(
    processed_data,
)
```

전처리 과정에서는 다음 항목을 확인합니다.

- 숫자형 변환 실패 건수
- 날짜형 변환 실패 건수
- 가격·수량·단가가 0 이하인 행
- 전체 행 중복과 주요 키 중복
- 전처리 전후 행 수 변화

- 파일 간 외래키 미매칭

단순히 `errors="coerce"`를 사용하고 끝내지 말고, 변환 실패로 생긴 결측치의 개수와 처리 기준을 기록해야 합니다.

8. 키 중복과 병합 검증하기

관계형 데이터에서는 전체 행 중복뿐 아니라 식별 키 중복을 확인해야 합니다.

```
from src.midterm_project import build_key_duplicate_checks

key_duplicate_checks = build_key_duplicate_checks(
    processed_data
)
```

`left merge`라고 해서 왼쪽 행 수가 자동으로 유지되는 것은 아닙니다. 오른쪽 키가 중복되어 있으면 왼쪽 한 행이 여러 행으로 늘어날 수 있습니다.

```
order_sales = processed_data["order_items"].merge(
    processed_data["orders"][
        [
            "order_id",
            "customer_id",
            "order_date",
            "order_status",
        ]
    ],
    on="order_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)

print(order_sales["_merge"].value_counts())
```

- `validate="many_to_one"`은 주문 상세의 `order_id`는 여러 번 나올 수 있지만 주문 테이블의 `order_id`는 한 번만 나와야 한다는 의미입니다.
- `indicator=True`는 각 행이 양쪽 데이터에서 매칭되었는지 확인할 수 있는 `_merge` 컬럼을 추가합니다.
- 병합 전후 행 수와 `left_only` 건수를 함께 확인합니다.

9. 완료 주문 기준 분석 데이터 만들기

주문 상세 금액을 만든 뒤 주문 상태를 연결합니다.

```
order_items_clean = processed_data["order_items"].copy()

if "line_total" not in order_items_clean.columns:
    order_items_clean["line_total"] = (
        order_items_clean["quantity"]
        * order_items_clean["unit_price"]
    )
```

완료 주문만 분석 대상으로 선택합니다.

```
order_sales = order_items_clean.merge(
    processed_data["orders"][
        [
            "order_id",
            "customer_id",
            "order_date",
            "order_month",
            "order_status",
        ]
    ],
    on="order_id",
    how="left",
    validate="many_to_one",
)

completed_order_sales = order_sales[
    order_sales["order_status"] == "completed"
].copy()
```

전체 주문 상세 금액과 완료 주문 매출을 구분해 비교합니다.

```
amount_scope_summary = pd.DataFrame({
    "scope": [
        "전체 주문 상세 금액",
        "완료 주문 매출",
        "취소·환불 등 제외 금액",
    ],
    "amount": [
        order_sales["line_total"].sum(),
        completed_order_sales["line_total"].sum(),
        (
            order_sales["line_total"].sum()
            - completed_order_sales["line_total"].sum()
        ),
    ],
})
```

10. 주요 지표 계산하기

공통 분석 함수로 완료 주문 기준 결과표를 생성합니다.

```
from src.midterm_project import build_analysis_tables

analysis_tables = build_analysis_tables(processed_data)

category_sales = analysis_tables["category_sales"]
monthly_sales = analysis_tables["monthly_sales"]
customer_sales = analysis_tables["customer_sales"]
order_status_summary = analysis_tables[
    "order_status_summary"
]
```

카테고리별 완료 주문 매출

```
category_sales
```

카테고리 매출이 높은 이유가 판매 수량 때문인지 평균 단가 때문인지는 이 표만으로 알 수 없습니다.

판매 수량과 가격 수준을 함께 확인합니다.

월별 완료 주문 매출

```
monthly_sales
```

`order_count`는 주문 상세 행 수가 아니라 고유한 주문 건수입니다. 월별 매출 변화가 주문 수 변화 때문인지 평균 주문 금액 변화 때문인지 구분합니다.

고객별 완료 주문 구매 금액

```
customer_sales[
  [
    "customer_label",
    "city",
    "order_count",
    "total_sales",
    "avg_order_value",
  ]
].head(10)
```

고객 이름은 결과 CSV와 보고서에 포함하지 않습니다. `customer_id` 기반 익명화 라벨을 사용하고, 원본 고객 정보를 외부 LLM에 전달하지 않습니다.

주문 상태별 주문 수

```
order_status_summary
```

주문 상태 분포는 전체 주문을 대상으로 계산합니다. 매출표의 완료 주문 기준과 분석 범위가 다르다는 점을 보고서에 명시합니다.

11. 결과 합계 일치 검증하기

카테고리별, 월별, 고객별 매출은 모두 같은 완료 주문 상세 데이터를 사용하므로 합계가 일치해야 합니다.

```

completed_total = analysis_tables[
    "completed_order_sales"
][["line_total"]].sum()

category_total = category_sales["total_sales"].sum()
monthly_total = monthly_sales["total_sales"].sum()
customer_total = customer_sales["total_sales"].sum()

assert (
    completed_total
    == category_total
    == monthly_total
    == customer_total
)

```

합계가 다르면 필터링 범위, 병합 누락, 중복 키, 그룹화 기준을 다시 확인합니다.

12. 시각화와 해석 메모 만들기

그래프 제목에도 완료 주문 기준임을 표시합니다.

```

from src.midterm_project import (
    build_interpretation_notes,
    create_project_figures,
)

interpretation_notes = build_interpretation_notes()

create_project_figures(
    analysis_tables,
    FIGURE_DIR,
    show=True,
)

```

해석은 다음 세 부분으로 나누어 작성합니다.

구분	작성 내용
관찰	데이터와 그래프에서 직접 확인한 사실
주의점	현재 결과만으로 단정할 수 없는 내용
다음 질문	추가로 확인할 지표나 데이터

예를 들어 “특정 월의 매출이 증가했다”는 관찰이지만, “프로모션 때문에 증가했다”는 현재 데이터만으로 확인할 수 없는 가설입니다.

13. 결과 파일과 보고서 저장하기

결과표와 보고서는 공통 함수를 사용해 저장합니다.

```
from src.midterm_project import (
    build_midterm_report,
    save_project_tables,
)

saved_tables = save_project_tables(
    dataset_summary,
    preprocessing_comparison,
    key_duplicate_checks,
    relationship_checks,
    analysis_tables,
    interpretation_notes,
    REPORT_DIR,
)

report_text = build_midterm_report(
    dataset_summary,
    preprocessing_comparison,
    key_duplicate_checks,
    relationship_checks,
    analysis_tables,
    interpretation_notes,
)

report_path = REPORT_DIR / "ch08_midterm_report.md"
report_path.write_text(report_text, encoding="utf-8")
```

보고서에는 다음 기준을 명시합니다.

- 매출은 완료 주문만 포함
- 취소·환불 주문을 포함한 금액과 완료 주문 매출을 구분
- 키 중복, 외래키 미매칭, 병합 전후 행 수 확인
- 고객 이름을 제외하고 익명화 라벨 사용

- 관찰과 원인 가설을 구분
- Notebook, CSV, 그래프, 보고서의 수치 일치 확인

14. LLM은 검토 파트너로 활용하기

LLM에는 실제 고객명이나 원본 행을 전달하지 않고 구조와 요약 결과만 제공합니다.

온라인 쇼핑물 중간 프로젝트 결과를 검토해 주세요.

분석 기준:

- 매출은 order_status가 completed인 주문만 포함
- 고객 이름은 제외하고 익명화 라벨 사용
- 병합은 validate와 indicator로 검증

검토 항목:

1. 분석 질문과 지표가 연결되어 있는가?
2. 취소/환불 주문이 매출에 포함되지 않았는가?
3. 병합 후 행 증가 또는 미매칭을 확인했는가?
4. 관찰과 원인 가설을 구분했는가?
5. 개인정보가 불필요하게 노출되지 않았는가?
6. Notebook, CSV, 그래프, 보고서의 수치가 일치하는가?

데이터에 없는 원인은 단정하지 말고,
필수 수정과 권장 개선을 구분해 주세요.

LLM의 제안은 실제 컬럼, 값의 종류, 집계 기준, 실행 결과와 비교해 검증합니다.

15. 제출물과 최종 검증

최종 산출물은 다음과 같습니다.

- notebooks/ch08_midterm_project.ipynb
- data/processed/customers_clean.csv
- data/processed/products_clean.csv
- data/processed/orders_clean.csv
- data/processed/order_items_clean.csv
- reports/ch08_midterm_report.md
- reports/ch08_dataset_summary.csv
- reports/ch08_preprocessing_comparison.csv
- reports/ch08_key_duplicate_checks.csv

- reports/ch08_relationship_checks.csv
- reports/ch08_merge_checks.csv
- reports/ch08_amount_scope_summary.csv
- reports/ch08_category_sales.csv
- reports/ch08_monthly_sales.csv
- reports/ch08_customer_sales.csv
- reports/ch08_order_status_summary.csv
- reports/figures/ch08_category_sales.png
- reports/figures/ch08_monthly_sales.png
- reports/figures/ch08_top_customers.png



그림 8-5. 중간 프로젝트 산출물 구성

제출 전에는 다음을 확인합니다.

검증 항목	확인 기준
실행 가능성	위에서부터 다시 실행해 오류 없이 완료됨
매출 기준	완료 주문만 카테고리·월별·고객별 매출에 포함
병합 검증	키 중복, 미매칭, 병합 전후 행 수 확인
개인정보	고객 이름·이메일 등 불필요한 정보가 결과에 없음
결과 일치	Notebook, CSV, 그래프, 보고서의 수치가 일치
해석 품질	관찰과 원인 가설이 구분됨

검증 항목	확인 기준
재현성	샘플 데이터 생성부터 전체 실행 순서가 기록됨
산출물	필수 파일이 모두 존재하고 비어 있지 않음

전체 파이프라인은 프로젝트 루트에서 다음 명령으로 확인할 수 있습니다.

```
python scripts/generate_sample_data.py
python scripts/run_midterm_project.py
```

16. 다음 장으로 이어지는 흐름

중간 프로젝트까지는 데이터를 불러오고, 전처리하고, 안전하게 병합하고, 질문에 맞는 지표를 계산하고, 시각화와 보고서로 정리하는 기본 흐름을 다루었습니다.

다음 장부터는 머신러닝으로 확장합니다. 그러나 머신러닝도 좋은 데이터, 명확한 목표 변수, 적절한 평가 기준에서 출발합니다. 이번 프로젝트에서 익힌 원본 보존, 데이터 품질 점검, 완료 주문 기준, 병합 검증, 결과 해석 습관은 이후 모델링에서도 그대로 이어집니다.

9장. 회귀 분석으로 숫자 예측하기

지금까지는 데이터를 불러오고, 정리하고, 질문을 만들고, 그래프로 확인하는 과정을 살펴보았습니다. 이러한 과정은 주로 **무슨 일이 있었는가**를 이해하는 데 초점을 둡니다. 이제 한 걸음 더 나아가, 이미 관찰한 데이터의 패턴을 바탕으로 새로운 숫자 값을 추정하는 회귀 모델링을 살펴보겠습니다.

회귀는 금액, 수량, 점수, 온도처럼 연속적인 숫자 값을 예측하는 머신러닝 문제입니다. 그러나 회귀 모델을 만드는 목적은 단순히 높은 점수를 얻는 것이 아닙니다. **예측 시점을 먼저 정하고, 그 시점에 실제로 사용할 수 있는 정보만 입력값으로 선택한 뒤, 단순한 기준보다 모델이 더 유용한지 검증하는 것이 중요합니다.**

이 장에서는 온라인 쇼핑몰 데이터로 주문 금액을 예측하는 교육용 예제를 다룹니다. 주문 상세 금액은 수량과 단가로 직접 계산되므로, 같은 주문에서 만든 `total_quantity`, `avg_unit_price`, `line_total` 같은 값을 입력값으로 사용하면 목표값을 사실상 미리 알려 주는 문제가 생깁니다. 따라서 이번 실습에서는 주문 상세 정보는 목표값을 만드는 데만 사용하고, 모델 입력에는 주문 시점·결제 수단·고객의 비식별 특성만 사용합니다.

이 예제는 실제 운영 모델을 완성하는 것이 아니라, 데이터 누수를 피하고 회귀 모델을 정직하게 평가하는 과정을 익히기 위한 실습입니다. 현재 가상 데이터는 변수 사이의 예측 패턴이 강하게 설계되어 있지 않으므로, 복잡한 모델도 단순 평균 기준보다 크게 좋아지지 않을 수 있습니다. 낮은 성능 역시 중요한 분석 결과입니다.



그림 9-1. 회귀 분석 전체 흐름도

이 장에서 생각해 볼 질문

- 어떤 숫자를 예측하려고 하는가?
- 모델은 어느 시점에 예측을 수행한다고 가정하는가?
- 예측 시점에 실제로 알 수 있는 입력값은 무엇인가?
- 목표값이나 목표값의 계산 재료가 입력값에 섞이지 않았는가?
- 데이터를 시간 순서대로 나누어야 하는 이유는 무엇인가?
- 모델은 단순 평균 예측보다 실제로 나은가?
- MAE, RMSE, R^2 는 각각 무엇을 알려 주는가?
- 훈련 성능과 테스트 성능의 차이는 과적합을 나타내는가?
- 성능이 낮을 때 모델을 억지로 운영에 사용하지 않을 수 있는가?
- LLM이 작성한 회귀 코드는 어떤 기준으로 검증해야 하는가?

1. 회귀 문제는 예측 시점부터 정의한다

회귀는 연속적인 숫자 값을 예측합니다. 범주를 예측하는 분류와 달리, 회귀의 결과는 금액이나 수량처럼 숫자로 표현됩니다.

온라인 쇼핑몰에서는 다음과 같은 회귀 문제를 생각할 수 있습니다.

예측 질문	예측 대상	예측 시점에 사용할 수 있는 정보 예시
새 주문의 금액은 어느 정도일까요?	주문 상세 금액 합계	주문 시점, 결제 수단, 고객 연령·지역
고객의 다음 30일 구매 금액은 얼마일까요?	미래 30일 구매 금액	예측 기준일 이전 구매 이력
상품의 다음 달 판매 수량은 얼마일까요?	다음 달 판매 수량	이전 기간 판매량, 가격, 카테고리
다음 달 완료 주문 금액은 얼마일까요?	다음 달 완료 주문 금액	이전 월 금액, 주문 수, 계절 정보

같은 목표값이라도 **언제 예측하는가**에 따라 사용할 수 있는 입력값이 달라집니다. 예를 들어 주문이 모두 완료된 뒤 주문 상세를 집계할 수 있다면 주문 금액은 예측할 필요 없이 계산하면 됩니다. 반대로 주문 상세 처리가 지연되어 주문 메타데이터만 먼저 도착하는 상황이라면, 주문 금액을 미리 추정하는 교육용 문제를 만들 수 있습니다.

이번 장에서는 다음과 같이 문제를 정의합니다.

- 예측 대상:
 - 한 주문의 주문 상세 금액 합계(order_total)
- 교육용 예측 시점:
 - 주문 메타데이터와 고객 특성은 확인할 수 있지만 주문 상세의 수량·단가·금액은 모델에 제공하지 않는 시점
- 사용할 입력값:
 - payment_method
 - order_month
 - order_dayofweek
 - gender
 - age
 - city
- 입력에서 제외할 값:
 - order_id, customer_id

- order_status
- product_id
- quantity, unit_price, line_total
- item_count, total_quantity, avg_unit_price

order_id와 customer_id는 식별자이므로 일반적인 숫자 변수처럼 사용하지 않습니다. order_status는 주문 처리 이후에 확정될 수 있으므로 예측 시점 이후 정보가 될 가능성이 있습니다. 주문 상세에서 계산한 수량·단가·금액 관련 값은 목표값과 직접 연결되므로 입력에서 제외합니다.

이 문제는 실제 서비스 배포를 위한 완성된 예측 문제라기보다, **입력값의 가용 시점과 데이터 누수**를 판단하는 연습입니다.

2. 데이터 누수는 정답 컬럼만의 문제가 아니다

데이터 누수(data leakage)는 모델이 실제 예측 시점에는 사용할 수 없는 정보를 훈련 과정에서 미리 보는 문제입니다.

가장 명확한 누수는 목표값 자체를 입력에 넣는 것입니다.

잘못된 예시

```
X = model_data[["order_total", "age", "city"]]
y = model_data["order_total"]
```

하지만 목표값의 계산 재료나 결과 이후에 만들어지는 컬럼도 누수가 될 수 있습니다.

입력값 후보	사용 여부	이유
order_total	제외	예측 대상 자체입니다.
line_total	제외	주문 금액을 구성하는 직접 계산값입니다.
quantity, unit_price	제외	목표값 계산에 직접 사용됩니다.
total_quantity, avg_unit_price	제외	같은 주문 상세에서 만든 목표값의 대리 변수입니다.
item_count	제외	주문 상세가 모두 확인된 뒤 계산되는 값입니다.
order_status	제외	예측 시점 이후에 확정될 수 있습니다.
order_id, customer_id	제외	식별자를 외우거나 특정 고객을 암기할 위험이 있습니다.
payment_method	사용	이 실습에서 예측 시점에 이미 확인된다고 가정합니다.
주문 월·요일	사용	주문일에서 만들 수 있는 시점 정보입니다.
연령·성별·지역	사용	고객 데이터에 있는 비식별 특성입니다.

여기서 중요한 기준은 “이 컬럼이 데이터 파일에 존재하는가?”가 아니라, **실제 예측 시점에 이 값을 사용할 수 있는가?**입니다.



그림 9-2. 입력값과 예측 대상 분리 개념도

3. 회귀 평가 지표와 베이스라인

회귀 모델은 한 가지 지표만으로 판단하지 않습니다. 이번 장에서는 MAE, RMSE, R²를 함께 확인합니다.

지표	의미	해석할 때 주의할 점
MAE	평균 절대 오차	목표값과 같은 단위로 평균적인 오차 크기를 보여 줍니다.
RMSE	평균 제곱근 오차	큰 오차에 더 민감합니다. MAE보다 훨씬 크면 일부 큰 오차를 확인합니다.
R ²	결정계수	기준 평균 예측과 비교해 변동을 얼마나 설명하는지 봅니다. 음수가 될 수도 있습니다.

주문 금액의 단위가 원이라면 MAE와 RMSE도 원 단위입니다. 예를 들어 MAE가 120,000이라면 예측값과 실제값이 평균적으로 약 12만 원 차이 난다고 해석할 수 있습니다.

R²는 보통 1에 가까울수록 좋지만 반드시 0과 1 사이에 있는 것은 아닙니다.

- $R^2 = 1$: 테스트 데이터를 완벽하게 예측했습니다.
- $R^2 = 0$: 테스트 데이터의 평균값을 예측하는 것과 비슷합니다.
- $R^2 < 0$: 단순 평균 예측보다도 성능이 낮습니다.

모델의 성능을 판단하려면 **베이스라인(baseline)**이 필요합니다. 이번 장에서는 훈련 데이터의 평균을 모든 테스트 주문에 예측하는 `DummyRegressor`를 기준 모델로 사용합니다. 복잡한 모델이 베이스라인보다 낮지 않다면, 현재 입력값만으로는 목표값을 충분히 예측하기 어렵다는 뜻일 수 있습니다.

4. 선형 회귀와 랜덤 포레스트 회귀

이번 장에서는 베이스라인과 함께 두 가지 회귀 모델을 비교합니다.

모델	역할	장점	주의할 점
DummyRegressor	단순 기준	모델이 최소한 넘어야 할 기준을 제공합니다.	입력값의 관계를 학습하지 않습니다.
LinearRegression	선형 모델	구조가 단순하고 비교적 해석하기 쉽습니다.	복잡한 비선형 관계를 표현하기 어렵습니다.
RandomForestRegressor	트리 앙상블	비선형 관계와 변수 간 상호작용을 표현할 수 있습니다.	과적합 가능성이 있고 해석이 더 어렵습니다.

복잡한 모델이 항상 더 좋은 것은 아닙니다. 입력값에 예측 정보가 거의 없다면 랜덤 포레스트도 베이스라인보다 나아지지 않을 수 있습니다. 반대로 훈련 성능만 지나치게 높고 테스트 성능이 낮다면 과적합을 의심해야 합니다.

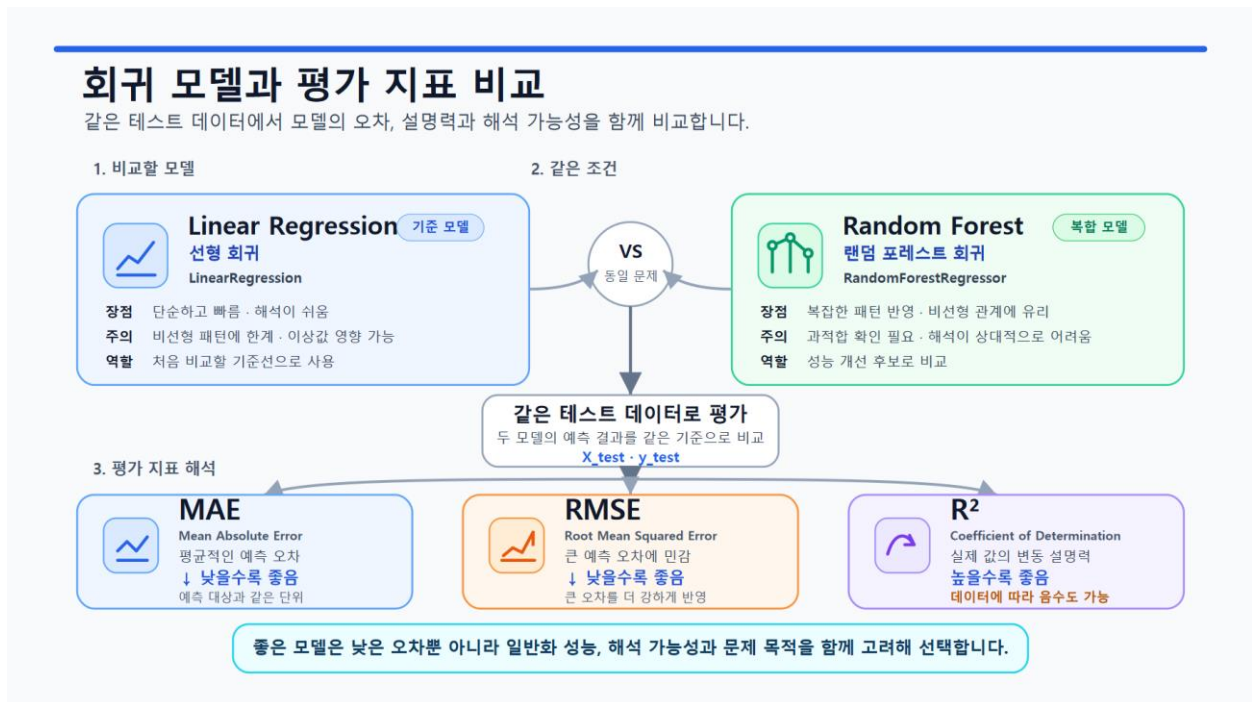


그림 9-3. 회귀 모델과 평가 지표 비교

5. 실습 환경과 데이터 불러오기

이번 장의 전체 코드는 `notebooks/ch09_regression_analysis.ipynb`에서 실행할 수 있습니다.

먼저 필요한 패키지를 불러옵니다.

```

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)
from sklearn.model_selection import (
    TimeSeriesSplit,
    cross_validate,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    OneHotEncoder,
    StandardScaler,
)

```

프로젝트 루트와 `notebooks` 폴더에서 실행하는 경우를 모두 처리합니다.

```

project_root = Path.cwd()

if project_root.name == "notebooks":
    project_root = project_root.parent

processed_dir = project_root / "data" / "processed"
report_dir = project_root / "reports"
figure_dir = report_dir / "figures"

report_dir.mkdir(parents=True, exist_ok=True)
figure_dir.mkdir(parents=True, exist_ok=True)

print("프로젝트 루트:", project_root.resolve())
print("전처리 데이터:", processed_dir.resolve())

```

5장에서 저장한 전처리 파일이 모두 존재하는지 확인합니다.

```
required_files = [  
    "customers_clean.csv",  
    "orders_clean.csv",  
    "order_items_clean.csv",  
]  
  
for file_name in required_files:  
    file_path = processed_dir / file_name  
    print(f"{file_name}: {file_path.exists()}")
```

하나라도 False라면 5장의 전처리 실습을 먼저 실행합니다. 원본 데이터로 바로 대체하면 5장에서 적용한 숫자 변환, 이상값 처리, 키 검증 기준이 달라질 수 있으므로 이 장에서는 전처리 파일만 사용합니다.

```
customers = pd.read_csv(  
    processed_dir / "customers_clean.csv"  
)  
orders = pd.read_csv(  
    processed_dir / "orders_clean.csv"  
)  
order_items = pd.read_csv(  
    processed_dir / "order_items_clean.csv"  
)
```

필요한 컬럼을 확인합니다.

```

required_columns = {
    "customers": {
        "customer_id",
        "gender",
        "age",
        "city",
    },
    "orders": {
        "order_id",
        "customer_id",
        "order_date",
        "payment_method",
        "order_status",
    },
    "order_items": {
        "order_id",
        "quantity",
        "unit_price",
    },
}

datasets = {
    "customers": customers,
    "orders": orders,
    "order_items": order_items,
}

for name, columns in required_columns.items():
    missing_columns = columns - set(
        datasets[name].columns
    )

    if missing_columns:
        raise KeyError(
            f"{name}에 필요한 컬럼이 없습니다: "
            f"{sorted(missing_columns)}"
        )

```

`line_total`이 없다면 수량과 단가로 다시 만듭니다. 이 컬럼은 목표값 생성에만 사용하며 모델 입력에는 넣지 않습니다.

```

if "line_total" not in order_items.columns:
    order_items["line_total"] = (
        pd.to_numeric(
            order_items["quantity"],
            errors="coerce",
        )
        * pd.to_numeric(
            order_items["unit_price"],
            errors="coerce",
        )
    )

```

6. 주문 단위 목표값 만들기

주문 상세는 한 주문에 여러 행이 있을 수 있으므로 주문별 금액으로 집계합니다.

```

order_totals = (
    order_items
    .groupby("order_id", as_index=False)
    .agg(
        order_total=("line_total", "sum"),
    )
)

```

order_total은 한 주문의 주문 상세 금액 합계입니다. 취소·환불 주문도 주문 당시의 상세 금액을 가질 수 있으므로, 이 값을 회계상의 매출이나 순매출이라고 부르지 않습니다.

주문 정보와 고객 특성을 연결합니다.

```
orders["order_date"] = pd.to_datetime(
    orders["order_date"],
    errors="coerce",
)

model_data = orders.merge(
    order_totals,
    on="order_id",
    how="inner",
    validate="one_to_one",
)

model_data = model_data.merge(
    customers[
        [
            "customer_id",
            "gender",
            "age",
            "city",
        ]
    ],
    on="customer_id",
    how="left",
    validate="many_to_one",
)
```

병합 결과를 확인합니다.

```
print("주문 수:", len(orders))
print("모델링 데이터 행 수:", len(model_data))
print(
    "주문일 변환 실패:",
    model_data["order_date"].isna().sum(),
)
print(
    "주문 금액 결측치:",
    model_data["order_total"].isna().sum(),
)
```

주문일에서 월과 요일을 만듭니다.

```
model_data["order_month"] = (
    model_data["order_date"].dt.month
)

model_data["order_dayofweek"] = (
    model_data["order_date"].dt.dayofweek
)
```

예측 대상의 분포를 확인합니다.

```
model_data["order_total"].describe()
```

주문 금액이 한쪽으로 치우쳐 있거나 매우 큰 주문이 일부 있다면 MAE와 RMSE의 차이가 커질 수 있습니다. 현재 장에서는 값을 자동으로 제거하지 않고, 5장의 전처리 기준과 실제 업무 의미를 먼저 확인합니다.

7. 입력값과 목표값 정의하기

이번 실습에서 사용할 입력값은 다음과 같습니다.

```
numeric_features = [
    "order_month",
    "order_dayofweek",
    "age",
]

categorical_features = [
    "payment_method",
    "gender",
    "city",
]

feature_columns = (
    numeric_features
    + categorical_features
)

target_column = "order_total"
```

누수 위험이 있는 컬럼이 입력에 포함되지 않았는지 확인합니다.


```

forbidden_features = {
    "order_total",
    "line_total",
    "quantity",
    "unit_price",
    "item_count",
    "total_quantity",
    "avg_unit_price",
    "order_status",
    "order_id",
    "customer_id",
}

leaked_features = (
    set(feature_columns)
    & forbidden_features
)

if leaked_features:
    raise ValueError(
        "입력값에 누수 위험 컬럼이 있습니다: "
        f"{sorted(leaked_features)}"
    )

```

날짜와 목표값이 없는 행은 사용할 수 없으므로 제외 건수를 기록합니다.

```

before_drop = len(model_data)

model_data = model_data.dropna(
    subset=[
        "order_date",
        target_column,
    ]
).copy()

print(
    "필수값 결측으로 제외된 행:",
    before_drop - len(model_data),
)

```

숫자형·범주형 입력의 결측치는 이후 파이프라인 안에서 처리합니다. 전체 데이터를 미리 대체하면 테스트 데이터의 분포가 훈련 과정에 섞일 수 있으므로, 훈련 데이터에 맞춘 대체 규칙을 모델 파이프라인 안에 넣습니다.

8. 시간 순서대로 훈련·테스트 데이터 나누기

이번 예제는 과거 주문으로 이후 주문을 예측하는 상황을 모방하기 위해 날짜 순서대로 나눕니다.

```
model_data = model_data.sort_values(
    ["order_date", "order_id"]
).reset_index(drop=True)

split_index = int(
    len(model_data) * 0.8
)

train_data = model_data.iloc[
    :split_index
].copy()

test_data = model_data.iloc[
    split_index:
].copy()
```

훈련·테스트 기간을 확인합니다.

```
print(
    "훈련 기간:",
    train_data["order_date"].min(),
    "~",
    train_data["order_date"].max(),
)

print(
    "테스트 기간:",
    test_data["order_date"].min(),
    "~",
    test_data["order_date"].max(),
)

print("훈련 행 수:", len(train_data))
print("테스트 행 수:", len(test_data))
```

입력값과 목표값을 분리합니다.

```

X_train = train_data[
    feature_columns
]

y_train = train_data[
    target_column
]

X_test = test_data[
    feature_columns
]

y_test = test_data[
    target_column
]

```

무작위 분할은 간단하지만 미래 주문이 훈련 데이터에 섞일 수 있습니다. 시간 흐름이 있는 업무 문제에서는 날짜 기준 분할이 더 현실적인 경우가 많습니다. 다만 실제 배포 목적과 데이터 생성 과정을 고려해 분할 방식을 결정해야 합니다.

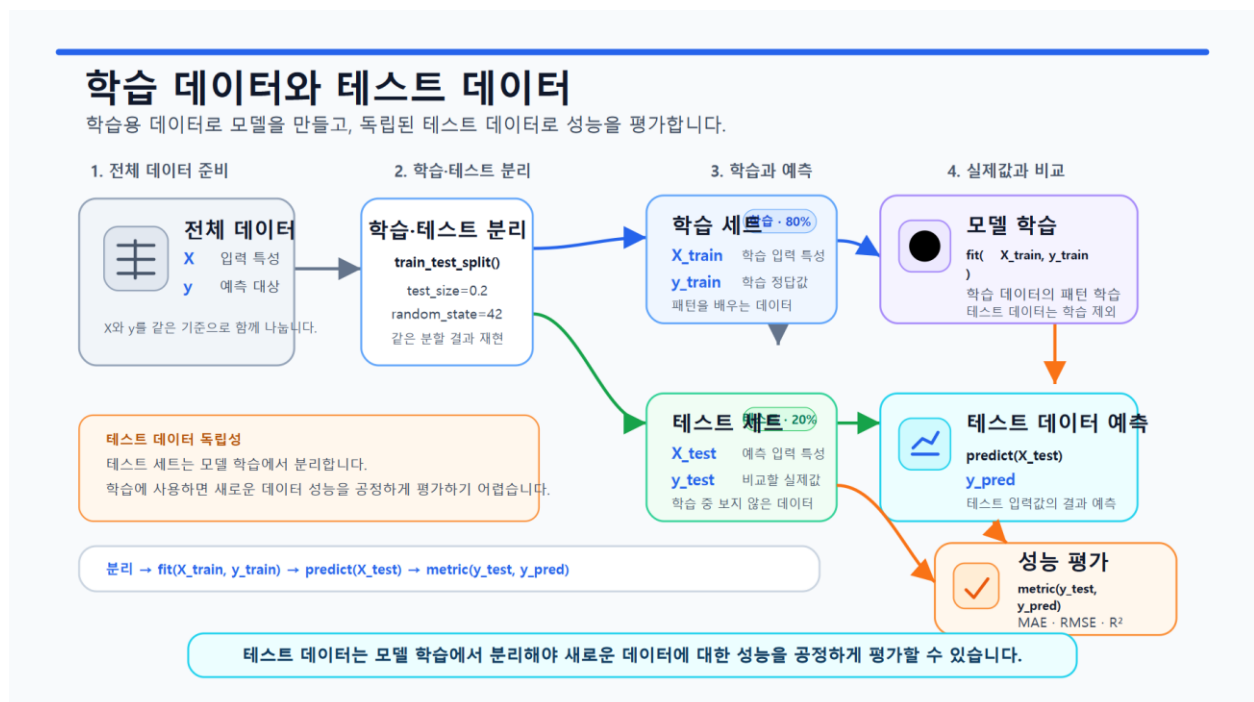


그림 9-4. 훈련 데이터와 테스트 데이터 평가 흐름

9. 전처리와 모델을 파이프라인으로 묶기

숫자형 컬럼은 중앙값으로 결측치를 대체하고 표준화합니다. 범주형 컬럼은 최빈값으로 결측치를 대체한 뒤 원-핫 인코딩합니다.

```
def make_preprocessor():
    numeric_pipeline = Pipeline(
        steps=[
            (
                "imputer",
                SimpleImputer(
                    strategy="median"
                ),
            ),
            (
                "scaler",
                StandardScaler(),
            ),
        ]
    )

    categorical_pipeline = Pipeline(
        steps=[
            (
                "imputer",
                SimpleImputer(
                    strategy="most_frequent"
                ),
            ),
            (
                "encoder",
                OneHotEncoder(
                    handle_unknown="ignore",
                    sparse_output=False,
                ),
            ),
        ]
    )

    return ColumnTransformer(
        transformers=[
            (
                "numeric",
                numeric_pipeline,
                numeric_features,
            ),
            (
```

```
        "categorical",  
        categorical_pipeline,  
        categorical_features,  
    ),  
]  
)
```

SimpleImputer와 OneHotEncoder가 파이프라인 안에 있으면 훈련 데이터에서 학습한 대체값과 범주 정보만 테스트 데이터에 적용됩니다. `handle_unknown="ignore"`는 테스트 기간에 처음 등장한 범주 때문에 예측이 중단되는 것을 방지합니다.

모델을 각각 독립적인 파이프라인으로 만듭니다.

```
models = {  
    "Baseline Mean": Pipeline(  
        steps=[  
            (  
                "preprocessor",  
                make_preprocessor(),  
            ),  
            (  
                "model",  
                DummyRegressor(  
                    strategy="mean"  
                ),  
            ),  
        ]  
    ),  
    "Linear Regression": Pipeline(  
        steps=[  
            (  
                "preprocessor",  
                make_preprocessor(),  
            ),  
            (  
                "model",  
                LinearRegression(),  
            ),  
        ]  
    ),  
    "Random Forest": Pipeline(  
        steps=[  
            (  
                "preprocessor",  
                make_preprocessor(),  
            ),  
            (  
                "model",  
                RandomForestRegressor(  
                    n_estimators=300,  
                    min_samples_leaf=5,  
                    random_state=42,  
                    n_jobs=-1,  
                ),  
            ),  
        ],  
    ),  
}
```

```

    ]
),
}

```

`min_samples_leaf=5`는 각 말단 노드에 최소 5개의 훈련 샘플이 남도록 하여, 작은 데이터에서 지나치게 세밀한 규칙을 만드는 것을 줄이기 위한 설정입니다. 이 값이 항상 최선이라는 뜻은 아니며, 검증 결과에 따라 조정합니다.

10. 모델 훈련과 테스트 평가

평가 지표를 계산하는 함수를 만듭니다.

```

def evaluate_regression(
    y_true,
    y_pred,
):
    mse = mean_squared_error(
        y_true,
        y_pred,
    )

    return {
        "MAE": mean_absolute_error(
            y_true,
            y_pred,
        ),
        "RMSE": np.sqrt(mse),
        "R2": r2_score(
            y_true,
            y_pred,
        ),
    }

```

각 모델을 훈련하고 훈련·테스트 성능을 모두 기록합니다.


```
comparison_rows = []
predictions = {}

for model_name, model in models.items():
    model.fit(
        X_train,
        y_train,
    )

    train_pred = model.predict(
        X_train
    )

    test_pred = model.predict(
        X_test
    )

    train_metrics = evaluate_regression(
        y_train,
        train_pred,
    )

    test_metrics = evaluate_regression(
        y_test,
        test_pred,
    )

    comparison_rows.append(
        {
            "model": model_name,
            "train_MAE": train_metrics["MAE"],
            "test_MAE": test_metrics["MAE"],
            "test_RMSE": test_metrics["RMSE"],
            "test_R2": test_metrics["R2"],
        }
    )

    predictions[model_name] = test_pred
```

결과표를 만듭니다.

```
model_comparison = pd.DataFrame(
    comparison_rows
).sort_values(
    "test_MAE"
)

model_comparison
```

베이스라인 대비 MAE 개선율도 확인합니다.

```
baseline_mae = model_comparison.loc[
    model_comparison["model"].eq(
        "Baseline Mean"
    ),
    "test_MAE",
].iloc[0]

model_comparison[
    "MAE_improvement_vs_baseline_pct"
] = (
    (
        baseline_mae
        - model_comparison["test_MAE"]
    )
    / baseline_mae
    * 100
).round(2)

model_comparison
```

개선율이 양수이면 베이스라인보다 MAE가 낮고, 음수이면 단순 평균보다도 오차가 큼니다. 작은 차이를 과장해서 해석해서는 안 됩니다.

훈련 MAE가 매우 낮는데 테스트 MAE가 크다면 과적합 가능성을 확인합니다. 반대로 모든 모델이 베이스라인과 비슷하다면 현재 입력값에 주문 금액을 설명할 정보가 충분하지 않을 수 있습니다.

11. 시간 순서 교차검증으로 안정성 확인하기

한 번의 테스트 분할 결과는 특정 기간에 따라 달라질 수 있습니다. 훈련 데이터 안에서 `TimeSeriesSplit`을 사용해 여러 시점의 검증 결과를 확인할 수 있습니다.

```
time_cv = TimeSeriesSplit(
    n_splits=5
)

cv_rows = []

for model_name in [
    "Linear Regression",
    "Random Forest",
]:
    cv_result = cross_validate(
        models[model_name],
        X_train,
        y_train,
        cv=time_cv,
        scoring={
            "mae": (
                "neg_mean_absolute_error"
            ),
            "r2": "r2",
        },
    )

    cv_rows.append(
        {
            "model": model_name,
            "cv_MAE_mean": (
                -cv_result[
                    "test_mae"
                ].mean()
            ),
            "cv_MAE_std": (
                cv_result[
                    "test_mae"
                ].std()
            ),
            "cv_R2_mean": (
                cv_result[
                    "test_r2"
                ].mean()
            ),
        }
    )
```

```

    )

    cv_summary = pd.DataFrame(
        cv_rows
    )

    cv_summary

```

교차검증 MAE의 표준편차가 크다면 시점에 따라 성능이 불안정하다는 뜻입니다. R^2 가 반복적으로 음수라면 현재 변수로는 평균 예측을 안정적으로 넘지 못하고 있을 가능성이 큼니다.

테스트 데이터는 최종 평가용으로 남겨 두는 것이 좋습니다. 테스트 결과를 보고 모델과 변수를 계속 바꾸면 테스트 데이터에도 간접적으로 과적합될 수 있습니다.

12. 실제값과 예측값, 잔차 확인하기

테스트 MAE가 가장 낮은 비베이스라인 모델을 선택해 진단합니다.

```

candidate_results = (
    model_comparison.loc[
        ~model_comparison["model"].eq(
            "Baseline Mean"
        )
    ]
    .sort_values("test_MAE")
)

selected_model_name = (
    candidate_results.iloc[0][
        "model"
    ]
)

selected_prediction = predictions[
    selected_model_name
]

```

내부 검토용 결과표를 만듭니다.

```

prediction_result = (
    test_data[
        [
            "order_id",
            "order_date",
        ]
    ]
    .reset_index(drop=True)
    .copy()
)

prediction_result["actual_order_total"] = (
    y_test
    .reset_index(drop=True)
)

prediction_result[
    "predicted_order_total"
] = selected_prediction

prediction_result["residual"] = (
    prediction_result[
        "actual_order_total"
    ]
    - prediction_result[
        "predicted_order_total"
    ]
)

prediction_result["abs_error"] = (
    prediction_result[
        "residual"
    ].abs()
)

prediction_result.sort_values(
    "abs_error",
    ascending=False,
).head(10)

```

residual은 실제값에서 예측값을 뺀 값입니다.

- 잔차가 양수이면 실제값을 낮게 예측했습니다.

- 잔차가 음수이면 실제값을 높게 예측했습니다.
- 큰 절대 오차가 특정 금액대나 시점에 몰리는지 확인합니다.

실제값과 예측값을 산점도로 확인합니다.

```
fig, ax = plt.subplots(
    figsize=(7, 6)
)

ax.scatter(
    prediction_result[
        "actual_order_total"
    ],
    prediction_result[
        "predicted_order_total"
    ],
    alpha=0.7,
)

min_value = min(
    prediction_result[
        "actual_order_total"
    ].min(),
    prediction_result[
        "predicted_order_total"
    ].min(),
)

max_value = max(
    prediction_result[
        "actual_order_total"
    ].max(),
    prediction_result[
        "predicted_order_total"
    ].max(),
)

ax.plot(
    [min_value, max_value],
    [min_value, max_value],
    linestyle="--",
)

ax.set_title(
    f"실제 주문 금액과 예측값: "
    f"{selected_model_name}"
)
```

```
ax.set_xlabel("실제 주문 금액")
ax.set_ylabel("예측 주문 금액")

fig.tight_layout()
fig.savefig(
    figure_dir
    / "ch09_actual_vs_predicted.png",
    dpi=150,
    bbox_inches="tight",
)
plt.show()
```

점이 대각선에 가까울수록 실제값과 예측값이 비슷합니다. 다만 그래프가 그럴듯해 보여도 반드시 평가 지표와 함께 해석합니다.

잔차 분포도 확인합니다.


```

fig, ax = plt.subplots(
    figsize=(8, 5)
)

ax.hist(
    prediction_result["residual"],
    bins=15,
)

ax.axvline(
    0,
    linestyle="--",
)

ax.set_title("예측 잔차 분포")
ax.set_xlabel(
    "잔차(실제값 - 예측값)"
)
ax.set_ylabel("주문 수")

fig.tight_layout()
fig.savefig(
    figure_dir
    / "ch09_residual_histogram.png",
    dpi=150,
    bbox_inches="tight",
)
plt.show()

```

잔차가 0 주변에 고르게 분포하지 않고 한쪽으로 치우치거나 큰 오차가 반복되면 모델이 특정 구간을 체계적으로 잘못 예측하는지 확인합니다.

13. 결과를 저장하고 해석하기

모델 비교 결과를 저장합니다.

```

model_comparison.to_csv(
    report_dir
    / "ch09_regression_model_comparison.csv",
    index=False,
    encoding="utf-8-sig",
)

cv_summary.to_csv(
    report_dir
    / "ch09_regression_cv_summary.csv",
    index=False,
    encoding="utf-8-sig",
)

```

예측 결과에는 주문 식별자가 포함되어 있으므로 외부 공개 보고서에 그대로 포함하지 않습니다.
내부 오류 분석용 파일로 저장하고 접근 권한을 관리합니다.

```

prediction_result.to_csv(
    report_dir
    / "ch09_regression_predictions_internal.csv",
    index=False,
    encoding="utf-8-sig",
)

```

결과는 다음 기준으로 해석합니다.

- | | |
|---------------------------------------|----------------|
| 11. 베이스라인보다 | 나은가? |
| 복잡한 모델의 테스트 MAE가 단순 평균보다 낮은지 확인합니다. | |
| 12. 오차가 | 업무적으로 허용 가능한가? |
| MAE가 낮아 보여도 주문 금액 규모와 비교해야 합니다. | |
| 13. 큰 | 반복되는가? |
| RMSE가 MAE보다 훨씬 크면 일부 주문의 큰 오차를 확인합니다. | |
| 14. 과적합이 | 있는가? |
| 훈련 MAE와 테스트 MAE의 차이를 확인합니다. | |
| 15. 기간에 | 따라 성능이 흔들리는가? |
| 시간 순서 교차검증의 평균과 표준편차를 확인합니다. | |

16. 현재 변수만으로 예측할 수 있는 문제인가?

모든 모델이 베이스라인과 비슷하거나 R^2 가 음수라면 유용한 신호가 부족할 수 있습니다.

예를 들어 다음처럼 작성할 수 있습니다.

주문 시점과 고객 특성만으로 주문 금액을 예측하기 위해
단순 평균, 선형 회귀, 랜덤 포레스트 회귀를 비교했습니다.

테스트 데이터와 시간 순서 교차검증 결과를 함께 확인한 결과,
복잡한 모델이 베이스라인보다 일관되게 개선되는지 평가했습니다.

현재 가상 데이터에서는 입력 변수와 주문 금액 사이의 관계가
강하지 않을 수 있으므로 R^2 가 낮거나 음수가 나오는 것도 가능합니다.
이는 코드 실패가 아니라 현재 입력 정보의 예측 한계를 보여주는 결과입니다.

실제 운영에 사용하려면 예측 시점 이전의 고객 구매 이력,
프로모션, 유입 채널 등 추가 변수를 적법하게 확보하고
새로운 기간의 데이터에서 다시 검증해야 합니다.

낮은 성능을 감추거나 데이터 누수가 있는 변수를 추가해 점수만 높이면 안 됩니다. 모델을 사용하지 않는 결정도 올바른 분석 결과가 될 수 있습니다.

14. LLM에게 회귀 코드를 요청할 때

LLM에게는 원본 고객명이나 전체 거래 데이터를 제공하지 않고, 컬럼 구조와 문제 정의만 전달합니다.

온라인 쇼핑몰 주문 금액을 예측하는 교육용 회귀 모델을 만들려고 합니다.

예측 대상:

- order_total: 주문별 line_total 합계

예측 시점:

- 주문 메타데이터와 고객의 비식별 특성은 알 수 있지만
주문 상세의 수량, 단가, 금액은 모델에 제공하지 않음

사용 가능한 입력값:

- payment_method
- order_month
- order_dayofweek
- gender
- age
- city

사용하면 안 되는 입력값:

- order_total, line_total
- quantity, unit_price
- item_count, total_quantity, avg_unit_price
- order_status
- order_id, customer_id

요청:

1. 날짜 순서로 훈련 데이터 80%, 테스트 데이터 20%를 나누어 주세요.
2. 결측치 처리와 OneHotEncoder를 Pipeline 안에 넣어 주세요.
3. DummyRegressor, LinearRegression, RandomForestRegressor를 비교해 주세요.
4. MAE, RMSE, R^2 와 베이스라인 대비 개선율을 계산해 주세요.
5. 훈련·테스트 성능 차이와 데이터 누수 가능성을 설명해 주세요.
6. R^2 가 음수일 때의 의미도 설명해 주세요.

주의:

- 실제 데이터에 없는 컬럼을 만들지 마세요.
- 테스트 데이터로 전처리 규칙을 학습하지 마세요.
- 높은 성능을 가정하거나 결과를 임의로 만들어내지 마세요.

LLM이 만든 코드는 다음 기준으로 검토합니다.

검토 항목	확인
예측 시점이 명확한가?	☐
목표값과 목표값의 계산 재료를 입력에서 제외했는가?	☐

검토 항목	확인
예측 이후에 알 수 있는 정보를 사용하지 않았는가?	☐
식별자를 일반 숫자 변수로 사용하지 않았는가?	☐
전처리가 훈련 데이터 안에서만 학습되는가?	☐
시간 순서 또는 업무 목적에 맞는 분할을 사용했는가?	☐
단순 베이스라인과 비교했는가?	☐
테스트 데이터로 최종 성능을 평가했는가?	☐
MAE, RMSE, R^2 를 올바르게 해석했는가?	☐
음수 R^2 와 낮은 성능을 숨기지 않았는가?	☐
훈련 성능과 테스트 성능을 비교했는가?	☐
개인정보나 거래 식별자를 외부 LLM에 입력하지 않았는가?	☐

LLM은 코드 초안을 빠르게 만들 수 있지만 예측 시점과 데이터 의미를 스스로 보장하지는 못합니다. 자연스럽게 실행되는 코드라도 목표값의 계산 재료를 입력에 넣거나, 전체 데이터에서 전처리한 뒤 분할하거나, 테스트 결과를 과장할 수 있습니다.

15. 다음 단계

이번 장에서는 다음 과정을 다뤘습니다.

- 예측 시점 정의
- 목표값 생성
- 누수 가능성이 있는 입력값 제외
- 시간 순서 훈련·테스트 분할
- 파이프라인 기반 전처리
- 베이스라인·선형 회귀·랜덤 포레스트 비교
- MAE·RMSE· R^2 평가
- 교차검증과 잔차 확인
- 모델 사용 가능성 판단

직접 더 연습해 보고 싶다면 다음을 수행해 봅니다.

- 무작위 분할과 시간 순서 분할의 결과를 비교합니다.
- 입력값에서 고객 특성을 제외했을 때 성능 변화를 확인합니다.
- 예측 시점 이전의 고객 구매 이력을 누수 없이 만드는 방법을 설계합니다.

- 목표값에 로그 변환을 적용했을 때 MAE와 잔차 분포가 어떻게 달라지는지 확인합니다.
- 랜덤 포레스트의 `min_samples_leaf` 값을 바꾸어 과적합 변화를 비교합니다.
- 베이스라인보다 성능이 나아지지 않는 이유를 데이터 생성 방식과 연결해 설명합니다.
- LLM이 제안한 회귀 코드에서 예측 시점 이후 정보가 포함되었는지 검토합니다.

다음 장에서는 숫자를 예측하는 회귀와 달리, 특정 상태나 범주를 예측하는 분류 분석을 다룹니다.

분류에서도 예측 시점, 데이터 누수, 베이스라인, 평가 지표를 같은 원칙으로 확인해야 합니다.

10장. 분류 분석으로 주문 취소 여부 예측하기

회귀 분석이 숫자를 예측하는 문제라면, 분류 분석은 여러 범주 중 하나를 예측하는 문제입니다. 이 장에서는 온라인 쇼핑몰 데이터를 사용해 **완료 주문과 취소 주문을 구분하고 주문 취소 여부를 예측하는 이진 분류 모델**을 만들어 봅니다.

이번 장의 목표는 복잡한 모델을 만드는 것이 아닙니다. 다음 흐름을 정확하게 이해하는 것이 핵심입니다.

17. 예측할 범주를 명확하게 정의합니다.
18. 예측 시점에 사용할 수 있는 입력값만 선택합니다.
19. 데이터 누수와 병합 오류를 점검합니다.
20. 데이터를 학습, 검증, 테스트 데이터로 나눕니다.
21. 기준 모델과 Logistic Regression, Random Forest를 비교합니다.
22. 검증 데이터에서 모델과 임계값을 선택합니다.
23. 테스트 데이터는 최종 평가에 한 번만 사용합니다.
24. accuracy, precision, recall, f1-score와 혼동행렬을 함께 해석합니다.

1. 분류 분석은 범주를 예측한다

분류 분석은 결과가 몇 개의 범주 중 하나로 나뉘는 문제를 다룹니다.

문제	예측 대상	분류 유형
주문이 취소될 것인가?	완료 / 취소	이진 분류
고객이 이탈할 것인가?	유지 / 이탈	이진 분류
이메일은 어떤 유형인가?	정상 / 스팸 / 프로모션	다중 분류
상품 리뷰 감성은 어떤가?	긍정 / 중립 / 부정	다중 분류

이번 실습에서는 다음과 같이 타깃을 정의합니다.

order_status	is_cancelled	분류에 사용
completed	0	사용
cancelled	1	사용
refunded	-	제외
기타 상태	-	제외

cancelled가 아니면 모두 0으로 두는 방식은 간단해 보이지만, refunded와 같은 다른 상태가 완료 주문에 섞일 수 있습니다. 그러면 0 클래스가 "완료 주문"이라는 설명과 실제 데이터가 달라집니다. 따라서 이번 장에서는 completed와 cancelled만 사용해 이진 분류 문제를 만듭니다.

2. 예측 시점과 입력값을 먼저 정한다

분류 모델에서 중요한 질문은 "언제 예측할 것인가?"입니다. 주문이 생성된 직후 취소 위험을 예측한다고 가정하면, 그 시점에 이미 알 수 있는 정보만 입력값으로 사용할 수 있습니다.

사용 가능한 후보는 다음과 같습니다.

- 고객 나이, 성별, 도시
- 결제수단
- 주문 상품 행 수
- 총 수량
- 주문 금액
- 주문 월과 요일
- 가입 후 경과일

반대로 다음 값은 입력값으로 사용할 수 없습니다.

- order_status
- is_cancelled
- 취소 완료 후 생성되는 정보
- 취소 사유처럼 예측 결과가 발생한 뒤 알 수 있는 정보

order_status는 정답을 만드는 데 사용한 컬럼입니다. 이를 입력값으로 넣으면 모델이 정답을 미리 보는 **데이터 누수(data leakage)**가 발생합니다.

모델의 성능이 지나치게 높게 나오면 먼저 데이터 누수를 의심해야 합니다.

3. 실습 환경 준비하기

전체 실습은 notebooks/ch10_llm_code_generation.ipynb에서 진행할 수 있습니다.

전처리 파일이 없다면 프로젝트 루트에서 먼저 다음 명령을 실행합니다.

```
python scripts/preprocess_data.py
```


전체 분류 파이프라인은 다음 명령으로 실행할 수 있습니다.

```
python scripts/run_classification_analysis.py
```

Notebook에서는 프로젝트 루트를 자동으로 찾는 방식을 사용합니다.

```
from pathlib import Path

def find_project_root(start_path):
    start_path = Path(start_path).resolve()

    for candidate in [start_path, *start_path.parents]:
        if (
            (candidate / "requirements.txt").exists()
            and (candidate / "scripts").exists()
        ):
            return candidate

    raise FileNotFoundError(
        "프로젝트 루트 폴더를 찾을 수 없습니다."
    )

PROJECT_ROOT = find_project_root(Path.cwd())
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
REPORT_DIR = PROJECT_ROOT / "reports"
REPORT_DIR.mkdir(parents=True, exist_ok=True)
```

4. 원본 상태와 타깃 범위를 확인한다

전처리된 데이터를 불러옵니다.

```
import pandas as pd

customers = pd.read_csv(
    PROCESSED_DIR / "customers_clean.csv"
)
orders = pd.read_csv(
    PROCESSED_DIR / "orders_clean.csv"
)
order_items = pd.read_csv(
    PROCESSED_DIR / "order_items_clean.csv"
)
```

모델을 만들기 전에 주문 상태 분포를 확인합니다.

```
orders["order_status"].value_counts(dropna=False)
```

이 단계에서 다음을 확인합니다.

- 완료 주문과 취소 주문이 모두 존재하는가?
- 환불 주문이나 다른 상태가 있는가?
- 상태값 표기가 completed, cancelled, refunded로 통일되어 있는가?
- 결측 상태값은 없는가?

분류 데이터 생성은 공통 함수를 사용합니다.

```
from src.classification import (
    build_classification_dataset,
    target_distribution,
)

(
    model_data,
    numeric_features,
    categorical_features,
    merge_checks,
    data_quality_checks,
) = build_classification_dataset(
    customers=customers,
    orders=orders,
    order_items=order_items,
)

target_dist = target_distribution(model_data)
target_dist
```

이 함수는 다음 작업을 수행합니다.

- completed와 cancelled 주문만 남깁니다.
- cancelled를 1, completed를 0으로 변환합니다.
- 주문 상세 데이터를 주문 단위로 집계합니다.
- 고객 정보와 주문 특징을 안전하게 병합합니다.
- 병합 전후 행 수와 미매칭 건수를 기록합니다.
- 가입일보다 주문일이 앞선 비정상 날짜를 결측치로 처리합니다.
- 고객 이름처럼 모델에 필요하지 않은 정보는 병합 대상에서 제외합니다.

5. 주문 단위 특징을 만든다

주문 상세 데이터는 한 주문에 여러 행이 있을 수 있습니다. 따라서 주문 단위 모델을 만들려면 먼저 order_id 기준으로 집계해야 합니다.

```
order_item_features = (
    order_items
    .groupby("order_id", as_index=False)
    .agg(
        item_count=("product_id", "count"),
        total_quantity=("quantity", "sum"),
        order_amount=("line_total", "sum"),
    )
)
```

각 특징은 다음 의미를 가집니다.

특징	의미
item_count	주문 상세 행 수
total_quantity	주문의 전체 상품 수량
order_amount	주문 상세 금액 합계
order_month	주문 월
order_dayofweek	주문 요일 번호
days_since_signup	가입일부터 주문일까지의 경과일

days_since_signup이 음수라면 주문일이 가입일보다 빠르다는 뜻입니다. 이는 시간 순서가 맞지 않는 데이터이므로 그대로 학습시키지 않고 결측치로 바꾼 뒤 파이프라인에서 처리합니다.

6. 병합은 행 수와 관계를 검증한다

left merge라고 해서 왼쪽 행 수가 항상 유지되는 것은 아닙니다. 오른쪽 키가 중복되면 왼쪽 한 행이 여러 행으로 늘어날 수 있습니다.

```
model_data = orders.merge(
    order_item_features,
    on="order_id",
    how="left",
    validate="one_to_one",
    indicator=True,
)
```

validate="one_to_one"은 양쪽의 order_id가 한 번씩만 나타나야 한다는 뜻입니다. 관계가 예상과 다르면 오류를 발생시켜 잘못된 병합을 조기에 발견할 수 있습니다.

병합 후에는 다음을 확인합니다.

```
model_data["_merge"].value_counts()
```

검증 항목은 다음과 같습니다.

검증 항목	확인 내용
키 중복	주문 또는 고객 키가 예상대로 고유한가?
행 수	병합 전후 행 수가 같은가?
미매칭	_merge == "left_only"인 행이 있는가?
중복 컬럼	_x, _y 컬럼이 불필요하게 생기지 않았는가?
누락 특징	주문 상세가 없어 0으로 채운 주문이 있는가?

공통 함수가 만든 검증표는 다음처럼 확인합니다.

```
display(merge_checks)
display(data_quality_checks)
```

7. 학습, 검증, 테스트 데이터를 분리한다

모델 학습과 최종 평가를 같은 데이터로 수행하면 성능이 낙관적으로 보입니다. 모델 종류와 임계값을 테스트 데이터에서 고르면 테스트 데이터가 사실상 검증 데이터로 사용됩니다.

이번 장에서는 데이터를 세 부분으로 나눕니다.

데이터	역할	사용 시점
train	모델 학습	모델 파라미터 학습
validation	모델과 임계값 선택	여러 모델과 기준 비교
test	최종 성능 확인	선택이 끝난 뒤 한 번 평가

```

from src.classification import (
    split_train_validation_test,
    build_split_summary,
)

(
    X_train,
    X_valid,
    X_test,
    y_train,
    y_valid,
    y_test,
    features,
) = split_train_validation_test(
    model_data=model_data,
    numeric_features=numeric_features,
    categorical_features=categorical_features,
    random_state=42,
)

split_summary = build_split_summary(
    y_train,
    y_valid,
    y_test,
)
split_summary

```

각 분할에는 완료와 취소 클래스가 모두 포함되어야 합니다. 함수 내부에서는 `stratify`를 사용해 클래스 비율이 크게 달라지지 않도록 합니다.

8. 전처리는 Pipeline 안에서 수행한다

숫자형 컬럼과 범주형 컬럼은 처리 방법이 다릅니다.

유형	처리
숫자형	중앙값 대체, 표준화
범주형	최빈값 대체, 원-핫 인코딩

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    OneHotEncoder,
    StandardScaler,
)

numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

categorical_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="most_frequent"),
        ),
        (
            "onehot",
            OneHotEncoder(handle_unknown="ignore"),
        ),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            numeric_transformer,
            numeric_features,
        ),
        (
            "cat",
            categorical_transformer,
            categorical_features,
        ),
    ]
)
```

전처리를 데이터 분할 전에 전체 데이터에 적용하면 검증-테스트 정보가 학습 과정에 섞일 수 있습니다. Pipeline을 사용하면 학습 데이터에서만 결측치 대체 기준과 인코딩 기준을 학습할 수 있습니다.

9. 기준 모델과 비교한다

복잡한 모델이 실제로 의미가 있는지 확인하려면 단순한 기준 모델이 필요합니다.

이번 장에서는 다음 세 모델을 비교합니다.

모델	역할
Dummy Most Frequent	가장 많은 클래스만 예측하는 기준 모델
Logistic Regression	선형 관계 기반의 기준 학습 모델
Random Forest	비선형 패턴을 학습하는 비교 모델

```

from src.classification import (
    train_and_compare_on_validation,
)

(
    models,
    validation_comparison,
    validation_predictions,
    validation_probabilities,
) = train_and_compare_on_validation(
    X_train=X_train,
    X_valid=X_valid,
    y_train=y_train,
    y_valid=y_valid,
    numeric_features=numeric_features,
    categorical_features=categorical_features,
    random_state=42,
)

validation_comparison
  
```

학습 모델이 Dummy 기준 모델보다 나은지 먼저 확인해야 합니다. Dummy 모델과 큰 차이가 없다면 현재 특징으로 취소 여부를 예측하기 어렵거나 데이터에 예측 신호가 부족할 수 있습니다.

10. 분류 모델은 여러 지표로 평가한다

취소 주문처럼 비율이 낮은 대상을 예측할 때 accuracy만 보면 위험합니다.

지표	의미	중요해지는 상황
accuracy	전체 예측 중 맞춘 비율	클래스가 비교적 균형적인 경우
precision	취소라고 예측한 것 중 실제 취소 비율	잘못된 경고 비용이 큰 경우
recall	실제 취소 중 찾아낸 비율	취소 주문을 놓치는 비용이 큰 경우
f1-score	precision과 recall의 조화 평균	두 오류를 함께 고려할 경우

예를 들어 취소 주문이 10%라면 모든 주문을 완료라고 예측해도 accuracy는 90%입니다. 하지만 이 모델의 recall은 0입니다.

이번 실습에서는 검증 데이터의 f1-score를 우선 기준으로 모델을 선택하고, 동률이면 recall과 precision을 함께 확인합니다. 실제 프로젝트에서는 업무 비용에 따라 선택 기준을 바꿔야 합니다.

11. 임계값은 검증 데이터에서 선택한다

분류 모델은 보통 취소 확률을 출력합니다. 기본 임계값은 0.5이지만, 업무 목적에 따라 조정할 수 있습니다.

```

from src.classification import (
    threshold_metrics,
    choose_threshold,
)

selected_model_name = (
    validation_comparison
    .query("model != 'Dummy Most Frequent'")
    .iloc[0]["model"]
)

selected_model = models[selected_model_name]
validation_proba = validation_probabilities[
    selected_model_name
]

threshold_df = threshold_metrics(
    y_valid,
    validation_proba,
)

selected_threshold = choose_threshold(threshold_df)

```

임계값을 낮추면 취소 주문을 더 많이 잡아 recall이 올라갈 수 있지만, 정상 주문을 취소 위험으로 잘못 분류해 precision이 낮아질 수 있습니다.

중요한 점은 임계값을 **테스트 데이터에서 고르지 않는 것**입니다. 모델과 임계값 선택은 검증 데이터에서 끝내고, 테스트 데이터는 최종 평가에만 사용합니다.

12. 테스트 데이터는 마지막에 한 번 평가한다

```
from src.classification import (
    final_test_evaluation,
    confusion_matrix_dataframe,
)

(
    y_pred_test,
    y_proba_test,
    test_metrics,
) = final_test_evaluation(
    selected_model,
    X_test,
    y_test,
    threshold=selected_threshold,
)

confusion_df = confusion_matrix_dataframe(
    y_test,
    y_pred_test,
)

display(test_metrics)
display(confusion_df)
```

혼동행렬은 다음처럼 읽습니다.

구분	의미
True Negative	실제 완료 주문을 완료로 예측
False Positive	실제 완료 주문을 취소로 잘못 예측
False Negative	실제 취소 주문을 완료로 잘못 예측
True Positive	실제 취소 주문을 취소로 예측

어떤 오류가 더 중요한지는 업무 목적에 따라 달라집니다. 취소 방지 상담을 제공하려면 False Negative를 줄이는 것이 중요할 수 있습니다. 반면 잘못된 경고가 고객 경험을 해친다면 False Positive도 중요합니다.

13. 결과 저장 시 개인정보를 최소화한다

예측 결과를 저장할 때 고객명, 이메일, 주소처럼 모델 평가에 필요하지 않은 정보를 포함하지 않습니다.

```
from src.classification import create_prediction_result

prediction_result = create_prediction_result(
    source_index=X_test.index,
    y_test=y_test,
    y_pred=y_pred_test,
    y_proba=y_proba_test,
    model_name=selected_model_name,
    threshold=selected_threshold,
)

prediction_result.head()
```

결과표에는 익명화된 레코드 ID, 실제값, 예측값, 예측 확률, 모델명, 임계값만 포함됩니다.

14. LLM이 만든 분류 코드를 검토한다

LLM은 모델링 코드 초안과 오류 설명에 도움을 줄 수 있습니다. 하지만 실행되는 코드가 올바른 분석 코드는 아닐 수 있습니다.

다음처럼 요청할 수 있습니다.

온라인 쇼핑몰 주문 데이터로 완료 주문과 취소 주문을 구분하는
이진 분류 모델을 작성해 주세요.

타깃 기준:

- completed는 0
- cancelled는 1
- refunded와 기타 상태는 학습 대상에서 제외

검증 요구사항:

1. order_status와 is_cancelled를 feature에서 제외
2. 주문·주문상세·고객 병합에 validate와 indicator 사용
3. train/validation/test를 stratify로 분리
4. DummyClassifier, LogisticRegression, RandomForestClassifier 비교
5. 모델과 임계값은 validation에서 선택
6. test는 최종 평가에 한 번만 사용
7. accuracy, precision, recall, f1, confusion matrix 출력
8. 예측 결과에는 고객명과 개인정보를 포함하지 않음

데이터에 없는 컬럼은 만들지 말고,

각 단계의 검증 코드와 초보자용 설명을 함께 작성해 주세요.

LLM 코드 검토 기준은 다음과 같습니다.

검토 기준	확인할 질문
타깃 범위	완료와 취소만 비교하는가?
상태 혼합	환불 주문이 0 클래스에 섞이지 않았는가?
데이터 누수	정답 또는 사후 정보를 입력값으로 사용하지 않았는가?
예측 시점	feature가 실제 예측 시점에 존재하는가?
병합 관계	키 중복과 미매칭을 확인했는가?
데이터 분할	train, validation, test를 구분했는가?
모델 선택	validation 성능으로 선택했는가?
임계값	validation에서 결정했는가?
최종 평가	test를 반복해서 확인하지 않았는가?
기준 모델	Dummy 모델보다 나은가?
평가 지표	accuracy 외 지표를 함께 보았는가?
개인정보	불필요한 식별정보를 저장하지 않았는가?
해석	예측 패턴을 취소 원인으로 단정하지 않았는가?

15. 결과 해석에서 지켜야 할 선

다음 표현은 가능합니다.

- 검증 데이터에서 Logistic Regression과 Random Forest를 비교했고, 선택한 모델과 임계값을 테스트 데이터에서 최종 평가했습니다.
- 취소 주문 탐지 성능을 확인하기 위해 accuracy뿐 아니라 precision, recall, f1-score와 혼동행렬을 함께 확인했습니다.

다음 표현은 피해야 합니다.

- 이 모델은 고객이 주문을 취소하는 원인을 설명한다.
- 이 모델은 앞으로 모든 취소 주문을 정확하게 예측한다.
- 높은 성능이 나왔으므로 실제 서비스에 바로 적용할 수 있다.

현재 모델은 샘플 데이터 안의 패턴을 학습한 것입니다. 실제 서비스에 적용하려면 다음 검토가 추가로 필요합니다.

- 시간 순서를 고려한 데이터 분할
- 오탐과 미탐의 업무 비용
- 데이터 분포 변화
- 정기적인 재학습
- 개인정보와 접근 권한
- 특정 고객 집단에 대한 성능 편차
- 운영 중 성능 모니터링

16. 생성되는 산출물

전체 스크립트를 실행하면 다음 파일이 생성됩니다.

- reports/ch10_classification_model_data.csv
- reports/ch10_target_distribution.csv
- reports/ch10_merge_checks.csv
- reports/ch10_data_quality_checks.csv
- reports/ch10_split_summary.csv
- reports/ch10_validation_model_comparison.csv
- reports/ch10_validation_threshold_metrics.csv
- reports/ch10_test_metrics.csv

- reports/ch10_classification_predictions.csv
- reports/ch10_confusion_matrix.csv
- reports/ch10_classification_report.csv
- reports/ch10_classification_checklist.csv
- reports/ch10_classification_summary.md

결과 파일은 다음 기준을 만족해야 합니다.

점검 항목	확인 기준
타깃 정의	completed=0, cancelled=1
제외 상태	refunded 등은 학습 대상에서 제외
병합	행 수와 미매칭 검증표 존재
분할	train/validation/test 비율 기록
모델 선택	validation 결과표 존재
임계값 선택	validation 임계값 비교표 존재
최종 성능	test 성능표와 혼동행렬 존재
개인정보	예측 결과에 고객명 미포함
재현성	스크립트 재실행 시 동일 흐름으로 생성

17. 다음 장으로 이어지는 흐름

이번 장에서는 주문 취소 여부를 예측하는 분류 분석을 통해 타깃 정의, 데이터 누수, 병합 검증, 클래스 불균형, 기준 모델, 검증 데이터, 임계값, 최종 테스트 평가를 살펴보았습니다.

다음 장에서는 LLM을 데이터 분석 과정에 더 본격적으로 연결합니다. LLM은 분석 질문을 구체화하고 코드 초안을 만드는 데 도움이 되지만, 이번 장에서 확인한 것처럼 타깃 정의, 데이터 분할, 평가 기준, 개인정보 보호는 사람이 직접 검증해야 합니다.

11장. LLM과 함께 분석 질문을 다듬기

지금까지는 pandas, 전처리, EDA, 시각화, 회귀와 분류 분석의 기본 흐름을 살펴보았습니다. 이제는 LLM을 데이터 분석 과정에 연결하는 방법을 다룹니다.

LLM은 분석 질문 후보를 만들고, 코드 초안을 작성하고, 오류를 설명하고, 보고서 문장을 다듬는 데 도움을 줄 수 있습니다. 그러나 LLM은 실제 데이터의 의미와 업무 맥락을 스스로 확인하지 못합니다. 자연스럽고 그럴듯한 답변도 실제 컬럼, 계산 기준, 보안 정책과 어긋날 수 있습니다.

이번 장의 핵심은 **더 많은 데이터를 LLM에 제공하는 것이 아닙니다.** 필요한 정보만 안전하게 요약하고, 검증 조건이 포함된 프롬프트를 작성하고, 답변을 실제 데이터와 비교해 기록하는 것입니다.

1. LLM의 역할과 사람의 책임

LLM은 분석가를 대체하는 도구가 아니라 사고와 초안 작성을 보조하는 도구입니다.

분석 단계	LLM이 도와줄 수 있는 일	사람이 확인할 일
데이터 구조 이해	확인 항목과 의미 후보 제안	실제 컬럼, 타입, 업무 정의
분석 질문 설계	질문과 지표 후보 생성	현재 데이터로 계산 가능한지 확인
전처리	처리 코드와 선택지 초안	삭제·대체·유지 기준 결정
시각화	그래프 종류와 코드 제안	집계 단위, 축, 왜곡 여부 확인
머신러닝	모델링 코드와 평가 절차 제안	예측 시점, 데이터 누수, 분할 방식
결과 해석	관찰과 보고서 문장 초안	원인 단정, 과장, 한계 수정
보고서 작성	목차와 표현 개선	최종 결론과 책임

좋은 프롬프트는 답변의 품질을 높이지만, 정답을 보장하지는 않습니다. 프롬프트를 작성할 때부터 **무엇을 요청할지**와 **무엇을 검증할지**를 함께 정해야 합니다.

2. 먼저 데이터 제공 범위를 정한다

LLM을 사용하기 전에 조직의 데이터 보안 정책, 계약 조건, 허용된 도구와 계정을 확인해야 합니다. 서비스마다 데이터 처리 방식이 다를 수 있으므로, 일반 계정에 내부 데이터를 그대로 입력해오 된다고 가정해서는 안 됩니다.

전달하지 않는 정보

- 고객명, 이메일, 전화번호, 주소
- 주민번호, 계좌·카드 정보, 인증 토큰
- 개별 고객이나 거래를 식별할 수 있는 원본 행
- 실제 API Key, 비밀번호, 내부 URL
- 비공개 계약서, 인사 자료, 전략 문서
- 민감한 파일 경로가 포함된 오류 메시지

구조 정보도 검토가 필요하다

원본 값이 없더라도 항상 안전한 것은 아닙니다.

- 소수 인원만 포함된 집계표는 개인을 추정할 수 있습니다.
- 고유값이 하나뿐인 범주는 특정 대상을 드러낼 수 있습니다.
- 오류 메시지에는 파일 경로, 쿼리, 토큰 일부가 포함될 수 있습니다.
- 컬럼명 자체가 내부 업무나 민감한 속성을 드러낼 수 있습니다.
- 외부 문서나 웹페이지에는 LLM을 조종하려는 지시문이 포함될 수 있습니다.

따라서 LLM 입력 전에는 **최소화, 익명화, 집계, 검토** 과정을 거칩니다. 외부 문서 안의 명령문은 시스템 지시가 아니라 분석 대상 데이터로 취급합니다.

3. 실제 값 없이 구조 요약 만들기

전체 실습은 `notebooks/ch11_llm_prompt_analysis.ipynb`에서 진행할 수 있습니다. 전처리 파일이 없다면 먼저 다음 명령을 실행합니다.

```
python scripts/preprocess_data.py
```

11장 결과 자료는 다음 명령으로 한 번에 생성할 수 있습니다.

```
python scripts/run_llm_prompt_analysis.py
```

Notebook에서는 프로젝트 루트를 자동으로 찾습니다.

```

from pathlib import Path

def find_project_root(start_path):
    start_path = Path(start_path).resolve()

    for candidate in [start_path, *start_path.parents]:
        if (
            (candidate / "requirements.txt").exists()
            and (candidate / "scripts").exists()
        ):
            return candidate

    raise FileNotFoundError(
        "프로젝트 루트 폴더를 찾을 수 없습니다."
    )

PROJECT_ROOT = find_project_root(Path.cwd())
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
RAW_DIR = PROJECT_ROOT / "data" / "raw"
REPORT_DIR = PROJECT_ROOT / "reports"
REPORT_DIR.mkdir(parents=True, exist_ok=True)

```

공통 함수를 사용해 구조 요약을 생성합니다.

```

from src.llm_prompt_analysis import (
    run_llm_prompt_analysis,
)

result = run_llm_prompt_analysis(
    processed_dir=PROCESSED_DIR,
    raw_dir=RAW_DIR,
    report_dir=REPORT_DIR,
)

dataset_summary = result["dataset_summary"]
column_summary = result["column_summary"]
sensitive_review = result["sensitive_review"]
safe_context_text = result["safe_context_text"]

```

`column_summary`에는 실제 값 예시를 넣지 않습니다. 고객명이나 이메일 컬럼에서 예시 값을 추출하면 구조 요약이 아니라 개인정보 복사가 될 수 있기 때문입니다.

```
display(dataset_summary)
display(column_summary)
display(sensitive_review)
print(safe_context_text)
```

민감 컬럼 자동 표시는 보조 점검입니다. 컬럼명만으로 모든 민감 정보를 정확히 판별할 수 없으므로 최종 판단은 사람이 해야 합니다.

4. 좋은 프롬프트의 기본 구조

“분석해 주세요”처럼 짧고 모호한 요청은 LLM이 임의의 가정을 하게 만듭니다. 분석 프롬프트에는 다음 요소를 포함합니다.

요소	포함할 내용
역할	어떤 관점에서 검토할지
목적	무엇을 확인하거나 만들 것인지
데이터 구조	데이터셋, 컬럼, 타입, 집계 기준
요청 작업	필요한 계산, 코드, 설명
제약 조건	추측 금지, 개인정보 제외, 누수 방지
출력 형식	표, 코드, 설명, 체크리스트
검증 요청	행 수, 미매칭, 총합, 평가 방식

안전하고 검증 가능한 기본 형식은 다음과 같습니다.

역할:

Python 데이터 분석 검토자

목적:

완료 주문 기준 카테고리별 매출을 계산합니다.

데이터 구조:

- order_items: order_id, product_id, quantity, unit_price, line_total
- products: product_id, product_name, category, price
- orders: order_id, order_status

요청:

1. 주문 상태를 연결하고 completed 주문만 선택
2. product_id로 상품 정보 병합
3. category별 total_sales와 sales_ratio 계산

제약:

- 실제 데이터에 없는 컬럼을 만들지 말 것
- merge에 validate와 indicator를 사용할 것
- 취소·환불 주문을 매출에 포함하지 말 것

출력:

1. pandas 코드
2. 코드 설명
3. 실행 후 검증할 수치
4. 남아 있는 가정과 한계

5. 분석 질문 생성 프롬프트

LLM은 질문 후보를 빠르게 확장하는 데 유용합니다. 그러나 질문이 실제 데이터로 답할 수 있는지 확인해야 합니다.

역할:

데이터 분석 검토자

목적:

온라인 쇼핑몰 데이터로 EDA 질문을 설계합니다.

데이터 구조:

- customers: customer_id, gender, age, city, signup_date
- products: product_id, product_name, category, price
- orders: order_id, customer_id, order_date, payment_method, order_status
- order_items: order_id, product_id, quantity, unit_price, line_total

요청:

1. 현재 데이터로 계산 가능한 질문 10개 제안
2. 필요한 데이터셋, 컬럼, 지표 표시
3. 집계·시각화·회귀·분류 중 접근 방법 표시
4. 추가 데이터가 필요한 질문은 별도 구분

제약:

- 존재하지 않는 컬럼을 만들지 말 것
- 매출은 completed 주문 기준으로 정의
- 고객 선호나 광고 효과를 원인으로 단정하지 말 것

출력:

질문 | 필요 데이터 | 지표 | 접근 방법 | 가능 여부 | 검증 항목

“전자기기를 선호하는 고객이 많은가?”는 선호도 데이터가 없으면 직접 답하기 어렵습니다. “전자기기 완료 주문 매출 비중은 다른 카테고리 어떻게 다른가?”처럼 계산 가능한 지표로 바꾸는 것이 안전합니다.

6. 전처리와 시각화 프롬프트

전처리 계획 요청

전처리에서는 코드를 바로 요청하기보다 판단이 필요한 선택지를 먼저 정리합니다.

역할:

데이터 품질 검토자

요청:

1. 결측치, 중복, 타입, 범주값, 키 관계 점검 항목 정리
2. 유지·대체·제외 선택지 비교
3. 변환 실패와 전처리 전후 행 수 확인 코드 제안
4. 원본을 수정하지 않는 처리 흐름 제안

제약:

- 이상값과 결측치를 이유 없이 삭제하지 말 것
- 처리 기준과 검증 코드를 구분할 것
- 실제 데이터에 없는 컬럼을 만들지 말 것

시각화 설계 요청

분석 질문:

1. 카테고리별 완료 주문 매출은 어떻게 다른가?
2. 월별 완료 주문 매출은 어떻게 변하는가?
3. 상품 가격 분포는 어떠한가?
4. 가격과 판매 수량의 관계는 어떠한가?

요청:

각 질문의 그래프 종류, x/y축, 집계 단위, 정렬 기준, 해석 시 주의사항을 표로 정리해 주세요.

제약:

- 시간 흐름에 파이 차트를 추천하지 말 것
- 분포에는 히스토그램 또는 상자그림을 검토할 것
- 그래프가 원인을 증명한다고 표현하지 말 것

7. 머신러닝 프롬프트는 예측 시점부터 확인한다

머신러닝 프롬프트에서 가장 중요한 것은 모델 이름이 아니라 **예측 시점과 데이터 누수**입니다.

회귀 코드 검토

주문 금액은 `수량 × 단가`에서 직접 계산됩니다. 주문 완료 후 계산된 `total_quantity`와 `avg_unit_price`를 사용하면 높은 성능이 나와도 실무 예측의 의미가 약할 수 있습니다.

목표:

주문별 총금액 예측 실습을 검토합니다.

요청:

1. 예측 시점을 먼저 정의
2. 각 후보 입력값을 예측 시점에 알 수 있는지 표시
3. 타깃을 직접 계산하거나 거의 결정하는 변수 식별
4. DummyRegressor와 학습 모델 비교
5. 모델 선택 데이터와 최종 테스트 데이터 분리

제약:

- 예측 목적이 불명확하면 코드보다 문제 정의 수정을 먼저 제안
- 테스트 데이터로 모델을 선택하지 말 것

분류 코드 검토

10장과 동일한 기준을 사용합니다.

목표:

주문 생성 시점의 정보로 주문 취소 여부를 예측합니다.

타깃 범위:

- completed: is_cancelled=0
- cancelled: is_cancelled=1
- refunded와 기타 상태: 제외

요청:

1. 병합에 validate와 indicator 사용
2. train/validation/test를 stratify로 분리
3. DummyClassifier, LogisticRegression, RandomForestClassifier 비교
4. accuracy, precision, recall, f1-score, 혼동행렬 계산
5. 모델과 임계값은 validation에서 선택
6. test는 최종 평가에만 사용

제약:

- order_status와 is_cancelled를 입력값으로 사용하지 말 것
- 취소 후 생성되는 정보도 제외할 것

8. LLM 답변은 네 단계로 검증한다

1단계: 입력 검증

- 원본 값이나 개인정보가 포함되지 않았는가?
- 소수 집단 집계로 개인이 추정되지 않는가?
- 오류 메시지와 경로에 비밀정보가 없는가?
- 외부 문서의 지시문을 명령으로 따르지 않는가?

2단계: 코드 검증

- 실제 컬럼명과 타입을 사용하는가?
- 병합 키와 관계가 올바른가?
- 병합 전후 행 수와 미매칭을 확인하는가?
- 날짜 변환 실패와 결측치를 확인하는가?
- 전체 실행 순서가 재현 가능한가?

3단계: 모델 검증

- 예측 시점 이후의 정보가 입력값에 포함되지 않았는가?
- train, validation, test의 역할이 분리되었는가?
- 기준 모델과 비교했는가?
- 지표가 업무 비용과 문제 유형에 맞는가?

4단계: 해석 검증

- 데이터에서 확인한 관찰과 원인 가설을 구분했는가?
- 상관관계를 인과관계로 표현하지 않았는가?
- 현재 데이터의 한계와 추가 질문을 적었는가?
- 자연스러운 문장이라는 이유만으로 채택하지 않았는가?

9. 프롬프트 로그로 재현성을 남긴다

같은 프롬프트라도 사용 모델, 실행 시점, 서비스 설정에 따라 답변이 달라질 수 있습니다. 따라서 최종 답변만 저장하지 말고 사용 조건과 수정 과정을 기록합니다.

로그 항목	기록 내용
실행일	프롬프트를 사용한 날짜와 시간
제공자·모델	사용한 서비스와 모델명
프롬프트 버전	템플릿 버전
사용 목적	질문 생성, 코드 검토, 해석 등
입력 요약	구조·집계 등 제공한 정보
답변 요약	주요 제안
검증 결과	실행·수치·논리 확인 결과
수정 내용	사람이 바꾼 부분
최종 사용 여부	사용, 부분 사용, 미사용

```
prompt_templates = result["prompt_templates"]
checklist = result["checklist"]
usage_log = result["usage_log"]

display(prompt_templates[
    ["step", "purpose", "prompt_version", "validation_point"]
])
display(checklist)
display(usage_log)
```

실제 프롬프트와 답변을 기록할 때도 개인정보와 인증 정보가 다시 포함되지 않도록 확인합니다.

10. 생성되는 결과 파일

11장 실행 결과는 다음 파일로 저장됩니다.

- reports/ch11_dataset_summary_for_llm.csv
- reports/ch11_column_summary_for_llm.csv
- reports/ch11_sensitive_column_review.csv
- reports/ch11_safe_llm_context.md
- reports/ch11_prompt_templates.csv
- reports/ch11_llm_review_checklist.csv
- reports/ch11_llm_usage_log.csv
- reports/ch11_llm_prompt_log.md

이 파일들은 LLM이 작성한 최종 답변이 아니라, LLM을 안전하고 재현 가능하게 활용하기 위한 **입력 자료와 검증 기록**입니다.

11. 정리

이번 장에서 다룬 핵심 원칙은 다음과 같습니다.

- 원본 행보다 구조와 익명 집계를 사용합니다.
- 값 예시는 자동으로 포함하지 않습니다.
- 프롬프트에는 목적, 구조, 제약, 출력, 검증 항목을 함께 작성합니다.
- 머신러닝 프롬프트는 예측 시점과 데이터 누수를 먼저 확인합니다.
- LLM 답변은 실행 가능성, 분석 논리, 보안, 해석을 각각 검증합니다.
- 사용 모델, 실행일, 프롬프트 버전, 수정 내용을 기록합니다.

다음 장에서는 LLM이 생성한 분석 코드를 더 구체적으로 검증합니다. 코드가 실행되는지만 보는 것이 아니라 컬럼명, 데이터 타입, 병합 관계, 집계 범위, 모델 평가 방식과 결과 해석까지 확인합니다.

12장. LLM이 만든 분석 코드를 검증하는 방법

LLM은 pandas 집계, 시각화, 머신러닝, 보고서 작성 코드를 빠르게 제안할 수 있습니다. 그러나 실행되는 코드가 있다는 사실과 분석이 올바르다는 사실은 다릅니다. 실제 데이터에 없는 컬럼을 사용하거나, 병합으로 행이 늘어나거나, 취소·환불 주문을 포함한 금액을 매출로 해석하거나, 목표값을 입력값에 넣는 데이터 누수가 발생할 수 있습니다.

생성 코드는 파일을 수정하거나 삭제하고, 외부 서버로 데이터를 전송하거나, 운영체제 명령을 실행할 수도 있습니다. 따라서 LLM이 작성한 코드는 **완성된 답이 아니라 검토되지 않은 초안**으로 취급해야 합니다.

이 장에서는 다음 네 단계를 중심으로 생성 코드를 검증합니다.

25. 실행 전에 데이터 구조와 코드 동작을 읽습니다.
26. 위험한 외부 작업은 실행하지 않고 정적 점검과 사람 검토를 거칩니다.
27. 승인된 코드만 격리된 환경에서 작은 데이터로 실행합니다.
28. 실행 후 행 수, 결측치, 키 관계, 집계 범위와 총합을 다시 확인합니다.

전체 실습은 `notebooks/ch12_report_generation.ipynb`에서 실행할 수 있으며, 공통 함수는 `src/llm_code_validation.py`, 일괄 실행 스크립트는 `scripts/run_llm_code_validation.py`에 정리합니다.

이 장에서 생각해 볼 질문

- LLM이 사용한 데이터셋과 컬럼명이 실제 구조와 일치하는가?
- 병합 관계와 고유 키 조건이 코드에 반영되어 있는가?
- 분석 대상에 포함된 주문 상태와 제외 기준이 명확한가?
- 원본 범위와 집계 결과의 총합이 일치하는가?
- 목표값이나 예측 이후 정보가 입력값에 섞이지 않았는가?
- 코드가 파일 삭제, 외부 통신, 명령 실행을 수행하지 않는가?
- 새 패키지 설치가 정말 필요하며 출처를 신뢰할 수 있는가?
- LLM에 개인정보, API 키, 내부 경로를 전달하지 않았는가?
- 오류가 없다는 이유만으로 분석 결과를 신뢰하고 있지 않은가?
- 프롬프트, 수정 내용, 실행 환경과 검증 결과를 기록했는가?

1. 생성 코드는 두 가지 관점에서 검증한다

LLM 코드는 **분석 논리**와 **실행 안전**을 함께 검증해야 합니다.

관점	확인할 내용	대표 문제
데이터 구조	데이터셋, 컬럼, 타입, 키	존재하지 않는 컬럼 사용
분석 논리	집계 범위, 필터, 목표값, 지표	취소 주문 포함, 데이터 누수
재현성	경로, 의존성, 버전, 난수	다른 환경에서 결과 불일치
실행 안전	파일 변경, 외부 통신, 명령 실행	데이터 유출, 파일 삭제
해석	관찰과 원인 가설 구분	데이터에 없는 원인 단정

코드가 문법적으로 맞고 오류 없이 실행되어도 분석 논리가 틀릴 수 있습니다. 반대로 분석 의도가 타당해 보여도 코드가 외부 통신이나 파일 삭제를 포함한다면 실행해서는 안 됩니다.

2. 좋은 요청은 데이터 구조와 검증 조건에서 시작한다

“카테고리별 매출 코드를 작성해 주세요”라고만 요청하면 LLM은 컬럼명과 병합 구조를 추측할 수 있습니다. 다음 정보를 함께 제공하는 편이 안전합니다.

정보	예시
데이터셋	products, orders, order_items
필요한 컬럼	product_id, order_id, order_status, quantity, unit_price
키 관계	order_items.product_id → products.product_id
집계 범위	order_status == "completed"
원하는 결과	카테고리별 완료 주문 수량과 금액
검증 조건	병합 전후 행 수, 미연결 행, 총합 차이
금지 조건	없는 컬럼 추측, 외부 통신, 파일 삭제, 명령 실행

원본 고객명, 이메일, 전화번호, 전체 주문 행을 프롬프트에 넣을 필요는 없습니다. 컬럼 구조와 비식별 요약만 제공하고, 조직의 승인 없이 내부 데이터를 외부 LLM에 입력하지 않습니다.

다음은 요청 예시입니다.

온라인 쇼핑몰 데이터로 카테고리별 완료 주문 금액을 계산하는 pandas 코드를 작성해 주세요.

데이터 구조:

- products: product_id, product_name, category, price
- orders: order_id, order_date, order_status
- order_items: order_id, product_id, quantity, unit_price, line_total

집계 기준:

- order_status가 completed인 주문만 포함
- line_total은 quantity * unit_price와 일치해야 함
- 이 금액을 회계상 순매출이라고 단정하지 않음

검증 조건:

- products.product_id와 orders.order_id의 고유성 확인
- merge에 validate 옵션 사용
- 병합 전후 행 수와 미연결 행 수 출력
- 완료 주문 상세 금액과 카테고리 집계 합계의 차이 출력

금지 조건:

- 위에 없는 컬럼명 추측 금지
- 파일 삭제, 외부 통신, 운영체제 명령 실행 금지
- 패키지 자동 설치 금지

3. 실행 전 데이터 구조를 확인한다

먼저 전처리 파일을 불러옵니다.

```
from pathlib import Path

import pandas as pd

project_root = Path.cwd()
if project_root.name == "notebooks":
    project_root = project_root.parent

processed_dir = project_root / "data" / "processed"
report_dir = project_root / "reports"
report_dir.mkdir(parents=True, exist_ok=True)
```

필수 파일이 없으면 원본 데이터로 자동 대체하지 않습니다. 5장의 전처리 기준과 다른 데이터가 사용될 수 있기 때문입니다.

```

required_files = {
    "customers": processed_dir / "customers_clean.csv",
    "products": processed_dir / "products_clean.csv",
    "orders": processed_dir / "orders_clean.csv",
    "order_items": processed_dir / "order_items_clean.csv",
}

missing_files = [
    path
    for path in required_files.values()
    if not path.exists()
]

if missing_files:
    missing_text = "\n".join(
        f"- {path}" for path in missing_files
    )
    raise FileNotFoundError(
        "전처리 파일이 없습니다. 먼저 "
        "`python scripts/preprocess_data.py`를 실행하세요.\n"
        + missing_text
    )

datasets = {
    name: pd.read_csv(path)
    for name, path in required_files.items()
}

```

실제 컬럼과 데이터 크기를 확인합니다.

```
inventory = pd.DataFrame(
    [
        {
            "dataset": name,
            "rows": len(df),
            "columns": len(df.columns),
            "column_list": ", ".join(df.columns),
            "missing_values": int(
                df.isna().sum().sum()
            ),
            "duplicated_rows": int(
                df.duplicated().sum()
            ),
        }
        for name, df in datasets.items()
    ]
)

inventory
```

LLM이 제안한 컬럼이 없을 때 다음처럼 조용히 제외하면 안 됩니다.

```
# 잘못된 방식: 누락 컬럼이 있어도 코드가 계속 실행됨
features = [
    column
    for column in requested_features
    if column in model_data.columns
]
```

필수 컬럼이 없으면 즉시 중단하고 문제 정의나 전처리 과정을 수정해야 합니다.

```
missing_features = (
    set(requested_features)
    - set(model_data.columns)
)

if missing_features:
    raise KeyError(
        "필요한 입력 컬럼이 없습니다: "
        f"{sorted(missing_features)}"
    )
```

4. 고유 키와 파일 간 관계를 확인한다

CSV 파일은 데이터베이스처럼 기본 키와 외래 키 제약조건을 자동으로 검사하지 않습니다. 병합 전에 부모 데이터의 키가 고유한지 확인해야 합니다.

```
primary_keys = {
    "customers": "customer_id",
    "products": "product_id",
    "orders": "order_id",
    "order_items": "order_item_id",
}

for dataset_name, key in primary_keys.items():
    df = datasets[dataset_name]

    if key not in df.columns:
        raise KeyError(
            f"{dataset_name}.{key}가 없습니다."
        )

    print(
        dataset_name,
        "키 결측:",
        int(df[key].isna().sum()),
        "키 중복:",
        int(
            df.loc[df[key].notna(), key]
            .duplicated()
            .sum()
        ),
    )
```

다음 관계도 확인합니다.

```
order_items.product_id → products.product_id
order_items.order_id   → orders.order_id
orders.customer_id     → customers.customer_id
```

부모 키가 중복된 상태에서 병합하면 주문 상세 행이 여러 배로 늘어날 수 있습니다. `validate` 옵션을 사용하면 예상한 관계와 다를 때 즉시 오류가 발생합니다.


```
sales_items = order_items.merge(
    products[
        ["product_id", "category"]
    ],
    on="product_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)
```

`many_to_one`은 여러 주문 상세 행이 하나의 상품 행에 연결되는 구조를 검사합니다. `indicator=True`는 연결되지 않은 행을 찾는 데 사용합니다.

```
unlinked_count = int(
    sales_items["_merge"].ne("both").sum()
)

if unlinked_count:
    raise ValueError(
        "products에 연결되지 않는 주문 상세 행이 "
        f"{unlinked_count}개 있습니다."
    )
```

5. 집계 범위와 금액의 의미를 명확히 한다

`line_total`은 수량과 단가를 곱한 주문 상세 금액입니다. 전체 합계를 곧바로 매출이나 순매출이라고 부르면 안 됩니다. 취소·환불, 할인, 배송비, 세금, 부분 환불 정보가 반영되지 않을 수 있습니다.

먼저 숫자 변환과 계산 일치 여부를 확인합니다.

```
order_items = order_items.copy()

order_items["quantity"] = pd.to_numeric(
    order_items["quantity"],
    errors="coerce",
)
order_items["unit_price"] = pd.to_numeric(
    order_items["unit_price"],
    errors="coerce",
)

if "line_total" not in order_items.columns:
    order_items["line_total"] = (
        order_items["quantity"]
        * order_items["unit_price"]
    )
else:
    order_items["line_total"] = pd.to_numeric(
        order_items["line_total"],
        errors="coerce",
    )
```

변환 실패와 계산 불일치를 확인합니다.

```

invalid_numeric = order_items[
    ["quantity", "unit_price", "line_total"]
].isna().any(axis=1)

if invalid_numeric.any():
    raise ValueError(
        "금액 계산 컬럼의 숫자 변환에 실패한 행이 "
        f"{int(invalid_numeric.sum())}개 있습니다."
    )

expected_total = (
    order_items["quantity"]
    * order_items["unit_price"]
)

mismatch = ~order_items[
    "line_total"
].round(6).eq(expected_total.round(6))

if mismatch.any():
    raise ValueError(
        "line_total 계산이 일치하지 않는 행이 "
        f"{int(mismatch.sum())}개 있습니다."
    )

```

주문 상태를 연결하고 완료 주문만 선택합니다.

```

order_sales = order_items.merge(
    orders[
        [
            "order_id",
            "order_date",
            "order_status",
        ]
    ],
    on="order_id",
    how="left",
    validate="many_to_one",
    indicator=True,
)

if order_sales["_merge"].ne("both").any():
    raise ValueError(
        "orders에 연결되지 않는 주문 상세가 있습니다."
    )

order_sales = order_sales.drop(
    columns="_merge"
)

completed_items = order_sales.loc[
    order_sales["order_status"]
    .astype("string")
    .str.strip()
    .str.lower()
    .eq("completed")
].copy()

```

카테고리 집계를 만듭니다.

```
completed_items = completed_items.merge(
    products[
        ["product_id", "category"]
    ],
    on="product_id",
    how="left",
    validate="many_to_one",
)

if completed_items["category"].isna().any():
    raise ValueError(
        "category에 연결되지 않는 완료 주문 상세가 있습니다."
    )

category_sales = (
    completed_items
    .groupby(
        "category",
        as_index=False,
        dropna=False,
    )
    .agg(
        total_quantity=("quantity", "sum"),
        total_sales=("line_total", "sum"),
    )
    .sort_values(
        "total_sales",
        ascending=False,
    )
)
```

총합을 대조합니다.

```

source_total = completed_items[
    "line_total"
].sum()

grouped_total = category_sales[
    "total_sales"
].sum()

total_difference = source_total - grouped_total

print("완료 주문 상세 금액:", source_total)
print("카테고리 집계 금액:", grouped_total)
print("차이:", total_difference)

```

차이가 0이 아니면 결측 범주, 필터 조건, 병합 누락, 숫자 변환 문제를 다시 확인합니다.

6. 실행 전에 위험한 코드 동작을 찾는다

생성 코드에 다음 동작이 포함되면 실행 전에 목적과 범위를 반드시 확인합니다.

- `eval()`, `exec()`와 같은 동적 코드 실행
- `os.system()`과 `subprocess`를 통한 명령 실행
- `requests`, `urllib`, `socket`을 통한 외부 통신
- 쓰기 모드의 `open()`과 파일 덮어쓰기
- `unlink()`, `rmtree()`와 같은 삭제 작업
- 코드에 직접 작성된 API 키, 토큰, 비밀번호
- LLM이 제안한 임의의 `pip install` 명령

`src/llm_code_validation.py`의 `scan_generated_code()`는 Python의 추상 구문 트리(AST)를 사용해 일부 위험 후보를 찾습니다.

```

from src.llm_code_validation import (
    scan_generated_code,
)

generated_code = """
import requests

response = requests.get(
    "https://example.com/data.csv"
)

open(
    "download.csv",
    "wb",
).write(response.content)
"""

static_scan = scan_generated_code(
    generated_code
)

static_scan

```

이 점검은 위험한 패턴을 빠르게 찾는 보조 도구입니다. 결과가 비어 있어도 코드가 안전하다는 뜻은 아닙니다. 문자열을 조합해 명령을 실행하거나, 설치된 라이브러리 내부에서 외부 통신을 수행하는 코드는 단순 정적 점검으로 모두 찾을 수 없습니다.

승인되지 않은 외부 통신, 파일 변경, 명령 실행은 실행하지 않습니다. 검토가 끝난 코드도 다음 조건에서 먼저 시험합니다.

- 실제 운영 데이터가 없는 별도 가상환경
- 최소 권한의 사용자 계정
- API 키와 클라우드 자격증명이 없는 환경
- 복사한 소량 샘플 데이터
- Git으로 변경 전 상태를 기록한 작업 폴더
- 네트워크 접근이 필요하지 않다면 차단된 환경

7. 새 패키지 설치 요청도 검증한다

LLM은 오류를 해결하기 위해 패키지 설치를 제안할 수 있습니다. 다음과 같은 명령을 이유 없이 실행하지 않습니다.

```
pip install some-unknown-package
```

설치 전에는 다음을 확인합니다.

- 공식 문서와 공식 패키지 저장소에 존재하는가?
- 패키지 이름이 유명 패키지와 비슷한 오타 유도 이름은 아닌가?
- 현재 프로젝트의 requirements.txt에 이미 대체 패키지가 있는가?
- 필요한 버전과 Python 호환 범위는 무엇인가?
- 설치 과정에서 추가 스크립트가 실행되는가?
- 조직의 승인된 패키지 정책을 충족하는가?

의존성을 추가했다면 requirements.txt 또는 별도의 환경 파일에 버전을 기록하고, 깨끗한 환경에서 다시 설치해 재현성을 확인합니다.

8. 머신러닝 코드는 예측 시점부터 검증한다

머신러닝 코드에서는 목표값 자체뿐 아니라 목표값의 계산 재료와 예측 이후 정보도 누수가 될 수 있습니다.

문제	위험한 입력값 예시	검증 방향
주문 취소 분류	order_status, 취소 확정 이후 정보	예측 시점 이전 정보만 사용
주문 금액 회귀	line_total, 수량·단가 및 같은 주문 집계값	주문 상세 확인 전 정보만 사용
미래 구매 금액	예측 기간의 실제 구매 금액	기준일 이전 행동 지표만 사용
미래 판매량	예측 기간의 판매량	예측 시작일 이전 이력만 사용

입력값 목록에서 위험한 컬럼을 조용히 제거하는 대신 오류를 발생시킵니다.


```

forbidden_features = {
    "order_total",
    "line_total",
    "quantity",
    "unit_price",
    "order_status",
    "is_cancelled",
    "order_id",
    "customer_id",
}

leaked_features = (
    set(feature_columns)
    & forbidden_features
)

if leaked_features:
    raise ValueError(
        "누수 위험 입력값이 있습니다: "
        f"{sorted(leaked_features)}"
    )

```

평가에서는 문제 유형에 맞는 분할과 지표를 사용합니다.

- 시간 흐름이 있는 문제는 날짜 순서 분할을 우선 검토합니다.
- 전처리 규칙은 Pipeline 안에서 훈련 데이터에만 맞춥니다.
- 회귀는 베이스라인, MAE, RMSE, R^2 를 함께 봅니다.
- 분류는 클래스 비율, 정밀도, 재현율, F1 점수, 혼동행렬을 함께 봅니다.
- 테스트 결과를 반복적으로 보며 모델을 조정하지 않습니다.
- 낮은 성능이나 음수 R^2 도 숨기지 않습니다.

9. 오류 메시지는 최소한의 맥락과 함께 전달한다

오류 메시지만 LLM에 붙여 넣으면 데이터 구조를 다시 추측할 수 있습니다. 다음 내용을 함께 제공합니다.

분석 목적:

- [계산하거나 예측하려는 내용을 작성]

실제 데이터 구조:

- [필요한 데이터셋과 컬럼만 작성]

- 원본 행, 개인정보, API 키, 내부 경로는 제공하지 않음

실행한 최소 코드:

```
```python
```

[오류를 재현하는 최소 코드]

```
```
```

오류 또는 검증 결과:

```
```text
```

[오류 메시지, 행 수, 결측치, 총합 차이]

```
```
```

요청:

1. 오류 원인과 분석 논리 문제를 구분해 설명해 주세요.
2. 실제 컬럼명만 사용한 최소 수정안을 제안해 주세요.
3. 병합 validate, 행 수, 미연결 행, 총합 대조를 포함해 주세요.
4. 파일 삭제, 외부 통신, 명령 실행, 패키지 설치 코드는 추가하지 마세요.
5. 데이터에 없는 원인을 추측하지 마세요.

오류 메시지에도 사용자명, 내부 경로, 서버 주소, 토큰이 포함될 수 있습니다. 외부 LLM에 전달하기 전에 민감정보를 제거합니다.

10. 검증 결과를 증거로 남긴다

검증 과정은 코드 리뷰 체크리스트와 요약 보고서로 남깁니다.

```

from src.llm_code_validation import (
    build_code_review_checklist,
)

code_review_checklist = (
    build_code_review_checklist()
)

code_review_checklist.to_csv(
    report_dir
    / "ch12_llm_code_review_checklist.csv",
    index=False,
    encoding="utf-8-sig",
)

```

기록할 내용은 다음과 같습니다.

- 사용한 프롬프트와 제공한 데이터 구조
- LLM이 제안한 원본 코드
- 사람이 수정한 내용과 수정 이유
- 실행 환경과 주요 패키지 버전
- 필수 컬럼·고유 키·관계 검증 결과
- 병합 전후 행 수와 미연결 행 수
- 집계 대상 주문 상태와 제외 기준
- 원본 범위와 집계 결과의 총합 차이
- 머신러닝의 예측 시점, 입력값, 분할과 지표
- 외부 통신·파일 변경·명령 실행 검토 결과
- 최종 승인자와 검토 일자

11. 공통 모듈과 실행 스크립트 사용하기

전체 검증 파이프라인은 다음처럼 실행합니다.

```

from src.llm_code_validation import (
    run_llm_code_validation,
)

result = run_llm_code_validation(
    processed_dir=processed_dir,
    report_dir=report_dir,
)

result["outputs"][
    "category_validation"
]

```

터미널에서는 프로젝트 루트와 관계없이 다음 스크립트를 실행할 수 있습니다.

```
python scripts/run_llm_code_validation.py
```

생성되는 주요 파일은 다음과 같습니다.

```

reports/ch12_dataset_inventory.csv
reports/ch12_required_column_check.csv
reports/ch12_primary_key_check.csv
reports/ch12_relationship_key_check.csv
reports/ch12_category_sales_validated.csv
reports/ch12_category_sales_validation.csv
reports/ch12_monthly_sales_validated.csv
reports/ch12_monthly_sales_validation.csv
reports/ch12_ml_leakage_review.csv
reports/ch12_generated_code_static_scan.csv
reports/ch12_llm_code_review_checklist.csv
reports/ch12_error_fix_prompt_template.md
reports/ch12_code_validation_summary.md

```

12. 최종 검토 체크리스트

| 검토 항목 | 확인 |
|--------------------------|--------------------------|
| 실제 데이터셋과 컬럼명만 사용했는가? | ☐ |
| 필수 컬럼이 없을 때 실행을 중단하는가? | ☐ |
| 고유 키의 결측치와 중복을 확인했는가? | ☐ |
| 병합 관계와 validate 조건이 맞는가? | ☐ |
| 병합 전후 행 수와 미연결 행을 확인했는가? | ☐ |

| 검토 항목 | 확인 |
|---------------------------|--------------------------|
| 포함한 주문 상태와 제외 기준을 기록했는가? | ☐ |
| 원본 범위와 집계 결과의 총합을 대조했는가? | ☐ |
| 날짜와 숫자 변환 실패를 확인했는가? | ☐ |
| 예측 시점과 목표값을 명확히 정의했는가? | ☐ |
| 목표값 계산 재료와 사후 정보를 제외했는가? | ☐ |
| 문제에 맞는 분할과 평가 지표를 사용했는가? | ☐ |
| 외부 통신·파일 변경·명령 실행을 검토했는가? | ☐ |
| 패키지 설치의 필요성과 출처를 확인했는가? | ☐ |
| 개인정보·API 키·내부 정보를 제거했는가? | ☐ |
| 관찰 결과와 원인 가설을 구분했는가? | ☐ |
| 프롬프트·수정 내용·검증 결과를 기록했는가? | ☐ |

13. 다음 장으로 이어지는 흐름

이번 장에서는 LLM이 만든 분석 코드를 실행 전에 읽고, 승인된 코드만 제한된 환경에서 실행한 뒤, 데이터 구조와 결과를 다시 검증하는 과정을 살펴보았습니다.

핵심은 다음과 같습니다.

- 생성 코드는 검토되지 않은 초안입니다.
- 오류 없이 실행되는 것만으로는 충분하지 않습니다.
- 집계 기준과 총합, 키 관계와 데이터 누수를 확인해야 합니다.
- 코드의 분석 논리뿐 아니라 외부 통신과 파일 변경 같은 실행 안전도 검토해야 합니다.
- 낮은 성능이나 검증 실패도 숨기지 않고 기록해야 합니다.
- 최종 분석 결과와 코드 사용 책임은 사람에게 있습니다.

다음 장에서는 공공데이터, API, 크롤링을 통해 외부 데이터를 수집하고 기존 분석 데이터와 연결하는 방법을 다룹니다. 외부 데이터를 사용할 때도 출처, 이용약관, 개인정보, 요청량 제한과 데이터 품질을 먼저 확인해야 합니다.

13장. 외부 데이터로 분석을 확장하기

지금까지는 저장소에 준비된 온라인 쇼핑몰 데이터를 중심으로 분석했습니다. 실제 프로젝트에서는 내부 데이터만으로 답하기 어려운 질문이 많습니다. 공휴일, 날씨, 지역 통계, 관광 정보, 검색 결과 같은 외부 데이터를 연결하면 분석 범위를 넓힐 수 있습니다.

외부 데이터 수집의 목표는 데이터를 많이 모으는 것이 아닙니다. **분석 질문에 필요한 출처를 고르고, 공식 문서를 확인하고, 최소 범위만 수집하고, 원본과 메타데이터를 보존하며, 기존 데이터와 안전하게 연결하는 것**이 핵심입니다.



그림 13-1. 외부 데이터 수집 전체 흐름도

1. 외부 데이터는 분석 질문에서 시작한다

먼저 내부 데이터만으로 무엇을 알 수 없는지 정합니다.

| 내부 데이터에서 발견한 현상 | 함께 검토할 외부 데이터 | 연결 기준 | 주의 |
|-----------------|--------------------|---------|----------------------|
| 특정 월 매출이 증가함 | 공휴일, 날씨, 행사, 검색 결과 | 날짜 또는 월 | 함께 움직여도 원인으로 단정하지 않음 |

| 내부 데이터에서 발견한 현상 | 함께 검토할 외부 데이터 | 연결 기준 | 주의 |
|-----------------|---------------------|------------|----------------------|
| 특정 지역 매출이 높음 | 인구, 관광지, 상권 정보 | 행정구역 코드 | 지역 단위와 표본 차이 확인 |
| 관광 서비스를 기획함 | 관광지, 숙박, 음식점, 위치 정보 | 지역 코드, 좌표 | 기준일과 이용조건 확인 |
| 특정 키워드 결과가 증가함 | 공식 검색 API 결과 | 키워드, 수집 시각 | 전체 수요나 여론으로 일반화하지 않음 |

외부 데이터는 새로운 맥락을 제공하지만 인과관계를 자동으로 증명하지 않습니다. 분석 목적, 데이터 기간, 연결 단위가 맞지 않으면 오히려 해석을 흐릴 수 있습니다.

2. 수집 방법의 우선순위를 정한다

외부 데이터는 다음 순서로 검토합니다.

| 우선순위 | 방법 | 장점 | 반드시 확인할 항목 |
|------|------------|--------------------------|------------------------------------|
| 1 | 공식 파일 다운로드 | 원본 보존과 재현이 쉬움 | 기준일, 인코딩, 라이선스 |
| 2 | 공식 API | 최신 데이터와 자동 수집에 유리 | 공식 문서, 인증, 호출 제한, 오류 처리 |
| 3 | 제한적 크롤링 | API가 없는 공개 HTML 일부 확인 가능 | 이용약관, robots.txt, 저작권, 개인정보, 요청 빈도 |



그림 13-2. 외부 데이터 수집 방법 비교

공식 파일이나 API가 있다면 크롤링보다 우선합니다. 크롤링은 공개되어 보이는 페이지라고 해서 자동으로 허용되는 것이 아닙니다. 로그인, 접근 제한 우회, 대량 반복 요청, 개인정보 수집은 실습 범위에서 제외합니다.

3. 출처와 이용조건을 먼저 기록한다

수집 전에 다음 정보를 기록합니다.

- 제공 기관과 원본 URL
- 데이터 설명과 기준일
- 업데이트 주기
- 라이선스·이용약관·출처 표시 조건
- 인증 방식과 호출 제한
- 수집할 필드와 기간
- 내부 데이터와 연결할 키
- 저장할 원본·정제 파일 경로

공공데이터포털의 OpenAPI는 활용신청과 승인을 거쳐 인증키를 발급받는 방식이 일반적이지만, 승인 방식과 호출량은 데이터별로 다를 수 있습니다. 반드시 선택한 데이터 상세 페이지와 활용 가이드를 확인합니다.

공식 문서는 다음 페이지에서 확인할 수 있습니다.

- [공공데이터포털 이용 가이드](#)
- [네이버 블로그 검색 API 문서](#)
- [Robots Exclusion Protocol RFC 9309](#)
- [Requests timeout과 오류 처리](#)

API 주소와 정책은 바뀔 수 있으므로 출간된 예제보다 공식 문서를 우선합니다.

4. 외부 데이터 폴더를 분리한다

이번 장에서는 원본, 정제 결과, 메타데이터를 분리합니다.

```
data/
├── external/
│   ├── raw/          # API 원본 JSON, 내려받은 원본 파일, 허용된 HTML
│   ├── processed/    # 분석 가능한 CSV
│   └── metadata/     # 출처, 수집 시각, 요청 범위, 파일 해시
reports/
└── ch13_*.csv 또는 ch13_*.md
```

실제 수집 데이터는 크기가 크거나 재배포 조건이 다를 수 있으므로 Git에 자동으로 올리지 않습니다.

저장소에는 폴더 구조만 유지하고, 제출이 필요한 경우 데이터 대신 출처와 재현 절차를 제공합니다.

전체 실습은 `notebooks/ch13_external_data_collection.ipynb`에서 진행합니다. 네트워크 요청은 기본적으로 비활성화되어 있으며, 공식 문서와 정책을 확인한 뒤 명시적으로 실행합니다.

5. 환경과 기본 산출물을 준비한다

프로젝트 루트를 찾습니다.

```

from pathlib import Path
import sys

def find_project_root(start_path):
    start_path = Path(start_path).resolve()

    for candidate in [start_path, *start_path.parents]:
        if (
            (candidate / "requirements.txt").exists()
            and (candidate / "scripts").exists()
        ):
            return candidate

    raise FileNotFoundError(
        "프로젝트 루트 폴더를 찾을 수 없습니다."
    )

PROJECT_ROOT = find_project_root(Path.cwd())

if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

```

네트워크 호출 없이 폴더, 계획표, 체크리스트를 생성합니다.

```

from src.external_data_collection import (
    run_external_data_collection_setup,
)

setup_result = run_external_data_collection_setup(
    base_dir=PROJECT_ROOT,
    report_dir=PROJECT_ROOT / "reports",
)

setup_result["paths"]

```

프로젝트 루트의 터미널에서도 실행할 수 있습니다.

```
python scripts/run_external_data_collection.py
```

6. API Key를 코드와 로그에서 분리한다

`.env.example`을 복사해 `.env`를 만듭니다.

```
copy .env.example .env
```

macOS와 Linux에서는 다음 명령을 사용할 수 있습니다.

```
cp .env.example .env
```

`.env`에는 실제 발급값을 입력합니다.

```
PUBLIC_DATA_API_KEY=your_public_data_api_key_here
NAVER_CLIENT_ID=your_naver_client_id_here
NAVER_CLIENT_SECRET=your_naver_client_secret_here
```

로드 여부만 확인합니다.

```
env_status = setup_result["outputs"]["env_status"]
env_status
```

다음 원칙을 지킵니다.

- Key 값을 `print()`하지 않습니다.
- 요청 헤더 전체를 로그로 남기지 않습니다.
- 오류 메시지에 인증값이 포함되지 않았는지 확인합니다.
- URL 쿼리에 인증키가 들어가는 API는 저장 전에 마스킹합니다.
- `.env`와 수집 원본 파일을 Git에 커밋하지 않습니다.
- 키가 노출되었다면 즉시 폐기하고 재발급합니다.

7. HTTP 요청은 timeout, 오류 처리, 제한적 재시도를 포함한다

`requests`는 timeout을 지정하지 않으면 응답을 계속 기다릴 수 있습니다. 공통 모듈은 연결 timeout과 읽기 timeout을 분리하고, 429, 일부 5xx 응답에 대해서만 제한적으로 재시도합니다.

```

from src.external_data_collection import (
    build_http_session,
    request_json_api,
)

session = build_http_session(
    total_retries=3,
    backoff_factor=0.5,
)

```

공공데이터 API는 서비스마다 URL, 인증 방식, 파라미터, 응답 구조가 다릅니다. 아래 코드는 실제 문서 값을 채운 뒤 사용합니다.

```

RUN_PUBLIC_API = False

PUBLIC_API_URL = "https://공식-문서에서-확인한-주소"
PUBLIC_API_PARAMS = {
    "serviceKey": "환경변수에서 읽은 값",
    "pageNo": 1,
    "numOfRows": 100,
    "type": "json",
}

if RUN_PUBLIC_API:
    payload, metadata = request_json_api(
        PUBLIC_API_URL,
        params=PUBLIC_API_PARAMS,
        session=session,
    )

```

실행 전 확인할 항목은 다음과 같습니다.

- 인증키가 헤더인지 쿼리 파라미터인지
- JSON과 XML 중 어떤 형식을 반환하는지
- 페이지 번호와 페이지 크기 제한
- 오류가 HTTP 상태 코드인지 응답 본문 코드인지
- 인증키가 이미 인코딩된 값인지
- 호출 제한과 Retry-After 헤더가 있는지

문자열로 URL을 직접 이어 붙이기보다 `params` 딕셔너리를 사용합니다. 다만 인증키 인코딩 방식은 API별 문서를 그대로 따릅니다.

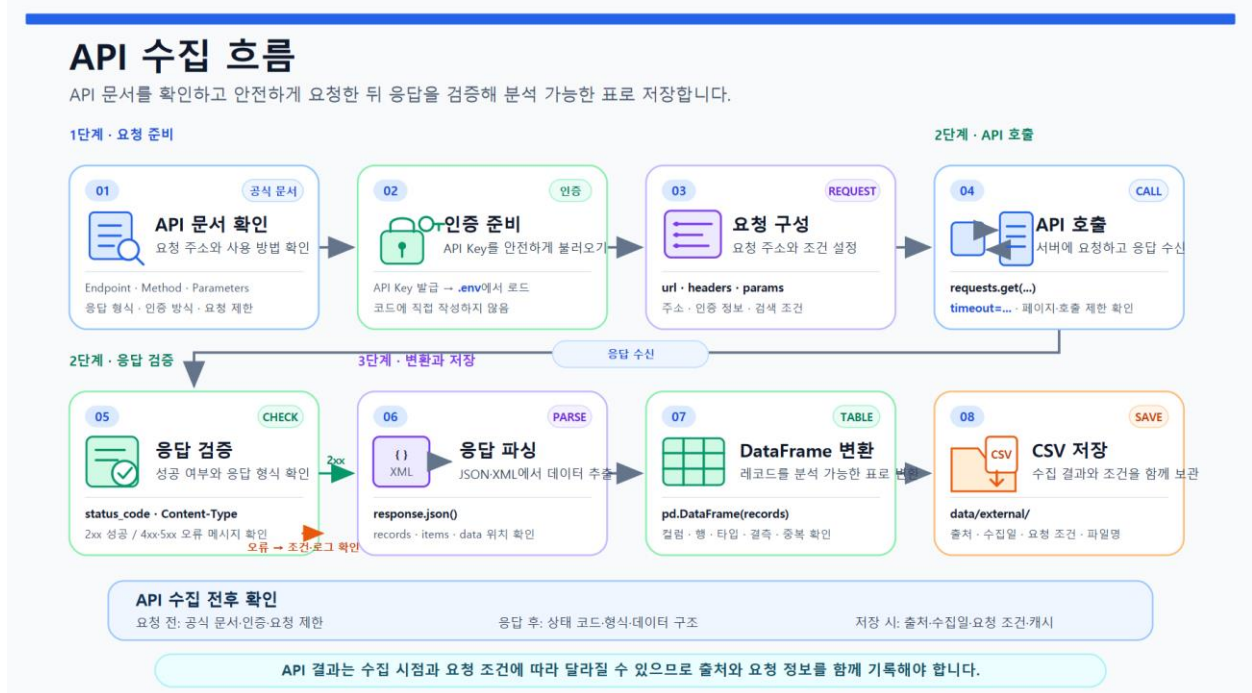


그림 13-3. API 수집과 검증 흐름

8. 네이버 블로그 검색 API를 사용한다

네이버 블로그 검색 API는 HTTP 헤더에 Client ID와 Client Secret을 전달합니다. 공식 문서 기준으로 display는 1~100, start는 1~1000이며, 정렬은 sim 또는 date를 사용합니다. 호출 한도와 정책은 변경될 수 있으므로 개발자 센터에서 다시 확인합니다.



그림 13-4. API Key 안전 관리

Notebook에서는 실제 네트워크 호출을 기본적으로 끕니다.

```

from src.external_data_collection import (
    naver_blog_items_to_dataframe,
    save_json_snapshot,
    search_naver_blog,
)

RUN_NAVER_API = False

if RUN_NAVER_API:
    result, metadata = search_naver_blog(
        "제주 여행",
        display=10,
        start=1,
        sort="sim",
        session=session,
    )

    raw_path = save_json_snapshot(
        result,
        setup_result["paths"]["raw"]
        / "naver_blog_jeju.json",
    )

    naver_blog_df = naver_blog_items_to_dataframe(
        result
    )
    processed_path = (
        setup_result["paths"]["processed"]
        / "naver_blog_jeju.csv"
    )
    naver_blog_df.to_csv(
        processed_path,
        index=False,
        encoding="utf-8-sig",
    )

    display(metadata)
    display(naver_blog_df.head())

```

제목과 설명에는 검색어 강조용 HTML 태그가 포함될 수 있으므로 정제 컬럼을 사용합니다. postdate는 날짜형으로 변환합니다.

검색 API 결과는 검색어, 정렬 방식, 페이지 범위, 수집 시각, 검색 정책의 영향을 받습니다. 전체 여론, 시장 규모, 실제 구매 수요를 대표한다고 단정하지 않습니다.

9. 크롤링은 robots.txt와 이용약관을 각각 확인한다

robots.txt는 자동 수집 클라이언트가 따라야 할 접근 규칙을 전달하는 표준입니다. 그러나 robots.txt 허용은 계약상 이용허락이나 저작권 사용 허가가 아닙니다. 이용약관, 라이선스, 개인정보, 콘텐츠 사용 범위는 별도로 확인해야 합니다.

공통 모듈의 크롤링 함수는 다음 조건을 적용합니다.

- http와 https만 허용
- localhost와 사설·루프백 주소 차단
- robots.txt 확인
- 이용약관 확인 여부를 `policy_confirmed=True`로 명시
- 리다이렉트 발생 시 자동 추적하지 않고 재검토
- HTML 응답만 허용
- 응답 크기 제한
- timeout과 HTTP 오류 처리


```

from src.external_data_collection import (
    extract_title_and_links,
    fetch_public_html,
    save_text_snapshot,
)

RUN_CRAWLING_EXAMPLE = False
POLICY_CONFIRMED = False
TARGET_URL = "https://example.com/"

if RUN_CRAWLING_EXAMPLE and POLICY_CONFIRMED:
    html, metadata = fetch_public_html(
        TARGET_URL,
        policy_confirmed=True,
        respect_robots=True,
        session=session,
    )

    raw_html_path = save_text_snapshot(
        html,
        setup_result["paths"]["raw"]
        / "example_page.html",
    )

    page_title, links_df = extract_title_and_links(
        html,
        base_url=TARGET_URL,
    )

    links_df.to_csv(
        setup_result["paths"]["processed"]
        / "example_links.csv",
        index=False,
        encoding="utf-8-sig",
    )

    print(page_title)
    display(metadata)
    display(links_df.head())

```

실제 사이트에서는 다음을 추가로 확인합니다.

- 로그인이나 유료 구독이 필요한가?

- 자동 수집을 금지하는 이용약관이 있는가?
- 페이지 원문을 재배포해도 되는가?
- 개인정보나 사용자 작성 콘텐츠가 포함되는가?
- 한 페이지 확인인지 반복·대량 수집인지
- 서버 부담을 줄일 요청 간격과 캐시 전략이 있는가?

User-Agent를 브라우저처럼 속여 접근 제한을 우회하지 않습니다. 자동 수집 주체를 식별할 수 있는 User-Agent를 사용합니다.

10. 원본, 정제 결과, 메타데이터를 함께 보관한다

같은 데이터를 다시 만들 수 있도록 다음을 구분해 저장합니다.

| 구분 | 예시 |
|-------|-----------------------------------|
| 원본 | API JSON, 내려받은 CSV, 허용된 HTML |
| 정제 결과 | 필요한 컬럼만 정리한 CSV |
| 메타데이터 | 제공 기관, URL, 기준일, 수집 UTC 시각, 요청 범위 |
| 무결성 | 원본 파일 SHA-256 |
| 검증 | 행·열 수, 결측치, 중복, 파싱 실패 |
| 정책 | 라이선스·이용약관·robots.txt 확인 결과 |

파일 해시를 기록하면 이후 원본이 바뀌었는지 확인할 수 있습니다.

```
from src.external_data_collection import sha256_file

# raw_file_hash = sha256_file(raw_path)
```

수집 일시와 데이터 기준일은 다릅니다. 오늘 내려받은 파일이 작년 기준 통계일 수 있으므로 두 값을 분리해 기록합니다.

11. 외부 데이터를 병합할 때 행 증가를 검증한다

외부 데이터는 날짜, 지역, 카테고리, 키워드, 위치 중 하나로 내부 데이터와 연결합니다.



그림 13-5. 외부 데이터와 내부 데이터 연결

연결 전에 분석 단위를 맞춥니다.

- 일별과 월별 데이터
- 한국 표준시와 UTC
- 서울, 서울특별시, 행정구역 코드
- 상품 카테고리 분류 체계
- 위도·경도 좌표계
- 검색어와 수집 기간

공통 병합 함수는 `validate`와 `indicator`를 사용해 행 수와 미매칭을 확인합니다.

```
from src.external_data_collection import (
    merge_external_data,
)

merged_monthly, merge_check = merge_external_data(
    monthly_sales,
    external_monthly,
    on="order_month",
    how="left",
    validate="many_to_one",
)

display(merge_check)
```

left merge도 오른쪽 키가 중복되면 행 수가 증가할 수 있습니다. 병합 전후 행 수, left_only 건수, 기간과 키의 중복을 반드시 확인합니다.

12. LLM에는 공식 문서 일부와 검증 조건을 함께 제공한다

LLM은 API 호출 코드 초안을 만드는 데 유용하지만 오래된 주소, 존재하지 않는 파라미터, 잘못된 응답 구조를 제안할 수 있습니다. API Key와 실제 내부 URL을 전달하지 않고, 공식 문서에서 확인한 구조만 제공합니다.

역할:

외부 데이터 수집 코드 검토자

목적:

공식 JSON API에서 필요한 필드만 수집합니다.

공식 문서에서 확인한 정보:

- 요청 URL: 실제 공식 URL
- HTTP 메서드: GET
- 인증 위치: 헤더 또는 serviceKey 파라미터
- 파라미터: 공식 문서에 있는 항목만 작성
- 응답 구조: 실제 JSON 경로
- 호출 제한: 공식 문서 기준

요청:

1. requests와 params 딕셔너리 사용
2. 연결·읽기 timeout 지정
3. 429와 일시적 5xx만 제한적으로 재시도
4. raise_for_status와 JSON 파싱 오류 처리
5. 원본 JSON과 정제 CSV 분리 저장
6. 출처, UTC 수집 시각, 요청 범위 기록

제약:

- API Key를 코드, URL 로그, 오류 메시지에 노출하지 말 것
- 공식 문서에 없는 파라미터를 만들지 말 것
- 로그인·접근 제한 우회 코드를 제안하지 말 것
- 검색·크롤링 결과를 전체 여론이나 인과관계로 해석하지 말 것

LLM 답변은 다음 순서로 검증합니다.

29. 공식 URL과 파라미터가 맞는가?
30. 인증 정보가 노출되지 않는가?
31. timeout과 오류 처리가 있는가?
32. 응답 JSON 경로가 실제와 일치하는가?
33. 페이지네이션과 호출 제한을 지키는가?
34. 원본과 정제 결과가 분리되는가?
35. 이용약관과 수집 범위를 우회하지 않는가?

13. 생성되는 결과 파일

네트워크를 호출하지 않아도 다음 준비 자료가 생성됩니다.

- reports/ch13_external_data_plan.csv
- reports/ch13_collection_method_summary.csv
- reports/ch13_external_integration_plan.csv
- reports/ch13_external_data_checklist.csv
- reports/ch13_external_data_log.csv
- reports/ch13_env_key_status.csv
- reports/ch13_external_data_summary.md

실제 수집을 수행하면 data/external/raw/, processed/, metadata/에 별도 파일을 저장합니다.

14. 제출 전 체크리스트

| 점검 항목 | 확인 |
|-----------------------------------|--------------------------|
| 분석 질문에 필요한 데이터만 수집했는가? | ☐ |
| 공식 파일 또는 공식 API를 먼저 확인했는가? | ☐ |
| 제공 기관, URL, 기준일, 수집 시각을 기록했는가? | ☐ |
| 라이선스와 이용약관을 확인했는가? | ☐ |
| API Key와 인증 헤더를 노출하지 않았는가? | ☐ |
| timeout, 오류 처리, 제한적 재시도를 적용했는가? | ☐ |
| 원본 응답과 정제 결과를 분리했는가? | ☐ |
| 크롤링 전 robots.txt와 이용약관을 각각 확인했는가? | ☐ |
| 로그인·접근 제한 우회를 하지 않았는가? | ☐ |
| 개인정보와 저작물 원문을 불필요하게 저장하지 않았는가? | ☐ |
| 병합 전후 행 수와 미매칭을 확인했는가? | ☐ |
| 검색 결과를 전체 수요나 여론으로 단정하지 않았는가? | ☐ |
| LLM 코드가 공식 문서와 일치하는지 확인했는가? | ☐ |

15. 정리

이번 장에서 익힌 핵심 원칙은 다음과 같습니다.

- 외부 데이터 수집은 분석 질문에서 시작합니다.

- 공식 파일과 공식 API를 크롤링보다 우선합니다.
- API 주소와 정책은 출간 예제가 아니라 현재 공식 문서를 기준으로 확인합니다.
- 인증 정보는 코드, 로그, URL, GitHub에서 분리합니다.
- 네트워크 요청에는 timeout, 오류 처리, 제한적 재시도를 포함합니다.
- robots.txt와 이용약관은 서로 다른 확인 항목입니다.
- 원본, 정제 결과, 메타데이터, 파일 해시를 함께 관리합니다.
- 외부 데이터 병합 후 행 수와 미매칭을 검증합니다.
- 검색 결과나 동시 변화를 인과관계로 단정하지 않습니다.

다음 장에서는 수집·분석·보고 흐름을 반복 실행할 수 있도록 자동화 파이프라인으로 연결합니다.

14장. 반복되는 분석 흐름을 안전하게 자동화하기

데이터 분석 업무는 한 번 실행하고 끝나는 경우보다 같은 흐름을 반복하는 경우가 더 많습니다. 새 파일을 확인하고, 전처리하고, 지표를 계산하고, 그래프와 보고서를 만든 뒤 결과가 정상인지 검증하는 과정이 매일 또는 매주 반복될 수 있습니다.

수동 실행은 단계가 적을 때는 편리합니다. 그러나 처리 단계가 늘어나면 실행 순서를 놓치거나, 이전 실행의 파일을 새 결과로 착각하거나, 일부 단계가 실패했는데도 전체 작업이 끝난 것으로 오해하기 쉽습니다. 분석 자동화는 이런 반복 작업을 **정해진 순서, 입력·출력 계약, 실패 기준, 재실행 규칙**에 따라 운영하는 과정입니다.

이 장에서는 Make, n8n, Airflow의 역할을 구분하고, 온라인 쇼핑몰 분석 흐름을 Python으로 먼저 검증한 뒤 Docker Compose 기반 Airflow 3.3.0 학습 환경에서 실행합니다.

이 장의 Docker Compose 구성은 로컬 학습과 탐색을 위한 것입니다. 운영 환경의 보안·고가용성·백업·모니터링을 보장하는 배포 템플릿이 아닙니다.



그림 14-1. 분석 자동화와 파이프라인 전체 흐름도

이 장에서 생각해 볼 질문

- 각 Task의 입력과 출력은 무엇인가?
- 같은 파이프라인을 다시 실행해도 결과가 중복되지 않는가?
- 파일이 존재한다는 사실만으로 성공이라고 판단해도 되는가?
- 취소·환불 주문을 포함한 금액을 매출이라고 불러도 되는가?
- 실패한 Task를 재실행할 때 이전의 부분 결과가 남지 않는가?
- DAG의 스케줄과 타임존은 명확한가?
- 비밀번호와 JWT secret이 저장소에 하드코딩되어 있지 않은가?
- Airflow Task 성공과 분석 결과의 타당성을 구분하고 있는가?

1. 자동화는 분석 코드를 대신 쓰는 일이 아니다

자동화는 분석 코드를 없애는 것이 아니라, 검증된 코드를 안정적인 순서로 실행하도록 만드는 일입니다. 전처리, 분석, 시각화, 보고서 생성 코드가 한 파일에 뒤섞여 있으면 어느 단계가 실패했는지 확인하기 어렵습니다.

이번 장의 파이프라인은 다음 계약을 사용합니다.

| Task | 주요 입력 | 주요 출력 | 실패 기준 |
|-------------------------|----------------|----------------------------|--------------------------------|
| check_input_files | data/raw/*.csv | 입력 점검 결과 | 파일 누락 또는 빈 파일 |
| run_preprocessing | 원본 CSV 4개 | data/processed/*_clean.csv | 컬럼·타입·키·참조
무결성 오류 |
| run_analysis | 전처리 CSV | 일자·카테고리 집계 CSV | 완료 주문 없음, 병합
오류 |
| generate_visualizations | 일자별 집계 | PNG 그래프 | 집계 파일 누락 또는
빈 결과 |
| generate_report | 집계·메타데이터·그래프 | Markdown 보고서 | 필수 결과 누락 |
| validate_outputs | 전체 산출물 | 검증 로그 CSV | 최신성·행
수·총합·범위 문구
오류 |

각 Task는 자신의 산출물을 전체 파일로 다시 만들고 원자적으로 교체합니다. 따라서 동일한 입력으로 재실행해도 행이 계속 추가되지 않습니다. 이런 성질을 멱등성(idempotency)이라고 합니다.

2. Make, n8n, Airflow의 역할

| 도구 | 잘 맞는 상황 | 예시 |
|---------|-----------------|------------------------------|
| Make | 외부 SaaS 연결과 알림 | 보고서 생성 후 Gmail·Slack 전달 |
| n8n | 노코드·로우코드 워크플로우 | API 호출, 내부 도구 연결, 승인 흐름 |
| Airflow | 코드 기반 데이터 파이프라인 | 전처리 → 분석 → 보고서 실행 순서와 재시도 관리 |

Airflow는 분석 결과를 자동으로 옮겨 만들어 주는 도구가 아닙니다. Airflow가 관리하는 것은 실행 순서, 스케줄, 재시도, 로그, 상태입니다. 금액 집계 기준과 결과 해석은 분석 코드와 사람이 책임집니다.

3. 이번 장의 금액 기준

`line_total`은 `quantity × unit_price`로 계산한 주문 상세 금액입니다. 전체 주문 상세 금액을 곧바로 회계상 순매출이라고 부를 수는 없습니다. 할인, 배송비, 세금, 부분 환불, 정산 시점이 반영되지 않을 수 있기 때문입니다.

이번 장에서는 다음 범위를 사용합니다.

- 집계 대상:
 - `order_status == "completed"`
- 금액 정의:
 - 주문 상세의 `quantity × unit_price` 합계
- 표현:
 - 완료 주문 기준 금액
 - 회계상 순매출로 단정하지 않음

일자별 집계와 카테고리별 집계는 동일한 완료 주문 범위를 사용하며, 마지막 검증 Task에서 두 집계의 총합이 일치하는지 확인합니다.

4. 프로젝트 구조

```

llm-data-analysis-course/
├── data/
│   ├── raw/
│   └── processed/
├── scripts/
│   ├── generate_sample_data.py
│   ├── ch14_preprocessing.py
│   ├── ch14_analysis.py
│   ├── ch14_visualization.py
│   ├── ch14_report.py
│   ├── ch14_validate_outputs.py
│   └── run_ch14_pipeline.py
├── src/
│   └── automation_pipeline.py
├── reports/
│   └── figures/
├── notebooks/
│   └── ch14_airflow_pipeline.ipynb
└── automation/
    ├── airflow/
    │   ├── Dockerfile
    │   ├── docker-compose.yml
    │   ├── requirements.txt
    │   ├── .env.example
    │   └── dags/
    │       └── ch14_local_analysis_pipeline.py

```

원본 데이터 폴더에는 다음 파일이 있어야 합니다.

```

customers.csv
products.csv
orders.csv
order_items.csv

```

파일이 없다면 프로젝트 루트에서 실행합니다.

```
python scripts/generate_sample_data.py
```

5. Airflow를 실행하기 전에 Python 파이프라인 검증

Airflow에서 실패하는 원인은 크게 두 종류입니다.

36. 데이터 또는 Python 분석 코드의 문제

37. Docker·Airflow 실행 환경의 문제

두 문제를 분리하기 위해 로컬 Python 파이프라인을 먼저 실행합니다.

```
python scripts/generate_sample_data.py
python scripts/run_ch14_pipeline.py
```

정상 실행되면 다음 주요 파일이 생성됩니다.

```
data/processed/customers_clean.csv
data/processed/products_clean.csv
data/processed/orders_clean.csv
data/processed/order_items_clean.csv
reports/ch14_daily_sales.csv
reports/ch14_category_sales.csv
reports/ch14_pipeline_task_summary.csv
reports/ch14_airflow_setup_guide.csv
reports/ch14_pipeline_run_metadata.csv
reports/figures/ch14_daily_sales.png
reports/ch14_airflow_report.md
reports/ch14_airflow_validation_log.csv
```

검증 로그는 파일의 존재와 크기만 확인하지 않습니다.

- 원본 파일보다 산출물이 오래되지 않았는지
- CSV에 실제 데이터 행이 있는지
- 일자별 금액과 카테고리별 금액의 총합이 일치하는지
- 카테고리 비율 합계가 약 100%인지
- 보고서에 완료 주문 기준과 순매출 한계가 명시되었는지

로컬 실행이 실패하면 Airflow를 시작하기 전에 데이터와 Python 코드를 먼저 수정합니다.

6. Docker Compose 요구사항

Airflow 공식 Docker Compose 빠른 시작은 학습과 탐색에 적합하지만 운영 배포용은 아닙니다. 실습 전 다음을 확인합니다.

```
docker --version
docker compose version
docker run --rm hello-world
```

확인할 기준은 다음과 같습니다.

- Compose V2를 사용합니다.
- Airflow 공식 빠른 시작 기준으로 Compose 2.14 이상이 필요합니다.
- Docker Engine에 최소 4GB, 가능하면 8GB 정도의 메모리를 할당합니다.
- Linux에서는 마운트 폴더 권한 문제를 줄이기 위해 `AIRFLOW_UID=$(id -u)`를 사용합니다.

Docker 설치에 다음 별도 가이드를 참고합니다.

```
https://blog.naver.com/dev-dog/224341211248
```

Airflow 공식 Docker Compose 문서도 함께 확인합니다.

```
https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html
```

7. 환경 파일과 비밀번호 준비

Airflow 폴더로 이동하고 예시 파일을 복사합니다.

```
cd automation/airflow
cp .env.example .env
```

Windows PowerShell:

```
Copy-Item .env.example .env
```

`.env.example`에는 변수 이름만 두고 비밀번호는 비워 둡니다.

```
AIRFLOW_UID=50000
AIRFLOW_PROJECT_DIR=../..
AIRFLOW_VERSION=3.3.0
AIRFLOW_UI_PORT=8080
AIRFLOW_DEFAULT_TIMEZONE=utc
AIRFLOW_DAG_TIMEZONE=Asia/Seoul
AIRFLOW_DB_PASSWORD=
AIRFLOW_API_JWT_SECRET=
_AIRFLOW_WWW_USER_USERNAME=airflow
_AIRFLOW_WWW_USER_PASSWORD=
```

비어 있는 `AIRFLOW_DB_PASSWORD`, `AIRFLOW_API_JWT_SECRET`, `_AIRFLOW_WWW_USER_PASSWORD` 값은 시작 전에 반드시 채웁니다. 세 항목에는 서로 다른 임의 값을 사용합니다. 빈 값이면 Docker Compose의 필수 환경변수 검사에서 실행이 중단됩니다.

Python으로 임의 문자열을 만들 수 있습니다.

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

중요한 원칙은 다음과 같습니다.

- 실제 `.env`는 Git에 커밋하지 않습니다.
- `.env.example`에는 변수 이름만 두고 비밀값은 비워 둡니다.
- 운영 비밀번호를 `airflow`, `admin`, `password`처럼 쉽게 추측할 수 있는 값으로 두지 않습니다.
- Docker Compose에 JWT secret이나 DB 비밀번호를 직접 적지 않습니다.
- 실제 비밀값이 Git에 포함되었다면 삭제만 하지 말고 즉시 폐기·교체합니다.

8. 이미지 빌드와 Airflow 초기화

이 저장소는 필요한 Python 패키지가 포함된 사용자 정의 이미지를 사용합니다.

```
docker compose build
docker compose up airflow-init
```

초기화가 정상적으로 끝나면 `airflow-init` 컨테이너가 종료 코드 0으로 끝납니다.

Airflow를 백그라운드에서 시작합니다.

```
docker compose up -d
docker compose ps
```

`postgres`, `airflow-api-server`, `airflow-scheduler`, `airflow-dag-processor`, `airflow-triggerer`가 실행 중인지 확인합니다. `unhealthy` 상태이거나 계속 재시작하는 서비스가 있으면 로그를 확인합니다.

```
docker compose logs --tail 100 airflow-api-server
docker compose logs --tail 100 airflow-scheduler
docker compose logs --tail 100 airflow-dag-processor
```

9. 로그인

브라우저에서 `.env`의 `AIRFLOW_UI_PORT`에 맞는 주소로 접속합니다.

http://localhost:8080

Username에는 `_AIRFLOW_WWW_USER_USERNAME`, Password에는 `_AIRFLOW_WWW_USER_PASSWORD`로 직접 설정한 값을 입력합니다.

Airflow 로그인 화면이 나타났을 때
저장소의 고정 비밀번호가 아니라, 본인이 .env에 설정한 값을 입력합니다.

http://localhost:\$(AIRFLOW_UI_PORT:-8080)

1. .env 확인

로그인 값 확인

Username
`_AIRFLOW_WWW_USER_USERNAME`

Password
`_AIRFLOW_WWW_USER_PASSWORD`

Sign into Airflow
Enter the values configured in your .env file.

Username
airflow

Password

Sign in

2. 실패 시 점검

설정과 로그 확인
계정을 추측하지 말고 현재 구성을 확인합니다.

```
docker compose config
docker compose logs --tail 100 airflow-init
docker compose logs --tail 100 airflow-api-server
```

환경값을 바꾼 뒤 기존 DB가 남아 있으면
학습 기록 삭제 가능 여부를 확인한 후 불륨을 초기화합니다.

로그인 확인 흐름
.env 값 확인 → 로그인 → 실패 시 Compose 설정과 로그 확인

실제 .env는 Git에 커밋하지 않으며, 기본 예시 비밀번호를 운영 환경에 재사용하지 않습니다.

로그인이 되지 않으면 하드코딩된 다른 계정을 추측하지 말고 현재 Compose 설정을 확인합니다.

```
docker compose config
docker compose logs --tail 100 airflow-init
docker compose logs --tail 100 airflow-api-server
```

.env를 수정한 뒤 이미 생성된 메타데이터 DB와 계정이 남아 있다면 새 값이 즉시 적용되지 않을 수 있습니다. 학습 환경을 완전히 초기화해도 되는 경우에만 다음 명령을 사용합니다.

```
docker compose down --volumes --remove-orphans
docker compose up airflow-init
docker compose up -d
```

이 명령은 DAG 실행 기록과 계정 정보도 삭제합니다.

10. Airflow 3 TaskFlow DAG 구조

DAG 파일은 다음 위치에 있습니다.

automation/airflow/dags/ch14_local_analysis_pipeline.py

Airflow 3의 공개 인터페이스인 airflow.sdk의 @dag, @task를 사용합니다.

```
import os
from datetime import timedelta
from pathlib import Path

import pendulum
from airflow.sdk import dag, task

BASE_DIR = Path(
    os.getenv("PROJECT_ROOT", "/opt/airflow/project")
).resolve()

DAG_TIMEZONE = os.getenv(
    "AIRFLOW_DAG_TIMEZONE",
    "Asia/Seoul",
)

@dag(
    dag_id="ch14_local_analysis_pipeline",
    schedule=None,
    start_date=pendulum.datetime(
        2026, 1, 1, tz=DAG_TIMEZONE
    ),
    catchup=False,
    max_active_runs=1,
)
def pipeline():
    @task(
        retries=1,
        retry_delay=timedelta(minutes=1),
        execution_timeout=timedelta(minutes=10),
    )
    def preprocessing():
        run_preprocessing(BASE_DIR)

    # 나머지 Task도 같은 방식으로 정의합니다.
```

핵심 설정은 다음과 같습니다.

| 설정 | 의미 |
|---------------|-----------------------|
| schedule=None | 수업에서는 자동 스케줄 없이 수동 실행 |

| 설정 | 의미 |
|-------------------|-----------------------------|
| 타임존 포함 start_date | 모호한 로컬 시각을 피함 |
| catchup=False | 과거 실행 구간을 자동 생성하지 않음 |
| max_active_runs=1 | 같은 DAG가 동시에 파일을 덮어쓰지 않도록 제한 |
| retries=1 | 일시적 오류에 한 번 재시도 |
| execution_timeout | 멈춘 Task가 무한정 실행되지 않도록 제한 |

매일 오전 9시 실행으로 바꾸려면 운영 목적과 데이터 도착 시각을 확인한 뒤 다음처럼 변경할 수 있습니다.

```
@dag(
    schedule="0 9 * * *",
    start_date=pendulum.datetime(
        2026, 1, 1, tz="Asia/Seoul"
    ),
    catchup=False,
)
```

자동 스케줄을 적용하기 전에 맥등성과 지연 데이터 처리 기준을 먼저 확인합니다.



그림 14-2. 분석 파이프라인의 Task 의존성 흐름

11. DAG 확인과 실행

UI의 **Dags** 메뉴에서 `ch14_local_analysis_pipeline`을 찾습니다.

38. DAG 상세 화면을 엽니다.
39. 수동 실행 버튼으로 실행합니다.
40. Grid 또는 Graph 화면에서 Task 순서를 확인합니다.
41. 실패한 Task를 클릭해 로그를 확인합니다.
42. 마지막 `validate_outputs`까지 성공했는지 확인합니다.

Airflow 로그인 후 처음 열리는 화면
 홈 대시보드에서 전체 상태를 확인한 뒤, 실제 실습은 왼쪽 Dags 메뉴에서 진행합니다.

처음 확인할 4가지

- 1 DAG 현황 요약**
실패·실행 중·활성 DAG 수를 먼저 확인합니다.
- 2 시스템 상태**
메타데이터 DB, 스케줄러 등 핵심 구성요소가 초록색인지 봅니다.
- 3 조회 기간**
지난 24시간 등 원하는 기간으로 통계 범위를 조정합니다.
- 4 실행 기록**
DAG 실행 결과와 성공·실패 상태를 확인하고, 상세 작업은 왼쪽 Dags 메뉴에서 진행합니다.

홈 화면은 전체 상태를 빠르게 확인하는 요약 화면이며, 실제 실습은 **Dags** 메뉴에서 진행합니다.

Airflow DAG 목록에서 실습 DAG 찾기

DAG 이름을 확인하고 활성화한 뒤 상세 화면으로 이동합니다.

1 DAG 이름 확인
'ch14_local_analysis_pipeline'인지 정확히 확인합니다.

2 활성화 토글 ON
일시 중지 상태라면 토글을 켜서 DAG를 활성화합니다.

3 상세 화면 이동
DAG 이름 또는 상세 버튼을 눌러 수동 실행 화면으로 이동합니다.

실습 DAG 확인 → 활성화 → 상세 화면 이동 → 수동 실행 순서로 진행합니다.

Airflow DAG 실행 성공 화면 확인

최근 실행, Task 상태, 검증 로그를 순서대로 확인합니다.

1 최근 실행 성공 확인
최근 실행 시간 옆의 초록색 체크와 성공 배지를 확인합니다.

2 모든 Task 성공 확인
입력 확인부터 결과 검증까지 여섯 개 Task가 모두 초록색인지 봅니다.

3 검증 로그 최종 확인
reports/ch14_airflow_validation_log.csv의 모든 status가 ok인지 확인합니다.

초록색 표시만 보지 말고, 최근 실행·Task 전체·검증 로그를 함께 확인합니다.

CLI로 DAG가 정상 파싱되는지 확인할 수도 있습니다.

```
docker compose exec airflow-scheduler W
airflow dags list | grep ch14_local_analysis_pipeline
```

특정 논리 시각으로 전체 DAG를 테스트하려면 다음 명령을 사용할 수 있습니다.

```
docker compose exec airflow-scheduler W
airflow dags test ch14_local_analysis_pipeline 2026-07-01
```

airflow dags test 성공은 DAG 코드의 단일 테스트 실행을 의미합니다. 웹 UI의 Scheduler-Executor-API 통합 실행과 완전히 같은 검증은 아니므로 UI 실행도 별도로 확인합니다.

12. 산출물 검증

프로젝트 루트에서 검증 로그를 확인합니다.

```
cat reports/ch14_airflow_validation_log.csv
```

Windows PowerShell:

```
Get-Content reports\ch14_airflow_validation_log.csv
```

모든 status가 ok인지 확인합니다. 특히 다음 행이 중요합니다.

- daily_vs_category_completed_order_amount
- category_amount_ratio_pct_sum
- report_scope

Task가 모두 초록색이더라도 검증 로그에 error가 있으면 분석 파이프라인은 완료된 것이 아닙니다.

13. 실패 상황 실습

자동화에서는 성공 화면보다 실패 원인을 읽는 능력이 중요합니다. 원본 파일을 삭제하지 말고 이름을 임시 변경합니다.

macOS·Linux·WSL2:

```
mv data/raw/customers.csv ₩
data/raw/customers.csv.exercise-backup
```

Windows PowerShell:

```
Rename-Item data\raw\customers.csv `
customers.csv.exercise-backup
```

DAG를 실행하면 check_input_files에서 실패해야 합니다. 로그에서 누락 파일 경로가 정확히 표시되는지 확인합니다.

실습 후 반드시 복구합니다.

```
Rename-Item data\raw\customers.csv.exercise-backup `
customers.csv
```

실제 업무 데이터에서는 이런 실패 실습을 진행하지 않습니다.

14. 자주 발생하는 문제

14.1 컨테이너가 계속 재시작함

Docker Desktop에 할당된 메모리가 부족할 수 있습니다. Airflow 공식 빠른 시작은 최소 4GB, 가능하면 8GB 정도를 권장합니다.

```
docker compose ps
docker compose logs --tail 200 airflow-api-server
```

14.2 8080 포트 충돌

.env에서 포트를 변경합니다.

```
AIRFLOW_UI_PORT=8081
```

브라우저 주소도 변경합니다.

```
http://localhost:8081
```

14.3 ModuleNotFoundError

사용자 정의 이미지를 다시 빌드합니다.

```
docker compose build --no-cache
docker compose up airflow-init
docker compose up -d
```

Dockerfile은 Airflow 이미지와 같은 버전의 apache-airflow를 함께 지정해 의존성 충돌 가능성을 줄입니다.

14.4 Linux 파일 권한 오류

.env의 AIRFLOW_UID를 현재 사용자 ID로 설정합니다.

```
echo "AIRFLOW_UID=$(id -u)" >> .env
```

기존 AIRFLOW_UID 행이 있다면 중복 추가하지 말고 값을 교체합니다. 무조건 `chmod -R 777`을 적용하는 방식은 피합니다.

14.5 JWT 또는 API 인증 오류

Scheduler와 API 서버가 같은 `AIRFLOW_API_JWT_SECRET`을 사용해야 합니다.

```
docker compose exec airflow-scheduler ₩
airflow config get-value api_auth jwt_secret

docker compose exec airflow-api-server ₩
airflow config get-value api_auth jwt_secret
```

값 자체를 화면 공유나 보고서에 노출하지 않습니다.

14.6 이전 실행 결과가 새 결과처럼 보임

검증 로그의 최신성 항목과 `ch14_pipeline_run_metadata.csv`의 생성 시각을 확인합니다. 원본 데이터보다 결과가 오래되었다면 파이프라인을 다시 실행합니다.

15. 종료와 초기화

실습을 종료하되 실행 기록을 유지하려면 다음 명령을 사용합니다.

```
docker compose down
```

DB 볼륨과 실행 기록까지 삭제하려면 다음 명령을 사용합니다.

```
docker compose down --volumes --remove-orphans
```

초기화 명령은 복구가 필요한 경우나 완전한 재실습이 필요한 경우에만 사용합니다.

16. Make와 n8n으로 결과 전달하기

Airflow는 코드 기반 데이터 처리와 검증을 담당하고, Make와 n8n은 승인된 외부 서비스로 결과를 전달하는 데 사용할 수 있습니다.

| 구간 | 담당 도구 예시 | 역할 |
|--------|---------------------|---------------------------|
| 데이터 처리 | Python, Airflow | 전처리, 분석, 시각화, 보고서 생성 |
| 결과 검증 | Python, Airflow | 최신성, 행 수, 총합, 파일 상태 확인 |
| 외부 전달 | Make, n8n | Gmail, Slack, Drive 등과 연결 |
| 운영 확인 | Airflow-Make-n8n 로그 | 실패 위치와 재실행 여부 확인 |

전달 단계에는 다음 조건을 둡니다.

- `validate_outputs`가 성공한 경우에만 전달합니다.
- 개인정보나 주문 식별자가 포함된 내부 파일을 공개 채널에 보내지 않습니다.
- API 키와 웹훅 주소는 각 도구의 비밀번호 저장 기능에 보관합니다.
- 재시도 시 동일한 보고서가 중복 발송되지 않도록 실행 ID를 기록합니다.

17. LLM에게 파이프라인 설계를 요청할 때

LLM은 DAG와 Docker Compose 초안을 만드는 데 도움을 줄 수 있지만, 생성한 설정을 바로 실행하면 안 됩니다.

온라인 쇼핑몰 분석 파이프라인을 설계해 주세요.

실행 환경:

- Airflow 3.3.0
- Docker Compose 기반 로컬 학습 환경
- 프로젝트 경로: /opt/airflow/project
- DAG 타임존: Asia/Seoul
- 수동 실행: schedule=None

금액 범위:

- order_status == completed
- quantity × unit_price 합계
- 회계상 순매출로 단정하지 않음

필요한 Task:

- 입력 파일 존재·스키마 검사
- 전처리와 키 무결성 검사
- 일자별·카테고리별 집계
- 그래프와 Markdown 보고서 생성
- 산출물 최신성·행 수·총합 검증

운영 조건:

- 재실행해도 중복되지 않아야 함
- max_active_runs=1
- Task별 retry와 execution_timeout 명시
- 비밀값은 .env로 분리하고 예시 파일에는 값 저장 금지
- Docker Compose는 운영 배포용이 아님을 명시

요청:

1. airflow.sdk의 @dag, @task를 사용해 주세요.
2. 타임존이 포함된 start_date를 사용해 주세요.
3. 각 Task의 입력·출력·실패 조건을 설명해 주세요.
4. 분석 결과 검증과 Airflow 실행 성공을 구분해 주세요.
5. 실제 비밀번호나 API 키를 예시로 만들지 마세요.

LLM이 만든 코드는 Airflow 버전, import 경로, 파일 마운트, 타임존, 권한, 비밀정보 노출 여부를 사람이 검토해야 합니다.

18. 최종 체크리스트

| 검토 항목 | 확인 |
|--|--------------------------|
| Python 파이프라인을 Airflow 전에 실행했는가? | ☐ |
| 완료 주문 기준과 금액 정의가 명시되었는가? | ☐ |
| 전처리 단계에서 필수 컬럼·고유 키·참조 관계를 검증하는가? | ☐ |
| 병합에 관계 검증이 적용되었는가? | ☐ |
| 재실행해도 데이터가 중복되지 않는가? | ☐ |
| 산출물의 존재뿐 아니라 최신성·행 수·총합을 검증하는가? | ☐ |
| start_date가 타임존을 포함하는가? | ☐ |
| catchup=False, max_active_runs=1을 의도적으로 설정했는가? | ☐ |
| 비밀값이 .env로 분리되어 있는가? | ☐ |
| .env.example의 비밀값이 비어 있는가? | ☐ |
| 실제 .env가 Git에 포함되지 않는가? | ☐ |
| Docker Compose 구성이 학습용임을 명시했는가? | ☐ |
| 분석 성공과 업무 해석의 타당성을 구분하는가? | ☐ |

19. 정리

이 장에서는 반복 분석을 단순히 예약 실행하는 것이 아니라, 입력·출력 계약과 실패 기준을 가진 파이프라인으로 구성했습니다.

- Python 코드로 전체 흐름 검증
- 완료 주문 기준의 일관된 금액 집계
- 필수 컬럼·타입·키·참조 관계 검사
- 원자적 파일 저장과 멍등성 확보
- 산출물 최신성·행 수·총합 검증
- Airflow 3 TaskFlow API 기반 DAG
- 타임존이 포함된 start_date
- catchup=False, max_active_runs=1, 재시도와 실행 제한
- .env를 통한 비밀정보 분리
- Docker Compose 학습 환경과 운영 배포의 구분
- Make·n8n을 통한 승인된 결과 전달

다음 장에서는 지금까지 배운 EDA, 시각화, 머신러닝, LLM, 외부 데이터, 자동화 흐름을 하나의 최종 프로젝트로 통합합니다.

15장. 하나의 데이터 분석 프로젝트로 완성하기

지금까지 데이터 구조 이해, pandas 분석, 전처리, EDA, 시각화, 머신러닝, LLM 활용, 외부 데이터 수집과 자동화 설계를 차례로 살펴보았습니다. 최종 프로젝트에서는 이 내용을 한 번에 많이 실행하는 것이 아니라, **하나의 분석 질문을 기준으로 데이터·코드·검증·보고서를 연결하는 방법**을 연습합니다. 이번 장의 기본 주제는 다음과 같습니다.

- 온라인 쇼핑몰의 완료 주문 매출을 분석하고, 주문 취소 위험을 예측하며, 선택한 외부 데이터를 연결해 분석 범위를 확장한다.

최종 프로젝트의 품질은 코드 줄 수나 모델 개수로 판단하지 않습니다. 분석 범위가 명확한지, 계산 기준이 일관적인지, 병합과 모델 평가가 검증되었는지, 개인정보와 API Key가 제외되었는지, 결과를 과장하지 않았는지가 더 중요합니다.



그림 15-1. 최종 프로젝트 전체 흐름도

1. 프로젝트 목표와 범위를 먼저 정한다

최종 프로젝트는 다음 질문에 답할 수 있어야 합니다.

- 어떤 의사결정을 돕기 위한 분석인가?

- 매출에 포함하는 주문 상태는 무엇인가?
- 현재 데이터로 계산 가능한 지표는 무엇인가?
- 병합 후 행 수와 미매칭을 확인했는가?
- 머신러닝의 예측 시점과 타깃이 명확한가?
- LLM을 사용했다면 어떤 답변을 어떻게 검증했는가?
- 외부 데이터의 출처와 연결 기준을 기록했는가?
- 보고서의 관찰과 원인 가설을 구분했는가?

이번 실습에서는 **완료 주문(completed)만 매출로 계산**합니다. 취소·환불 주문 금액은 매출에서 제외하고 별도 범위표로 기록합니다.

```

전체 주문 상세 금액
= 완료 주문 매출
+ 취소·환불 등 제외 금액
  
```

2. 필수 단계와 선택 단계를 구분한다

모든 기능을 억지로 넣을 필요는 없습니다. 필수 단계가 안정적으로 실행되고 검증된 뒤 선택 단계를 추가합니다.

| 구분 | 단계 | 기준 |
|----|-----------|-------------------|
| 필수 | 데이터 구조 점검 | 행·열 수, 결측치, 중복 확인 |
| 필수 | 전처리 | 타입·상태값·양수 조건 정리 |
| 필수 | 키 관계 검증 | 고객·주문·상품 외래키 확인 |
| 필수 | 완료 주문 EDA | 카테고리·월·고객·상품 집계 |
| 필수 | 시각화 | 분석 질문과 연결된 그래프 |
| 필수 | 프로젝트 검증 | 총합·행 수·익명화·파일 확인 |
| 필수 | 최종 보고서 | 목적, 결과, 한계, 다음 단계 |
| 선택 | 분류 모델 | 데이터가 충분할 때 실행 |
| 선택 | 외부 데이터 통합 | 출처 파일이 있을 때 실행 |
| 선택 | LLM 활용 | 실제 사용 내역만 기록 |
| 선택 | 자동화 구현 | 설계 이후 별도 구현 |

최종 프로젝트 기본 파이프라인은 외부 네트워크를 호출하지 않습니다. 외부 데이터 파일이 없거나 분류 데이터가 부족하면 해당 단계는 오류를 숨기지 않고 **skipped** 또는 **warning** 상태로 기록합니다.

3. 프로젝트 산출물 구조



그림 15-2. 최종 프로젝트 산출물 구성

주요 산출물은 다음과 같습니다.

| 영역 | 주요 파일 |
|-------------|---|
| 데이터 개요 | reports/ch15_dataset_summary.csv |
| 전처리·키 검증 | ch15_preprocessing_comparison.csv, ch15_relationship_checks.csv |
| 병합·범위 검증 | ch15_merge_checks.csv, ch15_amount_scope_summary.csv |
| 완료 주문 EDA | ch15_category_sales.csv, ch15_monthly_sales.csv |
| 익명 고객·상품 결과 | ch15_customer_sales.csv, ch15_product_sales.csv |
| 시각화 | reports/figures/ch15_*.png |
| 분류 모델 | ch15_classification_*.csv |
| 외부 데이터 | ch15_external_integration_status.csv |
| LLM 기록 | ch15_llm_usage_log.csv |
| 자동화 설계 | ch15_automation_plan.md |
| 프로젝트 검증 | ch15_project_validation.csv |
| 최종 보고서 | ch15_final_report.md |
| 산출물 무결성 | ch15_project_deliverables.csv |

ch15_project_deliverables.csv에는 각 파일의 존재 여부, 크기와 SHA-256 해시가 기록됩니다.

4. 실행 환경 준비

전체 실습은 `notebooks/ch15_final_project.ipynb`에서 진행할 수 있습니다. 원본 데이터가 없다면 프로젝트 루트에서 먼저 실행합니다.

```
python scripts/generate_sample_data.py
```

프로젝트 전체 파이프라인은 다음 명령으로 실행합니다.

```
python scripts/run_final_project.py
```

Notebook에서는 프로젝트 루트를 자동으로 찾습니다.

```
from pathlib import Path
import sys

def find_project_root(start_path):
    start_path = Path(start_path).resolve()

    for candidate in [start_path, *start_path.parents]:
        if (
            (candidate / "requirements.txt").exists()
            and (candidate / "scripts").exists()
        ):
            return candidate

    raise FileNotFoundError(
        "프로젝트 루트 폴더를 찾을 수 없습니다."
    )

PROJECT_ROOT = find_project_root(Path.cwd())

if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
```

5. 공통 파이프라인을 실행한다

최종 프로젝트는 앞 장의 검증된 공통 함수를 다시 사용합니다. 장마다 비슷한 전처리와 병합 코드를 복사하면 계산 기준이 달라질 수 있기 때문입니다.

```
from src.final_project import run_final_project

result = run_final_project(
    PROJECT_ROOT,
    random_state=42,
)
```

핵심 결과는 다음 키로 확인합니다.

```
core = result["core"]
classification_result = result["classification"]
external_result = result["external"]

display(core["dataset_summary"])
display(core["preprocessing_comparison"])
display(result["validation"])
```

6. 데이터 품질과 키 관계를 확인한다

최종 보고서를 작성하기 전에 데이터 구조와 관계 검증을 확인합니다.

```
display(core["key_duplicate_checks"])
display(core["relationship_checks"])
display(core["public_tables"]["merge_checks"])
```

확인할 내용은 다음과 같습니다.

- 고객, 상품, 주문의 기본 키가 중복되지 않는가?
- `orders.customer_id`가 고객 테이블에 존재하는가?
- `order_items.order_id`가 주문 테이블에 존재하는가?
- `order_items.product_id`가 상품 테이블에 존재하는가?
- 병합 후 행 수가 예상과 같은가?
- 병합되지 않은 주문·상품·고객이 있는가?

`left merge`를 사용해도 오른쪽 키가 중복되면 행 수가 증가할 수 있습니다. 따라서 `how="left"`만으로 안전하다고 판단하지 않고 `validate`와 `indicator` 결과를 함께 확인합니다.

7. 완료 주문 매출 기준을 검증한다

분석 범위표를 확인합니다.

```
amount_scope = core["public_tables"][
    "amount_scope_summary"
]
display(amount_scope)
```

핵심 집계는 모두 완료 주문 기준이어야 합니다.

```
category_total = core["public_tables"][
    "category_sales"
][["total_sales"].sum()

monthly_total = core["public_tables"][
    "monthly_sales"
][["total_sales"].sum()

customer_total = core["public_tables"][
    "customer_sales"
][["total_sales"].sum()

product_total = core["public_tables"][
    "product_sales"
][["total_sales"].sum()

category_total, monthly_total, customer_total, product_total
```

네 값이 완료 주문 매출과 일치하지 않으면 어느 단계에서 주문 상태 필터가 빠졌는지 확인해야 합니다.

8. EDA와 시각화를 연결한다

프로젝트는 다음 질문을 중심으로 분석합니다.

| 분석 질문 | 결과표 | 시각화 |
|---------------------------|----------------------|----------|
| 어떤 카테고리가 완료 주문 매출에 기여하는가? | category_sales | 막대그래프 |
| 월별 완료 주문 매출은 어떻게 변하는가? | monthly_sales | 선그래프 |
| 완료 주문 구매 금액 상위 고객군은 누구인가? | customer_sales | 가로 막대그래프 |
| 주문 상태별 주문 수는 어떻게 다른가? | order_status_summary | 막대그래프 |


```
display(
  core["public_tables"]["category_sales"].head(10)
)
display(
  core["public_tables"]["monthly_sales"].head(12)
)
display(
  core["public_tables"]["customer_sales"].head(10)
)
```

고객 결과에는 원본 고객 ID, 이름, 이메일, 전화번호를 저장하지 않습니다. Customer 1 같은 익명 라벨만 외부 배포용 결과에 포함합니다.

그래프 파일은 reports/figures/에 저장됩니다.



그림 15-3. 최종 프로젝트 주요 분석 대시보드 예시

9. 머신러닝은 분류 모델 하나를 기본으로 사용한다

최종 프로젝트 기본 모델은 10장에서 만든 주문 취소 분류 모델입니다.

타깃 범위는 다음과 같습니다.

| 주문 상태 | 타깃 | 사용 여부 |
|-----------|----|-------|
| completed | 0 | 사용 |
| cancelled | 1 | 사용 |
| refunded | - | 제외 |
| 기타 상태 | - | 제외 |

분류 단계는 다음 원칙을 따릅니다.

43. 병합에 `validate`와 `indicator`를 사용합니다.
44. `order_status`와 `is_cancelled`를 입력값에서 제외합니다.
45. `train`, `validation`, `test`로 분리합니다.
46. `DummyClassifier`를 기준 모델로 사용합니다.
47. 모델과 임계값은 `validation` 데이터에서 선택합니다.
48. `test` 데이터는 최종 평가에 한 번만 사용합니다.
49. `accuracy`, `precision`, `recall`, `f1-score`와 혼동행렬을 함께 봅니다.

```
display(classification_result["status"])
display(classification_result["validation_comparison"])
display(classification_result["test_metrics"])
display(classification_result["confusion_matrix"])
```

샘플 수가 부족하거나 각 클래스에 필요한 건수가 없으면 분류 단계는 `skipped`로 기록됩니다. 데이터를 억지로 복제하거나 테스트 데이터로 모델을 선택하지 않습니다.

회귀 분석을 추가할 때

회귀 모델은 선택 단계입니다. 주문 금액은 수량과 단가에서 직접 계산되는 값이므로, 주문 완료 후 만들어진 `total_quantity`, `avg_unit_price`를 입력값으로 사용하면 성능이 높아도 실무 예측 의미가 약할 수 있습니다. 회귀를 추가할 때는 반드시 다음을 설명합니다.

- 예측 시점
- 그 시점에 사용 가능한 입력값
- 타깃을 직접 또는 거의 결정하는 변수
- 기준 모델
- 모델 선택 데이터와 최종 테스트 데이터의 분리

10. 외부 데이터는 실제 파일이 있을 때만 연결한다

13장과 같은 폴더 구조를 사용합니다.

```
data/external/
├── raw/
├── processed/
└── metadata/
```

공휴일 예시를 연결하려면 다음 파일을 준비합니다.

```
data/external/processed/holidays.csv
```

필수 컬럼은 다음과 같습니다.

```
date,holiday_name,is_holiday
```

가능하면 다음 출처 정보도 함께 관리합니다.

```
provider,source_url,data_reference_date,license_or_terms
```

외부 파일이 없으면 파이프라인은 가짜 공휴일 데이터를 만들지 않습니다. 대신 다음 템플릿을 생성하고 통합 단계를 `skipped`로 기록합니다.

```
reports/ch15_holidays_template.csv
display(external_result["status"])
display(external_result["comparison"])
display(external_result["merge_check"])
```

외부 데이터를 연결할 때는 다음을 확인합니다.

- 날짜가 정상적으로 변환되는가?
- 같은 날짜가 중복되지 않는가?
- 병합 후 일자별 매출 행 수가 유지되는가?
- 내부 데이터 기간과 외부 데이터 기준일이 겹치는가?
- 출처 파일의 SHA-256이 기록되었는가?
- 공휴일과 일반일 표본이 모두 존재하는가?

공휴일 평균 매출 차이가 보여도 공휴일이 매출 변화의 원인이라고 단정할 수 없습니다.

11. LLM 사용 기록은 실제 사용 내역만 작성한다

파이프라인은 빈 로그 템플릿을 생성합니다.

```
display(result["llm_usage_log"])
```

주요 컬럼은 다음과 같습니다.

| 컬럼 | 기록 내용 |
|-------------------|--------------|
| executed_at | 사용 날짜와 시간 |
| provider | 사용 서비스 |
| model | 실제 모델명 |
| prompt_version | 프롬프트 버전 |
| step | 사용 단계 |
| input_summary | 제공한 구조·집계 정보 |
| response_summary | 답변 요약 |
| validation_result | 실행·수치·논리 검증 |
| revision_note | 사람이 수정한 내용 |
| final_use | 사용·부분 사용·미사용 |

LLM을 사용하지 않았다면 미사용 상태를 그대로 둡니다. 사용하지 않은 도구를 사용했다고 작성하거나, 빈 템플릿을 실제 사용 기록처럼 제출하지 않습니다.

LLM 입력에는 다음 내용을 포함하지 않습니다.

- 고객명과 연락처
- 개별 주문 원본
- 실제 API Key와 토큰
- 내부 URL과 민감한 파일 경로
- 계약상 외부 제공이 금지된 자료

12. 자동화는 검증 실패를 처리해야 한다

자동화 설계서는 다음 순서를 포함합니다.

- 원본 파일 확인
- 전처리
- 키 관계와 병합 검증
- 완료 주문 EDA
- 분류 모델 실행
- 외부 데이터 존재 여부 확인

- 프로젝트 검증
- 보고서와 manifest 생성
- 실패·경고 알림

단순히 Python 파일이 오류 없이 끝났다고 자동화가 성공한 것은 아닙니다.

- 필수 파일이 없으면 중단합니다.
- 키 관계가 깨지면 중단합니다.
- 완료 주문 매출 총합이 다르면 중단합니다.
- 외부 데이터가 없으면 선택 단계를 건너뛰니다.
- 모델 데이터가 부족하면 모델 단계를 건너뛰니다.
- 경고와 실패 상태를 로그와 알림에 남깁니다.

13. 프로젝트 검증표를 먼저 본다

```
validation = result["validation"]
display(validation)
```

검증 상태는 다음 의미로 사용합니다.

| 상태 | 의미 |
|------|--------------------|
| PASS | 필수 기준 충족 |
| WARN | 선택 단계 미실행 또는 해석 제한 |
| FAIL | 계산·관계·보안·산출물 오류 |

FAIL이 있다면 최종 보고서를 제출하기 전에 원인을 수정합니다. WARN은 무조건 오류가 아니지만, 보고서에 이유와 제한을 적어야 합니다.

14. 산출물 manifest로 재현성을 확인한다

```
manifest = result["manifest"]
display(manifest)
```

manifest에는 다음 정보가 포함됩니다.

- 산출물 이름
- 필수 여부
- 저장 경로
- 파일 존재 여부

- 파일 크기
- SHA-256

파일 해시는 분석 결과가 바뀌었는지 확인하는 보조 수단입니다. 해시가 같다고 분석 내용이 타당하다는 뜻은 아니므로 프로젝트 검증표와 함께 사용합니다.

15. 최종 보고서와 발표 자료

최종 보고서는 다음 구조를 권장합니다.

50. 프로젝트 목적과 분석 질문
51. 사용 데이터와 분석 범위
52. 전처리와 키 관계 검증
53. 완료 주문 기준 EDA
54. 시각화 결과
55. 분류 모델 결과 또는 미실행 사유
56. 외부 데이터 통합 결과 또는 미실행 사유
57. LLM 사용 및 검증 기록
58. 자동화 설계
59. 프로젝트 검증 결과
60. 결론, 한계, 다음 단계

발표에서는 코드 줄을 읽기보다 다음 내용을 설명합니다.

- 왜 완료 주문만 매출로 정의했는가?
- 어떤 병합 검증을 수행했는가?
- 핵심 지표가 같은 매출 총합을 사용하는가?
- 모델과 임계값을 어떻게 선택했는가?
- 외부 데이터가 실제 출처 데이터인가?
- 어떤 결과가 관찰이고 어떤 내용이 가설인가?
- WARN 또는 FAIL 항목은 무엇인가?

피해야 할 표현은 다음과 같습니다.

- 공휴일 때문에 매출이 증가했습니다.
- 모델이 고객 행동의 원인을 설명합니다.
- LLM이 생성했으므로 결과가 정확합니다.

- 높은 정확도이므로 바로 운영에 적용할 수 있습니다.

더 안전한 표현은 다음과 같습니다.

- 공휴일 여부에 따른 평균 매출 차이가 관찰되었습니다.
- 현재 입력값 범위에서 취소 여부와 관련된 패턴을 확인했습니다.
- LLM 제안은 실행과 수치 검증 후 일부 반영했습니다.
- 운영 적용 전 추가 데이터와 시간 순서 평가가 필요합니다.

16. 제출 전 체크리스트

| 영역 | 점검 항목 |
|--------|------------------------------|
| 목적 | 분석 질문과 의사결정 목적이 명확한가? |
| 범위 | 완료 주문 매출 기준을 설명했는가? |
| 데이터 | 구조, 결측치, 중복, 키 관계를 확인했는가? |
| 병합 | validate, 행 수, 미매칭을 확인했는가? |
| EDA | 카테고리·월·고객·상품 총합이 일치하는가? |
| 개인정보 | 고객 ID, 이름, 연락처를 결과에서 제외했는가? |
| 모델 | 타깃 범위, 분할, 기준 모델, 지표를 설명했는가? |
| 외부 데이터 | 출처, 기준일, 라이선스, 파일 해시를 기록했는가? |
| LLM | 실제 사용 조건과 수정 내용을 기록했는가? |
| 자동화 | 실패·경고 상태 처리 방법이 있는가? |
| 보고서 | 관찰, 가설, 한계를 구분했는가? |
| 산출물 | manifest의 필수 파일이 모두 존재하는가? |

17. 한 번에 실행하기

프로젝트 루트에서 실행합니다.

```
python scripts/generate_sample_data.py
python scripts/run_final_project.py
```

출력에서 다음 내용을 먼저 확인합니다.

[프로젝트 검증]
[분류 단계]
[외부 데이터 단계]
[최종 보고서]
[산출물 manifest]

최종 프로젝트의 목적은 완벽한 모델을 만드는 것이 아닙니다. **분석 질문부터 데이터 범위, 검증, 해석, 보고서까지 연결된 재현 가능한 분석 흐름을 만드는 것이 핵심입니다.**